PlussMic User Documentation

PlussMa Contributors

Feb 28, 2026

CONTENTS

1 Quickstart **3**
- 1.1 Include Paths . . . 3
- 1.2 Load A Table Fast . . . 3
- 1.3 Start With A Small In-Memory Frame . . . 3
- 1.4 Typical DataFrame Workflow . . . 3
- 1.5 Convert Between Models . . . 4
- 1.6 Save Results . . . 4

2 Demo Catalog **5**
- 2.1 Table Examples . . . 5
- 2.2 Kindle Examples . . . 5
- 2.3 Bundle Examples . . . 5

3 DataFrame Guide **7**
- 3.1 What DataFrame Is Good At . . . 7
- 3.2 Construction And Loading . . . 7
- 3.3 Inspect Rows, Features, And Cases . . . 7
- 3.4 Cast And Derive Features . . . 8
- 3.5 Filter, Query, And Sort . . . 8
- 3.6 Grouping, Pivoting, And Summaries . . . 9
- 3.7 Missing Data And Rolling Windows . . . 9
- 3.8 Join And Combine . . . 9
- 3.9 Time-Aware Table Verbs . . . 10
- 3.10 SQL And IO . . . 10

4 DataFrame API Reference **11**

5 Practical DataFrame API Reference **13**
- 5.1 API Reference Style . . . 13

6 ColumnRef API Reference **219**

7 Practical ColumnRef API Reference **221**
- 7.1 API Reference Style . . . 221

8 DataFramePlotAccessor API Reference **227**

9 Practical DataFramePlotAccessor API Reference **229**
- 9.1 API Reference Style . . . 229

10 DataFrameSqlAccessor API Reference **233**

11 Practical DataFrameSqlAccessor API Reference **235**
11.1 API Reference Style . . . 235

12 DataFrameSqlPrepared API Reference **237**

13 Practical DataFrameSqlPrepared API Reference **239**
13.1 API Reference Style . . . 239

14 Thread And Atlas Guide **243**
14.1 Thread . . . 243
14.2 Row-Space Helpers . . . 243
14.3 Atlas . . . 243
14.4 Atlas To Kindle . . . 244

15 Thread API Reference **245**

16 Practical Thread API Reference **247**
16.1 API Reference Style . . . 247

17 ThreadValue API Reference **291**

18 Practical ThreadValue API Reference **293**
18.1 API Reference Style . . . 293

19 Atlas API Reference **301**

20 Practical Atlas API Reference **303**
20.1 API Reference Style . . . 303

21 Kindle Model Guide **327**
21.1 When To Use Kindle Types . . . 327
21.2 Stream Basics . . . 327
21.3 Catalog Basics . . . 327
21.4 Text And Similarity Helpers . . . 328
21.5 Archive Basics . . . 328
21.6 Convert Back To Table Model . . . 328

22 Stream API Reference **329**

23 Practical Stream API Reference **331**
23.1 API Reference Style . . . 331

24 Catalog API Reference **367**

25 Practical Catalog API Reference **369**
25.1 API Reference Style . . . 369

26 Archive API Reference **455**

27 Practical Archive API Reference **457**
27.1 API Reference Style . . . 457

28 Bundle Model Guide **477**
28.1 When To Use Bundle Types . . . 477
28.2 Slot . . . 477
28.3 Registry . . . 477
28.4 Aisle . . . 478

28.5 Kindle Bridge . . . 478

29 Slot API Reference **479**

30 Practical Slot API Reference **481**
30.1 API Reference Style . . . 481

31 Registry API Reference **495**

32 Practical Registry API Reference **497**
32.1 API Reference Style . . . 497

33 Aisle API Reference **569**

34 Practical Aisle API Reference **571**
34.1 API Reference Style . . . 571

35 Build Outputs **587**
35.1 HTML . . . 587
35.2 PDF . . . 587
35.3 Notes . . . 587

PlussMic is the data-analysis layer in the Pluss stack. It gives you three related models:

- `table`: `DataFrame`, `Thread`, `Atlas`
- `kindle`: `Stream`, `Catalog`, `Archive`
- `bundle`: `Slot`, `Registry`, `Aisle`

This manual is organized the same way users usually approach the library:

- start with `DataFrame` for table-style analysis
- use `Thread` when you want typed column operations outside a frame
- use `Catalog` and `Archive` when streams/case-aligned data are a better fit
- use `Registry` and `Aisle` when you want heterogeneous record bundles

CHAPTER

ONE

QUICKSTART

1.1 Include Paths

```
#include <pluss/pluss.h>
// or, when you only want PlussMic:
#include <pluss/mic.h>
```

1.2 Load A Table Fast

```
auto df = pluss::table::DataFrame::load_csv(
    "PlussMic/data/messy_data.csv",
    {{"nrows", 16}});

std::cout << df << "\n";
df.info();
df.describe();
```

1.3 Start With A Small In-Memory Frame

```
auto sales = pluss::table::DataFrame({
    {"city", pluss::table::Thread(pluss::make_note({"Accra", "Tema", "Kumasi"}))},
    {"revenue", pluss::table::Thread(pluss::make_series({120.0, 95.0, 140.0}))},
    {"opened_at", pluss::table::Thread(pluss::make_timeline({
        "2024-01-01T09:00:00Z",
        "2024-01-02T09:00:00Z",
        "2024-01-03T09:00:00Z",
    }))},
});

sales.head();
```

1.4 Typical DataFrame Workflow

```
auto accidents = pluss::table::DataFrame::load_csv(
    "PlussMic/data/messy_data.csv",
    {{"nrows", 64}})
    .fillna(0.0, {"Lifecycle Cost", "Cost Variance (%)"})
```

(continues on next page)

(continued from previous page)

```
    .query("isnotnull(Agency Name)")
    .sort_values({"Lifecycle Cost"}, false)
    .head(10, false);

accidents.show();
```

1.5 Convert Between Models

```
auto df = pluss::table::DataFrame::load_csv("PlussMic/data/messy_data.csv", {{"nrows", 8}
↪});
auto catalog = df.kindelize();
auto roundtrip_df = catalog.tabelize();

auto registry = pluss::bundle::Registry::from_kindle(catalog);
auto back_catalog = registry.to_kindle();
```

1.6 Save Results

```
auto out = pluss::table::DataFrame::load_json("PlussMic/data/weather.json", {{"nrows",␣
↪12}});

out.save_csv("/tmp/weather_demo.csv");
out.save_excel("/tmp/weather_demo.xlsx");
out.save_json("/tmp/weather_demo.json");
```

CHAPTER

TWO

DEMO CATALOG

2.1 Table Examples

- `PlussMic/examples/table/dataframe_initial_demo.cpp`: loading CSV files, viewing rows, info, describe, and a simple `Atlas`
- `PlussMic/examples/table/dataframe_describe_demo.cpp`: summary statistics plus CSV loader benchmarking
- `PlussMic/examples/table/dataframe_view_demo.cpp`: row windows and quick inspection patterns
- `PlussMic/examples/table/dataframe_thread_values_demo.cpp`: working with typed `Thread` columns
- `PlussMic/examples/table/dataframe_versatile_methods_demo.cpp`: selection, transforms, grouping, joins, and reshaping verbs
- `PlussMic/examples/table/loader_saver_parity_demo.cpp`: CSV, TXT, JSON, Excel, and Parquet loader/saver parity
- `PlussMic/examples/table/dataframe_plot_demo.cpp`: plotting accessors and integration with PlussPl
- `PlussMic/examples/table/display_policy_demo.cpp`: terminal display policy and compact rendering
- `PlussMic/examples/table/eda_us_accidents.cpp`: end-to-end exploratory analysis workflow on the accidents dataset
- `PlussMic/examples/table/us_accidents_eda_verbs_demo.cpp`: richer EDA verbs and derived-feature workflow

2.2 Kindle Examples

- `PlussMic/examples/kindle/kindle_model_demo.cpp`: `Stream`, `Catalog`, and `Archive` workflows

2.3 Bundle Examples

- `PlussMic/examples/bundle/bundle_model_demo.cpp`: `Slot`, `Registry`, and `Aisle` workflows

CHAPTER

THREE

DATAFRAME GUIDE

3.1 What DataFrame Is Good At

`pluss::table::DataFrame` is the main table-analysis surface in PlussMic. Use it when you want:

- named features with mixed typed columns
- a case axis that can be numeric, note-based, timeline-based, or time-index based
- table verbs such as `query`, `assign`, `group_by`, `pivot_table`, `join`, `fillna`, and `describe`
- import/export across CSV, TXT, JSON, Excel, and Parquet

3.2 Construction And Loading

```
auto df = pluss::table::DataFrame({
    {"city", pluss::table::Thread(pluss::make_note({"Accra", "Tema", "Kumasi"}))},
    {"sales", pluss::table::Thread(pluss::make_series({100.0, 120.0, 90.0}))},
    {"opened", pluss::table::Thread(pluss::make_timeline({
        "2024-01-01T08:00:00Z",
        "2024-01-02T08:00:00Z",
        "2024-01-03T08:00:00Z",
    }))},
});

auto csv_df = pluss::table::DataFrame::load_csv(
    "PlussMic/data/messy_data.csv",
    {{"nrows", 32}});

auto excel_df = pluss::table::DataFrame::load_excel(
    "PlussMic/data/report.xlsx",
    {{"sheetindex", 0}, {"header", true}});

auto json_df = pluss::table::DataFrame::load_json(
    "PlussMic/data/weather.json",
    {{"nrows", 24}});
```

3.3 Inspect Rows, Features, And Cases

```
df.head();
df.tail();
```

(continues on next page)

(continued from previous page)

```
df.info();
df.describe();

auto city = df.features("city");
auto sales = df.features("sales");
auto case_axis = df.cases();

auto first_row = df.iloc(0);
auto first_value = df.at(0, "city");
```

Notes:

- `features("name")` returns a typed `Thread`
- `cases()` is the row axis, not just a display index
- `describe()` is the fastest way to check whether numeric casts and missing values look right

3.4 Cast And Derive Features

```
auto work = csv_df
    .cast_timeline({"Business Case ID"})
    .astype({{"Lifecycle Cost", pluss::codem::ScalarType::Float64}})
    .assign({
        pluss::table::DataFrame::DeriveSpec{
            "Cost_plus_buffer",
            [](const pluss::table::DataFrame& f) {
              return pluss::table::Thread(f.features("Lifecycle Cost").as_series().
↪add(500.0));
            },
        },
    });
```

3.5 Filter, Query, And Sort

```
auto positive = df.filter(df.features("sales").as_series().gt(95.0));

auto queried = df.query("sales > 95");

auto sorted = df.sort_values({"sales"}, false);

auto top3 = df.top("sales", 3);
```

Notes:

- use `filter(bool_mask)` when you already have a boolean `Series`
- use `query("...")` when the filter is easier to read as a table expression
- `top` and `bottom` are good for ranking tasks without writing extra sort/slice steps

3.6 Grouping, Pivoting, And Summaries

```
auto grouped = df.group_by(
    {"city"},
    {{"avg_sales", "sales", pluss::table::DataFrame::GroupOp::Mean}},
    true,
    false);

auto pivoted = df.pivot_table(
    "city",
    "city",
    "sales",
    pluss::table::DataFrame::GroupOp::Mean,
    true,
    false);

auto stats = df.aggregate({
    {"sales_mean", "sales", pluss::table::DataFrame::SummaryOp::Mean},
    {"sales_max", "sales", pluss::table::DataFrame::SummaryOp::Max},
}, false);
```

3.7 Missing Data And Rolling Windows

```
auto cleaned = df.fillna(0.0, {"sales"});
auto dropped = df.dropna();
auto carried = df.ffill();

auto rolling = df.window({
    {"sales_roll3", "sales", pluss::table::ThreadWindowOp::RollingMean, 3, 1},
});
```

3.8 Join And Combine

```
auto left = pluss::table::DataFrame({
    {"city", pluss::table::Thread(pluss::make_note({"Accra", "Tema"}))},
    {"sales", pluss::table::Thread(pluss::make_series({100.0, 120.0}))},
});

auto right = pluss::table::DataFrame({
    {"city", pluss::table::Thread(pluss::make_note({"Accra", "Tema"}))},
    {"manager", pluss::table::Thread(pluss::make_note({"Ama", "Kojo"}))},
});

auto joined = left.stitch_on({"city"}, right, {"city"}, "_r");
auto updated = left.update(right, true);
```

3.9 Time-Aware Table Verbs

```
auto schedule = pluss::table::DataFrame({
    {"ts", pluss::table::Thread(pluss::make_timeline({
        "2024-01-01T09:10:00Z",
        "2024-01-01T09:40:00Z",
        "2024-01-01T10:05:00Z",
    }))},
    {"value", pluss::table::Thread(pluss::make_series({10.0, 20.0, 30.0}))},
});

auto hourly = schedule.resample("ts", {"value"}, "1H", "mean", 1, false);
auto floored = schedule.floor_time({"ts"}, "hour");
auto utc = schedule.to_utc({"ts"});
```

3.10 SQL And IO

```
auto result = df.sql().query("SELECT city, sales FROM self ORDER BY sales DESC");

result.save_csv("/tmp/plussmic_sales.csv");
result.save_excel("/tmp/plussmic_sales.xlsx");
result.save_json("/tmp/plussmic_sales.json");
```

CHAPTER

FOUR

DATAFRAME API REFERENCE

CHAPTER

FIVE

PRACTICAL DATAFRAME API REFERENCE

5.1 API Reference Style

This page documents the full public `DataFrame` surface with usage-first examples generated from the current table API.

5.1.1 add

Signature

```
DataFrame add(self, const DataFrame& other) const
```

Parameters

`other` : `const DataFrame&` See the signature for the supported argument form.

Returns

`DataFrame` Value produced by this call.

Notes

This applies arithmetic across the compatible numeric payloads in the current object.

See also

- `DataFrame add(self, scalar other) const`

Examples

```
auto left = pluss::table::DataFrame({
    {"id", pluss::table::Thread(pluss::make_series({1, 2, 3}))},
    {"sales", pluss::table::Thread(pluss::make_series({100.0, 120.0, 90.0}))},
});
auto right = pluss::table::DataFrame({
    {"id", pluss::table::Thread(pluss::make_series({1, 2, 3}))},
    {"cost", pluss::table::Thread(pluss::make_series({60.0, 80.0, 55.0}))},
});

auto out = left.add(right);
```

Output

```
id  sales  city
1   100.0  Accra
2   120.0  Tema
3    90.0  Kumasi
```

5.1.2 add

Signature

```
DataFrame add(self, scalar other) const
```

Parameters

`other` : `scalar` See the signature for the supported argument form.

Returns

`DataFrame` Value produced by this call.

Notes

This applies arithmetic across the compatible numeric payloads in the current object.

See also

- `DataFrame add(self, const DataFrame& other) const`

Examples

```
auto df = pluss::table::DataFrame({
    {"id", pluss::table::Thread(pluss::make_series({1, 2, 3}))},
    {"sales", pluss::table::Thread(pluss::make_series({100.0, 120.0, 90.0}))},
    {"city", pluss::table::Thread(pluss::make_note({"Accra", "Tema", "Kumasi"}))},
});

auto out = df.add(10);
```

Output

```
id  sales  city
11  110.0  Accra
12  130.0  Tema
13  100.0  Kumasi
```

5.1.3 add_prefix

Signature

```
DataFrame add_prefix(self, std::string_view prefix) const
```

Parameters

`prefix` : `std::string_view` See the signature for the supported argument form.

Returns

`DataFrame` Value produced by this call.

Notes

This adjusts labels without changing the underlying payload values.

Examples

```
auto df = pluss::table::DataFrame({
    {"id", pluss::table::Thread(pluss::make_series({1, 2, 3}))},
    {"sales", pluss::table::Thread(pluss::make_series({100.0, 120.0, 90.0}))},
    {"city", pluss::table::Thread(pluss::make_note({"Accra", "Tema", "Kumasi"}))},
});

auto out = df.add_prefix("feat_");
```

Output

```
df.head():
id  sales  city
1   100.0  Accra
2   120.0  Tema
3    90.0  Kumasi

out.head():
feat_id  feat_sales  feat_city
1        100.0       Accra
2        120.0       Tema
3         90.0       Kumasi
```

5.1.4 add_suffix

Signature

```
DataFrame add_suffix(self, std::string_view suffix) const
```

Parameters

`suffix` : `std::string_view` See the signature for the supported argument form.

Returns

`DataFrame` Value produced by this call.

Notes

This adjusts labels without changing the underlying payload values.

Examples

```
auto df = pluss::table::DataFrame({
    {"id", pluss::table::Thread(pluss::make_series({1, 2, 3}))},
    {"sales", pluss::table::Thread(pluss::make_series({100.0, 120.0, 90.0}))},
    {"city", pluss::table::Thread(pluss::make_note({"Accra", "Tema", "Kumasi"}))},
});

auto out = df.add_suffix("_2024");
```

Output

```
df.head():
id  sales  city
1   100.0  Accra
2   120.0  Tema
3    90.0  Kumasi

out.head():
id_2024  sales_2024  city_2024
1        100.0       Accra
2        120.0       Tema
3         90.0       Kumasi
```

5.1.5 aggregate

Signature

```
DataFrame aggregate(self, std::initializer_list<SummarySpec> specs, bool render = true)␣
↪const
```

Parameters

`specs` : `std::initializer_list<SummarySpec>` See the signature for the supported argument form.

`render` : `bool` See the signature for the supported argument form.

Returns

`DataFrame` Value produced by this call.

Notes

Use this when you want a summary view instead of carrying the full input forward unchanged.

Examples

```
auto df = pluss::table::DataFrame({
    {"city", pluss::table::Thread(pluss::make_note({"Accra", "Accra", "Tema", "Tema"}))},
    {"product", pluss::table::Thread(pluss::make_note({"A", "B", "A", "B"}))},
```

(continues on next page)

(continued from previous page)

```
    {"sales", pluss::table::Thread(pluss::make_series({100.0, 120.0, 90.0, 150.0}))},
});

auto out = df.aggregate({{"sales_mean", "sales",␣
↪pluss::table::DataFrame::SummaryOp::Mean}}, false);
```

Output

```
metric       value
sales_mean    115.0
```

5.1.6 align_cases

Signature

```
DataFrame align_cases(self, const Thread& new_cases) const
```

Parameters

`new_cases` : `const Thread&` See the signature for the supported argument form.

Returns

`DataFrame` Value produced by this call.

Notes

This call is part of the public `DataFrame` API and is documented here using a concrete usage example.

Examples

```
auto df = pluss::table::DataFrame({
    {"id", pluss::table::Thread(pluss::make_series({1, 2, 3}))},
    {"sales", pluss::table::Thread(pluss::make_series({100.0, 120.0, 90.0}))},
    {"city", pluss::table::Thread(pluss::make_note({"Accra", "Tema", "Kumasi"}))},
});

auto cases = pluss::table::Thread(pluss::make_note({"cs1", "cs0", "cs2"}));
auto out = df.align_cases(cases);
```

Output

```
id  sales  city
1   100.0  Accra
2   120.0  Tema
3    90.0  Kumasi
```

5.1.7 assign

Signature

```
DataFrame assign(self, std::initializer_list<DeriveSpec> specs) const
```

Parameters

specs : std::initializer_list<DeriveSpec> See the signature for the supported argument form.

Returns

DataFrame Value produced by this call.

Notes

This mutates or derives structure while keeping the surrounding object model intact.

See also

- DataFrame assign(self, std::initializer_list<std::pair<std::string, Thread>> assignments) const

Examples

```
auto df = pluss::table::DataFrame({
    {"id", pluss::table::Thread(pluss::make_series({1, 2, 3}))},
    {"sales", pluss::table::Thread(pluss::make_series({100.0, 120.0, 90.0}))},
    {"city", pluss::table::Thread(pluss::make_note({"Accra", "Tema", "Kumasi"}))},
});

auto out = df.assign({{.feature = "profit", .expr = [](const pluss::table::DataFrame&␣
↪frame) { return pluss::table::Thread(frame.features("sales").as_series().sub(60)); }}}
↪);
```

Output

```
id  sales  city    profit
1   100.0  Accra   40.0
2   120.0  Tema    35.0
3    90.0  Kumasi  30.0
```

5.1.8 assign

Signature

```
DataFrame assign(self, std::initializer_list<std::pair<std::string, Thread>>␣
↪assignments) const
```

Parameters

assignments : std::initializer_list<std::pair<std::string, Thread>> See the signature for the supported argument form.

Returns

DataFrame Value produced by this call.

Notes

This mutates or derives structure while keeping the surrounding object model intact.

See also

- DataFrame assign(self, std::initializer_list<DeriveSpec> specs) const

Examples

```
auto df = pluss::table::DataFrame({
    {"id", pluss::table::Thread(pluss::make_series({1, 2, 3}))},
    {"sales", pluss::table::Thread(pluss::make_series({100.0, 120.0, 90.0}))},
    {"city", pluss::table::Thread(pluss::make_note({"Accra", "Tema", "Kumasi"}))},
});

auto out = df.assign({{"profit", pluss::table::Thread(pluss::make_series({40.0, 35.0, 30.
↪0}))}});
```

Output

```
id  sales  city    profit
1   100.0  Accra   40.0
2   120.0  Tema    35.0
3    90.0  Kumasi  30.0
```

5.1.9 astype

Signature

```
DataFrame astype(self,  std::initializer_list<std::pair<std::string_view,␣
↪pluss::codem::ScalarType>> feature_dtypes) const
```

Parameters

feature_dtypes : std::initializer_list<std::pair<std::string_view, pluss::codem::ScalarType>> See the signature for the supported argument form.

Returns

DataFrame Value produced by this call.

Notes

This changes the semantic or numeric type so later operations route through the correct implementation path.

See also

- DataFrame astype(self, std::initializer_list<std::string_view> features, pluss::codem::ScalarType dtype) const

Examples

```
auto df = pluss::table::DataFrame({{"sales", pluss::table::Thread(pluss::make_note({"100.
↪0", "120.0", "90.0"}))}});

auto out = df.astype({{"sales", pluss::codem::ScalarType::Float64}});
```

Output

```
id  sales  city
1   100.0  Accra
2   120.0  Tema
3    90.0  Kumasi
```

5.1.10 astype

Signature

```
DataFrame astype(self, std::initializer_list<std::string_view> features,␣
↪pluss::codem::ScalarType dtype) const
```

Parameters

features : std::initializer_list<std::string_view> See the signature for the supported argument form.

dtype : pluss::codem::ScalarType See the signature for the supported argument form.

Returns

DataFrame Value produced by this call.

Notes

This changes the semantic or numeric type so later operations route through the correct implementation path.

See also

- DataFrame astype(self, std::initializer_list<std::pair<std::string_view, pluss::codem::ScalarType>> feature_dtypes) const

Examples

```
auto df = pluss::table::DataFrame({
    {"id", pluss::table::Thread(pluss::make_series({1, 2, 3}))},
    {"sales", pluss::table::Thread(pluss::make_series({100.0, 120.0, 90.0}))},
    {"city", pluss::table::Thread(pluss::make_note({"Accra", "Tema", "Kumasi"}))},
});

auto out = df.astype();
```

Output

```
id  sales  city
1   100.0  Accra
2   120.0  Tema
3    90.0  Kumasi
```

5.1.11 at

Signature

```
std::string at(self, std::int64_t row, std::string_view feature_name) const
```

Parameters

`row` : `std::int64_t` See the signature for the supported argument form.

`feature_name` : `std::string_view` See the signature for the supported argument form.

Returns

`std::string` Rendered string result.

Notes

This call is part of the public `DataFrame` API and is documented here using a concrete usage example.

Examples

```
auto df = pluss::table::DataFrame({
    {"id", pluss::table::Thread(pluss::make_series({1, 2, 3}))},
    {"sales", pluss::table::Thread(pluss::make_series({100.0, 120.0, 90.0}))},
    {"city", pluss::table::Thread(pluss::make_note({"Accra", "Tema", "Kumasi"}))},
});

auto out = df.at(0, "city");
```

Output

```
1
```

5.1.12 bfill

Signature

```
DataFrame bfill(self) const
```

Parameters

None

Returns

`DataFrame` Value produced by this call.

Notes

This is part of the missing-data cleanup path and is usually applied before summaries, grouping, or export.

Examples

```
auto df = pluss::table::DataFrame({
    {"id", pluss::table::Thread(pluss::make_series({1, 2, 3}))},
    {"sales", pluss::table::Thread(pluss::make_series({100.0, 120.0, 90.0}))},
    {"city", pluss::table::Thread(pluss::make_note({"Accra", "Tema", "Kumasi"}))},
});

auto out = df.bfill();
```

Output

```
id  sales  city
1   100.0  Accra
2   120.0  Tema
3    90.0  Kumasi
```

5.1.13 bottom

Signature

```
DataFrame bottom(self, std::string_view feature, std::int64_t n) const
```

Parameters

`feature` : `std::string_view` See the signature for the supported argument form.

`n` : `std::int64_t` See the signature for the supported argument form.

Returns

`DataFrame` Value produced by this call.

Notes

This call is part of the public `DataFrame` API and is documented here using a concrete usage example.

Examples

```
auto df = pluss::table::DataFrame({
    {"id", pluss::table::Thread(pluss::make_series({1, 2, 3}))},
    {"sales", pluss::table::Thread(pluss::make_series({100.0, 120.0, 90.0}))},
    {"city", pluss::table::Thread(pluss::make_note({"Accra", "Tema", "Kumasi"}))},
});

auto out = df.bottom("sales", 2);
```

Output

```
id  sales  city
1   100.0  Accra
2   120.0  Tema
3    90.0  Kumasi
```

5.1.14 cases

Signature

```
const Thread& cases(self) const
```

Parameters

None

Returns

`const Thread&` Value produced by this call.

Notes

This call is part of the public `DataFrame` API and is documented here using a concrete usage example.

See also

- `DataFrame cases(self, std::int64_t row) const`
- `DataFrame cases(self, std::pair<std::int64_t, std::int64_t> row_span) const`
- `std::string cases(self, std::int64_t row, std::size_t feature_idx) const`
- `std::string cases(self, std::int64_t row, std::string_view feature_name) const`

Examples

```
auto df = pluss::table::DataFrame({
    {"id", pluss::table::Thread(pluss::make_series({1, 2, 3}))},
    {"sales", pluss::table::Thread(pluss::make_series({100.0, 120.0, 90.0}))},
    {"city", pluss::table::Thread(pluss::make_note({"Accra", "Tema", "Kumasi"}))},
});

auto out = df.cases(0);
```

5.1.15 cases

Signature

```
DataFrame cases(self, std::int64_t row) const
```

Parameters

`row` : `std::int64_t` See the signature for the supported argument form.

Returns

`DataFrame` Value produced by this call.

Notes

This call is part of the public `DataFrame` API and is documented here using a concrete usage example.

See also

- `const Thread& cases(self) const`
- `DataFrame cases(self, std::pair<std::int64_t, std::int64_t> row_span) const`
- `std::string cases(self, std::int64_t row, std::size_t feature_idx) const`
- `std::string cases(self, std::int64_t row, std::string_view feature_name) const`

Examples

```
auto df = pluss::table::DataFrame({
    {"id", pluss::table::Thread(pluss::make_series({1, 2, 3}))},
    {"sales", pluss::table::Thread(pluss::make_series({100.0, 120.0, 90.0}))},
    {"city", pluss::table::Thread(pluss::make_note({"Accra", "Tema", "Kumasi"}))},
});

auto out = df.cases(0);
```

Output

```
id  sales  city
1   100.0  Accra
2   120.0  Tema
3    90.0  Kumasi
```

5.1.16 cases

Signature

```
DataFrame cases(self, std::pair<std::int64_t, std::int64_t> row_span) const
```

Parameters

`row_span` : `std::pair<std::int64_t, std::int64_t>` See the signature for the supported argument form.

Returns

`DataFrame` Value produced by this call.

Notes

This call is part of the public `DataFrame` API and is documented here using a concrete usage example.

See also

- `const Thread& cases(self) const`
- `DataFrame cases(self, std::int64_t row) const`
- `std::string cases(self, std::int64_t row, std::size_t feature_idx) const`
- `std::string cases(self, std::int64_t row, std::string_view feature_name) const`

Examples

```
auto df = pluss::table::DataFrame({
    {"id", pluss::table::Thread(pluss::make_series({1, 2, 3}))},
    {"sales", pluss::table::Thread(pluss::make_series({100.0, 120.0, 90.0}))},
    {"city", pluss::table::Thread(pluss::make_note({"Accra", "Tema", "Kumasi"}))},
});

auto out = df.cases({0, 2});
```

Output

```
id  sales  city
1   100.0  Accra
2   120.0  Tema
3    90.0  Kumasi
```

5.1.17 cases

Signature

```
std::string cases(self, std::int64_t row, std::size_t feature_idx) const
```

Parameters

`row` : `std::int64_t` See the signature for the supported argument form.

`feature_idx` : `std::size_t` See the signature for the supported argument form.

Returns

`std::string` Rendered string result.

Notes

This call is part of the public `DataFrame` API and is documented here using a concrete usage example.

See also

- `const Thread& cases(self) const`
- `DataFrame cases(self, std::int64_t row) const`
- `DataFrame cases(self, std::pair<std::int64_t, std::int64_t> row_span) const`

- std::string cases(self, std::int64_t row, std::string_view feature_name) const

Examples

```
auto df = pluss::table::DataFrame({
    {"id", pluss::table::Thread(pluss::make_series({1, 2, 3}))},
    {"sales", pluss::table::Thread(pluss::make_series({100.0, 120.0, 90.0}))},
    {"city", pluss::table::Thread(pluss::make_note({"Accra", "Tema", "Kumasi"}))},
});

auto out = df.cases(0, 0);
```

Output

```
1
```

5.1.18 cases

Signature

```
std::string cases(self, std::int64_t row, std::string_view feature_name) const
```

Parameters

row : std::int64_t See the signature for the supported argument form.

feature_name : std::string_view See the signature for the supported argument form.

Returns

std::string Rendered string result.

Notes

This call is part of the public DataFrame API and is documented here using a concrete usage example.

See also

- const Thread& cases(self) const
- DataFrame cases(self, std::int64_t row) const
- DataFrame cases(self, std::pair<std::int64_t, std::int64_t> row_span) const
- std::string cases(self, std::int64_t row, std::size_t feature_idx) const

Examples

```
auto df = pluss::table::DataFrame({
    {"id", pluss::table::Thread(pluss::make_series({1, 2, 3}))},
    {"sales", pluss::table::Thread(pluss::make_series({100.0, 120.0, 90.0}))},
    {"city", pluss::table::Thread(pluss::make_note({"Accra", "Tema", "Kumasi"}))},
});

auto out = df.cases(0, "city");
```

Output

```
1
```

5.1.19 cases_series

Signature

```
pluss::Series cases_series(self) const
```

Parameters

None

Returns

`pluss::Series` Value produced by this call.

Notes

This call is part of the public `DataFrame` API and is documented here using a concrete usage example.

Examples

```
auto df = pluss::table::DataFrame({
    {"id", pluss::table::Thread(pluss::make_series({1, 2, 3}))},
    {"sales", pluss::table::Thread(pluss::make_series({100.0, 120.0, 90.0}))},
    {"city", pluss::table::Thread(pluss::make_note({"Accra", "Tema", "Kumasi"}))},
});

auto out = df.cases_series();
```

Output

```
[0, 1, 2]
```

5.1.20 cast

Signature

```
DataFrame cast(self,  std::initializer_list<std::pair<std::string_view,␣
↪pluss::codem::ScalarType>> feature_dtypes) const
```

Parameters

`feature_dtypes` : `std::initializer_list<std::pair<std::string_view, pluss::codem::ScalarType>>` See the signature for the supported argument form.

Returns

`DataFrame` Value produced by this call.

Notes

This changes the semantic or numeric type so later operations route through the correct implementation path.

See also

- `DataFrame cast(self, std::initializer_list<CastSpec> specs) const`

Examples

```
auto df = pluss::table::DataFrame({{"sales", pluss::table::Thread(pluss::make_note({"100.
↪0", "120.0", "90.0"}))}});

auto out = df.astype({{"sales", pluss::codem::ScalarType::Float64}});
```

Output

```
id  sales  city
1   100.0  Accra
2   120.0  Tema
3    90.0  Kumasi
```

5.1.21 cast

Signature

```
DataFrame cast(self, std::initializer_list<CastSpec> specs) const
```

Parameters

`specs` : `std::initializer_list<CastSpec>` See the signature for the supported argument form.

Returns

`DataFrame` Value produced by this call.

Notes

This changes the semantic or numeric type so later operations route through the correct implementation path.

See also

- `DataFrame cast(self, std::initializer_list<std::pair<std::string_view, pluss::codem::ScalarType>> feature_dtypes) const`

Examples

```
auto df = pluss::table::DataFrame({{"sales", pluss::table::Thread(pluss::make_note({"100.
↪0", "120.0", "90.0"}))}});

auto out = df.cast({{.feature = "sales", .target =␣
↪pluss::table::ThreadCastTarget::Series, .series_dtype =␣
↪pluss::codem::ScalarType::Float64}});
```

Output

```
id  sales  city
1   100.0  Accra
2   120.0  Tema
3    90.0  Kumasi
```

5.1.22 cast_note

Signature

```
DataFrame cast_note(self, std::initializer_list<std::string_view> features) const
```

Parameters

`features` : `std::initializer_list<std::string_view>` See the signature for the supported argument form.

Returns

`DataFrame` Value produced by this call.

Notes

This changes the semantic or numeric type so later operations route through the correct implementation path.

Examples

```
auto df = pluss::table::DataFrame({
    {"id", pluss::table::Thread(pluss::make_series({1, 2, 3}))},
    {"sales", pluss::table::Thread(pluss::make_series({100.0, 120.0, 90.0}))},
    {"city", pluss::table::Thread(pluss::make_note({"Accra", "Tema", "Kumasi"}))},
});

auto out = df.cast_note({"sales"});
```

Output

```
id  sales  city
1   100.0  Accra
2   120.0  Tema
3    90.0  Kumasi
```

5.1.23 cast_series

Signature

```
DataFrame cast_series(self, std::initializer_list<std::string_view> features,␣
↪std::optional<pluss::codem::ScalarType> dtype = std::nullopt) const
```

Parameters

`features` : `std::initializer_list<std::string_view>` See the signature for the supported argument form.

`dtype` : `std::optional<pluss::codem::ScalarType>` See the signature for the supported argument form.

Returns

`DataFrame` Value produced by this call.

Notes

This changes the semantic or numeric type so later operations route through the correct implementation path.

Examples

```
auto df = pluss::table::DataFrame({{"sales", pluss::table::Thread(pluss::make_note({"100.
↪0", "120.0", "90.0"}))}});

auto out = df.cast_series({"sales"}, pluss::codem::ScalarType::Float64);
```

Output

```
id  sales  city
1   100.0  Accra
2   120.0  Tema
3    90.0  Kumasi
```

5.1.24 `cast_timeindex`

Signature

```
DataFrame cast_timeindex(self, std::initializer_list<std::string_view> features) const
```

Parameters

`features` : `std::initializer_list<std::string_view>` See the signature for the supported argument form.

Returns

`DataFrame` Value produced by this call.

Notes

This changes the semantic or numeric type so later operations route through the correct implementation path.

Examples

```
auto df = pluss::table::DataFrame({
    {"id", pluss::table::Thread(pluss::make_series({1, 2, 3}))},
    {"sales", pluss::table::Thread(pluss::make_series({100.0, 120.0, 90.0}))},
    {"city", pluss::table::Thread(pluss::make_note({"Accra", "Tema", "Kumasi"}))},
});

auto out = df.cast_timeindex({"sales"});
```

Output

```
id  sales  city
1   100.0  Accra
2   120.0  Tema
3    90.0  Kumasi
```

5.1.25 cast_timeline

Signature

```
DataFrame cast_timeline(self, std::initializer_list<std::string_view> features) const
```

Parameters

`features` : `std::initializer_list<std::string_view>` See the signature for the supported argument form.

Returns

`DataFrame` Value produced by this call.

Notes

This changes the semantic or numeric type so later operations route through the correct implementation path.

Examples

```
auto df = pluss::table::DataFrame({{"ts", pluss::table::Thread(pluss::make_note({"2024-
↪01-01T09:10:00Z", "2024-01-01T09:40:00Z"}))}});

auto out = df.cast_timeline({"ts"});
```

Output

```
id  sales  city
1   100.0  Accra
2   120.0  Tema
3    90.0  Kumasi
```

5.1.26 ceil_time

Signature

```
DataFrame ceil_time(self, std::initializer_list<std::string_view> features, std::string_
↪view unit) const
```

Parameters

`features` : `std::initializer_list<std::string_view>` See the signature for the supported argument form.

`unit` : `std::string_view` See the signature for the supported argument form.

Returns

`DataFrame` Value produced by this call.

Notes

This is part of the time-aware surface and expects timeline or time-index semantics.

Examples

```
auto df = pluss::table::DataFrame({
    {"ts", pluss::table::Thread(pluss::make_timeline({
        "2024-01-01T09:10:00Z",
        "2024-01-01T09:40:00Z",
        "2024-01-01T10:05:00Z",
    }))},
    {"load", pluss::table::Thread(pluss::make_series({10.0, 20.0, 15.0}))},
});

auto out = df.ceil_time({"ts"}, "hour");
```

Output

```
ts
2024-01-01T09:00:00Z
2024-01-01T10:00:00Z
```

5.1.27 `column`

Signature

```
Thread column(self, std::string_view name) const
```

Parameters

`name` : `std::string_view` See the signature for the supported argument form.

Returns

`Thread` Value produced by this call.

Notes

This call is part of the public `DataFrame` API and is documented here using a concrete usage example.

Examples

```
auto df = pluss::table::DataFrame({
    {"id", pluss::table::Thread(pluss::make_series({1, 2, 3}))},
    {"sales", pluss::table::Thread(pluss::make_series({100.0, 120.0, 90.0}))},
    {"city", pluss::table::Thread(pluss::make_note({"Accra", "Tema", "Kumasi"}))},
});

auto out = df.column("sales");
```

Output

```
Thread([100.0, 120.0, 90.0])
```

5.1.28 column_names

Signature

```
const std::vector<std::string>& column_names(self) const
```

Parameters

None

Returns

`const std::vector<std::string>&` Borrowed list of names.

Notes

This call is part of the public `DataFrame` API and is documented here using a concrete usage example.

Examples

```
auto df = pluss::table::DataFrame({
    {"id", pluss::table::Thread(pluss::make_series({1, 2, 3}))},
    {"sales", pluss::table::Thread(pluss::make_series({100.0, 120.0, 90.0}))},
    {"city", pluss::table::Thread(pluss::make_note({"Accra", "Tema", "Kumasi"}))},
});

auto out = df.column_names();
```

Output

```
["id", "sales", "city"]
```

5.1.29 column_ref

Signature

```
ColumnRef column_ref(self, std::string_view name) const
```

Parameters

`name` : `std::string_view` See the signature for the supported argument form.

Returns

`ColumnRef` Value produced by this call.

Notes

This call is part of the public `DataFrame` API and is documented here using a concrete usage example.

Examples

```
auto df = pluss::table::DataFrame({
    {"id", pluss::table::Thread(pluss::make_series({1, 2, 3}))},
    {"sales", pluss::table::Thread(pluss::make_series({100.0, 120.0, 90.0}))},
    {"city", pluss::table::Thread(pluss::make_note({"Accra", "Tema", "Kumasi"}))},
});

auto out = df.column_ref("sales");
```

5.1.30 columns

Signature

```
std::vector<Thread> columns(self) const
```

Parameters

None

Returns

`std::vector<Thread>` Materialized collection of thread objects.

Notes

This call is part of the public `DataFrame` API and is documented here using a concrete usage example.

Examples

```
auto df = pluss::table::DataFrame({
    {"id", pluss::table::Thread(pluss::make_series({1, 2, 3}))},
    {"sales", pluss::table::Thread(pluss::make_series({100.0, 120.0, 90.0}))},
    {"city", pluss::table::Thread(pluss::make_note({"Accra", "Tema", "Kumasi"}))},
});

auto out = df.columns();
```

Output

```
[Thread(id), Thread(sales), Thread(city)]
```

5.1.31 combine

Signature

```
DataFrame combine(self, const DataFrame& other) const
```

Parameters

`other` : `const DataFrame&` See the signature for the supported argument form.

Returns

`DataFrame` Value produced by this call.

Notes

Use this when you need to merge compatible objects or extend the current one with another input.

See also

- `DataFrame combine(self, const DataFrame& other, const CombineSpec& spec) const`

Examples

```
auto left = pluss::table::DataFrame({
    {"id", pluss::table::Thread(pluss::make_series({1, 2, 3}))},
    {"sales", pluss::table::Thread(pluss::make_series({100.0, 120.0, 90.0}))},
});
auto right = pluss::table::DataFrame({
    {"id", pluss::table::Thread(pluss::make_series({1, 2, 3}))},
    {"cost", pluss::table::Thread(pluss::make_series({60.0, 80.0, 55.0}))},
});

auto out = left.combine(right);
```

Output

```
id  sales  cost
1   100.0  60.0
2   120.0  80.0
3    90.0  55.0
```

5.1.32 combine

Signature

```
DataFrame combine(self, const DataFrame& other, const CombineSpec& spec) const
```

Parameters

`other` : `const DataFrame&` See the signature for the supported argument form.

`spec` : `const CombineSpec&` See the signature for the supported argument form.

Returns

`DataFrame` Value produced by this call.

Notes

Use this when you need to merge compatible objects or extend the current one with another input.

See also

- `DataFrame combine(self, const DataFrame& other) const`

Examples

```cpp
auto left = pluss::table::DataFrame({
    {"id", pluss::table::Thread(pluss::make_series({1, 2, 3}))},
    {"sales", pluss::table::Thread(pluss::make_series({100.0, 120.0, 90.0}))},
});
auto right = pluss::table::DataFrame({
    {"id", pluss::table::Thread(pluss::make_series({1, 2, 3}))},
    {"cost", pluss::table::Thread(pluss::make_series({60.0, 80.0, 55.0}))},
});

pluss::table::DataFrame::CombineSpec spec;
spec.op = pluss::table::DataFrame::CombineOp::Update;
auto out = left.combine(right, spec);
```

Output

```
id  sales  cost
1   100.0  60.0
2   120.0  80.0
3    90.0  55.0
```

5.1.33 `compare`

Signature

```cpp
DataFrame compare(self, const DataFrame& other) const
```

Parameters

`other` : `const DataFrame&` See the signature for the supported argument form.

Returns

`DataFrame` Value produced by this call.

Notes

This produces a comparison-style result you can inspect directly or reuse as a mask.

See also

- `DataFrame compare(self, scalar other) const`

Examples

```
auto left = pluss::table::DataFrame({
    {"id", pluss::table::Thread(pluss::make_series({1, 2, 3}))},
    {"sales", pluss::table::Thread(pluss::make_series({100.0, 120.0, 90.0}))},
});
auto right = pluss::table::DataFrame({
    {"id", pluss::table::Thread(pluss::make_series({1, 2, 3}))},
    {"cost", pluss::table::Thread(pluss::make_series({60.0, 80.0, 55.0}))},
});

auto out = left.compare(right);
```

Output

```
id  sales
1   true
2   true
3   false
```

5.1.34 compare

Signature

```
DataFrame compare(self, scalar other) const
```

Parameters

`other` : `scalar` See the signature for the supported argument form.

Returns

`DataFrame` Value produced by this call.

Notes

This produces a comparison-style result you can inspect directly or reuse as a mask.

See also

- `DataFrame compare(self, const DataFrame& other) const`

Examples

```
auto df = pluss::table::DataFrame({
    {"id", pluss::table::Thread(pluss::make_series({1, 2, 3}))},
    {"sales", pluss::table::Thread(pluss::make_series({100.0, 120.0, 90.0}))},
    {"city", pluss::table::Thread(pluss::make_note({"Accra", "Tema", "Kumasi"}))},
});

auto out = df.compare(10);
```

Output

```
id  sales
1   true
2   true
3   false
```

5.1.35 conform_to

Signature

```
DataFrame conform_to(self, const DataFrame& other) const
```

Parameters

`other` : `const DataFrame&` See the signature for the supported argument form.

Returns

`DataFrame` Value produced by this call.

Notes

This call is part of the public `DataFrame` API and is documented here using a concrete usage example.

Examples

```
auto left = pluss::table::DataFrame({
    {"id", pluss::table::Thread(pluss::make_series({1, 2, 3}))},
    {"sales", pluss::table::Thread(pluss::make_series({100.0, 120.0, 90.0}))},
});
auto right = pluss::table::DataFrame({
    {"id", pluss::table::Thread(pluss::make_series({1, 2, 3}))},
    {"cost", pluss::table::Thread(pluss::make_series({60.0, 80.0, 55.0}))},
});

auto out = left.conform_to(right);
```

Output

```
id  sales  city
1   100.0  Accra
2   120.0  Tema
3    90.0  Kumasi
```

5.1.36 copy

Signature

```
DataFrame copy(self) const
```

Parameters

None

Returns

DataFrame Value produced by this call.

Notes

This call is part of the public DataFrame API and is documented here using a concrete usage example.

Examples

```
auto df = pluss::table::DataFrame({
    {"id", pluss::table::Thread(pluss::make_series({1, 2, 3}))},
    {"sales", pluss::table::Thread(pluss::make_series({100.0, 120.0, 90.0}))},
    {"city", pluss::table::Thread(pluss::make_note({"Accra", "Tema", "Kumasi"}))},
});

auto out = df.copy();
```

Output

```
id  sales  city
1   100.0  Accra
2   120.0  Tema
3    90.0  Kumasi
```

5.1.37 count

Signature

```
DataFrame count(self, bool render = true) const
```

Parameters

render : bool See the signature for the supported argument form.

Returns

DataFrame Value produced by this call.

Notes

Use this when you want a summary view instead of carrying the full input forward unchanged.

Examples

```
auto df = pluss::table::DataFrame({
    {"id", pluss::table::Thread(pluss::make_series({1, 2, 3}))},
    {"sales", pluss::table::Thread(pluss::make_series({100.0, 120.0, 90.0}))},
    {"city", pluss::table::Thread(pluss::make_note({"Accra", "Tema", "Kumasi"}))},
});
```

(continues on next page)

(continued from previous page)

```
auto out = df.count(false);
```

Output

```
feature  value
sales    310.0
```

5.1.38 count_by

Signature

```
DataFrame count_by(self, std::string_view feature, std::int64_t top_n = 0, bool dropna =␣
↪true, bool render = true) const
```

Parameters

`feature` : `std::string_view` See the signature for the supported argument form.

`top_n` : `std::int64_t` See the signature for the supported argument form.

`dropna` : `bool` See the signature for the supported argument form.

`render` : `bool` See the signature for the supported argument form.

Returns

`DataFrame` Value produced by this call.

Notes

Use this when you want a summary view instead of carrying the full input forward unchanged.

Examples

```
auto df = pluss::table::DataFrame({
    {"city", pluss::table::Thread(pluss::make_note({"Accra", "Accra", "Tema", "Tema"}))},
    {"product", pluss::table::Thread(pluss::make_note({"A", "B", "A", "B"}))},
    {"sales", pluss::table::Thread(pluss::make_series({100.0, 120.0, 90.0, 150.0}))},
});

auto out = df.count_by("city", 3, true, false);
```

Output

```
city    count
Accra   2
Tema    2
```

5.1.39 count_null_rows

Signature

```
std::int64_t count_null_rows(self) const
```

Parameters

None

Returns

std::int64_t Integer count, size, or positional result.

Notes

This call is part of the public DataFrame API and is documented here using a concrete usage example.

Examples

```
auto df = pluss::table::DataFrame({
    {"id", pluss::table::Thread(pluss::make_series({1, 2, 3}))},
    {"sales", pluss::table::Thread(pluss::make_series({100.0, 120.0, 90.0}))},
    {"city", pluss::table::Thread(pluss::make_note({"Accra", "Tema", "Kumasi"}))},
});

auto out = df.count_null_rows();
```

Output

```
3
```

5.1.40 defined

Signature

```
bool defined(self) const
```

Parameters

None

Returns

bool Boolean status or membership result.

Notes

This call is part of the public DataFrame API and is documented here using a concrete usage example.

Examples

```
auto df = pluss::table::DataFrame({
    {"id", pluss::table::Thread(pluss::make_series({1, 2, 3}))},
    {"sales", pluss::table::Thread(pluss::make_series({100.0, 120.0, 90.0}))},
    {"city", pluss::table::Thread(pluss::make_note({"Accra", "Tema", "Kumasi"}))},
});

auto out = df.defined();
```

Output

```
true
```

5.1.41 describe

Signature

```
DataFrame describe(self, bool render = true) const
```

Parameters

`render` : `bool` See the signature for the supported argument form.

Returns

`DataFrame` Value produced by this call.

Notes

Use this when you want a summary view instead of carrying the full input forward unchanged.

Examples

```
auto df = pluss::table::DataFrame({
    {"id", pluss::table::Thread(pluss::make_series({1, 2, 3}))},
    {"sales", pluss::table::Thread(pluss::make_series({100.0, 120.0, 90.0}))},
    {"city", pluss::table::Thread(pluss::make_note({"Accra", "Tema", "Kumasi"}))},
});

auto out = df.describe(false);
```

Output

```
feature  count  mean   min   max
sales    3      103.3  90.0  120.0
```

5.1.42 div

Signature

```
DataFrame div(self, const DataFrame& other) const
```

Parameters

`other` : `const DataFrame&` See the signature for the supported argument form.

Returns

`DataFrame` Value produced by this call.

Notes

This applies arithmetic across the compatible numeric payloads in the current object.

See also

- `DataFrame div(self, scalar other) const`

Examples

```
auto left = pluss::table::DataFrame({
    {"id", pluss::table::Thread(pluss::make_series({1, 2, 3}))},
    {"sales", pluss::table::Thread(pluss::make_series({100.0, 120.0, 90.0}))},
});
auto right = pluss::table::DataFrame({
    {"id", pluss::table::Thread(pluss::make_series({1, 2, 3}))},
    {"cost", pluss::table::Thread(pluss::make_series({60.0, 80.0, 55.0}))},
});

auto out = left.div(right);
```

Output

```
id  sales  city
1   100.0  Accra
2   120.0  Tema
3    90.0  Kumasi
```

5.1.43 `div`

Signature

```
DataFrame div(self, scalar other) const
```

Parameters

`other` : `scalar` See the signature for the supported argument form.

Returns

`DataFrame` Value produced by this call.

Notes

This applies arithmetic across the compatible numeric payloads in the current object.

See also

- `DataFrame div(self, const DataFrame& other) const`

Examples

```
auto df = pluss::table::DataFrame({
    {"id", pluss::table::Thread(pluss::make_series({1, 2, 3}))},
    {"sales", pluss::table::Thread(pluss::make_series({100.0, 120.0, 90.0}))},
    {"city", pluss::table::Thread(pluss::make_note({"Accra", "Tema", "Kumasi"}))},
});

auto out = df.div(10);
```

Output

```
id  sales  city
11  110.0  Accra
12  130.0  Tema
13  100.0  Kumasi
```

5.1.44 divide

Signature

```
DataFrame divide(self, const DataFrame& other) const
```

Parameters

`other` : `const DataFrame&` See the signature for the supported argument form.

Returns

`DataFrame` Value produced by this call.

Notes

This applies arithmetic across the compatible numeric payloads in the current object.

See also

- `DataFrame divide(self, scalar other) const`

Examples

```
auto left = pluss::table::DataFrame({
    {"id", pluss::table::Thread(pluss::make_series({1, 2, 3}))},
    {"sales", pluss::table::Thread(pluss::make_series({100.0, 120.0, 90.0}))},
});
auto right = pluss::table::DataFrame({
```

(continues on next page)

(continued from previous page)

```
    {"id", pluss::table::Thread(pluss::make_series({1, 2, 3}))},
    {"cost", pluss::table::Thread(pluss::make_series({60.0, 80.0, 55.0}))},
});

auto out = left.divide(right);
```

Output

```
id  sales  city
1   100.0  Accra
2   120.0  Tema
3    90.0  Kumasi
```

5.1.45 divide

Signature

```
DataFrame divide(self, scalar other) const
```

Parameters

`other` : `scalar` See the signature for the supported argument form.

Returns

`DataFrame` Value produced by this call.

Notes

This applies arithmetic across the compatible numeric payloads in the current object.

See also

- `DataFrame divide(self, const DataFrame& other) const`

Examples

```
auto df = pluss::table::DataFrame({
    {"id", pluss::table::Thread(pluss::make_series({1, 2, 3}))},
    {"sales", pluss::table::Thread(pluss::make_series({100.0, 120.0, 90.0}))},
    {"city", pluss::table::Thread(pluss::make_note({"Accra", "Tema", "Kumasi"}))},
});

auto out = df.divide(10);
```

Output

```
id  sales  city
11  110.0  Accra
12  130.0  Tema
13  100.0  Kumasi
```

5.1.46 drop

Signature

```
DataFrame drop(self, std::initializer_list<std::string_view> names) const
```

Parameters

names : std::initializer_list<std::string_view> See the signature for the supported argument form.

Returns

DataFrame Value produced by this call.

Notes

Use this to reduce the current object to the rows or fields that matter for the next step.

Examples

```
auto df = pluss::table::DataFrame({
    {"id", pluss::table::Thread(pluss::make_series({1, 2, 3}))},
    {"sales", pluss::table::Thread(pluss::make_series({100.0, 120.0, 90.0}))},
    {"city", pluss::table::Thread(pluss::make_note({"Accra", "Tema", "Kumasi"}))},
});

auto out = df.drop({"city"});
```

Output

```
id  sales
1   100.0
2   120.0
3    90.0
```

5.1.47 dropna

Signature

```
DataFrame dropna(self) const
```

Parameters

None

Returns

DataFrame Value produced by this call.

Notes

This is part of the missing-data cleanup path and is usually applied before summaries, grouping, or export.

Examples

```
auto df = pluss::table::DataFrame({
    {"id", pluss::table::Thread(pluss::make_series({1, 2, 3}))},
    {"sales", pluss::table::Thread(pluss::make_series({100.0, 120.0, 90.0}))},
    {"city", pluss::table::Thread(pluss::make_note({"Accra", "Tema", "Kumasi"}))},
});

auto out = df.dropna();
```

Output

```
id  sales  city
1   100.0  Accra
2   120.0  Tema
3    90.0  Kumasi
```

5.1.48 dropna_all

Signature

```
DataFrame dropna_all(self, std::initializer_list<std::string_view> features =
```

Parameters

`features` : `std::initializer_list<std::string_view>` See the signature for the supported argument form.

Returns

`DataFrame` Value produced by this call.

Notes

This is part of the missing-data cleanup path and is usually applied before summaries, grouping, or export.

Examples

```
auto df = pluss::table::DataFrame({
    {"id", pluss::table::Thread(pluss::make_series({1, 2, 3}))},
    {"sales", pluss::table::Thread(pluss::make_series({100.0, 120.0, 90.0}))},
    {"city", pluss::table::Thread(pluss::make_note({"Accra", "Tema", "Kumasi"}))},
});

auto out = df.dropna_all({"sales"});
```

Output

```
id  sales  city
1   100.0  Accra
2   120.0  Tema
3    90.0  Kumasi
```

5.1.49 eq

Signature

```
DataFrame eq(self, const DataFrame& other) const
```

Parameters

`other` : `const DataFrame&` See the signature for the supported argument form.

Returns

`DataFrame` Value produced by this call.

Notes

This produces a comparison-style result you can inspect directly or reuse as a mask.

See also

- `DataFrame eq(self, scalar other) const`

Examples

```
auto left = pluss::table::DataFrame({
    {"id", pluss::table::Thread(pluss::make_series({1, 2, 3}))},
    {"sales", pluss::table::Thread(pluss::make_series({100.0, 120.0, 90.0}))},
});
auto right = pluss::table::DataFrame({
    {"id", pluss::table::Thread(pluss::make_series({1, 2, 3}))},
    {"cost", pluss::table::Thread(pluss::make_series({60.0, 80.0, 55.0}))},
});

auto out = left.eq(right);
```

Output

```
id  sales
1   true
2   true
3   false
```

5.1.50 eq

Signature

```
DataFrame eq(self, scalar other) const
```

Parameters

`other` : `scalar` See the signature for the supported argument form.

Returns

`DataFrame` Value produced by this call.

Notes

This produces a comparison-style result you can inspect directly or reuse as a mask.

See also

- `DataFrame eq(self, const DataFrame& other) const`

Examples

```
auto df = pluss::table::DataFrame({
    {"id", pluss::table::Thread(pluss::make_series({1, 2, 3}))},
    {"sales", pluss::table::Thread(pluss::make_series({100.0, 120.0, 90.0}))},
    {"city", pluss::table::Thread(pluss::make_note({"Accra", "Tema", "Kumasi"}))},
});

auto out = df.eq(10);
```

Output

```
id  sales
1   true
2   true
3   false
```

5.1.51 `equals`

Signature

```
bool equals(self, const DataFrame& other) const
```

Parameters

`other` : `const DataFrame&` See the signature for the supported argument form.

Returns

`bool` Boolean status or membership result.

Notes

This call is part of the public `DataFrame` API and is documented here using a concrete usage example.

Examples

```
auto left = pluss::table::DataFrame({
    {"id", pluss::table::Thread(pluss::make_series({1, 2, 3}))},
    {"sales", pluss::table::Thread(pluss::make_series({100.0, 120.0, 90.0}))},
});
auto right = pluss::table::DataFrame({
```

(continues on next page)

(continued from previous page)

```
    {"id", pluss::table::Thread(pluss::make_series({1, 2, 3}))},
    {"cost", pluss::table::Thread(pluss::make_series({60.0, 80.0, 55.0}))},
});

auto out = left.equals(right);
```

Output

```
true
```

5.1.52 feature_names

Signature

```
const std::vector<std::string>& feature_names(self) const
```

Parameters

None

Returns

`const std::vector<std::string>&` Borrowed list of names.

Notes

This call is part of the public `DataFrame` API and is documented here using a concrete usage example.

Examples

```
auto df = pluss::table::DataFrame({
    {"id", pluss::table::Thread(pluss::make_series({1, 2, 3}))},
    {"sales", pluss::table::Thread(pluss::make_series({100.0, 120.0, 90.0}))},
    {"city", pluss::table::Thread(pluss::make_note({"Accra", "Tema", "Kumasi"}))},
});

auto out = df.feature_names();
```

Output

```
["id", "sales", "city"]
```

5.1.53 feature_ref

Signature

```
ColumnRef feature_ref(self, std::string_view name) const
```

Parameters

name : std::string_view See the signature for the supported argument form.

Returns

ColumnRef Value produced by this call.

Notes

This call is part of the public DataFrame API and is documented here using a concrete usage example.

Examples

```
auto df = pluss::table::DataFrame({
    {"id", pluss::table::Thread(pluss::make_series({1, 2, 3}))},
    {"sales", pluss::table::Thread(pluss::make_series({100.0, 120.0, 90.0}))},
    {"city", pluss::table::Thread(pluss::make_note({"Accra", "Tema", "Kumasi"}))},
});

auto out = df.feature_ref("sales");
```

5.1.54 features

Signature

```
DataFrame features(self, std::initializer_list<std::string_view> names)
```

Parameters

names : std::initializer_list<std::string_view> See the signature for the supported argument form.

Returns

DataFrame Value produced by this call.

Notes

This call is part of the public DataFrame API and is documented here using a concrete usage example.

See also

- std::vector<Thread> features(self) const
- Thread features(self, std::string_view name)
- Thread features(self, std::string_view name) const
- DataFrame features(self, std::initializer_list<std::string_view> names) const

Examples

```
auto df = pluss::table::DataFrame({
    {"id", pluss::table::Thread(pluss::make_series({1, 2, 3}))},
    {"sales", pluss::table::Thread(pluss::make_series({100.0, 120.0, 90.0}))},
    {"city", pluss::table::Thread(pluss::make_note({"Accra", "Tema", "Kumasi"}))},
```

(continues on next page)

(continued from previous page)

```
});

auto out = df.features({"id", "sales"});
```

Output

```
Thread([100.0, 120.0, 90.0])
```

5.1.55 features

Signature

```
DataFrame features(self, std::initializer_list<std::string_view> names) const
```

Parameters

names : std::initializer_list<std::string_view> See the signature for the supported argument form.

Returns

DataFrame Value produced by this call.

Notes

This call is part of the public DataFrame API and is documented here using a concrete usage example.

See also

- std::vector<Thread> features(self) const
- Thread features(self, std::string_view name)
- Thread features(self, std::string_view name) const
- DataFrame features(self, std::initializer_list<std::string_view> names)

Examples

```
auto df = pluss::table::DataFrame({
    {"id", pluss::table::Thread(pluss::make_series({1, 2, 3}))},
    {"sales", pluss::table::Thread(pluss::make_series({100.0, 120.0, 90.0}))},
    {"city", pluss::table::Thread(pluss::make_note({"Accra", "Tema", "Kumasi"}))},
});

auto out = df.features({"id", "sales"});
```

Output

```
Thread([100.0, 120.0, 90.0])
```

5.1.56 features

Signature

```
std::vector<Thread> features(self) const
```

Parameters

None

Returns

std::vector<Thread> Materialized collection of thread objects.

Notes

This call is part of the public DataFrame API and is documented here using a concrete usage example.

See also

- Thread features(self, std::string_view name)
- Thread features(self, std::string_view name) const
- DataFrame features(self, std::initializer_list<std::string_view> names)
- DataFrame features(self, std::initializer_list<std::string_view> names) const

Examples

```
auto df = pluss::table::DataFrame({
    {"id", pluss::table::Thread(pluss::make_series({1, 2, 3}))},
    {"sales", pluss::table::Thread(pluss::make_series({100.0, 120.0, 90.0}))},
    {"city", pluss::table::Thread(pluss::make_note({"Accra", "Tema", "Kumasi"}))},
});

auto out = df.features();
```

Output

```
[Thread(id), Thread(sales), Thread(city)]
```

5.1.57 features

Signature

```
Thread features(self, std::string_view name)
```

Parameters

name : std::string_view See the signature for the supported argument form.

Returns

Thread Value produced by this call.

Notes

This call is part of the public DataFrame API and is documented here using a concrete usage example.

See also

- std::vector<Thread> features(self) const
- Thread features(self, std::string_view name) const
- DataFrame features(self, std::initializer_list<std::string_view> names)
- DataFrame features(self, std::initializer_list<std::string_view> names) const

Examples

```
auto df = pluss::table::DataFrame({
    {"id", pluss::table::Thread(pluss::make_series({1, 2, 3}))},
    {"sales", pluss::table::Thread(pluss::make_series({100.0, 120.0, 90.0}))},
    {"city", pluss::table::Thread(pluss::make_note({"Accra", "Tema", "Kumasi"}))},
});

auto out = df.features("sales");
```

Output

```
Thread([100.0, 120.0, 90.0])
```

5.1.58 features

Signature

```
Thread features(self, std::string_view name) const
```

Parameters

name : std::string_view See the signature for the supported argument form.

Returns

Thread Value produced by this call.

Notes

This call is part of the public DataFrame API and is documented here using a concrete usage example.

See also

- std::vector<Thread> features(self) const
- Thread features(self, std::string_view name)
- DataFrame features(self, std::initializer_list<std::string_view> names)

- DataFrame features(self, std::initializer_list<std::string_view> names) const

Examples

```
auto df = pluss::table::DataFrame({
    {"id", pluss::table::Thread(pluss::make_series({1, 2, 3}))},
    {"sales", pluss::table::Thread(pluss::make_series({100.0, 120.0, 90.0}))},
    {"city", pluss::table::Thread(pluss::make_note({"Accra", "Tema", "Kumasi"}))},
});

auto out = df.features("sales");
```

Output

```
Thread([100.0, 120.0, 90.0])
```

5.1.59 ffill

Signature

```
DataFrame ffill(self) const
```

Parameters

None

Returns

DataFrame Value produced by this call.

Notes

This is part of the missing-data cleanup path and is usually applied before summaries, grouping, or export.

Examples

```
auto df = pluss::table::DataFrame({
    {"id", pluss::table::Thread(pluss::make_series({1, 2, 3}))},
    {"sales", pluss::table::Thread(pluss::make_series({100.0, 120.0, 90.0}))},
    {"city", pluss::table::Thread(pluss::make_note({"Accra", "Tema", "Kumasi"}))},
});

auto out = df.ffill();
```

Output

```
id  sales  city
1   100.0  Accra
2   120.0  Tema
3    90.0  Kumasi
```

5.1.60 fillna

Signature

```
DataFrame fillna(self, const char* value) const
```

Parameters

`value` : `const char*` See the signature for the supported argument form.

Returns

`DataFrame` Value produced by this call.

Notes

This is part of the missing-data cleanup path and is usually applied before summaries, grouping, or export.

See also

- `DataFrame fillna(self, scalar value) const`
- `DataFrame fillna(self, scalar value, std::initializer_list<std::string_view> features) const`
- `DataFrame fillna(self, std::string_view value) const`
- `DataFrame fillna(self, std::string_view value, std::initializer_list<std::string_view> features) const`
- `DataFrame fillna(self, const char* value, std::initializer_list<std::string_view> features) const`
- `DataFrame fillna(self, const pluss::Timeline& value) const`
- `DataFrame fillna(self, const pluss::Timeline& value, std::initializer_list<std::string_view> features) const`
- `DataFrame fillna(self, const pluss::TimeIndex& value) const`
- `DataFrame fillna(self, const pluss::TimeIndex& value, std::initializer_list<std::string_view> features) const`

Examples

```
auto df = pluss::table::DataFrame({{"city", pluss::table::Thread(pluss::make_note({"Accra
↪", "Tema", "Kumasi"}))}});

auto out = df.fillna("Unknown");
```

Output

```
id  sales  city
1   100.0  Accra
2   120.0  Tema
3    90.0  Kumasi
```

5.1.61 fillna

Signature

```
DataFrame fillna(self, const char* value, std::initializer_list<std::string_view>␣
↪features) const
```

Parameters

`value` : `const char*` See the signature for the supported argument form.

`features` : `std::initializer_list<std::string_view>` See the signature for the supported argument form.

Returns

`DataFrame` Value produced by this call.

Notes

This is part of the missing-data cleanup path and is usually applied before summaries, grouping, or export.

See also

- `DataFrame fillna(self, scalar value) const`
- `DataFrame fillna(self, scalar value, std::initializer_list<std::string_view> features) const`
- `DataFrame fillna(self, std::string_view value) const`
- `DataFrame fillna(self, std::string_view value, std::initializer_list<std::string_view> features) const`
- `DataFrame fillna(self, const char* value) const`
- `DataFrame fillna(self, const pluss::Timeline& value) const`
- `DataFrame fillna(self, const pluss::Timeline& value, std::initializer_list<std::string_view> features) const`
- `DataFrame fillna(self, const pluss::TimeIndex& value) const`
- `DataFrame fillna(self, const pluss::TimeIndex& value, std::initializer_list<std::string_view> features) const`

Examples

```
auto df = pluss::table::DataFrame({{"city", pluss::table::Thread(pluss::make_note({"Accra
↪", "Tema", "Kumasi"}))}});

auto out = df.fillna("Unknown");
```

Output

```
id  sales  city
1   100.0  Accra
2   120.0  Tema
3    90.0  Kumasi
```

5.1.62 fillna

Signature

```
DataFrame fillna(self, const pluss::TimeIndex& value) const
```

Parameters

`value` : `const pluss::TimeIndex&` See the signature for the supported argument form.

Returns

`DataFrame` Value produced by this call.

Notes

This is part of the missing-data cleanup path and is usually applied before summaries, grouping, or export.

See also

- `DataFrame fillna(self, scalar value) const`
- `DataFrame fillna(self, scalar value, std::initializer_list<std::string_view> features) const`
- `DataFrame fillna(self, std::string_view value) const`
- `DataFrame fillna(self, std::string_view value, std::initializer_list<std::string_view> features) const`
- `DataFrame fillna(self, const char* value) const`
- `DataFrame fillna(self, const char* value, std::initializer_list<std::string_view> features) const`
- `DataFrame fillna(self, const pluss::Timeline& value) const`
- `DataFrame fillna(self, const pluss::Timeline& value, std::initializer_list<std::string_view> features) const`
- `DataFrame fillna(self, const pluss::TimeIndex& value, std::initializer_list<std::string_view> features) const`

Examples

```
auto df = pluss::table::DataFrame({{"idx", pluss::table::Thread(pluss::make_timeindex({
↪"2024-01-01T09:10:00Z", "2024-01-01T09:40:00Z"}))}});

auto out = df.fillna(pluss::make_timeindex("2024-01-01T00:00:00Z"));
```

Output

```
id  sales  city
1   100.0  Accra
2   120.0  Tema
3    90.0  Kumasi
```

5.1.63 fillna

Signature

```
DataFrame fillna(self, const pluss::TimeIndex& value, std::initializer_list<std::string_
↪view> features) const
```

Parameters

`value` : `const pluss::TimeIndex&` See the signature for the supported argument form.

`features` : `std::initializer_list<std::string_view>` See the signature for the supported argument form.

Returns

`DataFrame` Value produced by this call.

Notes

This is part of the missing-data cleanup path and is usually applied before summaries, grouping, or export.

See also

- `DataFrame fillna(self, scalar value) const`
- `DataFrame fillna(self, scalar value, std::initializer_list<std::string_view> features) const`
- `DataFrame fillna(self, std::string_view value) const`
- `DataFrame fillna(self, std::string_view value, std::initializer_list<std::string_view> features) const`
- `DataFrame fillna(self, const char* value) const`
- `DataFrame fillna(self, const char* value, std::initializer_list<std::string_view> features) const`
- `DataFrame fillna(self, const pluss::Timeline& value) const`
- `DataFrame fillna(self, const pluss::Timeline& value, std::initializer_list<std::string_view> features) const`
- `DataFrame fillna(self, const pluss::TimeIndex& value) const`

Examples

```
auto df = pluss::table::DataFrame({{"city", pluss::table::Thread(pluss::make_note({"Accra
↪", "Tema", "Kumasi"}))}});

auto out = df.fillna("Unknown");
```

Output

```
id  sales  city
1   100.0  Accra
2   120.0  Tema
3    90.0  Kumasi
```

5.1.64 fillna

Signature

```
DataFrame fillna(self, const pluss::Timeline& value) const
```

Parameters

`value` : `const pluss::Timeline&` See the signature for the supported argument form.

Returns

`DataFrame` Value produced by this call.

Notes

This is part of the missing-data cleanup path and is usually applied before summaries, grouping, or export.

See also

- `DataFrame fillna(self, scalar value) const`
- `DataFrame fillna(self, scalar value, std::initializer_list<std::string_view> features) const`
- `DataFrame fillna(self, std::string_view value) const`
- `DataFrame fillna(self, std::string_view value, std::initializer_list<std::string_view> features) const`
- `DataFrame fillna(self, const char* value) const`
- `DataFrame fillna(self, const char* value, std::initializer_list<std::string_view> features) const`
- `DataFrame fillna(self, const pluss::Timeline& value, std::initializer_list<std::string_view> features) const`
- `DataFrame fillna(self, const pluss::TimeIndex& value) const`
- `DataFrame fillna(self, const pluss::TimeIndex& value, std::initializer_list<std::string_view> features) const`

Examples

```
auto df = pluss::table::DataFrame({{"ts", pluss::table::Thread(pluss::make_timeline({
↪"2024-01-01T09:10:00Z", "2024-01-01T09:40:00Z"}))}});

auto out = df.fillna(pluss::make_timeline("2024-01-01T00:00:00Z"));
```

Output

```
id  sales  city
1   100.0  Accra
2   120.0  Tema
3    90.0  Kumasi
```

5.1.65 fillna

Signature

```
DataFrame fillna(self, const pluss::Timeline& value, std::initializer_list<std::string_
↪view> features) const
```

Parameters

`value` : `const pluss::Timeline&` See the signature for the supported argument form.

`features` : `std::initializer_list<std::string_view>` See the signature for the supported argument form.

Returns

`DataFrame` Value produced by this call.

Notes

This is part of the missing-data cleanup path and is usually applied before summaries, grouping, or export.

See also

- `DataFrame fillna(self, scalar value) const`
- `DataFrame fillna(self, scalar value, std::initializer_list<std::string_view> features) const`
- `DataFrame fillna(self, std::string_view value) const`
- `DataFrame fillna(self, std::string_view value, std::initializer_list<std::string_view> features) const`
- `DataFrame fillna(self, const char* value) const`
- `DataFrame fillna(self, const char* value, std::initializer_list<std::string_view> features) const`
- `DataFrame fillna(self, const pluss::Timeline& value) const`
- `DataFrame fillna(self, const pluss::TimeIndex& value) const`
- `DataFrame fillna(self, const pluss::TimeIndex& value, std::initializer_list<std::string_view> features) const`

Examples

```
auto df = pluss::table::DataFrame({{"city", pluss::table::Thread(pluss::make_note({"Accra
↪", "Tema", "Kumasi"}))}});

auto out = df.fillna("Unknown");
```

Output

```
id  sales  city
1   100.0  Accra
2   120.0  Tema
3    90.0  Kumasi
```

5.1.66 fillna

Signature

```
DataFrame fillna(self, scalar value) const
```

Parameters

`value` : `scalar` See the signature for the supported argument form.

Returns

`DataFrame` Value produced by this call.

Notes

This is part of the missing-data cleanup path and is usually applied before summaries, grouping, or export.

See also

- `DataFrame fillna(self, scalar value, std::initializer_list<std::string_view> features) const`
- `DataFrame fillna(self, std::string_view value) const`
- `DataFrame fillna(self, std::string_view value, std::initializer_list<std::string_view> features) const`
- `DataFrame fillna(self, const char* value) const`
- `DataFrame fillna(self, const char* value, std::initializer_list<std::string_view> features) const`
- `DataFrame fillna(self, const pluss::Timeline& value) const`
- `DataFrame fillna(self, const pluss::Timeline& value, std::initializer_list<std::string_view> features) const`
- `DataFrame fillna(self, const pluss::TimeIndex& value) const`
- `DataFrame fillna(self, const pluss::TimeIndex& value, std::initializer_list<std::string_view> features) const`

Examples

```
auto df = pluss::table::DataFrame({
    {"id", pluss::table::Thread(pluss::make_series({1, 2, 3}))},
    {"sales", pluss::table::Thread(pluss::make_series({100.0, 120.0, 90.0}))},
    {"city", pluss::table::Thread(pluss::make_note({"Accra", "Tema", "Kumasi"}))},
});

auto out = df.fillna(0.0, {"sales"});
```

Output

```
id  sales  city
1   100.0  Accra
2   120.0  Tema
3    90.0  Kumasi
```

5.1.67 fillna

Signature

```
DataFrame fillna(self, scalar value, std::initializer_list<std::string_view> features)␣
↪const
```

Parameters

`value` : `scalar` See the signature for the supported argument form.

`features` : `std::initializer_list<std::string_view>` See the signature for the supported argument form.

Returns

`DataFrame` Value produced by this call.

Notes

This is part of the missing-data cleanup path and is usually applied before summaries, grouping, or export.

See also

- `DataFrame fillna(self, scalar value) const`
- `DataFrame fillna(self, std::string_view value) const`
- `DataFrame fillna(self, std::string_view value, std::initializer_list<std::string_view> features) const`
- `DataFrame fillna(self, const char* value) const`
- `DataFrame fillna(self, const char* value, std::initializer_list<std::string_view> features) const`
- `DataFrame fillna(self, const pluss::Timeline& value) const`
- `DataFrame fillna(self, const pluss::Timeline& value, std::initializer_list<std::string_view> features) const`
- `DataFrame fillna(self, const pluss::TimeIndex& value) const`
- `DataFrame fillna(self, const pluss::TimeIndex& value, std::initializer_list<std::string_view> features) const`

Examples

```
auto df = pluss::table::DataFrame({{"city", pluss::table::Thread(pluss::make_note({"Accra
↪", "Tema", "Kumasi"}))}});

auto out = df.fillna("Unknown");
```

Output

```
id  sales  city
1   100.0  Accra
2   120.0  Tema
3    90.0  Kumasi
```

5.1.68 fillna

Signature

```
DataFrame fillna(self, std::string_view value) const
```

Parameters

value : std::string_view See the signature for the supported argument form.

Returns

DataFrame Value produced by this call.

Notes

This is part of the missing-data cleanup path and is usually applied before summaries, grouping, or export.

See also

- DataFrame fillna(self, scalar value) const
- DataFrame fillna(self, scalar value, std::initializer_list<std::string_view> features) const
- DataFrame fillna(self, std::string_view value, std::initializer_list<std::string_view> features) const
- DataFrame fillna(self, const char* value) const
- DataFrame fillna(self, const char* value, std::initializer_list<std::string_view> features) const
- DataFrame fillna(self, const pluss::Timeline& value) const
- DataFrame fillna(self, const pluss::Timeline& value, std::initializer_list<std::string_view> features) const
- DataFrame fillna(self, const pluss::TimeIndex& value) const
- DataFrame fillna(self, const pluss::TimeIndex& value, std::initializer_list<std::string_view> features) const

Examples

```
auto df = pluss::table::DataFrame({{"city", pluss::table::Thread(pluss::make_note({"Accra
↪", "Tema", "Kumasi"}))}});

auto out = df.fillna("Unknown");
```

Output

```
id  sales  city
1   100.0  Accra
2   120.0  Tema
3    90.0  Kumasi
```

5.1.69 fillna

Signature

```
DataFrame fillna(self, std::string_view value, std::initializer_list<std::string_view>␣
↪features) const
```

Parameters

`value` : `std::string_view` See the signature for the supported argument form.

`features` : `std::initializer_list<std::string_view>` See the signature for the supported argument form.

Returns

`DataFrame` Value produced by this call.

Notes

This is part of the missing-data cleanup path and is usually applied before summaries, grouping, or export.

See also

- `DataFrame fillna(self, scalar value) const`
- `DataFrame fillna(self, scalar value, std::initializer_list<std::string_view> features) const`
- `DataFrame fillna(self, std::string_view value) const`
- `DataFrame fillna(self, const char* value) const`
- `DataFrame fillna(self, const char* value, std::initializer_list<std::string_view> features) const`
- `DataFrame fillna(self, const pluss::Timeline& value) const`
- `DataFrame fillna(self, const pluss::Timeline& value, std::initializer_list<std::string_view> features) const`
- `DataFrame fillna(self, const pluss::TimeIndex& value) const`
- `DataFrame fillna(self, const pluss::TimeIndex& value, std::initializer_list<std::string_view> features) const`

Examples

```
auto df = pluss::table::DataFrame({{"city", pluss::table::Thread(pluss::make_note({"Accra
↪", "Tema", "Kumasi"}))}});

auto out = df.fillna("Unknown");
```

Output

```
id  sales  city
1   100.0  Accra
2   120.0  Tema
3    90.0  Kumasi
```

5.1.70 fillna_

Signature

```
DataFrame& fillna_(self, const char* value)
```

Parameters

`value` : `const char*` See the signature for the supported argument form.

Returns

`DataFrame&` Value produced by this call.

Notes

This is part of the missing-data cleanup path and is usually applied before summaries, grouping, or export.

See also

- `DataFrame& fillna_(self, scalar value)`
- `DataFrame& fillna_(self, std::string_view value)`
- `DataFrame& fillna_(self, const pluss::Timeline& value)`
- `DataFrame& fillna_(self, const pluss::TimeIndex& value)`

Examples

```
auto df = pluss::table::DataFrame({{"city", pluss::table::Thread(pluss::make_note({"Accra
↪", "Tema", "Kumasi"}))}});

df.fillna_("Unknown");
```

Output

```
id  sales  city
1   100.0  Accra
2   120.0  Tema
3    90.0  Kumasi
```

5.1.71 fillna_

Signature

```
DataFrame& fillna_(self, const pluss::TimeIndex& value)
```

Parameters

`value` : `const pluss::TimeIndex&` See the signature for the supported argument form.

Returns

DataFrame& Value produced by this call.

Notes

This is part of the missing-data cleanup path and is usually applied before summaries, grouping, or export.

See also

- DataFrame& fillna_(self, scalar value)
- DataFrame& fillna_(self, std::string_view value)
- DataFrame& fillna_(self, const char* value)
- DataFrame& fillna_(self, const pluss::Timeline& value)

Examples

```
auto df = pluss::table::DataFrame({{"idx", pluss::table::Thread(pluss::make_timeindex({
↪"2024-01-01T09:10:00Z", "2024-01-01T09:40:00Z"}))}});

df.fillna_(pluss::make_timeindex("2024-01-01T00:00:00Z"));
```

Output

```
id  sales  city
1   100.0  Accra
2   120.0  Tema
3    90.0  Kumasi
```

5.1.72 fillna_

Signature

```
DataFrame& fillna_(self, const pluss::Timeline& value)
```

Parameters

value : const pluss::Timeline& See the signature for the supported argument form.

Returns

DataFrame& Value produced by this call.

Notes

This is part of the missing-data cleanup path and is usually applied before summaries, grouping, or export.

See also

- DataFrame& fillna_(self, scalar value)
- DataFrame& fillna_(self, std::string_view value)
- DataFrame& fillna_(self, const char* value)

- DataFrame& fillna_(self, const pluss::TimeIndex& value)

Examples

```
auto df = pluss::table::DataFrame({{"ts", pluss::table::Thread(pluss::make_timeline({
↪"2024-01-01T09:10:00Z", "2024-01-01T09:40:00Z"}))}});

df.fillna_(pluss::make_timeline("2024-01-01T00:00:00Z"));
```

Output

```
id  sales  city
1   100.0  Accra
2   120.0  Tema
3    90.0  Kumasi
```

5.1.73 fillna_

Signature

```
DataFrame& fillna_(self, scalar value)
```

Parameters

value : scalar See the signature for the supported argument form.

Returns

DataFrame& Value produced by this call.

Notes

This is part of the missing-data cleanup path and is usually applied before summaries, grouping, or export.

See also

- DataFrame& fillna_(self, std::string_view value)
- DataFrame& fillna_(self, const char* value)
- DataFrame& fillna_(self, const pluss::Timeline& value)
- DataFrame& fillna_(self, const pluss::TimeIndex& value)

Examples

```
auto df = pluss::table::DataFrame({
    {"id", pluss::table::Thread(pluss::make_series({1, 2, 3}))},
    {"sales", pluss::table::Thread(pluss::make_series({100.0, 120.0, 90.0}))},
    {"city", pluss::table::Thread(pluss::make_note({"Accra", "Tema", "Kumasi"}))},
});

df.fillna_(0.0);
```

Output

```
id  sales  city
1   100.0  Accra
2   120.0  Tema
3    90.0  Kumasi
```

5.1.74 fillna_

Signature

```
DataFrame& fillna_(self, std::string_view value)
```

Parameters

`value` : `std::string_view` See the signature for the supported argument form.

Returns

`DataFrame&` Value produced by this call.

Notes

This is part of the missing-data cleanup path and is usually applied before summaries, grouping, or export.

See also

- `DataFrame& fillna_(self, scalar value)`
- `DataFrame& fillna_(self, const char* value)`
- `DataFrame& fillna_(self, const pluss::Timeline& value)`
- `DataFrame& fillna_(self, const pluss::TimeIndex& value)`

Examples

```
auto df = pluss::table::DataFrame({{"city", pluss::table::Thread(pluss::make_note({"Accra
↪", "Tema", "Kumasi"}))}});

df.fillna_("Unknown");
```

Output

```
id  sales  city
1   100.0  Accra
2   120.0  Tema
3    90.0  Kumasi
```

5.1.75 filter

Signature

```
DataFrame filter(self, const pluss::Series& bool_mask) const
```

Parameters

`bool_mask` : `const pluss::Series&` See the signature for the supported argument form.

Returns

`DataFrame` Value produced by this call.

Notes

Use this to reduce the current object to the rows or fields that matter for the next step.

Examples

```
auto df = pluss::table::DataFrame({
    {"id", pluss::table::Thread(pluss::make_series({1, 2, 3}))},
    {"sales", pluss::table::Thread(pluss::make_series({100.0, 120.0, 90.0}))},
    {"city", pluss::table::Thread(pluss::make_note({"Accra", "Tema", "Kumasi"}))},
});

auto mask = pluss::make_series({true, false, true}, pluss::codem::ScalarType::Bool);
auto out = df.filter(mask);
```

Output

```
id  sales  city
1   100.0  Accra
3    90.0  Kumasi
```

5.1.76 `floor_time`

Signature

```
DataFrame floor_time(self, std::initializer_list<std::string_view> features, std::string_
↪view unit) const
```

Parameters

`features` : `std::initializer_list<std::string_view>` See the signature for the supported argument form.

`unit` : `std::string_view` See the signature for the supported argument form.

Returns

`DataFrame` Value produced by this call.

Notes

This is part of the time-aware surface and expects timeline or time-index semantics.

Examples

```
auto df = pluss::table::DataFrame({
    {"ts", pluss::table::Thread(pluss::make_timeline({
        "2024-01-01T09:10:00Z",
        "2024-01-01T09:40:00Z",
        "2024-01-01T10:05:00Z",
    }))},
    {"load", pluss::table::Thread(pluss::make_series({10.0, 20.0, 15.0}))},
});

auto out = df.floor_time({"ts"}, "hour");
```

Output

```
ts
2024-01-01T09:00:00Z
2024-01-01T10:00:00Z
```

5.1.77 `floordiv`

Signature

```
DataFrame floordiv(self, const DataFrame& other) const
```

Parameters

`other` : `const DataFrame&` See the signature for the supported argument form.

Returns

`DataFrame` Value produced by this call.

Notes

This applies arithmetic across the compatible numeric payloads in the current object.

See also

- `DataFrame floordiv(self, scalar other) const`

Examples

```
auto left = pluss::table::DataFrame({
    {"id", pluss::table::Thread(pluss::make_series({1, 2, 3}))},
    {"sales", pluss::table::Thread(pluss::make_series({100.0, 120.0, 90.0}))},
});
auto right = pluss::table::DataFrame({
    {"id", pluss::table::Thread(pluss::make_series({1, 2, 3}))},
    {"cost", pluss::table::Thread(pluss::make_series({60.0, 80.0, 55.0}))},
});

auto out = left.floordiv(right);
```

Output

```
id  sales  city
1   100.0  Accra
2   120.0  Tema
3    90.0  Kumasi
```

5.1.78 floordiv

Signature

```
DataFrame floordiv(self, scalar other) const
```

Parameters

`other` : `scalar` See the signature for the supported argument form.

Returns

`DataFrame` Value produced by this call.

Notes

This applies arithmetic across the compatible numeric payloads in the current object.

See also

- `DataFrame floordiv(self, const DataFrame& other) const`

Examples

```
auto df = pluss::table::DataFrame({
    {"id", pluss::table::Thread(pluss::make_series({1, 2, 3}))},
    {"sales", pluss::table::Thread(pluss::make_series({100.0, 120.0, 90.0}))},
    {"city", pluss::table::Thread(pluss::make_note({"Accra", "Tema", "Kumasi"}))},
});

auto out = df.floordiv(10);
```

Output

```
id  sales  city
11  110.0  Accra
12  130.0  Tema
13  100.0  Kumasi
```

5.1.79 ge

Signature

```
DataFrame ge(self, const DataFrame& other) const
```

Parameters

`other` : `const DataFrame&` See the signature for the supported argument form.

Returns

`DataFrame` Value produced by this call.

Notes

This produces a comparison-style result you can inspect directly or reuse as a mask.

See also

- `DataFrame ge(self, scalar other) const`

Examples

```cpp
auto left = pluss::table::DataFrame({
    {"id", pluss::table::Thread(pluss::make_series({1, 2, 3}))},
    {"sales", pluss::table::Thread(pluss::make_series({100.0, 120.0, 90.0}))},
});
auto right = pluss::table::DataFrame({
    {"id", pluss::table::Thread(pluss::make_series({1, 2, 3}))},
    {"cost", pluss::table::Thread(pluss::make_series({60.0, 80.0, 55.0}))},
});

auto out = left.ge(right);
```

Output

```
id  sales
1   true
2   true
3   false
```

5.1.80 ge

Signature

```cpp
DataFrame ge(self, scalar other) const
```

Parameters

`other` : `scalar` See the signature for the supported argument form.

Returns

`DataFrame` Value produced by this call.

Notes

This produces a comparison-style result you can inspect directly or reuse as a mask.

See also

- `DataFrame ge(self, const DataFrame& other) const`

Examples

```
auto df = pluss::table::DataFrame({
    {"id", pluss::table::Thread(pluss::make_series({1, 2, 3}))},
    {"sales", pluss::table::Thread(pluss::make_series({100.0, 120.0, 90.0}))},
    {"city", pluss::table::Thread(pluss::make_note({"Accra", "Tema", "Kumasi"}))},
});

auto out = df.ge(10);
```

Output

```
id  sales
1   true
2   true
3   false
```

5.1.81 `get`

Signature

```
std::optional<Thread> get(self, std::string_view feature_name) const
```

Parameters

`feature_name` : `std::string_view` See the signature for the supported argument form.

Returns

`std::optional<Thread>` Value produced by this call.

Notes

This call is part of the public `DataFrame` API and is documented here using a concrete usage example.

Examples

```
auto df = pluss::table::DataFrame({
    {"id", pluss::table::Thread(pluss::make_series({1, 2, 3}))},
    {"sales", pluss::table::Thread(pluss::make_series({100.0, 120.0, 90.0}))},
    {"city", pluss::table::Thread(pluss::make_note({"Accra", "Tema", "Kumasi"}))},
});

auto out = df.get("sales");
```

Output

```
Thread([100.0, 120.0, 90.0])
```

5.1.82 group_by

Signature

```
DataFrame group_by(self, std::initializer_list<std::string_view> keys, std::initializer_
↪list<GroupSpec> specs, bool dropna_keys = true, bool render = true) const
```

Parameters

`keys` : `std::initializer_list<std::string_view>` See the signature for the supported argument form.

`specs` : `std::initializer_list<GroupSpec>` See the signature for the supported argument form.

`dropna_keys` : `bool` See the signature for the supported argument form.

`render` : `bool` See the signature for the supported argument form.

Returns

`DataFrame` Value produced by this call.

Notes

Use this when you want a summary view instead of carrying the full input forward unchanged.

Examples

```
auto df = pluss::table::DataFrame({
    {"city", pluss::table::Thread(pluss::make_note({"Accra", "Accra", "Tema", "Tema"}))},
    {"product", pluss::table::Thread(pluss::make_note({"A", "B", "A", "B"}))},
    {"sales", pluss::table::Thread(pluss::make_series({100.0, 120.0, 90.0, 150.0}))},
});

auto out = df.group_by({"city"}, {{"sales_sum", "sales",␣
↪pluss::table::DataFrame::GroupOp::Sum}}, true, false);
```

Output

```
city    sales_sum
Accra   220.0
Tema    240.0
```

5.1.83 gt

Signature

```
DataFrame gt(self, const DataFrame& other) const
```

Parameters

`other` : `const DataFrame&` See the signature for the supported argument form.

Returns

`DataFrame` Value produced by this call.

Notes

This produces a comparison-style result you can inspect directly or reuse as a mask.

See also

- `DataFrame gt(self, scalar other) const`

Examples

```cpp
auto left = pluss::table::DataFrame({
    {"id", pluss::table::Thread(pluss::make_series({1, 2, 3}))},
    {"sales", pluss::table::Thread(pluss::make_series({100.0, 120.0, 90.0}))},
});
auto right = pluss::table::DataFrame({
    {"id", pluss::table::Thread(pluss::make_series({1, 2, 3}))},
    {"cost", pluss::table::Thread(pluss::make_series({60.0, 80.0, 55.0}))},
});

auto out = left.gt(right);
```

Output

```
id  sales
1   true
2   true
3   false
```

5.1.84 `gt`

Signature

```cpp
DataFrame gt(self, scalar other) const
```

Parameters

`other` : `scalar` See the signature for the supported argument form.

Returns

`DataFrame` Value produced by this call.

Notes

This produces a comparison-style result you can inspect directly or reuse as a mask.

See also

- `DataFrame gt(self, const DataFrame& other) const`

Examples

```cpp
auto df = pluss::table::DataFrame({
    {"id", pluss::table::Thread(pluss::make_series({1, 2, 3}))},
    {"sales", pluss::table::Thread(pluss::make_series({100.0, 120.0, 90.0}))},
    {"city", pluss::table::Thread(pluss::make_note({"Accra", "Tema", "Kumasi"}))},
});

auto out = df.gt(10);
```

Output

```
id  sales
1   true
2   true
3   false
```

5.1.85 has_column

Signature

```cpp
bool has_column(self, std::string_view name) const
```

Parameters

`name` : `std::string_view` See the signature for the supported argument form.

Returns

`bool` Boolean status or membership result.

Notes

This call is part of the public `DataFrame` API and is documented here using a concrete usage example.

Examples

```cpp
auto df = pluss::table::DataFrame({
    {"id", pluss::table::Thread(pluss::make_series({1, 2, 3}))},
    {"sales", pluss::table::Thread(pluss::make_series({100.0, 120.0, 90.0}))},
    {"city", pluss::table::Thread(pluss::make_note({"Accra", "Tema", "Kumasi"}))},
});

auto out = df.has_column("sales");
```

Output

```
true
```

5.1.86 has_feature

Signature

```
bool has_feature(self, std::string_view name) const
```

Parameters

`name` : `std::string_view` See the signature for the supported argument form.

Returns

`bool` Boolean status or membership result.

Notes

This call is part of the public `DataFrame` API and is documented here using a concrete usage example.

Examples

```
auto df = pluss::table::DataFrame({
    {"id", pluss::table::Thread(pluss::make_series({1, 2, 3}))},
    {"sales", pluss::table::Thread(pluss::make_series({100.0, 120.0, 90.0}))},
    {"city", pluss::table::Thread(pluss::make_note({"Accra", "Tema", "Kumasi"}))},
});

auto out = df.has_feature("sales");
```

Output

```
true
```

5.1.87 head

Signature

```
DataFrame head(self, std::int64_t n = 5, bool render = true) const
```

Parameters

`n` : `std::int64_t` See the signature for the supported argument form.

`render` : `bool` See the signature for the supported argument form.

Returns

`DataFrame` Value produced by this call.

Notes

Use this to narrow the current object to the rows or depth range you want to inspect next.

Examples

```
auto df = pluss::table::DataFrame({
    {"id", pluss::table::Thread(pluss::make_series({1, 2, 3}))},
    {"sales", pluss::table::Thread(pluss::make_series({100.0, 120.0, 90.0}))},
    {"city", pluss::table::Thread(pluss::make_note({"Accra", "Tema", "Kumasi"}))},
});

auto out = df.head(2, false);
```

Output

```
id  sales  city
1   100.0  Accra
2   120.0  Tema
3    90.0  Kumasi
```

5.1.88 `iat`

Signature

```
std::string iat(self, std::int64_t row, std::size_t feature_idx) const
```

Parameters

`row` : `std::int64_t` See the signature for the supported argument form.

`feature_idx` : `std::size_t` See the signature for the supported argument form.

Returns

`std::string` Rendered string result.

Notes

This call is part of the public `DataFrame` API and is documented here using a concrete usage example.

Examples

```
auto df = pluss::table::DataFrame({
    {"id", pluss::table::Thread(pluss::make_series({1, 2, 3}))},
    {"sales", pluss::table::Thread(pluss::make_series({100.0, 120.0, 90.0}))},
    {"city", pluss::table::Thread(pluss::make_note({"Accra", "Tema", "Kumasi"}))},
});

auto out = df.iat(0, 0);
```

Output

```
1
```

5.1.89 iloc

Signature

```
DataFrame iloc(self, std::int64_t row) const
```

Parameters

`row` : `std::int64_t` See the signature for the supported argument form.

Returns

`DataFrame` Value produced by this call.

Notes

Use this to narrow the current object to the rows or depth range you want to inspect next.

See also

- `DataFrame iloc(self, std::int64_t start, std::int64_t stop) const`

Examples

```
auto df = pluss::table::DataFrame({
    {"id", pluss::table::Thread(pluss::make_series({1, 2, 3}))},
    {"sales", pluss::table::Thread(pluss::make_series({100.0, 120.0, 90.0}))},
    {"city", pluss::table::Thread(pluss::make_note({"Accra", "Tema", "Kumasi"}))},
});

auto out = df.iloc(1);
```

Output

```
id  sales  city
1   100.0  Accra
2   120.0  Tema
3    90.0  Kumasi
```

5.1.90 iloc

Signature

```
DataFrame iloc(self, std::int64_t start, std::int64_t stop) const
```

Parameters

`start` : `std::int64_t` See the signature for the supported argument form.

`stop` : `std::int64_t` See the signature for the supported argument form.

Returns

`DataFrame` Value produced by this call.

Notes

Use this to narrow the current object to the rows or depth range you want to inspect next.

See also

- `DataFrame iloc(self, std::int64_t row) const`

Examples

```
auto df = pluss::table::DataFrame({
    {"id", pluss::table::Thread(pluss::make_series({1, 2, 3}))},
    {"sales", pluss::table::Thread(pluss::make_series({100.0, 120.0, 90.0}))},
    {"city", pluss::table::Thread(pluss::make_note({"Accra", "Tema", "Kumasi"}))},
});

auto out = df.iloc(0, 2);
```

Output

```
id  sales  city
1   100.0  Accra
2   120.0  Tema
3    90.0  Kumasi
```

5.1.91 `info`

Signature

```
void info(self, const display::DisplayOptions& options =
```

Parameters

`options` : `const display::DisplayOptions&` See the signature for the supported argument form.

Returns

`void` Performs the side effect and returns no value.

Notes

This renders a human-readable view for inspection instead of creating a new analytic object.

See also

- void info(self, std::ostream& os, const display::DisplayOptions& options =

Examples

```
auto df = pluss::table::DataFrame({
    {"id", pluss::table::Thread(pluss::make_series({1, 2, 3}))},
    {"sales", pluss::table::Thread(pluss::make_series({100.0, 120.0, 90.0}))},
    {"city", pluss::table::Thread(pluss::make_note({"Accra", "Tema", "Kumasi"}))},
});

df.info();
```

Output

```
rows: 3
features: 3
numeric: id, sales
note: city
```

5.1.92 info

Signature

```
void info(self, std::ostream& os, const display::DisplayOptions& options =
```

Parameters

os : std::ostream& See the signature for the supported argument form.

options : const display::DisplayOptions& See the signature for the supported argument form.

Returns

void Performs the side effect and returns no value.

Notes

This renders a human-readable view for inspection instead of creating a new analytic object.

See also

- void info(self, const display::DisplayOptions& options =

Examples

```
auto df = pluss::table::DataFrame({
    {"id", pluss::table::Thread(pluss::make_series({1, 2, 3}))},
    {"sales", pluss::table::Thread(pluss::make_series({100.0, 120.0, 90.0}))},
    {"city", pluss::table::Thread(pluss::make_note({"Accra", "Tema", "Kumasi"}))},
});

df.info();
```

Output

```
rows: 3
features: 3
numeric: id, sales
note: city
```

5.1.93 info_string

Signature

```
std::string info_string(self, const display::DisplayOptions& options =
```

Parameters

`options` : `const display::DisplayOptions&` See the signature for the supported argument form.

Returns

`std::string` Rendered string result.

Notes

This call is part of the public `DataFrame` API and is documented here using a concrete usage example.

Examples

```
auto df = pluss::table::DataFrame({
    {"id", pluss::table::Thread(pluss::make_series({1, 2, 3}))},
    {"sales", pluss::table::Thread(pluss::make_series({100.0, 120.0, 90.0}))},
    {"city", pluss::table::Thread(pluss::make_note({"Accra", "Tema", "Kumasi"}))},
});

auto out = df.info_string();
```

Output

```
id  sales  city
1   100.0  Accra
2   120.0  Tema
3    90.0  Kumasi
```

5.1.94 insert

Signature

```
DataFrame insert(self, std::int64_t feature_index, std::string name, const Thread&␣
↪feature) const
```

Parameters

`feature_index` : `std::int64_t` See the signature for the supported argument form.

`name` : `std::string` See the signature for the supported argument form.

`feature` : `const Thread&` See the signature for the supported argument form.

Returns

`DataFrame` Value produced by this call.

Notes

This call is part of the public `DataFrame` API and is documented here using a concrete usage example.

Examples

```
auto df = pluss::table::DataFrame({
    {"id", pluss::table::Thread(pluss::make_series({1, 2, 3}))},
    {"sales", pluss::table::Thread(pluss::make_series({100.0, 120.0, 90.0}))},
    {"city", pluss::table::Thread(pluss::make_note({"Accra", "Tema", "Kumasi"}))},
});

auto out = df.insert(1, "profit", pluss::table::Thread(pluss::make_series({40.0, 35.0,␣
↪30.0})));
```

Output

```
id  sales  city    profit
1   100.0  Accra   40.0
2   120.0  Tema    35.0
3    90.0  Kumasi  30.0
```

5.1.95 `isna`

Signature

```
DataFrame isna(self, std::initializer_list<std::string_view> features =
```

Parameters

`features` : `std::initializer_list<std::string_view>` See the signature for the supported argument form.

Returns

`DataFrame` Value produced by this call.

Notes

This is part of the missing-data cleanup path and is usually applied before summaries, grouping, or export.

Examples

```
auto df = pluss::table::DataFrame({
    {"id", pluss::table::Thread(pluss::make_series({1, 2, 3}))},
    {"sales", pluss::table::Thread(pluss::make_series({100.0, 120.0, 90.0}))},
    {"city", pluss::table::Thread(pluss::make_note({"Accra", "Tema", "Kumasi"}))},
});

auto out = df.isna({"sales"});
```

Output

```
id  sales  city
1   100.0  Accra
2   120.0  Tema
3    90.0  Kumasi
```

5.1.96 isnull

Signature

```
DataFrame isnull(self, std::initializer_list<std::string_view> features =
```

Parameters

`features` : `std::initializer_list<std::string_view>` See the signature for the supported argument form.

Returns

`DataFrame` Value produced by this call.

Notes

This is part of the missing-data cleanup path and is usually applied before summaries, grouping, or export.

Examples

```
auto df = pluss::table::DataFrame({
    {"id", pluss::table::Thread(pluss::make_series({1, 2, 3}))},
    {"sales", pluss::table::Thread(pluss::make_series({100.0, 120.0, 90.0}))},
    {"city", pluss::table::Thread(pluss::make_note({"Accra", "Tema", "Kumasi"}))},
});

auto out = df.isnull({"sales"});
```

Output

```
id  sales  city
1   100.0  Accra
2   120.0  Tema
3    90.0  Kumasi
```

5.1.97 join

Signature

```
DataFrame join(self, const DataFrame& other, std::string collision_suffix = "_right")␣
↪const
```

Parameters

`other` : `const DataFrame&` See the signature for the supported argument form.

`collision_suffix` : `std::string` See the signature for the supported argument form.

Returns

`DataFrame` Value produced by this call.

Notes

Use this when you need to merge compatible objects or extend the current one with another input.

Examples

```
auto left = pluss::table::DataFrame({
    {"id", pluss::table::Thread(pluss::make_series({1, 2, 3}))},
    {"sales", pluss::table::Thread(pluss::make_series({100.0, 120.0, 90.0}))},
});
auto right = pluss::table::DataFrame({
    {"id", pluss::table::Thread(pluss::make_series({1, 2, 3}))},
    {"cost", pluss::table::Thread(pluss::make_series({60.0, 80.0, 55.0}))},
});

auto out = left.join(right);
```

Output

```
id  sales  cost
1   100.0  60.0
2   120.0  80.0
3    90.0  55.0
```

5.1.98 kindelize

Signature

```
pluss::kindle::Catalog kindelize(self) const
```

Parameters

None

Returns

`pluss::kindle::Catalog` Value produced by this call.

Notes

This is an explicit bridge between PlussMic models so you can switch to the representation that fits the next task.

See also

- `pluss::kindle::Catalog kindelize(self, const pluss::kindle::KindelizePolicy& policy) const`

Examples

```
auto df = pluss::table::DataFrame({
    {"id", pluss::table::Thread(pluss::make_series({1, 2, 3}))},
    {"sales", pluss::table::Thread(pluss::make_series({100.0, 120.0, 90.0}))},
    {"city", pluss::table::Thread(pluss::make_note({"Accra", "Tema", "Kumasi"}))},
});

auto out = df.kindelize();
```

5.1.99 `kindelize`

Signature

```
pluss::kindle::Catalog kindelize(self, const pluss::kindle::KindelizePolicy& policy)␣
↪const
```

Parameters

`policy` : `const pluss::kindle::KindelizePolicy&` See the signature for the supported argument form.

Returns

`pluss::kindle::Catalog` Value produced by this call.

Notes

This is an explicit bridge between PlussMic models so you can switch to the representation that fits the next task.

See also

- `pluss::kindle::Catalog kindelize(self) const`

Examples

```
auto df = pluss::table::DataFrame({
    {"id", pluss::table::Thread(pluss::make_series({1, 2, 3}))},
    {"sales", pluss::table::Thread(pluss::make_series({100.0, 120.0, 90.0}))},
    {"city", pluss::table::Thread(pluss::make_note({"Accra", "Tema", "Kumasi"}))},
});

auto out = df.kindelize();
```

5.1.100 le

Signature

```
DataFrame le(self, const DataFrame& other) const
```

Parameters

`other` : `const DataFrame&` See the signature for the supported argument form.

Returns

`DataFrame` Value produced by this call.

Notes

This produces a comparison-style result you can inspect directly or reuse as a mask.

See also

- `DataFrame le(self, scalar other) const`

Examples

```
auto left = pluss::table::DataFrame({
    {"id", pluss::table::Thread(pluss::make_series({1, 2, 3}))},
    {"sales", pluss::table::Thread(pluss::make_series({100.0, 120.0, 90.0}))},
});
auto right = pluss::table::DataFrame({
    {"id", pluss::table::Thread(pluss::make_series({1, 2, 3}))},
    {"cost", pluss::table::Thread(pluss::make_series({60.0, 80.0, 55.0}))},
});

auto out = left.le(right);
```

Output

```
id  sales
1   true
2   true
3   false
```

5.1.101 le

Signature

```
DataFrame le(self, scalar other) const
```

Parameters

`other` : `scalar` See the signature for the supported argument form.

Returns

`DataFrame` Value produced by this call.

Notes

This produces a comparison-style result you can inspect directly or reuse as a mask.

See also

- `DataFrame le(self, const DataFrame& other) const`

Examples

```
auto df = pluss::table::DataFrame({
    {"id", pluss::table::Thread(pluss::make_series({1, 2, 3}))},
    {"sales", pluss::table::Thread(pluss::make_series({100.0, 120.0, 90.0}))},
    {"city", pluss::table::Thread(pluss::make_note({"Accra", "Tema", "Kumasi"}))},
});

auto out = df.le(10);
```

Output

```
id  sales
1   true
2   true
3   false
```

5.1.102 `load`

Signature

```
static DataFrame load(std::string_view source_type, std::string_view filepath)
```

Parameters

`source_type` : `std::string_view` See the signature for the supported argument form.

`filepath` : `std::string_view` See the signature for the supported argument form.

Returns

`DataFrame` Value produced by this call.

Notes

Loads the object from disk using the corresponding PlussMic loader route.

See also

- `static DataFrame load(std::string_view source_type, std::string_view filepath, const pluss::loader::Config& config)`
- `static DataFrame load(std::string_view source_type, std::string_view filepath, std::initializer_list<pluss::loader::ConfigEntry> config_entries)`

- static DataFrame load(std::string_view source_type, std::string_view filepath, const pluss::loader::ReadCsvOptions& options)
- static DataFrame load(std::string_view source_type, std::string_view filepath, char sep, std::optional<std::size_t> nrows = std::nullopt, std::vector<std::string> usecols =
- static DataFrame load(std::string_view source_type, std::string_view filepath, char sep, std::optional<std::size_t> nrows, std::initializer_list<std::string_view> usecols, std::optional<std::string> case_col = std::nullopt, pluss::loader::BadLinePolicy on_bad_lines = pluss::loader::BadLinePolicy::Error)

Examples

```
auto out = pluss::table::DataFrame::load("csv", "PlussMic/data/messy_data.csv");
```

Output

```
id  sales  city
1   100.0  Accra
2   120.0  Tema
3    90.0  Kumasi
```

5.1.103 load

Signature

```
static DataFrame load(std::string_view source_type, std::string_view filepath, char sep,␣
↪std::optional<std::size_t> nrows = std::nullopt, std::vector<std::string> usecols =
```

Parameters

source_type : std::string_view See the signature for the supported argument form.

filepath : std::string_view See the signature for the supported argument form.

sep : char See the signature for the supported argument form.

nrows : std::optional<std::size_t> See the signature for the supported argument form.

usecols : std::vector<std::string> See the signature for the supported argument form.

Returns

DataFrame Value produced by this call.

Notes

Loads the object from disk using the corresponding PlussMic loader route.

See also

- static DataFrame load(std::string_view source_type, std::string_view filepath)
- static DataFrame load(std::string_view source_type, std::string_view filepath, const pluss::loader::Config& config)

- `static DataFrame load(std::string_view source_type, std::string_view filepath, std::initializer_list<pluss::loader::ConfigEntry> config_entries)`
- `static DataFrame load(std::string_view source_type, std::string_view filepath, const pluss::loader::ReadCsvOptions& options)`
- `static DataFrame load(std::string_view source_type, std::string_view filepath, char sep, std::optional<std::size_t> nrows, std::initializer_list<std::string_view> usecols, std::optional<std::string> case_col = std::nullopt, pluss::loader::BadLinePolicy on_bad_lines = pluss::loader::BadLinePolicy::Error)`

Examples

```
auto out = pluss::table::DataFrame::load_csv("PlussMic/data/messy_data.csv", ',', 16,␣
↪std::vector<std::string>{"Agency Name"}, std::nullopt);
```

Output

```
id  sales  city
1   100.0  Accra
2   120.0  Tema
3    90.0  Kumasi
```

5.1.104 load

Signature

```
static DataFrame load(std::string_view source_type, std::string_view filepath, char sep,␣
↪std::optional<std::size_t> nrows, std::initializer_list<std::string_view> usecols,␣
↪std::optional<std::string> case_col = std::nullopt, pluss::loader::BadLinePolicy on_
↪bad_lines = pluss::loader::BadLinePolicy::Error)
```

Parameters

`source_type` : `std::string_view` See the signature for the supported argument form.

`filepath` : `std::string_view` See the signature for the supported argument form.

`sep` : `char` See the signature for the supported argument form.

`nrows` : `std::optional<std::size_t>` See the signature for the supported argument form.

`usecols` : `std::initializer_list<std::string_view>` See the signature for the supported argument form.

`case_col` : `std::optional<std::string>` See the signature for the supported argument form.

`on_bad_lines` : `pluss::loader::BadLinePolicy` See the signature for the supported argument form.

Returns

`DataFrame` Value produced by this call.

Notes

Loads the object from disk using the corresponding PlussMic loader route.

See also

- static DataFrame load(std::string_view source_type, std::string_view filepath)
- static DataFrame load(std::string_view source_type, std::string_view filepath, const pluss::loader::Config& config)
- static DataFrame load(std::string_view source_type, std::string_view filepath, std::initializer_list<pluss::loader::ConfigEntry> config_entries)
- static DataFrame load(std::string_view source_type, std::string_view filepath, const pluss::loader::ReadCsvOptions& options)
- static DataFrame load(std::string_view source_type, std::string_view filepath, char sep, std::optional<std::size_t> nrows = std::nullopt, std::vector<std::string> usecols =

Examples

```
auto out = pluss::table::DataFrame::load_csv("PlussMic/data/messy_data.csv", ',', 16,␣
↪std::vector<std::string>{"Agency Name"}, std::nullopt);
```

Output

```
id  sales  city
1   100.0  Accra
2   120.0  Tema
3    90.0  Kumasi
```

5.1.105 load

Signature

```
static DataFrame load(std::string_view source_type, std::string_view filepath, const␣
↪pluss::loader::Config& config)
```

Parameters

source_type : std::string_view See the signature for the supported argument form.

filepath : std::string_view See the signature for the supported argument form.

config : const pluss::loader::Config& See the signature for the supported argument form.

Returns

DataFrame Value produced by this call.

Notes

Loads the object from disk using the corresponding PlussMic loader route.

See also

- static DataFrame load(std::string_view source_type, std::string_view filepath)
- static DataFrame load(std::string_view source_type, std::string_view filepath, std::initializer_list<pluss::loader::ConfigEntry> config_entries)

- static DataFrame load(std::string_view source_type, std::string_view filepath, const pluss::loader::ReadCsvOptions& options)
- static DataFrame load(std::string_view source_type, std::string_view filepath, char sep, std::optional<std::size_t> nrows = std::nullopt, std::vector<std::string> usecols =
- static DataFrame load(std::string_view source_type, std::string_view filepath, char sep, std::optional<std::size_t> nrows, std::initializer_list<std::string_view> usecols, std::optional<std::string> case_col = std::nullopt, pluss::loader::BadLinePolicy on_bad_lines = pluss::loader::BadLinePolicy::Error)

Examples

```
auto out = pluss::table::DataFrame::load_csv("PlussMic/data/messy_data.csv", {{"nrows",␣
↪16}});
```

Output

```
id  sales  city
1   100.0  Accra
2   120.0  Tema
3    90.0  Kumasi
```

5.1.106 load

Signature

```
static DataFrame load(std::string_view source_type, std::string_view filepath, const␣
↪pluss::loader::ReadCsvOptions& options)
```

Parameters

source_type : std::string_view See the signature for the supported argument form.

filepath : std::string_view See the signature for the supported argument form.

options : const pluss::loader::ReadCsvOptions& See the signature for the supported argument form.

Returns

DataFrame Value produced by this call.

Notes

Loads the object from disk using the corresponding PlussMic loader route.

See also

- static DataFrame load(std::string_view source_type, std::string_view filepath)
- static DataFrame load(std::string_view source_type, std::string_view filepath, const pluss::loader::Config& config)
- static DataFrame load(std::string_view source_type, std::string_view filepath, std::initializer_list<pluss::loader::ConfigEntry> config_entries)

- static DataFrame load(std::string_view source_type, std::string_view filepath, char sep, std::optional<std::size_t> nrows = std::nullopt, std::vector<std::string> usecols =
- static DataFrame load(std::string_view source_type, std::string_view filepath, char sep, std::optional<std::size_t> nrows, std::initializer_list<std::string_view> usecols, std::optional<std::string> case_col = std::nullopt, pluss::loader::BadLinePolicy on_bad_lines = pluss::loader::BadLinePolicy::Error)

Examples

```
pluss::loader::ReadCsvOptions options;
auto out = pluss::table::DataFrame::load_csv("PlussMic/data/messy_data.csv", options);
```

Output

```
id  sales  city
1   100.0  Accra
2   120.0  Tema
3    90.0  Kumasi
```

5.1.107 load

Signature

```
static DataFrame load(std::string_view source_type, std::string_view filepath,␣
↪std::initializer_list<pluss::loader::ConfigEntry> config_entries)
```

Parameters

`source_type` : `std::string_view` See the signature for the supported argument form.

`filepath` : `std::string_view` See the signature for the supported argument form.

`config_entries` : `std::initializer_list<pluss::loader::ConfigEntry>` See the signature for the supported argument form.

Returns

`DataFrame` Value produced by this call.

Notes

Loads the object from disk using the corresponding PlussMic loader route.

See also

- static DataFrame load(std::string_view source_type, std::string_view filepath)
- static DataFrame load(std::string_view source_type, std::string_view filepath, const pluss::loader::Config& config)
- static DataFrame load(std::string_view source_type, std::string_view filepath, const pluss::loader::ReadCsvOptions& options)

- static DataFrame load(std::string_view source_type, std::string_view filepath, char sep, std::optional<std::size_t> nrows = std::nullopt, std::vector<std::string> usecols =
- static DataFrame load(std::string_view source_type, std::string_view filepath, char sep, std::optional<std::size_t> nrows, std::initializer_list<std::string_view> usecols, std::optional<std::string> case_col = std::nullopt, pluss::loader::BadLinePolicy on_bad_lines = pluss::loader::BadLinePolicy::Error)

Examples

```
auto out = pluss::table::DataFrame::load_csv("PlussMic/data/messy_data.csv", {{"nrows",␣
↪16}});
```

Output

```
id  sales  city
1   100.0  Accra
2   120.0  Tema
3    90.0  Kumasi
```

5.1.108 load_csv

Signature

```
static DataFrame load_csv(std::string_view filepath)
```

Parameters

`filepath` : `std::string_view` See the signature for the supported argument form.

Returns

`DataFrame` Value produced by this call.

Notes

Loads the object from disk using the corresponding PlussMic loader route.

See also

- static DataFrame load_csv(std::string_view filepath, const pluss::loader::ReadCsvOptions& options)
- static DataFrame load_csv(std::string_view filepath, const pluss::loader::Config& config)
- static DataFrame load_csv(std::string_view filepath, std::initializer_list<pluss::loader::ConfigEnt config_entries)
- static DataFrame load_csv(std::string_view filepath, std::string_view header, std::string_view case_feature = "none", std::string_view engine = "auto", std::optional<std::size_t> ncases = std::nullopt)
- static DataFrame load_csv(std::string_view filepath, std::string_view header, std::size_t ncases)

- static DataFrame load_csv(std::string_view filepath, char sep, std::optional<std::size_t> nrows = std::nullopt, std::vector<std::string> usecols =
- static DataFrame load_csv(std::string_view filepath, char sep, std::optional<std::size_t> nrows, std::initializer_list<std::string_view> usecols, std::optional<std::string> case_col = std::nullopt, pluss::loader::BadLinePolicy on_bad_lines = pluss::loader::BadLinePolicy::Error)

Examples

```
auto out = pluss::table::DataFrame::load_csv("PlussMic/data/messy_data.csv");
```

Output

```
id  sales  city
1   100.0  Accra
2   120.0  Tema
3    90.0  Kumasi
```

5.1.109 load_csv

Signature

```
static DataFrame load_csv(std::string_view filepath, char sep, std::optional<std::size_t>
↪ nrows = std::nullopt, std::vector<std::string> usecols =
```

Parameters

`filepath` : `std::string_view` See the signature for the supported argument form.

`sep` : `char` See the signature for the supported argument form.

`nrows` : `std::optional<std::size_t>` See the signature for the supported argument form.

`usecols` : `std::vector<std::string>` See the signature for the supported argument form.

Returns

`DataFrame` Value produced by this call.

Notes

Loads the object from disk using the corresponding PlussMic loader route.

See also

- static DataFrame load_csv(std::string_view filepath)
- static DataFrame load_csv(std::string_view filepath, const pluss::loader::ReadCsvOptions& options)
- static DataFrame load_csv(std::string_view filepath, const pluss::loader::Config& config)
- static DataFrame load_csv(std::string_view filepath, std::initializer_list<pluss::loader::ConfigEnt config_entries)

- `static DataFrame load_csv(std::string_view filepath, std::string_view header, std::string_view case_feature = "none", std::string_view engine = "auto", std::optional<std::size_t> ncases = std::nullopt)`
- `static DataFrame load_csv(std::string_view filepath, std::string_view header, std::size_t ncases)`
- `static DataFrame load_csv(std::string_view filepath, char sep, std::optional<std::size_t> nrows, std::initializer_list<std::string_view> usecols, std::optional<std::string> case_col = std::nullopt, pluss::loader::BadLinePolicy on_bad_lines = pluss::loader::BadLinePolicy::Error)`

Examples

```
auto out = pluss::table::DataFrame::load_csv("PlussMic/data/messy_data.csv", ',', 16,␣
↪std::vector<std::string>{"Agency Name"}, std::nullopt);
```

Output

```
id  sales  city
1   100.0  Accra
2   120.0  Tema
3    90.0  Kumasi
```

5.1.110 load_csv

Signature

```
static DataFrame load_csv(std::string_view filepath, char sep, std::optional<std::size_t>
↪ nrows, std::initializer_list<std::string_view> usecols, std::optional<std::string>␣
↪case_col = std::nullopt, pluss::loader::BadLinePolicy on_bad_lines =␣
↪pluss::loader::BadLinePolicy::Error)
```

Parameters

`filepath` : `std::string_view` See the signature for the supported argument form.

`sep` : `char` See the signature for the supported argument form.

`nrows` : `std::optional<std::size_t>` See the signature for the supported argument form.

`usecols` : `std::initializer_list<std::string_view>` See the signature for the supported argument form.

`case_col` : `std::optional<std::string>` See the signature for the supported argument form.

`on_bad_lines` : `pluss::loader::BadLinePolicy` See the signature for the supported argument form.

Returns

`DataFrame` Value produced by this call.

Notes

Loads the object from disk using the corresponding PlussMic loader route.

See also

- static DataFrame load_csv(std::string_view filepath)
- static DataFrame load_csv(std::string_view filepath, const pluss::loader::ReadCsvOptions& options)
- static DataFrame load_csv(std::string_view filepath, const pluss::loader::Config& config)
- static DataFrame load_csv(std::string_view filepath, std::initializer_list<pluss::loader::ConfigEnt config_entries)
- static DataFrame load_csv(std::string_view filepath, std::string_view header, std::string_view case_feature = "none", std::string_view engine = "auto", std::optional<std::size_t> ncases = std::nullopt)
- static DataFrame load_csv(std::string_view filepath, std::string_view header, std::size_t ncases)
- static DataFrame load_csv(std::string_view filepath, char sep, std::optional<std::size_t> nrows = std::nullopt, std::vector<std::string> usecols =

Examples

```
auto out = pluss::table::DataFrame::load_csv("PlussMic/data/messy_data.csv", ',', 16,␣
↪std::vector<std::string>{"Agency Name"}, std::nullopt);
```

Output

```
id  sales  city
1   100.0  Accra
2   120.0  Tema
3    90.0  Kumasi
```

5.1.111 load_csv

Signature

```
static DataFrame load_csv(std::string_view filepath, const pluss::loader::Config& config)
```

Parameters

`filepath` : `std::string_view` See the signature for the supported argument form.

`config` : `const pluss::loader::Config&` See the signature for the supported argument form.

Returns

`DataFrame` Value produced by this call.

Notes

Loads the object from disk using the corresponding PlussMic loader route.

See also

- static DataFrame load_csv(std::string_view filepath)
- static DataFrame load_csv(std::string_view filepath, const pluss::loader::ReadCsvOptions& options)
- static DataFrame load_csv(std::string_view filepath, std::initializer_list<pluss::loader::ConfigEnt config_entries)
- static DataFrame load_csv(std::string_view filepath, std::string_view header, std::string_view case_feature = "none", std::string_view engine = "auto", std::optional<std::size_t> ncases = std::nullopt)
- static DataFrame load_csv(std::string_view filepath, std::string_view header, std::size_t ncases)
- static DataFrame load_csv(std::string_view filepath, char sep, std::optional<std::size_t> nrows = std::nullopt, std::vector<std::string> usecols =
- static DataFrame load_csv(std::string_view filepath, char sep, std::optional<std::size_t> nrows, std::initializer_list<std::string_view> usecols, std::optional<std::string> case_col = std::nullopt, pluss::loader::BadLinePolicy on_bad_lines = pluss::loader::BadLinePolicy::Error)

Examples

```
auto out = pluss::table::DataFrame::load_csv("PlussMic/data/messy_data.csv", {{"nrows",␣
↪16}});
```

Output

```
id  sales  city
1   100.0  Accra
2   120.0  Tema
3    90.0  Kumasi
```

5.1.112 load_csv

Signature

```
static DataFrame load_csv(std::string_view filepath, const pluss::loader::ReadCsvOptions&
↪ options)
```

Parameters

`filepath` : `std::string_view` See the signature for the supported argument form.

`options` : `const pluss::loader::ReadCsvOptions&` See the signature for the supported argument form.

Returns

`DataFrame` Value produced by this call.

Notes

Loads the object from disk using the corresponding PlussMic loader route.

See also

- `static DataFrame load_csv(std::string_view filepath)`
- `static DataFrame load_csv(std::string_view filepath, const pluss::loader::Config& config)`
- `static DataFrame load_csv(std::string_view filepath, std::initializer_list<pluss::loader::ConfigEnt config_entries)`
- `static DataFrame load_csv(std::string_view filepath, std::string_view header, std::string_view case_feature = "none", std::string_view engine = "auto", std::optional<std::size_t> ncases = std::nullopt)`
- `static DataFrame load_csv(std::string_view filepath, std::string_view header, std::size_t ncases)`
- `static DataFrame load_csv(std::string_view filepath, char sep, std::optional<std::size_t> nrows = std::nullopt, std::vector<std::string> usecols =`
- `static DataFrame load_csv(std::string_view filepath, char sep, std::optional<std::size_t> nrows, std::initializer_list<std::string_view> usecols, std::optional<std::string> case_col = std::nullopt, pluss::loader::BadLinePolicy on_bad_lines = pluss::loader::BadLinePolicy::Error)`

Examples

```
pluss::loader::ReadCsvOptions options;
auto out = pluss::table::DataFrame::load_csv("PlussMic/data/messy_data.csv", options);
```

Output

```
id  sales  city
1   100.0  Accra
2   120.0  Tema
3    90.0  Kumasi
```

5.1.113 load_csv

Signature

```
static DataFrame load_csv(std::string_view filepath, std::initializer_list
↪<pluss::loader::ConfigEntry> config_entries)
```

Parameters

`filepath` : `std::string_view` See the signature for the supported argument form.

`config_entries` : `std::initializer_list<pluss::loader::ConfigEntry>` See the signature for the supported argument form.

Returns

`DataFrame` Value produced by this call.

Notes

Loads the object from disk using the corresponding PlussMic loader route.

See also

- `static DataFrame load_csv(std::string_view filepath)`
- `static DataFrame load_csv(std::string_view filepath, const pluss::loader::ReadCsvOptions& options)`
- `static DataFrame load_csv(std::string_view filepath, const pluss::loader::Config& config)`
- `static DataFrame load_csv(std::string_view filepath, std::string_view header, std::string_view case_feature = "none", std::string_view engine = "auto", std::optional<std::size_t> ncases = std::nullopt)`
- `static DataFrame load_csv(std::string_view filepath, std::string_view header, std::size_t ncases)`
- `static DataFrame load_csv(std::string_view filepath, char sep, std::optional<std::size_t> nrows = std::nullopt, std::vector<std::string> usecols =`
- `static DataFrame load_csv(std::string_view filepath, char sep, std::optional<std::size_t> nrows, std::initializer_list<std::string_view> usecols, std::optional<std::string> case_col = std::nullopt, pluss::loader::BadLinePolicy on_bad_lines = pluss::loader::BadLinePolicy::Error)`

Examples

```
auto out = pluss::table::DataFrame::load_csv("PlussMic/data/messy_data.csv", {{"nrows",␣
↪16}});
```

Output

```
id  sales  city
1   100.0  Accra
2   120.0  Tema
3    90.0  Kumasi
```

5.1.114 load_csv

Signature

```
static DataFrame load_csv(std::string_view filepath, std::string_view header, std::size_
↪t ncases)
```

Parameters

`filepath` : `std::string_view` See the signature for the supported argument form.

`header` : `std::string_view` See the signature for the supported argument form.

`ncases` : `std::size_t` See the signature for the supported argument form.

Returns

`DataFrame` Value produced by this call.

Notes

Loads the object from disk using the corresponding PlussMic loader route.

See also

- `static DataFrame load_csv(std::string_view filepath)`
- `static DataFrame load_csv(std::string_view filepath, const pluss::loader::ReadCsvOptions& options)`
- `static DataFrame load_csv(std::string_view filepath, const pluss::loader::Config& config)`
- `static DataFrame load_csv(std::string_view filepath, std::initializer_list<pluss::loader::ConfigEnt config_entries)`
- `static DataFrame load_csv(std::string_view filepath, std::string_view header, std::string_view case_feature = "none", std::string_view engine = "auto", std::optional<std::size_t> ncases = std::nullopt)`
- `static DataFrame load_csv(std::string_view filepath, char sep, std::optional<std::size_t> nrows = std::nullopt, std::vector<std::string> usecols =`
- `static DataFrame load_csv(std::string_view filepath, char sep, std::optional<std::size_t> nrows, std::initializer_list<std::string_view> usecols, std::optional<std::string> case_col = std::nullopt, pluss::loader::BadLinePolicy on_bad_lines = pluss::loader::BadLinePolicy::Error)`

Examples

```
auto out = pluss::table::DataFrame::load_csv("PlussMic/data/messy_data.csv");
```

Output

```
id  sales  city
1   100.0  Accra
2   120.0  Tema
3    90.0  Kumasi
```

5.1.115 load_csv

Signature

```
static DataFrame load_csv(std::string_view filepath, std::string_view header,␣
↪std::string_view case_feature = "none", std::string_view engine = "auto", std::optional
↪<std::size_t> ncases = std::nullopt)
```

Parameters

`filepath` : `std::string_view` See the signature for the supported argument form.

`header` : `std::string_view` See the signature for the supported argument form.

`case_feature` : `std::string_view` See the signature for the supported argument form.

`engine` : `std::string_view` See the signature for the supported argument form.

`ncases` : `std::optional<std::size_t>` See the signature for the supported argument form.

Returns

`DataFrame` Value produced by this call.

Notes

Loads the object from disk using the corresponding PlussMic loader route.

See also

- `static DataFrame load_csv(std::string_view filepath)`
- `static DataFrame load_csv(std::string_view filepath, const pluss::loader::ReadCsvOptions& options)`
- `static DataFrame load_csv(std::string_view filepath, const pluss::loader::Config& config)`
- `static DataFrame load_csv(std::string_view filepath, std::initializer_list<pluss::loader::ConfigEnt config_entries)`
- `static DataFrame load_csv(std::string_view filepath, std::string_view header, std::size_t ncases)`
- `static DataFrame load_csv(std::string_view filepath, char sep, std::optional<std::size_t> nrows = std::nullopt, std::vector<std::string> usecols =`
- `static DataFrame load_csv(std::string_view filepath, char sep, std::optional<std::size_t> nrows, std::initializer_list<std::string_view> usecols, std::optional<std::string> case_col = std::nullopt, pluss::loader::BadLinePolicy on_bad_lines = pluss::loader::BadLinePolicy::Error)`

Examples

```
auto out = pluss::table::DataFrame::load_csv("PlussMic/data/messy_data.csv");
```

Output

```
id  sales  city
1   100.0  Accra
```

(continues on next page)

(continued from previous page)

```
2   120.0  Tema
3    90.0  Kumasi
```

5.1.116 load_excel

Signature

```
static DataFrame load_excel(std::string_view filepath)
```

Parameters

`filepath` : `std::string_view` See the signature for the supported argument form.

Returns

`DataFrame` Value produced by this call.

Notes

Loads the object from disk using the corresponding PlussMic loader route.

See also

- `static DataFrame load_excel(std::string_view filepath, const pluss::loader::ReadExcelOptions& options)`
- `static DataFrame load_excel(std::string_view filepath, const pluss::loader::Config& config)`
- `static DataFrame load_excel(std::string_view filepath, std::initializer_list<pluss::loader::ConfigE config_entries)`
- `static DataFrame load_excel(std::string_view filepath, std::string_view sheet_name, bool header = true)`
- `static DataFrame load_excel(std::string_view filepath, std::int64_t sheet_index, bool header = true)`
- `static pluss::loader::LoadResult load_excel(std::string_view filepath, std::nullptr_t, bool header = true)`
- `static pluss::loader::LoadResult load_excel(std::string_view filepath, const std::vector<std::string>& sheet_names, bool header = true)`
- `static pluss::loader::LoadResult load_excel(std::string_view filepath, std::initializer_list<std::string_view> sheet_names, bool header = true)`
- `static pluss::loader::LoadResult load_excel(std::string_view filepath, const std::vector<std::int64_t>& sheet_indices, bool header = true)`
- `static pluss::loader::LoadResult load_excel(std::string_view filepath, std::initializer_list<std::int64_t> sheet_indices, bool header = true)`

Examples

```
auto out = pluss::table::DataFrame::load_excel("PlussMic/data/messy_data.csv");
```

Output

```
id  sales  city
1   100.0  Accra
2   120.0  Tema
3    90.0  Kumasi
```

5.1.117 load_excel

Signature

```
static DataFrame load_excel(std::string_view filepath, const pluss::loader::Config&␣
↪config)
```

Parameters

`filepath` : `std::string_view` See the signature for the supported argument form.

`config` : `const pluss::loader::Config&` See the signature for the supported argument form.

Returns

`DataFrame` Value produced by this call.

Notes

Loads the object from disk using the corresponding PlussMic loader route.

See also

- `static DataFrame load_excel(std::string_view filepath)`
- `static DataFrame load_excel(std::string_view filepath, const pluss::loader::ReadExcelOptions& options)`
- `static DataFrame load_excel(std::string_view filepath, std::initializer_list<pluss::loader::ConfigE config_entries)`
- `static DataFrame load_excel(std::string_view filepath, std::string_view sheet_name, bool header = true)`
- `static DataFrame load_excel(std::string_view filepath, std::int64_t sheet_index, bool header = true)`
- `static pluss::loader::LoadResult load_excel(std::string_view filepath, std::nullptr_t, bool header = true)`
- `static pluss::loader::LoadResult load_excel(std::string_view filepath, const std::vector<std::string>& sheet_names, bool header = true)`
- `static pluss::loader::LoadResult load_excel(std::string_view filepath, std::initializer_list<std::string_view> sheet_names, bool header = true)`
- `static pluss::loader::LoadResult load_excel(std::string_view filepath, const std::vector<std::int64_t>& sheet_indices, bool header = true)`

- `static pluss::loader::LoadResult load_excel(std::string_view filepath, std::initializer_list<std::int64_t> sheet_indices, bool header = true)`

Examples

```
auto out = pluss::table::DataFrame::load_csv("PlussMic/data/messy_data.csv", {{"nrows",␣
↪16}});
```

Output

```
id  sales  city
1   100.0  Accra
2   120.0  Tema
3    90.0  Kumasi
```

5.1.118 load_excel

Signature

```
static DataFrame load_excel(std::string_view filepath, const␣
↪pluss::loader::ReadExcelOptions& options)
```

Parameters

`filepath` : `std::string_view` See the signature for the supported argument form.

`options` : `const pluss::loader::ReadExcelOptions&` See the signature for the supported argument form.

Returns

`DataFrame` Value produced by this call.

Notes

Loads the object from disk using the corresponding PlussMic loader route.

See also

- `static DataFrame load_excel(std::string_view filepath)`
- `static DataFrame load_excel(std::string_view filepath, const pluss::loader::Config& config)`
- `static DataFrame load_excel(std::string_view filepath, std::initializer_list<pluss::loader::ConfigE config_entries)`
- `static DataFrame load_excel(std::string_view filepath, std::string_view sheet_name, bool header = true)`
- `static DataFrame load_excel(std::string_view filepath, std::int64_t sheet_index, bool header = true)`
- `static pluss::loader::LoadResult load_excel(std::string_view filepath, std::nullptr_t, bool header = true)`
- `static pluss::loader::LoadResult load_excel(std::string_view filepath, const std::vector<std::string>& sheet_names, bool header = true)`

- `static pluss::loader::LoadResult load_excel(std::string_view filepath, std::initializer_list<std::string_view> sheet_names, bool header = true)`
- `static pluss::loader::LoadResult load_excel(std::string_view filepath, const std::vector<std::int64_t>& sheet_indices, bool header = true)`
- `static pluss::loader::LoadResult load_excel(std::string_view filepath, std::initializer_list<std::int64_t> sheet_indices, bool header = true)`

Examples

```
pluss::loader::ReadExcelOptions options;
auto out = pluss::table::DataFrame::load_excel("PlussMic/data/report.xlsx", options);
```

Output

```
id  sales  city
1   100.0  Accra
2   120.0  Tema
3    90.0  Kumasi
```

5.1.119 load_excel

Signature

```
static DataFrame load_excel(std::string_view filepath, std::initializer_list
↪<pluss::loader::ConfigEntry> config_entries)
```

Parameters

`filepath` : `std::string_view` See the signature for the supported argument form.

`config_entries` : `std::initializer_list<pluss::loader::ConfigEntry>` See the signature for the supported argument form.

Returns

`DataFrame` Value produced by this call.

Notes

Loads the object from disk using the corresponding PlussMic loader route.

See also

- `static DataFrame load_excel(std::string_view filepath)`
- `static DataFrame load_excel(std::string_view filepath, const pluss::loader::ReadExcelOptions& options)`
- `static DataFrame load_excel(std::string_view filepath, const pluss::loader::Config& config)`
- `static DataFrame load_excel(std::string_view filepath, std::string_view sheet_name, bool header = true)`
- `static DataFrame load_excel(std::string_view filepath, std::int64_t sheet_index, bool header = true)`

- `static pluss::loader::LoadResult load_excel(std::string_view filepath, std::nullptr_t, bool header = true)`
- `static pluss::loader::LoadResult load_excel(std::string_view filepath, const std::vector<std::string>& sheet_names, bool header = true)`
- `static pluss::loader::LoadResult load_excel(std::string_view filepath, std::initializer_list<std::string_view> sheet_names, bool header = true)`
- `static pluss::loader::LoadResult load_excel(std::string_view filepath, const std::vector<std::int64_t>& sheet_indices, bool header = true)`
- `static pluss::loader::LoadResult load_excel(std::string_view filepath, std::initializer_list<std::int64_t> sheet_indices, bool header = true)`

Examples

```
auto out = pluss::table::DataFrame::load_csv("PlussMic/data/messy_data.csv", {{"nrows",␣
↪16}});
```

Output

```
id  sales  city
1   100.0  Accra
2   120.0  Tema
3    90.0  Kumasi
```

5.1.120 load_excel

Signature

```
static DataFrame load_excel(std::string_view filepath, std::int64_t sheet_index, bool␣
↪header = true)
```

Parameters

`filepath` : `std::string_view` See the signature for the supported argument form.

`sheet_index` : `std::int64_t` See the signature for the supported argument form.

`header` : `bool` See the signature for the supported argument form.

Returns

`DataFrame` Value produced by this call.

Notes

Loads the object from disk using the corresponding PlussMic loader route.

See also

- `static DataFrame load_excel(std::string_view filepath)`
- `static DataFrame load_excel(std::string_view filepath, const pluss::loader::ReadExcelOptions& options)`

- static DataFrame load_excel(std::string_view filepath, const pluss::loader::Config& config)
- static DataFrame load_excel(std::string_view filepath, std::initializer_list<pluss::loader::ConfigE config_entries)
- static DataFrame load_excel(std::string_view filepath, std::string_view sheet_name, bool header = true)
- static pluss::loader::LoadResult load_excel(std::string_view filepath, std::nullptr_t, bool header = true)
- static pluss::loader::LoadResult load_excel(std::string_view filepath, const std::vector<std::string>& sheet_names, bool header = true)
- static pluss::loader::LoadResult load_excel(std::string_view filepath, std::initializer_list<std::string_view> sheet_names, bool header = true)
- static pluss::loader::LoadResult load_excel(std::string_view filepath, const std::vector<std::int64_t>& sheet_indices, bool header = true)
- static pluss::loader::LoadResult load_excel(std::string_view filepath, std::initializer_list<std::int64_t> sheet_indices, bool header = true)

Examples

```
auto out = pluss::table::DataFrame::load_excel("PlussMic/data/report.xlsx", 0, true);
```

Output

```
id  sales  city
1   100.0  Accra
2   120.0  Tema
3    90.0  Kumasi
```

5.1.121 load_excel

Signature

```
static DataFrame load_excel(std::string_view filepath, std::string_view sheet_name, bool␣
↪header = true)
```

Parameters

`filepath` : `std::string_view` See the signature for the supported argument form.

`sheet_name` : `std::string_view` See the signature for the supported argument form.

`header` : `bool` See the signature for the supported argument form.

Returns

`DataFrame` Value produced by this call.

Notes

Loads the object from disk using the corresponding PlussMic loader route.

See also

- `static DataFrame load_excel(std::string_view filepath)`
- `static DataFrame load_excel(std::string_view filepath, const pluss::loader::ReadExcelOptions& options)`
- `static DataFrame load_excel(std::string_view filepath, const pluss::loader::Config& config)`
- `static DataFrame load_excel(std::string_view filepath, std::initializer_list<pluss::loader::ConfigE config_entries)`
- `static DataFrame load_excel(std::string_view filepath, std::int64_t sheet_index, bool header = true)`
- `static pluss::loader::LoadResult load_excel(std::string_view filepath, std::nullptr_t, bool header = true)`
- `static pluss::loader::LoadResult load_excel(std::string_view filepath, const std::vector<std::string>& sheet_names, bool header = true)`
- `static pluss::loader::LoadResult load_excel(std::string_view filepath, std::initializer_list<std::string_view> sheet_names, bool header = true)`
- `static pluss::loader::LoadResult load_excel(std::string_view filepath, const std::vector<std::int64_t>& sheet_indices, bool header = true)`
- `static pluss::loader::LoadResult load_excel(std::string_view filepath, std::initializer_list<std::int64_t> sheet_indices, bool header = true)`

Examples

```
auto out = pluss::table::DataFrame::load_excel("PlussMic/data/report.xlsx", "Sheet1",␣
↪true);
```

Output

```
id  sales  city
1   100.0  Accra
2   120.0  Tema
3    90.0  Kumasi
```

5.1.122 load_excel

Signature

```
static pluss::loader::LoadResult load_excel(std::string_view filepath, const std::vector
↪<std::int64_t>& sheet_indices, bool header = true)
```

Parameters

`filepath` : `std::string_view` See the signature for the supported argument form.

`sheet_indices` : `const std::vector<std::int64_t>&` See the signature for the supported argument form.

`header` : `bool` See the signature for the supported argument form.

Returns

`pluss::loader::LoadResult` Value produced by this call.

Notes

Loads the object from disk using the corresponding PlussMic loader route.

See also

- `static DataFrame load_excel(std::string_view filepath)`
- `static DataFrame load_excel(std::string_view filepath, const pluss::loader::ReadExcelOptions& options)`
- `static DataFrame load_excel(std::string_view filepath, const pluss::loader::Config& config)`
- `static DataFrame load_excel(std::string_view filepath, std::initializer_list<pluss::loader::ConfigE config_entries)`
- `static DataFrame load_excel(std::string_view filepath, std::string_view sheet_name, bool header = true)`
- `static DataFrame load_excel(std::string_view filepath, std::int64_t sheet_index, bool header = true)`
- `static pluss::loader::LoadResult load_excel(std::string_view filepath, std::nullptr_t, bool header = true)`
- `static pluss::loader::LoadResult load_excel(std::string_view filepath, const std::vector<std::string>& sheet_names, bool header = true)`
- `static pluss::loader::LoadResult load_excel(std::string_view filepath, std::initializer_list<std::string_view> sheet_names, bool header = true)`
- `static pluss::loader::LoadResult load_excel(std::string_view filepath, std::initializer_list<std::int64_t> sheet_indices, bool header = true)`

Examples

```
auto out = pluss::table::DataFrame::load_excel("PlussMic/data/messy_data.csv");
```

5.1.123 load_excel

Signature

```
static pluss::loader::LoadResult load_excel(std::string_view filepath, const std::vector
↪<std::string>& sheet_names, bool header = true)
```

Parameters

`filepath` : `std::string_view` See the signature for the supported argument form.

`sheet_names` : `const std::vector<std::string>&` See the signature for the supported argument form.

`header` : `bool` See the signature for the supported argument form.

Returns

`pluss::loader::LoadResult` Value produced by this call.

Notes

Loads the object from disk using the corresponding PlussMic loader route.

See also

- `static DataFrame load_excel(std::string_view filepath)`
- `static DataFrame load_excel(std::string_view filepath, const pluss::loader::ReadExcelOptions& options)`
- `static DataFrame load_excel(std::string_view filepath, const pluss::loader::Config& config)`
- `static DataFrame load_excel(std::string_view filepath, std::initializer_list<pluss::loader::ConfigE config_entries)`
- `static DataFrame load_excel(std::string_view filepath, std::string_view sheet_name, bool header = true)`
- `static DataFrame load_excel(std::string_view filepath, std::int64_t sheet_index, bool header = true)`
- `static pluss::loader::LoadResult load_excel(std::string_view filepath, std::nullptr_t, bool header = true)`
- `static pluss::loader::LoadResult load_excel(std::string_view filepath, std::initializer_list<std::string_view> sheet_names, bool header = true)`
- `static pluss::loader::LoadResult load_excel(std::string_view filepath, const std::vector<std::int64_t>& sheet_indices, bool header = true)`
- `static pluss::loader::LoadResult load_excel(std::string_view filepath, std::initializer_list<std::int64_t> sheet_indices, bool header = true)`

Examples

```
auto out = pluss::table::DataFrame::load_excel("PlussMic/data/report.xlsx", "Sheet1",␣
↪true);
```

5.1.124 load_excel

Signature

```
static pluss::loader::LoadResult load_excel(std::string_view filepath, std::initializer_
↪list<std::int64_t> sheet_indices, bool header = true)
```

Parameters

`filepath` : `std::string_view` See the signature for the supported argument form.

`sheet_indices` : `std::initializer_list<std::int64_t>` See the signature for the supported argument form.

`header` : `bool` See the signature for the supported argument form.

Returns

`pluss::loader::LoadResult` Value produced by this call.

Notes

Loads the object from disk using the corresponding PlussMic loader route.

See also

- `static DataFrame load_excel(std::string_view filepath)`
- `static DataFrame load_excel(std::string_view filepath, const pluss::loader::ReadExcelOptions& options)`
- `static DataFrame load_excel(std::string_view filepath, const pluss::loader::Config& config)`
- `static DataFrame load_excel(std::string_view filepath, std::initializer_list<pluss::loader::ConfigE config_entries)`
- `static DataFrame load_excel(std::string_view filepath, std::string_view sheet_name, bool header = true)`
- `static DataFrame load_excel(std::string_view filepath, std::int64_t sheet_index, bool header = true)`
- `static pluss::loader::LoadResult load_excel(std::string_view filepath, std::nullptr_t, bool header = true)`
- `static pluss::loader::LoadResult load_excel(std::string_view filepath, const std::vector<std::string>& sheet_names, bool header = true)`
- `static pluss::loader::LoadResult load_excel(std::string_view filepath, std::initializer_list<std::string_view> sheet_names, bool header = true)`
- `static pluss::loader::LoadResult load_excel(std::string_view filepath, const std::vector<std::int64_t>& sheet_indices, bool header = true)`

Examples

```
auto out = pluss::table::DataFrame::load_excel("PlussMic/data/messy_data.csv");
```

5.1.125 load_excel

Signature

```
static pluss::loader::LoadResult load_excel(std::string_view filepath, std::initializer_
↪list<std::string_view> sheet_names, bool header = true)
```

Parameters

`filepath` : `std::string_view` See the signature for the supported argument form.

`sheet_names` : `std::initializer_list<std::string_view>` See the signature for the supported argument form.

`header` : `bool` See the signature for the supported argument form.

Returns

`pluss::loader::LoadResult` Value produced by this call.

Notes

Loads the object from disk using the corresponding PlussMic loader route.

See also

- `static DataFrame load_excel(std::string_view filepath)`
- `static DataFrame load_excel(std::string_view filepath, const pluss::loader::ReadExcelOptions& options)`
- `static DataFrame load_excel(std::string_view filepath, const pluss::loader::Config& config)`
- `static DataFrame load_excel(std::string_view filepath, std::initializer_list<pluss::loader::ConfigE config_entries)`
- `static DataFrame load_excel(std::string_view filepath, std::string_view sheet_name, bool header = true)`
- `static DataFrame load_excel(std::string_view filepath, std::int64_t sheet_index, bool header = true)`
- `static pluss::loader::LoadResult load_excel(std::string_view filepath, std::nullptr_t, bool header = true)`
- `static pluss::loader::LoadResult load_excel(std::string_view filepath, const std::vector<std::string>& sheet_names, bool header = true)`
- `static pluss::loader::LoadResult load_excel(std::string_view filepath, const std::vector<std::int64_t>& sheet_indices, bool header = true)`
- `static pluss::loader::LoadResult load_excel(std::string_view filepath, std::initializer_list<std::int64_t> sheet_indices, bool header = true)`

Examples

```
auto out = pluss::table::DataFrame::load_excel("PlussMic/data/report.xlsx", "Sheet1",␣
↪true);
```

5.1.126 load_excel

Signature

```
static pluss::loader::LoadResult load_excel(std::string_view filepath, std::nullptr_t,␣
↪bool header = true)
```

Parameters

`filepath` : `std::string_view` See the signature for the supported argument form.

`arg2` : `std::nullptr_t` See the signature for the supported argument form.

`header` : `bool` See the signature for the supported argument form.

Returns

`pluss::loader::LoadResult` Value produced by this call.

Notes

Loads the object from disk using the corresponding PlussMic loader route.

See also

- `static DataFrame load_excel(std::string_view filepath)`
- `static DataFrame load_excel(std::string_view filepath, const pluss::loader::ReadExcelOptions& options)`
- `static DataFrame load_excel(std::string_view filepath, const pluss::loader::Config& config)`
- `static DataFrame load_excel(std::string_view filepath, std::initializer_list<pluss::loader::ConfigE config_entries)`
- `static DataFrame load_excel(std::string_view filepath, std::string_view sheet_name, bool header = true)`
- `static DataFrame load_excel(std::string_view filepath, std::int64_t sheet_index, bool header = true)`
- `static pluss::loader::LoadResult load_excel(std::string_view filepath, const std::vector<std::string>& sheet_names, bool header = true)`
- `static pluss::loader::LoadResult load_excel(std::string_view filepath, std::initializer_list<std::string_view> sheet_names, bool header = true)`
- `static pluss::loader::LoadResult load_excel(std::string_view filepath, const std::vector<std::int64_t>& sheet_indices, bool header = true)`
- `static pluss::loader::LoadResult load_excel(std::string_view filepath, std::initializer_list<std::int64_t> sheet_indices, bool header = true)`

Examples

```
auto out = pluss::table::DataFrame::load_excel("PlussMic/data/report.xlsx", nullptr,␣
↪true);
```

5.1.127 load_json

Signature

```
static DataFrame load_json(std::string_view filepath)
```

Parameters

`filepath` : `std::string_view` See the signature for the supported argument form.

Returns

`DataFrame` Value produced by this call.

Notes

Loads the object from disk using the corresponding PlussMic loader route.

See also

- `static DataFrame load_json(std::string_view filepath, const pluss::loader::Config& config)`
- `static DataFrame load_json(std::string_view filepath, std::initializer_list<pluss::loader::ConfigEr config_entries)`

Examples

```
auto out = pluss::table::DataFrame::load_json("PlussMic/data/messy_data.csv");
```

Output

```
id  sales  city
1   100.0  Accra
2   120.0  Tema
3    90.0  Kumasi
```

5.1.128 load_json

Signature

```
static DataFrame load_json(std::string_view filepath, const pluss::loader::Config&␣
↪config)
```

Parameters

`filepath` : `std::string_view` See the signature for the supported argument form.

`config` : `const pluss::loader::Config&` See the signature for the supported argument form.

Returns

`DataFrame` Value produced by this call.

Notes

Loads the object from disk using the corresponding PlussMic loader route.

See also

- `static DataFrame load_json(std::string_view filepath)`
- `static DataFrame load_json(std::string_view filepath, std::initializer_list<pluss::loader::ConfigEr config_entries)`

Examples

```
auto out = pluss::table::DataFrame::load_csv("PlussMic/data/messy_data.csv", {{"nrows",␣
↪16}});
```

Output

```
id  sales  city
1   100.0  Accra
2   120.0  Tema
3    90.0  Kumasi
```

5.1.129 load_json

Signature

```
static DataFrame load_json(std::string_view filepath, std::initializer_list
↪<pluss::loader::ConfigEntry> config_entries)
```

Parameters

`filepath` : `std::string_view` See the signature for the supported argument form.

`config_entries` : `std::initializer_list<pluss::loader::ConfigEntry>` See the signature for the supported argument form.

Returns

`DataFrame` Value produced by this call.

Notes

Loads the object from disk using the corresponding PlussMic loader route.

See also

- `static DataFrame load_json(std::string_view filepath)`
- `static DataFrame load_json(std::string_view filepath, const pluss::loader::Config& config)`

Examples

```
auto out = pluss::table::DataFrame::load_csv("PlussMic/data/messy_data.csv", {{"nrows",␣
↪16}});
```

Output

```
id  sales  city
1   100.0  Accra
2   120.0  Tema
3    90.0  Kumasi
```

5.1.130 load_parquet

Signature

```
static DataFrame load_parquet(std::string_view filepath)
```

Parameters

`filepath` : `std::string_view` See the signature for the supported argument form.

Returns

`DataFrame` Value produced by this call.

Notes

Loads the object from disk using the corresponding PlussMic loader route.

See also

- `static DataFrame load_parquet(std::string_view filepath, const pluss::loader::Config& config)`
- `static DataFrame load_parquet(std::string_view filepath, std::initializer_list<pluss::loader::Confi config_entries)`

Examples

```
auto out = pluss::table::DataFrame::load_parquet("PlussMic/data/messy_data.csv");
```

Output

```
id  sales  city
1   100.0  Accra
2   120.0  Tema
3    90.0  Kumasi
```

5.1.131 load_parquet

Signature

```
static DataFrame load_parquet(std::string_view filepath, const pluss::loader::Config&␣
↪config)
```

Parameters

`filepath` : `std::string_view` See the signature for the supported argument form.

`config` : `const pluss::loader::Config&` See the signature for the supported argument form.

Returns

`DataFrame` Value produced by this call.

Notes

Loads the object from disk using the corresponding PlussMic loader route.

See also

- `static DataFrame load_parquet(std::string_view filepath)`
- `static DataFrame load_parquet(std::string_view filepath, std::initializer_list<pluss::loader::Confi config_entries)`

Examples

```
auto out = pluss::table::DataFrame::load_csv("PlussMic/data/messy_data.csv", {{"nrows",␣
↪16}});
```

Output

```
id  sales  city
1   100.0  Accra
2   120.0  Tema
3    90.0  Kumasi
```

5.1.132 load_parquet

Signature

```
static DataFrame load_parquet(std::string_view filepath, std::initializer_list
↪<pluss::loader::ConfigEntry> config_entries)
```

Parameters

`filepath` : `std::string_view` See the signature for the supported argument form.

`config_entries` : `std::initializer_list<pluss::loader::ConfigEntry>` See the signature for the supported argument form.

Returns

`DataFrame` Value produced by this call.

Notes

Loads the object from disk using the corresponding PlussMic loader route.

See also

- `static DataFrame load_parquet(std::string_view filepath)`
- `static DataFrame load_parquet(std::string_view filepath, const pluss::loader::Config& config)`

Examples

```
auto out = pluss::table::DataFrame::load_csv("PlussMic/data/messy_data.csv", {{"nrows", 16}});
```

Output

```
id  sales  city
1   100.0  Accra
2   120.0  Tema
3    90.0  Kumasi
```

5.1.133 load_txt

Signature

```
static DataFrame load_txt(std::string_view filepath)
```

Parameters

`filepath` : `std::string_view` See the signature for the supported argument form.

Returns

`DataFrame` Value produced by this call.

Notes

Loads the object from disk using the corresponding PlussMic loader route.

See also

- `static DataFrame load_txt(std::string_view filepath, const pluss::loader::Config& config)`
- `static DataFrame load_txt(std::string_view filepath, std::initializer_list<pluss::loader::ConfigEnt config_entries)`

Examples

```
auto out = pluss::table::DataFrame::load_txt("PlussMic/data/messy_data.csv");
```

Output

```
id  sales  city
1   100.0  Accra
2   120.0  Tema
3    90.0  Kumasi
```

5.1.134 load_txt

Signature

```
static DataFrame load_txt(std::string_view filepath, const pluss::loader::Config& config)
```

Parameters

`filepath` : `std::string_view` See the signature for the supported argument form.

`config` : `const pluss::loader::Config&` See the signature for the supported argument form.

Returns

`DataFrame` Value produced by this call.

Notes

Loads the object from disk using the corresponding PlussMic loader route.

See also

- `static DataFrame load_txt(std::string_view filepath)`
- `static DataFrame load_txt(std::string_view filepath, std::initializer_list<pluss::loader::ConfigEnt config_entries)`

Examples

```
auto out = pluss::table::DataFrame::load_csv("PlussMic/data/messy_data.csv", {{"nrows",␣
↪16}});
```

Output

```
id  sales  city
1   100.0  Accra
2   120.0  Tema
3    90.0  Kumasi
```

5.1.135 load_txt

Signature

```
static DataFrame load_txt(std::string_view filepath, std::initializer_list
↪<pluss::loader::ConfigEntry> config_entries)
```

Parameters

`filepath` : `std::string_view` See the signature for the supported argument form.

`config_entries` : `std::initializer_list<pluss::loader::ConfigEntry>` See the signature for the supported argument form.

Returns

DataFrame Value produced by this call.

Notes

Loads the object from disk using the corresponding PlussMic loader route.

See also

- static DataFrame load_txt(std::string_view filepath)
- static DataFrame load_txt(std::string_view filepath, const pluss::loader::Config& config)

Examples

```
auto out = pluss::table::DataFrame::load_csv("PlussMic/data/messy_data.csv", {{"nrows",␣
↪16}});
```

Output

```
id  sales  city
1   100.0  Accra
2   120.0  Tema
3    90.0  Kumasi
```

5.1.136 lt

Signature

```
DataFrame lt(self, const DataFrame& other) const
```

Parameters

other : const DataFrame& See the signature for the supported argument form.

Returns

DataFrame Value produced by this call.

Notes

This produces a comparison-style result you can inspect directly or reuse as a mask.

See also

- DataFrame lt(self, scalar other) const

Examples

```
auto left = pluss::table::DataFrame({
    {"id", pluss::table::Thread(pluss::make_series({1, 2, 3}))},
    {"sales", pluss::table::Thread(pluss::make_series({100.0, 120.0, 90.0}))},
});
```

(continues on next page)

(continued from previous page)

```
auto right = pluss::table::DataFrame({
    {"id", pluss::table::Thread(pluss::make_series({1, 2, 3}))},
    {"cost", pluss::table::Thread(pluss::make_series({60.0, 80.0, 55.0}))},
});

auto out = left.lt(right);
```

Output

```
id  sales
1   true
2   true
3   false
```

5.1.137 lt

Signature

```
DataFrame lt(self, scalar other) const
```

Parameters

`other` : `scalar` See the signature for the supported argument form.

Returns

`DataFrame` Value produced by this call.

Notes

This produces a comparison-style result you can inspect directly or reuse as a mask.

See also

- `DataFrame lt(self, const DataFrame& other) const`

Examples

```
auto df = pluss::table::DataFrame({
    {"id", pluss::table::Thread(pluss::make_series({1, 2, 3}))},
    {"sales", pluss::table::Thread(pluss::make_series({100.0, 120.0, 90.0}))},
    {"city", pluss::table::Thread(pluss::make_note({"Accra", "Tema", "Kumasi"}))},
});

auto out = df.lt(10);
```

Output

```
id  sales
1   true
2   true
3   false
```

5.1.138 max

Signature

```
DataFrame max(self, bool render = true) const
```

Parameters

render : bool See the signature for the supported argument form.

Returns

DataFrame Value produced by this call.

Notes

Use this when you want a summary view instead of carrying the full input forward unchanged.

Examples

```
auto df = pluss::table::DataFrame({
    {"id", pluss::table::Thread(pluss::make_series({1, 2, 3}))},
    {"sales", pluss::table::Thread(pluss::make_series({100.0, 120.0, 90.0}))},
    {"city", pluss::table::Thread(pluss::make_note({"Accra", "Tema", "Kumasi"}))},
});

auto out = df.max(false);
```

Output

```
feature  value
sales    310.0
```

5.1.139 mean

Signature

```
DataFrame mean(self, bool render = true) const
```

Parameters

render : bool See the signature for the supported argument form.

Returns

DataFrame Value produced by this call.

Notes

Use this when you want a summary view instead of carrying the full input forward unchanged.

Examples

```
auto df = pluss::table::DataFrame({
    {"id", pluss::table::Thread(pluss::make_series({1, 2, 3}))},
    {"sales", pluss::table::Thread(pluss::make_series({100.0, 120.0, 90.0}))},
    {"city", pluss::table::Thread(pluss::make_note({"Accra", "Tema", "Kumasi"}))},
});

auto out = df.mean(false);
```

Output

```
feature  value
sales    310.0
```

5.1.140 mean_by

Signature

```
DataFrame mean_by(self, std::string_view group_feature, std::string_view value_feature,␣
↪std::int64_t top_n = 0, std::int64_t min_count = 1, bool dropna_group = true, bool␣
↪render = true) const
```

Parameters

`group_feature` : `std::string_view` See the signature for the supported argument form.

`value_feature` : `std::string_view` See the signature for the supported argument form.

`top_n` : `std::int64_t` See the signature for the supported argument form.

`min_count` : `std::int64_t` See the signature for the supported argument form.

`dropna_group` : `bool` See the signature for the supported argument form.

`render` : `bool` See the signature for the supported argument form.

Returns

`DataFrame` Value produced by this call.

Notes

Use this when you want a summary view instead of carrying the full input forward unchanged.

Examples

```
auto df = pluss::table::DataFrame({
    {"city", pluss::table::Thread(pluss::make_note({"Accra", "Accra", "Tema", "Tema"}))},
    {"product", pluss::table::Thread(pluss::make_note({"A", "B", "A", "B"}))},
    {"sales", pluss::table::Thread(pluss::make_series({100.0, 120.0, 90.0, 150.0}))},
});

auto out = df.mean_by("city", "sales", 3, 1, true, false);
```

Output

```
city    sales_mean
Accra   110.0
Tema    120.0
```

5.1.141 min

Signature

```
DataFrame min(self, bool render = true) const
```

Parameters

`render` : `bool` See the signature for the supported argument form.

Returns

`DataFrame` Value produced by this call.

Notes

Use this when you want a summary view instead of carrying the full input forward unchanged.

Examples

```
auto df = pluss::table::DataFrame({
    {"id", pluss::table::Thread(pluss::make_series({1, 2, 3}))},
    {"sales", pluss::table::Thread(pluss::make_series({100.0, 120.0, 90.0}))},
    {"city", pluss::table::Thread(pluss::make_note({"Accra", "Tema", "Kumasi"}))},
});

auto out = df.min(false);
```

Output

```
feature  value
sales    310.0
```

5.1.142 mod

Signature

```
DataFrame mod(self, const DataFrame& other) const
```

Parameters

`other` : `const DataFrame&` See the signature for the supported argument form.

Returns

`DataFrame` Value produced by this call.

Notes

This applies arithmetic across the compatible numeric payloads in the current object.

See also

- `DataFrame mod(self, scalar other) const`

Examples

```
auto left = pluss::table::DataFrame({
    {"id", pluss::table::Thread(pluss::make_series({1, 2, 3}))},
    {"sales", pluss::table::Thread(pluss::make_series({100.0, 120.0, 90.0}))},
});
auto right = pluss::table::DataFrame({
    {"id", pluss::table::Thread(pluss::make_series({1, 2, 3}))},
    {"cost", pluss::table::Thread(pluss::make_series({60.0, 80.0, 55.0}))},
});

auto out = left.mod(right);
```

Output

```
id  sales  city
1   100.0  Accra
2   120.0  Tema
3    90.0  Kumasi
```

5.1.143 `mod`

Signature

```
DataFrame mod(self, scalar other) const
```

Parameters

`other` : `scalar` See the signature for the supported argument form.

Returns

`DataFrame` Value produced by this call.

Notes

This applies arithmetic across the compatible numeric payloads in the current object.

See also

- `DataFrame mod(self, const DataFrame& other) const`

Examples

```
auto df = pluss::table::DataFrame({
    {"id", pluss::table::Thread(pluss::make_series({1, 2, 3}))},
    {"sales", pluss::table::Thread(pluss::make_series({100.0, 120.0, 90.0}))},
    {"city", pluss::table::Thread(pluss::make_note({"Accra", "Tema", "Kumasi"}))},
});

auto out = df.mod(10);
```

Output

```
id  sales  city
11  110.0  Accra
12  130.0  Tema
13  100.0  Kumasi
```

5.1.144 `mul`

Signature

```
DataFrame mul(self, const DataFrame& other) const
```

Parameters

`other` : `const DataFrame&` See the signature for the supported argument form.

Returns

`DataFrame` Value produced by this call.

Notes

This applies arithmetic across the compatible numeric payloads in the current object.

See also

- `DataFrame mul(self, scalar other) const`

Examples

```
auto left = pluss::table::DataFrame({
    {"id", pluss::table::Thread(pluss::make_series({1, 2, 3}))},
    {"sales", pluss::table::Thread(pluss::make_series({100.0, 120.0, 90.0}))},
});
auto right = pluss::table::DataFrame({
    {"id", pluss::table::Thread(pluss::make_series({1, 2, 3}))},
    {"cost", pluss::table::Thread(pluss::make_series({60.0, 80.0, 55.0}))},
});
```

(continues on next page)

(continued from previous page)

```
auto out = left.mul(right);
```

Output

```
id  sales  city
1   100.0  Accra
2   120.0  Tema
3    90.0  Kumasi
```

5.1.145 mul

Signature

```
DataFrame mul(self, scalar other) const
```

Parameters

`other` : `scalar` See the signature for the supported argument form.

Returns

`DataFrame` Value produced by this call.

Notes

This applies arithmetic across the compatible numeric payloads in the current object.

See also

- `DataFrame mul(self, const DataFrame& other) const`

Examples

```
auto df = pluss::table::DataFrame({
    {"id", pluss::table::Thread(pluss::make_series({1, 2, 3}))},
    {"sales", pluss::table::Thread(pluss::make_series({100.0, 120.0, 90.0}))},
    {"city", pluss::table::Thread(pluss::make_note({"Accra", "Tema", "Kumasi"}))},
});

auto out = df.mul(10);
```

Output

```
id  sales  city
11  110.0  Accra
12  130.0  Tema
13  100.0  Kumasi
```

5.1.146 multiply

Signature

```
DataFrame multiply(self, const DataFrame& other) const
```

Parameters

`other` : `const DataFrame&` See the signature for the supported argument form.

Returns

`DataFrame` Value produced by this call.

Notes

This applies arithmetic across the compatible numeric payloads in the current object.

See also

- `DataFrame multiply(self, scalar other) const`

Examples

```
auto left = pluss::table::DataFrame({
    {"id", pluss::table::Thread(pluss::make_series({1, 2, 3}))},
    {"sales", pluss::table::Thread(pluss::make_series({100.0, 120.0, 90.0}))},
});
auto right = pluss::table::DataFrame({
    {"id", pluss::table::Thread(pluss::make_series({1, 2, 3}))},
    {"cost", pluss::table::Thread(pluss::make_series({60.0, 80.0, 55.0}))},
});

auto out = left.multiply(right);
```

Output

```
id  sales  city
1   100.0  Accra
2   120.0  Tema
3    90.0  Kumasi
```

5.1.147 multiply

Signature

```
DataFrame multiply(self, scalar other) const
```

Parameters

`other` : `scalar` See the signature for the supported argument form.

Returns

`DataFrame` Value produced by this call.

Notes

This applies arithmetic across the compatible numeric payloads in the current object.

See also

- `DataFrame multiply(self, const DataFrame& other) const`

Examples

```
auto df = pluss::table::DataFrame({
    {"id", pluss::table::Thread(pluss::make_series({1, 2, 3}))},
    {"sales", pluss::table::Thread(pluss::make_series({100.0, 120.0, 90.0}))},
    {"city", pluss::table::Thread(pluss::make_note({"Accra", "Tema", "Kumasi"}))},
});

auto out = df.multiply(10);
```

Output

```
id  sales  city
11  110.0  Accra
12  130.0  Tema
13  100.0  Kumasi
```

5.1.148 ncols

Signature

```
std::int64_t ncols(self) const
```

Parameters

None

Returns

`std::int64_t` Integer count, size, or positional result.

Notes

This call is part of the public `DataFrame` API and is documented here using a concrete usage example.

Examples

```
auto df = pluss::table::DataFrame({
    {"id", pluss::table::Thread(pluss::make_series({1, 2, 3}))},
    {"sales", pluss::table::Thread(pluss::make_series({100.0, 120.0, 90.0}))},
    {"city", pluss::table::Thread(pluss::make_note({"Accra", "Tema", "Kumasi"}))},
});
```

(continues on next page)

(continued from previous page)

```
auto out = df.ncols();
```

Output

```
3
```

5.1.149 ne

Signature

```
DataFrame ne(self, const DataFrame& other) const
```

Parameters

`other` : `const DataFrame&` See the signature for the supported argument form.

Returns

`DataFrame` Value produced by this call.

Notes

This produces a comparison-style result you can inspect directly or reuse as a mask.

See also

- `DataFrame ne(self, scalar other) const`

Examples

```
auto left = pluss::table::DataFrame({
    {"id", pluss::table::Thread(pluss::make_series({1, 2, 3}))},
    {"sales", pluss::table::Thread(pluss::make_series({100.0, 120.0, 90.0}))},
});
auto right = pluss::table::DataFrame({
    {"id", pluss::table::Thread(pluss::make_series({1, 2, 3}))},
    {"cost", pluss::table::Thread(pluss::make_series({60.0, 80.0, 55.0}))},
});

auto out = left.ne(right);
```

Output

```
id  sales
1   true
2   true
3   false
```

5.1.150 ne

Signature

```
DataFrame ne(self, scalar other) const
```

Parameters

`other` : `scalar` See the signature for the supported argument form.

Returns

`DataFrame` Value produced by this call.

Notes

This produces a comparison-style result you can inspect directly or reuse as a mask.

See also

- `DataFrame ne(self, const DataFrame& other) const`

Examples

```
auto df = pluss::table::DataFrame({
    {"id", pluss::table::Thread(pluss::make_series({1, 2, 3}))},
    {"sales", pluss::table::Thread(pluss::make_series({100.0, 120.0, 90.0}))},
    {"city", pluss::table::Thread(pluss::make_note({"Accra", "Tema", "Kumasi"}))},
});

auto out = df.ne(10);
```

Output

```
id  sales
1   true
2   true
3   false
```

5.1.151 nfeatures

Signature

```
std::int64_t nfeatures(self) const
```

Parameters

None

Returns

`std::int64_t` Integer count, size, or positional result.

Notes

This call is part of the public `DataFrame` API and is documented here using a concrete usage example.

Examples

```
auto df = pluss::table::DataFrame({
    {"id", pluss::table::Thread(pluss::make_series({1, 2, 3}))},
    {"sales", pluss::table::Thread(pluss::make_series({100.0, 120.0, 90.0}))},
    {"city", pluss::table::Thread(pluss::make_note({"Accra", "Tema", "Kumasi"}))},
});

auto out = df.nfeatures();
```

Output

```
3
```

5.1.152 note_features

Signature

```
DataFrame note_features(self) const
```

Parameters

None

Returns

`DataFrame` Value produced by this call.

Notes

This call is part of the public `DataFrame` API and is documented here using a concrete usage example.

Examples

```
auto df = pluss::table::DataFrame({
    {"id", pluss::table::Thread(pluss::make_series({1, 2, 3}))},
    {"sales", pluss::table::Thread(pluss::make_series({100.0, 120.0, 90.0}))},
    {"city", pluss::table::Thread(pluss::make_note({"Accra", "Tema", "Kumasi"}))},
});

auto out = df.note_features();
```

Output

```
id  sales  city
1   100.0  Accra
2   120.0  Tema
3    90.0  Kumasi
```

5.1.153 notna

Signature

```
DataFrame notna(self, std::initializer_list<std::string_view> features =
```

Parameters

`features` : `std::initializer_list<std::string_view>` See the signature for the supported argument form.

Returns

`DataFrame` Value produced by this call.

Notes

This is part of the missing-data cleanup path and is usually applied before summaries, grouping, or export.

Examples

```
auto df = pluss::table::DataFrame({
    {"id", pluss::table::Thread(pluss::make_series({1, 2, 3}))},
    {"sales", pluss::table::Thread(pluss::make_series({100.0, 120.0, 90.0}))},
    {"city", pluss::table::Thread(pluss::make_note({"Accra", "Tema", "Kumasi"}))},
});

auto out = df.notna({"sales"});
```

Output

```
id  sales  city
1   100.0  Accra
2   120.0  Tema
3    90.0  Kumasi
```

5.1.154 notnull

Signature

```
DataFrame notnull(self, std::initializer_list<std::string_view> features =
```

Parameters

`features` : `std::initializer_list<std::string_view>` See the signature for the supported argument form.

Returns

`DataFrame` Value produced by this call.

Notes

This is part of the missing-data cleanup path and is usually applied before summaries, grouping, or export.

Examples

```
auto df = pluss::table::DataFrame({
    {"id", pluss::table::Thread(pluss::make_series({1, 2, 3}))},
    {"sales", pluss::table::Thread(pluss::make_series({100.0, 120.0, 90.0}))},
    {"city", pluss::table::Thread(pluss::make_note({"Accra", "Tema", "Kumasi"}))},
});

auto out = df.notnull({"sales"});
```

Output

```
id  sales  city
1   100.0  Accra
2   120.0  Tema
3    90.0  Kumasi
```

5.1.155 nrows

Signature

```
std::int64_t nrows(self) const
```

Parameters

None

Returns

`std::int64_t` Integer count, size, or positional result.

Notes

This call is part of the public `DataFrame` API and is documented here using a concrete usage example.

Examples

```
auto df = pluss::table::DataFrame({
    {"id", pluss::table::Thread(pluss::make_series({1, 2, 3}))},
    {"sales", pluss::table::Thread(pluss::make_series({100.0, 120.0, 90.0}))},
    {"city", pluss::table::Thread(pluss::make_note({"Accra", "Tema", "Kumasi"}))},
});

auto out = df.nrows();
```

Output

```
3
```

5.1.156 nunique

Signature

```
DataFrame nunique(self, bool render = true) const
```

Parameters

`render` : `bool` See the signature for the supported argument form.

Returns

`DataFrame` Value produced by this call.

Notes

Use this when you want a summary view instead of carrying the full input forward unchanged.

Examples

```
auto df = pluss::table::DataFrame({
    {"id", pluss::table::Thread(pluss::make_series({1, 2, 3}))},
    {"sales", pluss::table::Thread(pluss::make_series({100.0, 120.0, 90.0}))},
    {"city", pluss::table::Thread(pluss::make_note({"Accra", "Tema", "Kumasi"}))},
});

auto out = df.nunique(false);
```

Output

```
feature  value
sales    310.0
```

5.1.157 pivot

Signature

```
DataFrame pivot(self, std::string_view case_feature, std::string_view feature_feature,␣
↪std::string_view value_feature, bool dropna_keys = true, bool render = true) const
```

Parameters

`case_feature` : `std::string_view` See the signature for the supported argument form.

`feature_feature` : `std::string_view` See the signature for the supported argument form.

`value_feature` : `std::string_view` See the signature for the supported argument form.

`dropna_keys` : `bool` See the signature for the supported argument form.

`render` : `bool` See the signature for the supported argument form.

Returns

DataFrame Value produced by this call.

Notes

This call is part of the public DataFrame API and is documented here using a concrete usage example.

Examples

```
auto df = pluss::table::DataFrame({
    {"city", pluss::table::Thread(pluss::make_note({"Accra", "Accra", "Tema", "Tema"}))},
    {"product", pluss::table::Thread(pluss::make_note({"A", "B", "A", "B"}))},
    {"sales", pluss::table::Thread(pluss::make_series({100.0, 120.0, 90.0, 150.0}))},
});

auto out = df.pivot("city", "product", "sales", true, false);
```

Output

```
city   A      B
Accra  100.0  120.0
Tema    90.0  150.0
```

5.1.158 pivot_table

Signature

```
DataFrame pivot_table(self, std::string_view case_feature, std::string_view feature_
↪feature, std::string_view value_feature, GroupOp op = GroupOp::Mean, bool dropna_keys␣
↪= true, bool render = true) const
```

Parameters

case_feature : std::string_view See the signature for the supported argument form.

feature_feature : std::string_view See the signature for the supported argument form.

value_feature : std::string_view See the signature for the supported argument form.

op : GroupOp See the signature for the supported argument form.

dropna_keys : bool See the signature for the supported argument form.

render : bool See the signature for the supported argument form.

Returns

DataFrame Value produced by this call.

Notes

This call is part of the public DataFrame API and is documented here using a concrete usage example.

Examples

```
auto df = pluss::table::DataFrame({
    {"city", pluss::table::Thread(pluss::make_note({"Accra", "Accra", "Tema", "Tema"}))},
    {"product", pluss::table::Thread(pluss::make_note({"A", "B", "A", "B"}))},
    {"sales", pluss::table::Thread(pluss::make_series({100.0, 120.0, 90.0, 150.0}))},
});

auto out = df.pivot_table("city", "product", "sales",␣
↪pluss::table::DataFrame::GroupOp::Mean, true, false);
```

Output

```
city   A      B
Accra  100.0  120.0
Tema    90.0  150.0
```

5.1.159 plot

Signature

```
DataFramePlotAccessor plot(self) const
```

Parameters

None

Returns

`DataFramePlotAccessor` Value produced by this call.

Notes

This opens the plotting accessor for the current data object.

Examples

```
auto df = pluss::table::DataFrame({
    {"id", pluss::table::Thread(pluss::make_series({1, 2, 3}))},
    {"sales", pluss::table::Thread(pluss::make_series({100.0, 120.0, 90.0}))},
    {"city", pluss::table::Thread(pluss::make_note({"Accra", "Tema", "Kumasi"}))},
});

auto fig = df.plot().line("sales");
```

Output

```
Figure(line/scatter/bar/hist)
```

5.1.160 pow

Signature

```
DataFrame pow(self, const DataFrame& other) const
```

Parameters

`other` : `const DataFrame&` See the signature for the supported argument form.

Returns

`DataFrame` Value produced by this call.

Notes

This applies arithmetic across the compatible numeric payloads in the current object.

See also

- `DataFrame pow(self, scalar other) const`

Examples

```
auto left = pluss::table::DataFrame({
    {"id", pluss::table::Thread(pluss::make_series({1, 2, 3}))},
    {"sales", pluss::table::Thread(pluss::make_series({100.0, 120.0, 90.0}))},
});
auto right = pluss::table::DataFrame({
    {"id", pluss::table::Thread(pluss::make_series({1, 2, 3}))},
    {"cost", pluss::table::Thread(pluss::make_series({60.0, 80.0, 55.0}))},
});

auto out = left.pow(right);
```

Output

```
id  sales  city
1   100.0  Accra
2   120.0  Tema
3    90.0  Kumasi
```

5.1.161 pow

Signature

```
DataFrame pow(self, scalar other) const
```

Parameters

`other` : `scalar` See the signature for the supported argument form.

Returns

`DataFrame` Value produced by this call.

Notes

This applies arithmetic across the compatible numeric payloads in the current object.

See also

- `DataFrame pow(self, const DataFrame& other) const`

Examples

```
auto df = pluss::table::DataFrame({
    {"id", pluss::table::Thread(pluss::make_series({1, 2, 3}))},
    {"sales", pluss::table::Thread(pluss::make_series({100.0, 120.0, 90.0}))},
    {"city", pluss::table::Thread(pluss::make_note({"Accra", "Tema", "Kumasi"}))},
});

auto out = df.pow(10);
```

Output

```
id  sales  city
11  110.0  Accra
12  130.0  Tema
13  100.0  Kumasi
```

5.1.162 prepare_sql

Signature

```
DataFrameSqlPrepared prepare_sql(self, std::string_view statement) const
```

Parameters

`statement` : `std::string_view` See the signature for the supported argument form.

Returns

`DataFrameSqlPrepared` Value produced by this call.

Notes

Use this when the transformation is easier to express as SQL than as a chained sequence of table verbs.

See also

- `DataFrameSqlPrepared prepare_sql(self, std::string_view statement, SqlOptions options) const`

Examples

```
auto df = pluss::table::DataFrame({
    {"id", pluss::table::Thread(pluss::make_series({1, 2, 3}))},
    {"sales", pluss::table::Thread(pluss::make_series({100.0, 120.0, 90.0}))},
    {"city", pluss::table::Thread(pluss::make_note({"Accra", "Tema", "Kumasi"}))},
});

auto stmt = df.prepare_sql("SELECT id, sales FROM self WHERE sales > {min_sales}");
```

Output

```
SQL accessor / prepared statement created
```

5.1.163 prepare_sql

Signature

```
DataFrameSqlPrepared prepare_sql(self, std::string_view statement, SqlOptions options)␣
↪const
```

Parameters

`statement` : `std::string_view` See the signature for the supported argument form.

`options` : `SqlOptions` See the signature for the supported argument form.

Returns

`DataFrameSqlPrepared` Value produced by this call.

Notes

Use this when the transformation is easier to express as SQL than as a chained sequence of table verbs.

See also

- `DataFrameSqlPrepared prepare_sql(self, std::string_view statement) const`

Examples

```
auto df = pluss::table::DataFrame({
    {"id", pluss::table::Thread(pluss::make_series({1, 2, 3}))},
    {"sales", pluss::table::Thread(pluss::make_series({100.0, 120.0, 90.0}))},
    {"city", pluss::table::Thread(pluss::make_note({"Accra", "Tema", "Kumasi"}))},
});

auto stmt = df.prepare_sql("SELECT id, sales FROM self WHERE sales > {min_sales}");
```

Output

```
SQL accessor / prepared statement created
```

5.1.164 query

Signature

```
DataFrame query(self, std::string_view expression) const
```

Parameters

`expression` : `std::string_view` See the signature for the supported argument form.

Returns

`DataFrame` Value produced by this call.

Notes

Use this to reduce the current object to the rows or fields that matter for the next step.

Examples

```
auto df = pluss::table::DataFrame({
    {"id", pluss::table::Thread(pluss::make_series({1, 2, 3}))},
    {"sales", pluss::table::Thread(pluss::make_series({100.0, 120.0, 90.0}))},
    {"city", pluss::table::Thread(pluss::make_note({"Accra", "Tema", "Kumasi"}))},
});

auto out = df.query("sales > 95");
```

Output

```
id  sales  city
1   100.0  Accra
3    90.0  Kumasi
```

5.1.165 radd

Signature

```
DataFrame radd(self, scalar other) const
```

Parameters

`other` : `scalar` See the signature for the supported argument form.

Returns

`DataFrame` Value produced by this call.

Notes

This applies arithmetic across the compatible numeric payloads in the current object.

Examples

```
auto df = pluss::table::DataFrame({
    {"id", pluss::table::Thread(pluss::make_series({1, 2, 3}))},
    {"sales", pluss::table::Thread(pluss::make_series({100.0, 120.0, 90.0}))},
    {"city", pluss::table::Thread(pluss::make_note({"Accra", "Tema", "Kumasi"}))},
});

auto out = df.radd(10);
```

Output

```
id  sales  city
11  110.0  Accra
12  130.0  Tema
13  100.0  Kumasi
```

5.1.166 rdiv

Signature

```
DataFrame rdiv(self, scalar other) const
```

Parameters

`other` : `scalar` See the signature for the supported argument form.

Returns

`DataFrame` Value produced by this call.

Notes

This applies arithmetic across the compatible numeric payloads in the current object.

Examples

```
auto df = pluss::table::DataFrame({
    {"id", pluss::table::Thread(pluss::make_series({1, 2, 3}))},
    {"sales", pluss::table::Thread(pluss::make_series({100.0, 120.0, 90.0}))},
    {"city", pluss::table::Thread(pluss::make_note({"Accra", "Tema", "Kumasi"}))},
});

auto out = df.rdiv(10);
```

Output

```
id  sales  city
11  110.0  Accra
12  130.0  Tema
13  100.0  Kumasi
```

5.1.167 reindex_cases

Signature

```
DataFrame reindex_cases(self, const Thread& new_cases) const
```

Parameters

`new_cases` : `const Thread&` See the signature for the supported argument form.

Returns

`DataFrame` Value produced by this call.

Notes

This call is part of the public `DataFrame` API and is documented here using a concrete usage example.

Examples

```
auto df = pluss::table::DataFrame({
    {"id", pluss::table::Thread(pluss::make_series({1, 2, 3}))},
    {"sales", pluss::table::Thread(pluss::make_series({100.0, 120.0, 90.0}))},
    {"city", pluss::table::Thread(pluss::make_note({"Accra", "Tema", "Kumasi"}))},
});

auto cases = pluss::table::Thread(pluss::make_note({"cs1", "cs0", "cs2"}));
auto out = df.reindex_cases(cases);
```

Output

```
id  sales  city
1   100.0  Accra
2   120.0  Tema
3    90.0  Kumasi
```

5.1.168 reindex_like

Signature

```
DataFrame reindex_like(self, const DataFrame& other) const
```

Parameters

`other` : `const DataFrame&` See the signature for the supported argument form.

Returns

`DataFrame` Value produced by this call.

Notes

This call is part of the public `DataFrame` API and is documented here using a concrete usage example.

Examples

```
auto left = pluss::table::DataFrame({
    {"id", pluss::table::Thread(pluss::make_series({1, 2, 3}))},
    {"sales", pluss::table::Thread(pluss::make_series({100.0, 120.0, 90.0}))},
});
auto right = pluss::table::DataFrame({
    {"id", pluss::table::Thread(pluss::make_series({1, 2, 3}))},
    {"cost", pluss::table::Thread(pluss::make_series({60.0, 80.0, 55.0}))},
});

auto out = left.reindex_like(right);
```

Output

```
id  sales  city
1   100.0  Accra
2   120.0  Tema
3    90.0  Kumasi
```

5.1.169 relabel_features

Signature

```
DataFrame relabel_features(self, std::initializer_list<std::string_view> feature_names)␣
↪const
```

Parameters

`feature_names` : `std::initializer_list<std::string_view>` See the signature for the supported argument form.

Returns

`DataFrame` Value produced by this call.

Notes

This adjusts labels without changing the underlying payload values.

Examples

```
auto df = pluss::table::DataFrame({
    {"id", pluss::table::Thread(pluss::make_series({1, 2, 3}))},
    {"sales", pluss::table::Thread(pluss::make_series({100.0, 120.0, 90.0}))},
    {"city", pluss::table::Thread(pluss::make_note({"Accra", "Tema", "Kumasi"}))},
});

auto out = df.relabel_features({"revenue", "location", "id_new"});
```

Output

```
id  revenue  city
1   100.0    Accra
2   120.0    Tema
3    90.0    Kumasi
```

5.1.170 rename

Signature

```
DataFrame rename(self, std::string_view old_name, std::string new_name) const
```

Parameters

old_name : std::string_view See the signature for the supported argument form.

new_name : std::string See the signature for the supported argument form.

Returns

DataFrame Value produced by this call.

Notes

This adjusts labels without changing the underlying payload values.

Examples

```
auto df = pluss::table::DataFrame({
    {"id", pluss::table::Thread(pluss::make_series({1, 2, 3}))},
    {"sales", pluss::table::Thread(pluss::make_series({100.0, 120.0, 90.0}))},
    {"city", pluss::table::Thread(pluss::make_note({"Accra", "Tema", "Kumasi"}))},
});

auto out = df.rename("sales", "revenue");
```

Output

```
id  revenue  city
1   100.0    Accra
2   120.0    Tema
3    90.0    Kumasi
```

5.1.171 rename_

Signature

```
DataFrame& rename_(self, std::string_view old_name, std::string new_name)
```

Parameters

`old_name` : `std::string_view` See the signature for the supported argument form.

`new_name` : `std::string` See the signature for the supported argument form.

Returns

`DataFrame&` Value produced by this call.

Notes

This adjusts labels without changing the underlying payload values.

Examples

```
auto df = pluss::table::DataFrame({
    {"id", pluss::table::Thread(pluss::make_series({1, 2, 3}))},
    {"sales", pluss::table::Thread(pluss::make_series({100.0, 120.0, 90.0}))},
    {"city", pluss::table::Thread(pluss::make_note({"Accra", "Tema", "Kumasi"}))},
});

df.rename_("sales", "revenue");
```

Output

```
id  revenue  city
1   100.0    Accra
2   120.0    Tema
3    90.0    Kumasi
```

5.1.172 rename_features

Signature

```
DataFrame rename_features(self,  std::initializer_list<std::pair<std::string_view,␣
↪std::string_view>> renames) const
```

Parameters

`renames` : `std::initializer_list<std::pair<std::string_view, std::string_view>>` See the signature for the supported argument form.

Returns

`DataFrame` Value produced by this call.

Notes

This adjusts labels without changing the underlying payload values.

Examples

```
auto df = pluss::table::DataFrame({
    {"id", pluss::table::Thread(pluss::make_series({1, 2, 3}))},
    {"sales", pluss::table::Thread(pluss::make_series({100.0, 120.0, 90.0}))},
    {"city", pluss::table::Thread(pluss::make_note({"Accra", "Tema", "Kumasi"}))},
});

auto out = df.rename_features({{"sales", "revenue"}, {"city", "location"}});
```

Output

```
id  revenue  city
1   100.0    Accra
2   120.0    Tema
3    90.0    Kumasi
```

5.1.173 `reorder`

Signature

```
DataFrame reorder(self, std::initializer_list<std::string_view> names) const
```

Parameters

`names` : `std::initializer_list<std::string_view>` See the signature for the supported argument form.

Returns

`DataFrame` Value produced by this call.

Notes

This call is part of the public `DataFrame` API and is documented here using a concrete usage example.

Examples

```
auto df = pluss::table::DataFrame({
    {"id", pluss::table::Thread(pluss::make_series({1, 2, 3}))},
    {"sales", pluss::table::Thread(pluss::make_series({100.0, 120.0, 90.0}))},
    {"city", pluss::table::Thread(pluss::make_note({"Accra", "Tema", "Kumasi"}))},
});

auto out = df.reorder({"sales", "id", "city"});
```

Output

```
sales  id  city
100.0  1  Accra
120.0  2  Tema
90.0   3  Kumasi
```

5.1.174 repeated_cases

Signature

```
pluss::Series repeated_cases(self, std::initializer_list<std::string_view> features =
```

Parameters

`features` : `std::initializer_list<std::string_view>` See the signature for the supported argument form.

Returns

`pluss::Series` Value produced by this call.

Notes

This call is part of the public `DataFrame` API and is documented here using a concrete usage example.

Examples

```
auto df = pluss::table::DataFrame({
    {"id", pluss::table::Thread(pluss::make_series({1, 2, 3}))},
    {"sales", pluss::table::Thread(pluss::make_series({100.0, 120.0, 90.0}))},
    {"city", pluss::table::Thread(pluss::make_note({"Accra", "Tema", "Kumasi"}))},
});

auto out = df.repeated_cases();
```

5.1.175 resample

Signature

```
DataFrame resample(self, std::string_view time_feature, std::initializer_list
↪<std::string_view> value_features, std::string_view freq, std::string_view op = "mean",
↪ std::int64_t step = 1, bool render = true) const
```

Parameters

`time_feature` : `std::string_view` See the signature for the supported argument form.

`value_features` : `std::initializer_list<std::string_view>` See the signature for the supported argument form.

`freq` : `std::string_view` See the signature for the supported argument form.

`op` : `std::string_view` See the signature for the supported argument form.

`step` : `std::int64_t` See the signature for the supported argument form.

`render` : `bool` See the signature for the supported argument form.

Returns

`DataFrame` Value produced by this call.

Notes

This is part of the time-aware surface and expects timeline or time-index semantics.

Examples

```
auto df = pluss::table::DataFrame({
    {"ts", pluss::table::Thread(pluss::make_timeline({
        "2024-01-01T09:10:00Z",
        "2024-01-01T09:40:00Z",
        "2024-01-01T10:05:00Z",
    }))},
    {"load", pluss::table::Thread(pluss::make_series({10.0, 20.0, 15.0}))},
});

auto out = df.resample("ts", {"load"}, "1h", "mean", 1, false);
```

Output

```
ts                   load
2024-01-01T09:00:00Z  15.0
2024-01-01T10:00:00Z  15.0
```

5.1.176 reset_cases

Signature

```
DataFrame reset_cases(self, std::string case_feature = "case") const
```

Parameters

`case_feature` : `std::string` See the signature for the supported argument form.

Returns

`DataFrame` Value produced by this call.

Notes

This call is part of the public `DataFrame` API and is documented here using a concrete usage example.

Examples

```
auto df = pluss::table::DataFrame({
    {"id", pluss::table::Thread(pluss::make_series({1, 2, 3}))},
    {"sales", pluss::table::Thread(pluss::make_series({100.0, 120.0, 90.0}))},
    {"city", pluss::table::Thread(pluss::make_note({"Accra", "Tema", "Kumasi"}))},
});

auto with_cases = df.set_cases("city", true);
auto out = with_cases.reset_cases("case");
```

Output

```
id  sales  city
1   100.0  Accra
2   120.0  Tema
3    90.0  Kumasi
```

5.1.177 rfloordiv

Signature

```
DataFrame rfloordiv(self, scalar other) const
```

Parameters

`other` : `scalar` See the signature for the supported argument form.

Returns

`DataFrame` Value produced by this call.

Notes

This applies arithmetic across the compatible numeric payloads in the current object.

Examples

```
auto df = pluss::table::DataFrame({
    {"id", pluss::table::Thread(pluss::make_series({1, 2, 3}))},
    {"sales", pluss::table::Thread(pluss::make_series({100.0, 120.0, 90.0}))},
    {"city", pluss::table::Thread(pluss::make_note({"Accra", "Tema", "Kumasi"}))},
});

auto out = df.rfloordiv(10);
```

Output

```
id  sales  city
11  110.0  Accra
12  130.0  Tema
13  100.0  Kumasi
```

5.1.178 rmod

Signature

```
DataFrame rmod(self, scalar other) const
```

Parameters

`other` : `scalar` See the signature for the supported argument form.

Returns

`DataFrame` Value produced by this call.

Notes

This applies arithmetic across the compatible numeric payloads in the current object.

Examples

```
auto df = pluss::table::DataFrame({
    {"id", pluss::table::Thread(pluss::make_series({1, 2, 3}))},
    {"sales", pluss::table::Thread(pluss::make_series({100.0, 120.0, 90.0}))},
    {"city", pluss::table::Thread(pluss::make_note({"Accra", "Tema", "Kumasi"}))},
});

auto out = df.rmod(10);
```

Output

```
id  sales  city
11  110.0  Accra
12  130.0  Tema
13  100.0  Kumasi
```

5.1.179 `rmul`

Signature

```
DataFrame rmul(self, scalar other) const
```

Parameters

`other` : `scalar` See the signature for the supported argument form.

Returns

`DataFrame` Value produced by this call.

Notes

This applies arithmetic across the compatible numeric payloads in the current object.

Examples

```
auto df = pluss::table::DataFrame({
    {"id", pluss::table::Thread(pluss::make_series({1, 2, 3}))},
    {"sales", pluss::table::Thread(pluss::make_series({100.0, 120.0, 90.0}))},
    {"city", pluss::table::Thread(pluss::make_note({"Accra", "Tema", "Kumasi"}))},
});

auto out = df.rmul(10);
```

Output

```
id  sales  city
11  110.0  Accra
12  130.0  Tema
13  100.0  Kumasi
```

5.1.180 rolling

Signature

```
DataFrame rolling(self, std::initializer_list<WindowSpec> specs, bool keep_input = true)␣
↪const
```

Parameters

`specs` : `std::initializer_list<WindowSpec>` See the signature for the supported argument form.

`keep_input` : `bool` See the signature for the supported argument form.

Returns

`DataFrame` Value produced by this call.

Notes

This call is part of the public `DataFrame` API and is documented here using a concrete usage example.

Examples

```
auto df = pluss::table::DataFrame({
    {"id", pluss::table::Thread(pluss::make_series({1, 2, 3}))},
    {"sales", pluss::table::Thread(pluss::make_series({100.0, 120.0, 90.0}))},
    {"city", pluss::table::Thread(pluss::make_note({"Accra", "Tema", "Kumasi"}))},
});

auto out = df.rolling({{.output = "sales_roll", .feature = "sales", .op =␣
↪pluss::table::ThreadWindowOp::RollingMean, .window = 2, .min_periods = 1}}, true);
```

Output

```
id  sales  city
1   100.0  Accra
2   120.0  Tema
3    90.0  Kumasi
```

5.1.181 round_time

Signature

```
DataFrame round_time(self, std::initializer_list<std::string_view> features, std::string_
↪view unit) const
```

Parameters

`features` : `std::initializer_list<std::string_view>` See the signature for the supported argument form.

`unit` : `std::string_view` See the signature for the supported argument form.

Returns

`DataFrame` Value produced by this call.

Notes

This is part of the time-aware surface and expects timeline or time-index semantics.

Examples

```
auto df = pluss::table::DataFrame({
    {"ts", pluss::table::Thread(pluss::make_timeline({
        "2024-01-01T09:10:00Z",
        "2024-01-01T09:40:00Z",
        "2024-01-01T10:05:00Z",
    }))},
    {"load", pluss::table::Thread(pluss::make_series({10.0, 20.0, 15.0}))},
});

auto out = df.round_time({"ts"}, "hour");
```

Output

```
ts
2024-01-01T09:00:00Z
2024-01-01T10:00:00Z
```

5.1.182 `rpow`

Signature

```
DataFrame rpow(self, scalar other) const
```

Parameters

`other` : `scalar` See the signature for the supported argument form.

Returns

`DataFrame` Value produced by this call.

Notes

This applies arithmetic across the compatible numeric payloads in the current object.

Examples

```
auto df = pluss::table::DataFrame({
    {"id", pluss::table::Thread(pluss::make_series({1, 2, 3}))},
    {"sales", pluss::table::Thread(pluss::make_series({100.0, 120.0, 90.0}))},
    {"city", pluss::table::Thread(pluss::make_note({"Accra", "Tema", "Kumasi"}))},
});

auto out = df.rpow(10);
```

Output

```
id  sales  city
11  110.0  Accra
12  130.0  Tema
13  100.0  Kumasi
```

5.1.183 `rsub`

Signature

```
DataFrame rsub(self, scalar other) const
```

Parameters

`other` : `scalar` See the signature for the supported argument form.

Returns

`DataFrame` Value produced by this call.

Notes

This applies arithmetic across the compatible numeric payloads in the current object.

Examples

```
auto df = pluss::table::DataFrame({
    {"id", pluss::table::Thread(pluss::make_series({1, 2, 3}))},
    {"sales", pluss::table::Thread(pluss::make_series({100.0, 120.0, 90.0}))},
    {"city", pluss::table::Thread(pluss::make_note({"Accra", "Tema", "Kumasi"}))},
});

auto out = df.rsub(10);
```

Output

```
id  sales  city
11  110.0  Accra
12  130.0  Tema
13  100.0  Kumasi
```

5.1.184 rtruediv

Signature

```
DataFrame rtruediv(self, scalar other) const
```

Parameters

other : scalar See the signature for the supported argument form.

Returns

DataFrame Value produced by this call.

Notes

This applies arithmetic across the compatible numeric payloads in the current object.

Examples

```
auto df = pluss::table::DataFrame({
    {"id", pluss::table::Thread(pluss::make_series({1, 2, 3}))},
    {"sales", pluss::table::Thread(pluss::make_series({100.0, 120.0, 90.0}))},
    {"city", pluss::table::Thread(pluss::make_note({"Accra", "Tema", "Kumasi"}))},
});

auto out = df.rtruediv(10);
```

Output

```
id  sales  city
11  110.0  Accra
12  130.0  Tema
13  100.0  Kumasi
```

5.1.185 sample

Signature

```
DataFrame sample(self, std::int64_t n, std::optional<std::uint64_t> random_state =␣
↪std::nullopt) const
```

Parameters

n : std::int64_t See the signature for the supported argument form.

random_state : std::optional<std::uint64_t> See the signature for the supported argument form.

Returns

DataFrame Value produced by this call.

Notes

Use this to narrow the current object to the rows or depth range you want to inspect next.

Examples

```
auto df = pluss::table::DataFrame({
    {"id", pluss::table::Thread(pluss::make_series({1, 2, 3}))},
    {"sales", pluss::table::Thread(pluss::make_series({100.0, 120.0, 90.0}))},
    {"city", pluss::table::Thread(pluss::make_note({"Accra", "Tema", "Kumasi"}))},
});

auto out = df.sample(2, 7);
```

Output

```
id  sales  city
1   100.0  Accra
2   120.0  Tema
3    90.0  Kumasi
```

5.1.186 save_as

Signature

```
void save_as(self, std::string_view target_type, std::string_view filepath, const␣
↪pluss::loader::Config& config =
```

Parameters

`target_type` : `std::string_view` See the signature for the supported argument form.

`filepath` : `std::string_view` See the signature for the supported argument form.

`config` : `const pluss::loader::Config&` See the signature for the supported argument form.

Returns

`void` Performs the side effect and returns no value.

Notes

Writes the current object to disk using the corresponding PlussMic saver route.

See also

- `void save_as(self, std::string_view target_type, std::string_view filepath, std::initializer_list<pluss::loader::ConfigEntry> config_entries) const`

Examples

```
auto df = pluss::table::DataFrame({
    {"id", pluss::table::Thread(pluss::make_series({1, 2, 3}))},
    {"sales", pluss::table::Thread(pluss::make_series({100.0, 120.0, 90.0}))},
    {"city", pluss::table::Thread(pluss::make_note({"Accra", "Tema", "Kumasi"}))},
```

(continues on next page)

(continued from previous page)

```
});

df.save_as("json", "out.json");
```

Output

```
saved: out.*
```

5.1.187 save_as

Signature

```
void save_as(self, std::string_view target_type, std::string_view filepath,␣
↪std::initializer_list<pluss::loader::ConfigEntry> config_entries) const
```

Parameters

`target_type` : `std::string_view` See the signature for the supported argument form.

`filepath` : `std::string_view` See the signature for the supported argument form.

`config_entries` : `std::initializer_list<pluss::loader::ConfigEntry>` See the signature for the supported argument form.

Returns

`void` Performs the side effect and returns no value.

Notes

Writes the current object to disk using the corresponding PlussMic saver route.

See also

- `void save_as(self, std::string_view target_type, std::string_view filepath, const pluss::loader::Config& config =`

Examples

```
auto df = pluss::table::DataFrame({
    {"id", pluss::table::Thread(pluss::make_series({1, 2, 3}))},
    {"sales", pluss::table::Thread(pluss::make_series({100.0, 120.0, 90.0}))},
    {"city", pluss::table::Thread(pluss::make_note({"Accra", "Tema", "Kumasi"}))},
});

df.save_as("json", "out.json");
```

Output

```
saved: out.*
```

5.1.188 save_csv

Signature

```
void save_csv(self, std::string_view filepath) const
```

Parameters

`filepath` : `std::string_view` See the signature for the supported argument form.

Returns

`void` Performs the side effect and returns no value.

Notes

Writes the current object to disk using the corresponding PlussMic saver route.

See also

- `void save_csv(self, std::string_view filepath, bool add_cases, bool header = true, char sep = ',', std::string na_rep = "NA", std::optional<std::string> case_label = std::nullopt, std::optional<std::string> float_format = std::nullopt, char quotechar = '"', bool scalarquote = true, std::string line_terminator = "\n", char decimal = '.', std::string encoding = "utf-8") const`
- `void save_csv(self, std::string_view filepath, const pluss::loader::WriteCsvOptions& options) const`
- `void save_csv(self, std::string_view filepath, const pluss::loader::Config& config) const`
- `void save_csv(self, std::string_view filepath, std::initializer_list<pluss::loader::ConfigEntry> config_entries) const`

Examples

```
auto df = pluss::table::DataFrame({
    {"id", pluss::table::Thread(pluss::make_series({1, 2, 3}))},
    {"sales", pluss::table::Thread(pluss::make_series({100.0, 120.0, 90.0}))},
    {"city", pluss::table::Thread(pluss::make_note({"Accra", "Tema", "Kumasi"}))},
});

df.save_csv("out.csv");
```

Output

```
saved: out.*
```

5.1.189 save_csv

Signature

```
void save_csv(self, std::string_view filepath, bool add_cases, bool header = true, char␣
↪sep = ',', std::string na_rep = "NA", std::optional<std::string> case_label =␣
↪std::nullopt, std::optional<std::string> float_format = std::nullopt, char quotechar =
↪'"', bool scalarquote = true, std::string line_terminator = "\n", char decimal = '.',␣
↪std::string encoding = "utf-8") const
```

Parameters

`filepath` : `std::string_view` See the signature for the supported argument form.

`add_cases` : `bool` See the signature for the supported argument form.

`header` : `bool` See the signature for the supported argument form.

`sep` : `char` See the signature for the supported argument form.

`arg5` : `'` See the signature for the supported argument form.

`na_rep` : `std::string` See the signature for the supported argument form.

`case_label` : `std::optional<std::string>` See the signature for the supported argument form.

`float_format` : `std::optional<std::string>` See the signature for the supported argument form.

`quotechar` : `char` See the signature for the supported argument form.

`doublequote` : `bool` See the signature for the supported argument form.

`line_terminator` : `std::string` See the signature for the supported argument form.

`decimal` : `char` See the signature for the supported argument form.

`encoding` : `std::string` See the signature for the supported argument form.

Returns

`void` Performs the side effect and returns no value.

Notes

Writes the current object to disk using the corresponding PlussMic saver route.

See also

- `void save_csv(self, std::string_view filepath) const`
- `void save_csv(self, std::string_view filepath, const pluss::loader::WriteCsvOptions& options) const`
- `void save_csv(self, std::string_view filepath, const pluss::loader::Config& config) const`
- `void save_csv(self, std::string_view filepath, std::initializer_list<pluss::loader::ConfigEntry> config_entries) const`

Examples

```
auto df = pluss::table::DataFrame({
    {"id", pluss::table::Thread(pluss::make_series({1, 2, 3}))},
    {"sales", pluss::table::Thread(pluss::make_series({100.0, 120.0, 90.0}))},
    {"city", pluss::table::Thread(pluss::make_note({"Accra", "Tema", "Kumasi"}))},
```

(continues on next page)

(continued from previous page)

```
});

df.save_csv("out.csv");
```

Output

```
saved: out.*
```

5.1.190 save_csv

Signature

```
void save_csv(self, std::string_view filepath, const pluss::loader::Config& config) const
```

Parameters

`filepath` : `std::string_view` See the signature for the supported argument form.

`config` : `const pluss::loader::Config&` See the signature for the supported argument form.

Returns

`void` Performs the side effect and returns no value.

Notes

Writes the current object to disk using the corresponding PlussMic saver route.

See also

- `void save_csv(self, std::string_view filepath) const`
- `void save_csv(self, std::string_view filepath, bool add_cases, bool header = true, char sep = ',', std::string na_rep = "NA", std::optional<std::string> case_label = std::nullopt, std::optional<std::string> float_format = std::nullopt, char quotechar = '"', bool scalarquote = true, std::string line_terminator = "\n", char decimal = '.', std::string encoding = "utf-8") const`
- `void save_csv(self, std::string_view filepath, const pluss::loader::WriteCsvOptions& options) const`
- `void save_csv(self, std::string_view filepath, std::initializer_list<pluss::loader::ConfigEntry> config_entries) const`

Examples

```
auto df = pluss::table::DataFrame({
    {"id", pluss::table::Thread(pluss::make_series({1, 2, 3}))},
    {"sales", pluss::table::Thread(pluss::make_series({100.0, 120.0, 90.0}))},
    {"city", pluss::table::Thread(pluss::make_note({"Accra", "Tema", "Kumasi"}))},
});

df.save_csv("out.csv");
```

Output

```
saved: out.*
```

5.1.191 save_csv

Signature

```
void save_csv(self, std::string_view filepath, const pluss::loader::WriteCsvOptions&␣
↪options) const
```

Parameters

`filepath` : `std::string_view` See the signature for the supported argument form.

`options` : `const pluss::loader::WriteCsvOptions&` See the signature for the supported argument form.

Returns

`void` Performs the side effect and returns no value.

Notes

Writes the current object to disk using the corresponding PlussMic saver route.

See also

- `void save_csv(self, std::string_view filepath) const`
- `void save_csv(self, std::string_view filepath, bool add_cases, bool header = true, char sep = ',', std::string na_rep = "NA", std::optional<std::string> case_label = std::nullopt, std::optional<std::string> float_format = std::nullopt, char quotechar = '"', bool scalarquote = true, std::string line_terminator = "\n", char decimal = '.', std::string encoding = "utf-8") const`
- `void save_csv(self, std::string_view filepath, const pluss::loader::Config& config) const`
- `void save_csv(self, std::string_view filepath, std::initializer_list<pluss::loader::ConfigEntry> config_entries) const`

Examples

```
auto df = pluss::table::DataFrame({
    {"id", pluss::table::Thread(pluss::make_series({1, 2, 3}))},
    {"sales", pluss::table::Thread(pluss::make_series({100.0, 120.0, 90.0}))},
    {"city", pluss::table::Thread(pluss::make_note({"Accra", "Tema", "Kumasi"}))},
});

df.save_csv("out.csv");
```

Output

```
saved: out.*
```

5.1.192 save_csv

Signature

```
void save_csv(self, std::string_view filepath, std::initializer_list
↪<pluss::loader::ConfigEntry> config_entries) const
```

Parameters

`filepath` : `std::string_view` See the signature for the supported argument form.

`config_entries` : `std::initializer_list<pluss::loader::ConfigEntry>` See the signature for the supported argument form.

Returns

`void` Performs the side effect and returns no value.

Notes

Writes the current object to disk using the corresponding PlussMic saver route.

See also

- `void save_csv(self, std::string_view filepath) const`
- `void save_csv(self, std::string_view filepath, bool add_cases, bool header = true, char sep = ',', std::string na_rep = "NA", std::optional<std::string> case_label = std::nullopt, std::optional<std::string> float_format = std::nullopt, char quotechar = '"', bool scalarquote = true, std::string line_terminator = "\n", char decimal = '.', std::string encoding = "utf-8") const`
- `void save_csv(self, std::string_view filepath, const pluss::loader::WriteCsvOptions& options) const`
- `void save_csv(self, std::string_view filepath, const pluss::loader::Config& config) const`

Examples

```
auto df = pluss::table::DataFrame({
    {"id", pluss::table::Thread(pluss::make_series({1, 2, 3}))},
    {"sales", pluss::table::Thread(pluss::make_series({100.0, 120.0, 90.0}))},
    {"city", pluss::table::Thread(pluss::make_note({"Accra", "Tema", "Kumasi"}))},
});

df.save_csv("out.csv");
```

Output

```
saved: out.*
```

5.1.193 save_excel

Signature

```
void save_excel(self, std::string_view filepath) const
```

Parameters

`filepath` : `std::string_view` See the signature for the supported argument form.

Returns

`void` Performs the side effect and returns no value.

Notes

Writes the current object to disk using the corresponding PlussMic saver route.

See also

- `void save_excel(self, std::string_view filepath, std::string sheet_name, bool add_cases = false, bool header = true, std::string na_rep = "NA", std::optional<std::string> case_label = std::nullopt) const`
- `void save_excel(self, std::string_view filepath, const pluss::loader::WriteExcelOptions& options) const`
- `void save_excel(self, std::string_view filepath, const pluss::loader::Config& config) const`
- `void save_excel(self, std::string_view filepath, std::initializer_list<pluss::loader::ConfigEntry> config_entries) const`

Examples

```
auto df = pluss::table::DataFrame({
    {"id", pluss::table::Thread(pluss::make_series({1, 2, 3}))},
    {"sales", pluss::table::Thread(pluss::make_series({100.0, 120.0, 90.0}))},
    {"city", pluss::table::Thread(pluss::make_note({"Accra", "Tema", "Kumasi"}))},
});

df.save_excel("out.xlsx");
```

Output

```
saved: out.*
```

5.1.194 save_excel

Signature

```
void save_excel(self, std::string_view filepath, const pluss::loader::Config& config)␣
↪const
```

Parameters

`filepath` : `std::string_view` See the signature for the supported argument form.

`config` : `const pluss::loader::Config&` See the signature for the supported argument form.

Returns

`void` Performs the side effect and returns no value.

Notes

Writes the current object to disk using the corresponding PlussMic saver route.

See also

- `void save_excel(self, std::string_view filepath) const`
- `void save_excel(self, std::string_view filepath, std::string sheet_name, bool add_cases = false, bool header = true, std::string na_rep = "NA", std::optional<std::string> case_label = std::nullopt) const`
- `void save_excel(self, std::string_view filepath, const pluss::loader::WriteExcelOptions& options) const`
- `void save_excel(self, std::string_view filepath, std::initializer_list<pluss::loader::ConfigEntry> config_entries) const`

Examples

```
auto df = pluss::table::DataFrame({
    {"id", pluss::table::Thread(pluss::make_series({1, 2, 3}))},
    {"sales", pluss::table::Thread(pluss::make_series({100.0, 120.0, 90.0}))},
    {"city", pluss::table::Thread(pluss::make_note({"Accra", "Tema", "Kumasi"}))},
});

df.save_excel("out.xlsx");
```

Output

```
saved: out.*
```

5.1.195 save_excel

Signature

```
void save_excel(self, std::string_view filepath, const pluss::loader::WriteExcelOptions&␣
↪options) const
```

Parameters

`filepath` : `std::string_view` See the signature for the supported argument form.

`options` : `const pluss::loader::WriteExcelOptions&` See the signature for the supported argument form.

Returns

`void` Performs the side effect and returns no value.

Notes

Writes the current object to disk using the corresponding PlussMic saver route.

See also

- `void save_excel(self, std::string_view filepath) const`
- `void save_excel(self, std::string_view filepath, std::string sheet_name, bool add_cases = false, bool header = true, std::string na_rep = "NA", std::optional<std::string> case_label = std::nullopt) const`
- `void save_excel(self, std::string_view filepath, const pluss::loader::Config& config) const`
- `void save_excel(self, std::string_view filepath, std::initializer_list<pluss::loader::ConfigEntry> config_entries) const`

Examples

```
auto df = pluss::table::DataFrame({
    {"id", pluss::table::Thread(pluss::make_series({1, 2, 3}))},
    {"sales", pluss::table::Thread(pluss::make_series({100.0, 120.0, 90.0}))},
    {"city", pluss::table::Thread(pluss::make_note({"Accra", "Tema", "Kumasi"}))},
});

df.save_excel("out.xlsx");
```

Output

```
saved: out.*
```

5.1.196 save_excel

Signature

```
void save_excel(self, std::string_view filepath, std::initializer_list
↪<pluss::loader::ConfigEntry> config_entries) const
```

Parameters

`filepath` : `std::string_view` See the signature for the supported argument form.

`config_entries` : `std::initializer_list<pluss::loader::ConfigEntry>` See the signature for the supported argument form.

Returns

`void` Performs the side effect and returns no value.

Notes

Writes the current object to disk using the corresponding PlussMic saver route.

See also

- `void save_excel(self, std::string_view filepath) const`
- `void save_excel(self, std::string_view filepath, std::string sheet_name, bool add_cases = false, bool header = true, std::string na_rep = "NA", std::optional<std::string> case_label = std::nullopt) const`
- `void save_excel(self, std::string_view filepath, const pluss::loader::WriteExcelOptions& options) const`
- `void save_excel(self, std::string_view filepath, const pluss::loader::Config& config) const`

Examples

```
auto df = pluss::table::DataFrame({
    {"id", pluss::table::Thread(pluss::make_series({1, 2, 3}))},
    {"sales", pluss::table::Thread(pluss::make_series({100.0, 120.0, 90.0}))},
    {"city", pluss::table::Thread(pluss::make_note({"Accra", "Tema", "Kumasi"}))},
});

df.save_excel("out.xlsx");
```

Output

```
saved: out.*
```

5.1.197 save_excel

Signature

```
void save_excel(self, std::string_view filepath, std::string sheet_name, bool add_cases␣
↪= false, bool header = true, std::string na_rep = "NA", std::optional<std::string>␣
↪case_label = std::nullopt) const
```

Parameters

`filepath` : `std::string_view` See the signature for the supported argument form.

`sheet_name` : `std::string` See the signature for the supported argument form.

`add_cases` : `bool` See the signature for the supported argument form.

`header` : `bool` See the signature for the supported argument form.

`na_rep` : `std::string` See the signature for the supported argument form.

`case_label` : `std::optional<std::string>` See the signature for the supported argument form.

Returns

`void` Performs the side effect and returns no value.

Notes

Writes the current object to disk using the corresponding PlussMic saver route.

See also

- `void save_excel(self, std::string_view filepath) const`
- `void save_excel(self, std::string_view filepath, const pluss::loader::WriteExcelOptions& options) const`
- `void save_excel(self, std::string_view filepath, const pluss::loader::Config& config) const`
- `void save_excel(self, std::string_view filepath, std::initializer_list<pluss::loader::ConfigEntry> config_entries) const`

Examples

```
auto df = pluss::table::DataFrame({
    {"id", pluss::table::Thread(pluss::make_series({1, 2, 3}))},
    {"sales", pluss::table::Thread(pluss::make_series({100.0, 120.0, 90.0}))},
    {"city", pluss::table::Thread(pluss::make_note({"Accra", "Tema", "Kumasi"}))},
});

df.save_excel("out.xlsx");
```

Output

```
saved: out.*
```

5.1.198 save_json

Signature

```
void save_json(self, std::string_view filepath) const
```

Parameters

`filepath` : `std::string_view` See the signature for the supported argument form.

Returns

`void` Performs the side effect and returns no value.

Notes

Writes the current object to disk using the corresponding PlussMic saver route.

See also

- void save_json(self, std::string_view filepath, const pluss::loader::Config& config) const
- void save_json(self, std::string_view filepath, std::initializer_list<pluss::loader::ConfigEntry> config_entries) const

Examples

```
auto df = pluss::table::DataFrame({
    {"id", pluss::table::Thread(pluss::make_series({1, 2, 3}))},
    {"sales", pluss::table::Thread(pluss::make_series({100.0, 120.0, 90.0}))},
    {"city", pluss::table::Thread(pluss::make_note({"Accra", "Tema", "Kumasi"}))},
});

df.save_json("out.json");
```

Output

```
saved: out.*
```

5.1.199 save_json

Signature

```
void save_json(self, std::string_view filepath, const pluss::loader::Config& config)␣
↪const
```

Parameters

`filepath` : `std::string_view` See the signature for the supported argument form.

`config` : `const pluss::loader::Config&` See the signature for the supported argument form.

Returns

`void` Performs the side effect and returns no value.

Notes

Writes the current object to disk using the corresponding PlussMic saver route.

See also

- void save_json(self, std::string_view filepath) const
- void save_json(self, std::string_view filepath, std::initializer_list<pluss::loader::ConfigEntry> config_entries) const

Examples

```
auto df = pluss::table::DataFrame({
    {"id", pluss::table::Thread(pluss::make_series({1, 2, 3}))},
    {"sales", pluss::table::Thread(pluss::make_series({100.0, 120.0, 90.0}))},
```

(continues on next page)

(continued from previous page)

```
    {"city", pluss::table::Thread(pluss::make_note({"Accra", "Tema", "Kumasi"}))},
});

df.save_json("out.json");
```

Output

```
saved: out.*
```

5.1.200 save_json

Signature

```
void save_json(self, std::string_view filepath, std::initializer_list
↪<pluss::loader::ConfigEntry> config_entries) const
```

Parameters

`filepath` : `std::string_view` See the signature for the supported argument form.

`config_entries` : `std::initializer_list<pluss::loader::ConfigEntry>` See the signature for the supported argument form.

Returns

`void` Performs the side effect and returns no value.

Notes

Writes the current object to disk using the corresponding PlussMic saver route.

See also

- `void save_json(self, std::string_view filepath) const`
- `void save_json(self, std::string_view filepath, const pluss::loader::Config& config) const`

Examples

```
auto df = pluss::table::DataFrame({
    {"id", pluss::table::Thread(pluss::make_series({1, 2, 3}))},
    {"sales", pluss::table::Thread(pluss::make_series({100.0, 120.0, 90.0}))},
    {"city", pluss::table::Thread(pluss::make_note({"Accra", "Tema", "Kumasi"}))},
});

df.save_json("out.json");
```

Output

```
saved: out.*
```

5.1.201 save_parquet

Signature

```
void save_parquet(self, std::string_view filepath) const
```

Parameters

`filepath` : `std::string_view` See the signature for the supported argument form.

Returns

`void` Performs the side effect and returns no value.

Notes

Writes the current object to disk using the corresponding PlussMic saver route.

See also

- `void save_parquet(self, std::string_view filepath, const pluss::loader::Config& config) const`
- `void save_parquet(self, std::string_view filepath, std::initializer_list<pluss::loader::ConfigEntry config_entries) const`

Examples

```
auto df = pluss::table::DataFrame({
    {"id", pluss::table::Thread(pluss::make_series({1, 2, 3}))},
    {"sales", pluss::table::Thread(pluss::make_series({100.0, 120.0, 90.0}))},
    {"city", pluss::table::Thread(pluss::make_note({"Accra", "Tema", "Kumasi"}))},
});

df.save_parquet("out.parquet");
```

Output

```
saved: out.*
```

5.1.202 save_parquet

Signature

```
void save_parquet(self, std::string_view filepath, const pluss::loader::Config& config)␣
↪const
```

Parameters

`filepath` : `std::string_view` See the signature for the supported argument form.

`config` : `const pluss::loader::Config&` See the signature for the supported argument form.

Returns

`void` Performs the side effect and returns no value.

Notes

Writes the current object to disk using the corresponding PlussMic saver route.

See also

- `void save_parquet(self, std::string_view filepath) const`
- `void save_parquet(self, std::string_view filepath, std::initializer_list<pluss::loader::ConfigEntry config_entries) const`

Examples

```cpp
auto df = pluss::table::DataFrame({
    {"id", pluss::table::Thread(pluss::make_series({1, 2, 3}))},
    {"sales", pluss::table::Thread(pluss::make_series({100.0, 120.0, 90.0}))},
    {"city", pluss::table::Thread(pluss::make_note({"Accra", "Tema", "Kumasi"}))},
});

df.save_parquet("out.parquet");
```

Output

```
saved: out.*
```

5.1.203 save_parquet

Signature

```cpp
void save_parquet(self, std::string_view filepath, std::initializer_list
↪<pluss::loader::ConfigEntry> config_entries) const
```

Parameters

`filepath` : `std::string_view` See the signature for the supported argument form.

`config_entries` : `std::initializer_list<pluss::loader::ConfigEntry>` See the signature for the supported argument form.

Returns

`void` Performs the side effect and returns no value.

Notes

Writes the current object to disk using the corresponding PlussMic saver route.

See also

- void save_parquet(self, std::string_view filepath) const
- void save_parquet(self, std::string_view filepath, const pluss::loader::Config& config) const

Examples

```
auto df = pluss::table::DataFrame({
    {"id", pluss::table::Thread(pluss::make_series({1, 2, 3}))},
    {"sales", pluss::table::Thread(pluss::make_series({100.0, 120.0, 90.0}))},
    {"city", pluss::table::Thread(pluss::make_note({"Accra", "Tema", "Kumasi"}))},
});

df.save_parquet("out.parquet");
```

Output

```
saved: out.*
```

5.1.204 save_txt

Signature

```
void save_txt(self, std::string_view filepath) const
```

Parameters

`filepath` : `std::string_view` See the signature for the supported argument form.

Returns

`void` Performs the side effect and returns no value.

Notes

Writes the current object to disk using the corresponding PlussMic saver route.

See also

- void save_txt(self, std::string_view filepath, const pluss::loader::Config& config) const
- void save_txt(self, std::string_view filepath, std::initializer_list<pluss::loader::ConfigEntry> config_entries) const

Examples

```
auto df = pluss::table::DataFrame({
    {"id", pluss::table::Thread(pluss::make_series({1, 2, 3}))},
    {"sales", pluss::table::Thread(pluss::make_series({100.0, 120.0, 90.0}))},
    {"city", pluss::table::Thread(pluss::make_note({"Accra", "Tema", "Kumasi"}))},
});
```

(continues on next page)

(continued from previous page)

```
df.save_txt("out.txt");
```

Output

```
saved: out.*
```

5.1.205 save_txt

Signature

```
void save_txt(self, std::string_view filepath, const pluss::loader::Config& config) const
```

Parameters

`filepath` : `std::string_view` See the signature for the supported argument form.

`config` : `const pluss::loader::Config&` See the signature for the supported argument form.

Returns

`void` Performs the side effect and returns no value.

Notes

Writes the current object to disk using the corresponding PlussMic saver route.

See also

- `void save_txt(self, std::string_view filepath) const`
- `void save_txt(self, std::string_view filepath, std::initializer_list<pluss::loader::ConfigEntry> config_entries) const`

Examples

```
auto df = pluss::table::DataFrame({
    {"id", pluss::table::Thread(pluss::make_series({1, 2, 3}))},
    {"sales", pluss::table::Thread(pluss::make_series({100.0, 120.0, 90.0}))},
    {"city", pluss::table::Thread(pluss::make_note({"Accra", "Tema", "Kumasi"}))},
});

df.save_txt("out.txt");
```

Output

```
saved: out.*
```

5.1.206 save_txt

Signature

```
void save_txt(self, std::string_view filepath, std::initializer_list
↪<pluss::loader::ConfigEntry> config_entries) const
```

Parameters

`filepath` : `std::string_view` See the signature for the supported argument form.

`config_entries` : `std::initializer_list<pluss::loader::ConfigEntry>` See the signature for the supported argument form.

Returns

`void` Performs the side effect and returns no value.

Notes

Writes the current object to disk using the corresponding PlussMic saver route.

See also

- `void save_txt(self, std::string_view filepath) const`
- `void save_txt(self, std::string_view filepath, const pluss::loader::Config& config) const`

Examples

```
auto df = pluss::table::DataFrame({
    {"id", pluss::table::Thread(pluss::make_series({1, 2, 3}))},
    {"sales", pluss::table::Thread(pluss::make_series({100.0, 120.0, 90.0}))},
    {"city", pluss::table::Thread(pluss::make_note({"Accra", "Tema", "Kumasi"}))},
});

df.save_txt("out.txt");
```

Output

```
saved: out.*
```

5.1.207 series_features

Signature

```
DataFrame series_features(self) const
```

Parameters

None

Returns

`DataFrame` Value produced by this call.

Notes

This call is part of the public `DataFrame` API and is documented here using a concrete usage example.

Examples

```
auto df = pluss::table::DataFrame({
    {"id", pluss::table::Thread(pluss::make_series({1, 2, 3}))},
    {"sales", pluss::table::Thread(pluss::make_series({100.0, 120.0, 90.0}))},
    {"city", pluss::table::Thread(pluss::make_note({"Accra", "Tema", "Kumasi"}))},
});

auto out = df.series_features();
```

Output

```
id  sales  city
1   100.0  Accra
2   120.0  Tema
3    90.0  Kumasi
```

5.1.208 `set`

Signature

```
LetSlot set(self, std::string_view name)
```

Parameters

`name` : `std::string_view` See the signature for the supported argument form.

Returns

`LetSlot` Value produced by this call.

Notes

This call is part of the public `DataFrame` API and is documented here using a concrete usage example.

See also

- `LetSlots set(self, std::initializer_list<std::string_view> names)`

Examples

```
auto df = pluss::table::DataFrame({
    {"id", pluss::table::Thread(pluss::make_series({1, 2, 3}))},
    {"sales", pluss::table::Thread(pluss::make_series({100.0, 120.0, 90.0}))},
    {"city", pluss::table::Thread(pluss::make_note({"Accra", "Tema", "Kumasi"}))},
});
```

(continues on next page)

(continued from previous page)

```
df.set("profit") = pluss::table::Thread(pluss::make_series({40.0, 35.0, 30.0}));
```

5.1.209 set

Signature

```
LetSlots set(self, std::initializer_list<std::string_view> names)
```

Parameters

names : std::initializer_list<std::string_view> See the signature for the supported argument form.

Returns

LetSlots Value produced by this call.

Notes

This call is part of the public DataFrame API and is documented here using a concrete usage example.

See also

- LetSlot set(self, std::string_view name)

Examples

```
auto df = pluss::table::DataFrame({
    {"id", pluss::table::Thread(pluss::make_series({1, 2, 3}))},
    {"sales", pluss::table::Thread(pluss::make_series({100.0, 120.0, 90.0}))},
    {"city", pluss::table::Thread(pluss::make_note({"Accra", "Tema", "Kumasi"}))},
});

df.set({"profit", "cost"}) = {pluss::table::Thread(pluss::make_series({40.0, 35.0, 30.0}
↪)), pluss::table::Thread(pluss::make_series({60.0, 85.0, 60.0}))};
```

5.1.210 set_axis

Signature

```
DataFrame set_axis(self, std::initializer_list<std::string_view> feature_names) const
```

Parameters

feature_names : std::initializer_list<std::string_view> See the signature for the supported argument form.

Returns

DataFrame Value produced by this call.

Notes

This adjusts labels without changing the underlying payload values.

Examples

```
auto df = pluss::table::DataFrame({
    {"id", pluss::table::Thread(pluss::make_series({1, 2, 3}))},
    {"sales", pluss::table::Thread(pluss::make_series({100.0, 120.0, 90.0}))},
    {"city", pluss::table::Thread(pluss::make_note({"Accra", "Tema", "Kumasi"}))},
});

auto out = df.set_axis({"revenue", "location", "id_new"});
```

Output

```
id  revenue  city
1   100.0    Accra
2   120.0    Tema
3    90.0    Kumasi
```

5.1.211 set_cases

Signature

```
DataFrame set_cases(self, std::string_view feature_name, bool drop_feature = true) const
```

Parameters

`feature_name` : `std::string_view` See the signature for the supported argument form.

`drop_feature` : `bool` See the signature for the supported argument form.

Returns

`DataFrame` Value produced by this call.

Notes

This call is part of the public `DataFrame` API and is documented here using a concrete usage example.

Examples

```
auto df = pluss::table::DataFrame({
    {"id", pluss::table::Thread(pluss::make_series({1, 2, 3}))},
    {"sales", pluss::table::Thread(pluss::make_series({100.0, 120.0, 90.0}))},
    {"city", pluss::table::Thread(pluss::make_note({"Accra", "Tema", "Kumasi"}))},
});

auto out = df.set_cases("city", false);
```

Output

```
id  sales  city
1   100.0  Accra
2   120.0  Tema
3    90.0  Kumasi
```

5.1.212 set_feature_axis

Signature

```
DataFrame set_feature_axis(self, std::initializer_list<std::string_view> feature_names)␣
↪const
```

Parameters

`feature_names` : `std::initializer_list<std::string_view>` See the signature for the supported argument form.

Returns

`DataFrame` Value produced by this call.

Notes

This adjusts labels without changing the underlying payload values.

Examples

```
auto df = pluss::table::DataFrame({
    {"id", pluss::table::Thread(pluss::make_series({1, 2, 3}))},
    {"sales", pluss::table::Thread(pluss::make_series({100.0, 120.0, 90.0}))},
    {"city", pluss::table::Thread(pluss::make_note({"Accra", "Tema", "Kumasi"}))},
});

auto out = df.set_feature_axis({"revenue", "location", "id_new"});
```

Output

```
id  revenue  city
1   100.0    Accra
2   120.0    Tema
3    90.0    Kumasi
```

5.1.213 set_features

Signature

```
DataFrame set_features(self, std::initializer_list<std::string_view> names,␣
↪std::initializer_list<Thread> values) const
```

Parameters

names : std::initializer_list<std::string_view> See the signature for the supported argument form.

values : std::initializer_list<Thread> See the signature for the supported argument form.

Returns

DataFrame Value produced by this call.

Notes

This mutates or derives structure while keeping the surrounding object model intact.

See also

- DataFrame set_features(self, std::string_view name, const Thread& value) const

Examples

```
auto df = pluss::table::DataFrame({
    {"id", pluss::table::Thread(pluss::make_series({1, 2, 3}))},
    {"sales", pluss::table::Thread(pluss::make_series({100.0, 120.0, 90.0}))},
    {"city", pluss::table::Thread(pluss::make_note({"Accra", "Tema", "Kumasi"}))},
});

auto out = df.set_features(
    {"profit", "cost"},
    {
        pluss::table::Thread(pluss::make_series({40.0, 35.0, 30.0})),
        pluss::table::Thread(pluss::make_series({60.0, 85.0, 60.0})),
    });
```

Output

```
id  sales  city    profit
1   100.0  Accra   40.0
2   120.0  Tema    35.0
3    90.0  Kumasi  30.0
```

5.1.214 set_features

Signature

```
DataFrame set_features(self, std::string_view name, const Thread& value) const
```

Parameters

name : std::string_view See the signature for the supported argument form.

value : const Thread& See the signature for the supported argument form.

Returns

DataFrame Value produced by this call.

Notes

This mutates or derives structure while keeping the surrounding object model intact.

See also

- DataFrame set_features(self, std::initializer_list<std::string_view> names, std::initializer_list<Thread> values) const

Examples

```
auto df = pluss::table::DataFrame({
    {"id", pluss::table::Thread(pluss::make_series({1, 2, 3}))},
    {"sales", pluss::table::Thread(pluss::make_series({100.0, 120.0, 90.0}))},
    {"city", pluss::table::Thread(pluss::make_note({"Accra", "Tema", "Kumasi"}))},
});

auto out = df.set_features("profit", pluss::table::Thread(pluss::make_series({40.0, 35.0,
↪ 30.0})));
```

Output

```
id  sales  city    profit
1   100.0  Accra   40.0
2   120.0  Tema    35.0
3    90.0  Kumasi  30.0
```

5.1.215 set_features_

Signature

```
DataFrame& set_features_(self, std::initializer_list<std::string_view> names,␣
↪std::initializer_list<Thread> values)
```

Parameters

names : std::initializer_list<std::string_view> See the signature for the supported argument form.

values : std::initializer_list<Thread> See the signature for the supported argument form.

Returns

DataFrame& Value produced by this call.

Notes

This mutates or derives structure while keeping the surrounding object model intact.

See also

- DataFrame& set_features_(self, std::string_view name, const Thread& value)

Examples

```
auto df = pluss::table::DataFrame({
    {"id", pluss::table::Thread(pluss::make_series({1, 2, 3}))},
    {"sales", pluss::table::Thread(pluss::make_series({100.0, 120.0, 90.0}))},
    {"city", pluss::table::Thread(pluss::make_note({"Accra", "Tema", "Kumasi"}))},
});

df.set_features_(
    {"profit", "cost"},
    {
        pluss::table::Thread(pluss::make_series({40.0, 35.0, 30.0})),
        pluss::table::Thread(pluss::make_series({60.0, 85.0, 60.0})),
    });
```

Output

```
id  sales  city    profit
1   100.0  Accra   40.0
2   120.0  Tema    35.0
3    90.0  Kumasi  30.0
```

5.1.216 set_features_

Signature

```
DataFrame& set_features_(self, std::string_view name, const Thread& value)
```

Parameters

name : std::string_view See the signature for the supported argument form.

value : const Thread& See the signature for the supported argument form.

Returns

DataFrame& Value produced by this call.

Notes

This mutates or derives structure while keeping the surrounding object model intact.

See also

- DataFrame& set_features_(self, std::initializer_list<std::string_view> names, std::initializer_list<Thread> values)

Examples

```
auto df = pluss::table::DataFrame({
    {"id", pluss::table::Thread(pluss::make_series({1, 2, 3}))},
    {"sales", pluss::table::Thread(pluss::make_series({100.0, 120.0, 90.0}))},
    {"city", pluss::table::Thread(pluss::make_note({"Accra", "Tema", "Kumasi"}))},
});

df.set_features_("profit", pluss::table::Thread(pluss::make_series({40.0, 35.0, 30.0})));
```

Output

```
id  sales  city    profit
1   100.0  Accra   40.0
2   120.0  Tema    35.0
3    90.0  Kumasi  30.0
```

5.1.217 shift_time

Signature

```
DataFrame shift_time(self, std::initializer_list<std::string_view> features, int periods,
↪ std::string_view freq) const
```

Parameters

`features` : `std::initializer_list<std::string_view>` See the signature for the supported argument form.

`periods` : `int` See the signature for the supported argument form.

`freq` : `std::string_view` See the signature for the supported argument form.

Returns

`DataFrame` Value produced by this call.

Notes

This is part of the time-aware surface and expects timeline or time-index semantics.

Examples

```
auto df = pluss::table::DataFrame({
    {"ts", pluss::table::Thread(pluss::make_timeline({
        "2024-01-01T09:10:00Z",
        "2024-01-01T09:40:00Z",
        "2024-01-01T10:05:00Z",
    }))},
    {"load", pluss::table::Thread(pluss::make_series({10.0, 20.0, 15.0}))},
});

auto out = df.shift_time({"ts"}, 1, "hour");
```

Output

```
ts
2024-01-01T09:00:00Z
2024-01-01T10:00:00Z
```

5.1.218 slice

Signature

```
DataFrame slice(self, std::int64_t start, std::int64_t stop) const
```

Parameters

`start` : `std::int64_t` See the signature for the supported argument form.

`stop` : `std::int64_t` See the signature for the supported argument form.

Returns

`DataFrame` Value produced by this call.

Notes

Use this to narrow the current object to the rows or depth range you want to inspect next.

Examples

```
auto df = pluss::table::DataFrame({
    {"id", pluss::table::Thread(pluss::make_series({1, 2, 3}))},
    {"sales", pluss::table::Thread(pluss::make_series({100.0, 120.0, 90.0}))},
    {"city", pluss::table::Thread(pluss::make_note({"Accra", "Tema", "Kumasi"}))},
});

auto out = df.slice(0, 2);
```

Output

```
id  sales  city
1   100.0  Accra
2   120.0  Tema
3    90.0  Kumasi
```

5.1.219 sort_cases

Signature

```
DataFrame sort_cases(self, bool ascending = true, bool nulls_last = true) const
```

Parameters

ascending : bool See the signature for the supported argument form.

nulls_last : bool See the signature for the supported argument form.

Returns

DataFrame Value produced by this call.

Notes

This call is part of the public DataFrame API and is documented here using a concrete usage example.

Examples

```
auto df = pluss::table::DataFrame({
    {"id", pluss::table::Thread(pluss::make_series({1, 2, 3}))},
    {"sales", pluss::table::Thread(pluss::make_series({100.0, 120.0, 90.0}))},
    {"city", pluss::table::Thread(pluss::make_note({"Accra", "Tema", "Kumasi"}))},
});

auto with_cases = df.with_cases(pluss::table::Thread(pluss::make_note({"cs2", "cs0", "cs1
↪"})));
auto out = with_cases.sort_cases(true, true);
```

Output

```
id  sales  city
1   100.0  Accra
2   120.0  Tema
3    90.0  Kumasi
```

5.1.220 sort_values

Signature

```
DataFrame sort_values(self, std::initializer_list<ArrangeSpec> specs) const
```

Parameters

specs : std::initializer_list<ArrangeSpec> See the signature for the supported argument form.

Returns

DataFrame Value produced by this call.

Notes

This call is part of the public DataFrame API and is documented here using a concrete usage example.

See also

- DataFrame sort_values(self, std::initializer_list<std::string_view> features, bool ascending = true, bool nulls_last = true) const

Examples

```
auto df = pluss::table::DataFrame({
    {"id", pluss::table::Thread(pluss::make_series({1, 2, 3}))},
    {"sales", pluss::table::Thread(pluss::make_series({100.0, 120.0, 90.0}))},
    {"city", pluss::table::Thread(pluss::make_note({"Accra", "Tema", "Kumasi"}))},
});

auto out = df.sort_values({{.feature = "sales", .ascending = false}});
```

Output

```
id  sales  city
1   100.0  Accra
2   120.0  Tema
3    90.0  Kumasi
```

5.1.221 sort_values

Signature

```
DataFrame sort_values(self, std::initializer_list<std::string_view> features, bool␣
↪ascending = true, bool nulls_last = true) const
```

Parameters

features : std::initializer_list<std::string_view> See the signature for the supported argument form.

ascending : bool See the signature for the supported argument form.

nulls_last : bool See the signature for the supported argument form.

Returns

DataFrame Value produced by this call.

Notes

This call is part of the public DataFrame API and is documented here using a concrete usage example.

See also

- DataFrame sort_values(self, std::initializer_list<ArrangeSpec> specs) const

Examples

```
auto df = pluss::table::DataFrame({
    {"id", pluss::table::Thread(pluss::make_series({1, 2, 3}))},
    {"sales", pluss::table::Thread(pluss::make_series({100.0, 120.0, 90.0}))},
    {"city", pluss::table::Thread(pluss::make_note({"Accra", "Tema", "Kumasi"}))},
```

(continues on next page)

(continued from previous page)

```
});

auto out = df.sort_values({"sales"}, false, true);
```

Output

```
id  sales  city
1   100.0  Accra
2   120.0  Tema
3    90.0  Kumasi
```

5.1.222 sql

Signature

```
DataFrameSqlAccessor sql(self) const
```

Parameters

None

Returns

`DataFrameSqlAccessor` Value produced by this call.

Notes

Use this when the transformation is easier to express as SQL than as a chained sequence of table verbs.

See also

- `DataFrameSqlAccessor sql(self, SqlOptions options) const`

Examples

```
auto df = pluss::table::DataFrame({
    {"id", pluss::table::Thread(pluss::make_series({1, 2, 3}))},
    {"sales", pluss::table::Thread(pluss::make_series({100.0, 120.0, 90.0}))},
    {"city", pluss::table::Thread(pluss::make_note({"Accra", "Tema", "Kumasi"}))},
});

auto out = df.sql().execute("SELECT id, sales FROM self WHERE sales > 95");
```

Output

```
SQL accessor / prepared statement created
```

5.1.223 sql

Signature

```
DataFrameSqlAccessor sql(self, SqlOptions options) const
```

Parameters

`options` : `SqlOptions` See the signature for the supported argument form.

Returns

`DataFrameSqlAccessor` Value produced by this call.

Notes

Use this when the transformation is easier to express as SQL than as a chained sequence of table verbs.

See also

- `DataFrameSqlAccessor sql(self) const`

Examples

```
auto df = pluss::table::DataFrame({
    {"id", pluss::table::Thread(pluss::make_series({1, 2, 3}))},
    {"sales", pluss::table::Thread(pluss::make_series({100.0, 120.0, 90.0}))},
    {"city", pluss::table::Thread(pluss::make_note({"Accra", "Tema", "Kumasi"}))},
});

auto out = df.sql().execute("SELECT id, sales FROM self WHERE sales > 95");
```

Output

```
SQL accessor / prepared statement created
```

5.1.224 `std`

Signature

```
DataFrame std(self, bool render = true) const
```

Parameters

`render` : `bool` See the signature for the supported argument form.

Returns

`DataFrame` Value produced by this call.

Notes

Use this when you want a summary view instead of carrying the full input forward unchanged.

Examples

```
auto df = pluss::table::DataFrame({
    {"id", pluss::table::Thread(pluss::make_series({1, 2, 3}))},
    {"sales", pluss::table::Thread(pluss::make_series({100.0, 120.0, 90.0}))},
    {"city", pluss::table::Thread(pluss::make_note({"Accra", "Tema", "Kumasi"}))},
});

auto out = df.std(false);
```

Output

```
feature  value
sales    310.0
```

5.1.225 stitch_on

Signature

```
DataFrame stitch_on(self, std::initializer_list<std::string_view> left_features, const␣
↪DataFrame& right, std::initializer_list<std::string_view> right_features =
```

Parameters

`left_features` : `std::initializer_list<std::string_view>` See the signature for the supported argument form.

`right` : `const DataFrame&` See the signature for the supported argument form.

`right_features` : `std::initializer_list<std::string_view>` See the signature for the supported argument form.

Returns

`DataFrame` Value produced by this call.

Notes

Use this when you need to merge compatible objects or extend the current one with another input.

See also

- `DataFrame stitch_on(self, std::string_view left_feature, const DataFrame& right, std::string_view right_feature = "", std::string right_suffix = "_right") const`

Examples

```
auto left = pluss::table::DataFrame({
    {"id", pluss::table::Thread(pluss::make_series({1, 2, 3}))},
    {"sales", pluss::table::Thread(pluss::make_series({100.0, 120.0, 90.0}))},
});
auto right = pluss::table::DataFrame({
    {"id", pluss::table::Thread(pluss::make_series({1, 2, 3}))},
    {"cost", pluss::table::Thread(pluss::make_series({60.0, 80.0, 55.0}))},
```

(continues on next page)

(continued from previous page)

```
});

auto out = left.stitch_on({"id"}, right, {"id"}, "_right");
```

Output

```
id  sales  cost
1   100.0  60.0
2   120.0  80.0
3    90.0  55.0
```

5.1.226 stitch_on

Signature

```
DataFrame stitch_on(self, std::string_view left_feature, const DataFrame& right,␣
↪std::string_view right_feature = "", std::string right_suffix = "_right") const
```

Parameters

`left_feature` : `std::string_view` See the signature for the supported argument form.

`right` : `const DataFrame&` See the signature for the supported argument form.

`right_feature` : `std::string_view` See the signature for the supported argument form.

`right_suffix` : `std::string` See the signature for the supported argument form.

Returns

`DataFrame` Value produced by this call.

Notes

Use this when you need to merge compatible objects or extend the current one with another input.

See also

- `DataFrame stitch_on(self, std::initializer_list<std::string_view> left_features, const DataFrame& right, std::initializer_list<std::string_view> right_features =`

Examples

```
auto left = pluss::table::DataFrame({
    {"id", pluss::table::Thread(pluss::make_series({1, 2, 3}))},
    {"sales", pluss::table::Thread(pluss::make_series({100.0, 120.0, 90.0}))},
});
auto right = pluss::table::DataFrame({
    {"id", pluss::table::Thread(pluss::make_series({1, 2, 3}))},
    {"cost", pluss::table::Thread(pluss::make_series({60.0, 80.0, 55.0}))},
});

auto out = left.stitch_on("id", right, "id", "_right");
```

Output

```
id  sales  cost
1   100.0  60.0
2   120.0  80.0
3    90.0  55.0
```

5.1.227 sub

Signature

```
DataFrame sub(self, const DataFrame& other) const
```

Parameters

`other` : `const DataFrame&` See the signature for the supported argument form.

Returns

`DataFrame` Value produced by this call.

Notes

This applies arithmetic across the compatible numeric payloads in the current object.

See also

- `DataFrame sub(self, scalar other) const`

Examples

```
auto left = pluss::table::DataFrame({
    {"id", pluss::table::Thread(pluss::make_series({1, 2, 3}))},
    {"sales", pluss::table::Thread(pluss::make_series({100.0, 120.0, 90.0}))},
});
auto right = pluss::table::DataFrame({
    {"id", pluss::table::Thread(pluss::make_series({1, 2, 3}))},
    {"cost", pluss::table::Thread(pluss::make_series({60.0, 80.0, 55.0}))},
});

auto out = left.sub(right);
```

Output

```
id  sales  city
1   100.0  Accra
2   120.0  Tema
3    90.0  Kumasi
```

5.1.228 sub

Signature

```
DataFrame sub(self, scalar other) const
```

Parameters

`other` : `scalar` See the signature for the supported argument form.

Returns

`DataFrame` Value produced by this call.

Notes

This applies arithmetic across the compatible numeric payloads in the current object.

See also

- `DataFrame sub(self, const DataFrame& other) const`

Examples

```
auto df = pluss::table::DataFrame({
    {"id", pluss::table::Thread(pluss::make_series({1, 2, 3}))},
    {"sales", pluss::table::Thread(pluss::make_series({100.0, 120.0, 90.0}))},
    {"city", pluss::table::Thread(pluss::make_note({"Accra", "Tema", "Kumasi"}))},
});

auto out = df.sub(10);
```

Output

```
id  sales  city
11  110.0  Accra
12  130.0  Tema
13  100.0  Kumasi
```

5.1.229 `subtract`

Signature

```
DataFrame subtract(self, const DataFrame& other) const
```

Parameters

`other` : `const DataFrame&` See the signature for the supported argument form.

Returns

`DataFrame` Value produced by this call.

Notes

This applies arithmetic across the compatible numeric payloads in the current object.

See also

- `DataFrame subtract(self, scalar other) const`

Examples

```cpp
auto left = pluss::table::DataFrame({
    {"id", pluss::table::Thread(pluss::make_series({1, 2, 3}))},
    {"sales", pluss::table::Thread(pluss::make_series({100.0, 120.0, 90.0}))},
});
auto right = pluss::table::DataFrame({
    {"id", pluss::table::Thread(pluss::make_series({1, 2, 3}))},
    {"cost", pluss::table::Thread(pluss::make_series({60.0, 80.0, 55.0}))},
});

auto out = left.subtract(right);
```

Output

```
id  sales  city
1   100.0  Accra
2   120.0  Tema
3    90.0  Kumasi
```

5.1.230 `subtract`

Signature

```cpp
DataFrame subtract(self, scalar other) const
```

Parameters

`other` : `scalar` See the signature for the supported argument form.

Returns

`DataFrame` Value produced by this call.

Notes

This applies arithmetic across the compatible numeric payloads in the current object.

See also

- `DataFrame subtract(self, const DataFrame& other) const`

Examples

```cpp
auto df = pluss::table::DataFrame({
    {"id", pluss::table::Thread(pluss::make_series({1, 2, 3}))},
    {"sales", pluss::table::Thread(pluss::make_series({100.0, 120.0, 90.0}))},
    {"city", pluss::table::Thread(pluss::make_note({"Accra", "Tema", "Kumasi"}))},
});

auto out = df.subtract(10);
```

Output

```
id  sales  city
11  110.0  Accra
12  130.0  Tema
13  100.0  Kumasi
```

5.1.231 sum

Signature

```cpp
DataFrame sum(self, bool render = true) const
```

Parameters

`render` : `bool` See the signature for the supported argument form.

Returns

`DataFrame` Value produced by this call.

Notes

Use this when you want a summary view instead of carrying the full input forward unchanged.

Examples

```cpp
auto df = pluss::table::DataFrame({
    {"id", pluss::table::Thread(pluss::make_series({1, 2, 3}))},
    {"sales", pluss::table::Thread(pluss::make_series({100.0, 120.0, 90.0}))},
    {"city", pluss::table::Thread(pluss::make_note({"Accra", "Tema", "Kumasi"}))},
});

auto out = df.sum(false);
```

Output

```
feature  value
sales    310.0
```

5.1.232 tabelize

Signature

```
static DataFrame tabelize(const pluss::kindle::Catalog& catalog)
```

Parameters

`catalog` : `const pluss::kindle::Catalog&` See the signature for the supported argument form.

Returns

`DataFrame` Value produced by this call.

Notes

This is an explicit bridge between PlussMic models so you can switch to the representation that fits the next task.

See also

- `static DataFrame tabelize(const pluss::kindle::Catalog& catalog, const pluss::kindle::TabelizePolicy& policy)`

Examples

```
using namespace pluss::kindle::literals;
auto catalog = pluss::kindle::Catalog({"sales"_k = pluss::kindle::Stream(pluss::make_
↪series({10.0, 20.0, 30.0}))});
auto out = pluss::table::DataFrame::tabelize(catalog);
```

Output

```
id  sales  city
1   100.0  Accra
2   120.0  Tema
3    90.0  Kumasi
```

5.1.233 tabelize

Signature

```
static DataFrame tabelize(const pluss::kindle::Catalog& catalog, const␣
↪pluss::kindle::TabelizePolicy& policy)
```

Parameters

`catalog` : `const pluss::kindle::Catalog&` See the signature for the supported argument form.

`policy` : `const pluss::kindle::TabelizePolicy&` See the signature for the supported argument form.

Returns

`DataFrame` Value produced by this call.

Notes

This is an explicit bridge between PlussMic models so you can switch to the representation that fits the next task.

See also

- `static DataFrame tabelize(const pluss::kindle::Catalog& catalog)`

Examples

```
using namespace pluss::kindle::literals;
auto catalog = pluss::kindle::Catalog({"sales"_k = pluss::kindle::Stream(pluss::make_
↪series({10.0, 20.0, 30.0}))});
auto out = pluss::table::DataFrame::tabelize(catalog);
```

Output

```
id  sales  city
1   100.0  Accra
2   120.0  Tema
3    90.0  Kumasi
```

5.1.234 `tail`

Signature

```
DataFrame tail(self, std::int64_t n = 5, bool render = true) const
```

Parameters

`n` : `std::int64_t` See the signature for the supported argument form.

`render` : `bool` See the signature for the supported argument form.

Returns

`DataFrame` Value produced by this call.

Notes

Use this to narrow the current object to the rows or depth range you want to inspect next.

Examples

```
auto df = pluss::table::DataFrame({
    {"id", pluss::table::Thread(pluss::make_series({1, 2, 3}))},
    {"sales", pluss::table::Thread(pluss::make_series({100.0, 120.0, 90.0}))},
    {"city", pluss::table::Thread(pluss::make_note({"Accra", "Tema", "Kumasi"}))},
});

auto out = df.tail(2, false);
```

Output

```
id  sales  city
1   100.0  Accra
2   120.0  Tema
3    90.0  Kumasi
```

5.1.235 take

Signature

```
DataFrame take(self, std::span<const std::int64_t> indices) const
```

Parameters

`indices` : `std::span<const std::int64_t>` See the signature for the supported argument form.

Returns

`DataFrame` Value produced by this call.

Notes

Use this to narrow the current object to the rows or depth range you want to inspect next.

Examples

```
auto df = pluss::table::DataFrame({
    {"id", pluss::table::Thread(pluss::make_series({1, 2, 3}))},
    {"sales", pluss::table::Thread(pluss::make_series({100.0, 120.0, 90.0}))},
    {"city", pluss::table::Thread(pluss::make_note({"Accra", "Tema", "Kumasi"}))},
});

std::vector<std::int64_t> picks{0, 2};
auto out = df.take(picks);
```

Output

```
id  sales  city
1   100.0  Accra
3    90.0  Kumasi
```

5.1.236 tally

Signature

```
DataFrame tally(self, std::string_view feature, std::int64_t top_n = 0, bool dropna =␣
↪true, bool render = true) const
```

Parameters

`feature` : `std::string_view` See the signature for the supported argument form.

`top_n` : `std::int64_t` See the signature for the supported argument form.

`dropna` : `bool` See the signature for the supported argument form.

`render` : `bool` See the signature for the supported argument form.

Returns

`DataFrame` Value produced by this call.

Notes

Use this when you want a summary view instead of carrying the full input forward unchanged.

Examples

```
auto df = pluss::table::DataFrame({
    {"city", pluss::table::Thread(pluss::make_note({"Accra", "Accra", "Tema", "Tema"}))},
    {"product", pluss::table::Thread(pluss::make_note({"A", "B", "A", "B"}))},
    {"sales", pluss::table::Thread(pluss::make_series({100.0, 120.0, 90.0, 150.0}))},
});

auto out = df.tally("city", 3, true, false);
```

Output

```
city    count
Accra   2
Tema    2
```

5.1.237 `timeline_features`

Signature

```
DataFrame timeline_features(self) const
```

Parameters

None

Returns

`DataFrame` Value produced by this call.

Notes

This call is part of the public `DataFrame` API and is documented here using a concrete usage example.

Examples

```
auto df = pluss::table::DataFrame({
    {"id", pluss::table::Thread(pluss::make_series({1, 2, 3}))},
    {"sales", pluss::table::Thread(pluss::make_series({100.0, 120.0, 90.0}))},
    {"city", pluss::table::Thread(pluss::make_note({"Accra", "Tema", "Kumasi"}))},
});

auto out = df.timeline_features();
```

Output

```
id  sales  city
1   100.0  Accra
2   120.0  Tema
3    90.0  Kumasi
```

5.1.238 to_string

Signature

```
std::string to_string(self, const display::DisplayOptions& options =
```

Parameters

`options` : `const display::DisplayOptions&` See the signature for the supported argument form.

Returns

`std::string` Rendered string result.

Notes

This call is part of the public `DataFrame` API and is documented here using a concrete usage example.

Examples

```
auto df = pluss::table::DataFrame({
    {"id", pluss::table::Thread(pluss::make_series({1, 2, 3}))},
    {"sales", pluss::table::Thread(pluss::make_series({100.0, 120.0, 90.0}))},
    {"city", pluss::table::Thread(pluss::make_note({"Accra", "Tema", "Kumasi"}))},
});

auto out = df.to_string();
```

Output

```
id  sales  city
1   100.0  Accra
2   120.0  Tema
3    90.0  Kumasi
```

5.1.239 to_utc

Signature

```
DataFrame to_utc(self, std::initializer_list<std::string_view> features) const
```

Parameters

`features` : `std::initializer_list<std::string_view>` See the signature for the supported argument form.

Returns

`DataFrame` Value produced by this call.

Notes

This is part of the time-aware surface and expects timeline or time-index semantics.

Examples

```
auto df = pluss::table::DataFrame({
    {"ts", pluss::table::Thread(pluss::make_timeline({
        "2024-01-01T09:10:00Z",
        "2024-01-01T09:40:00Z",
        "2024-01-01T10:05:00Z",
    }))},
    {"load", pluss::table::Thread(pluss::make_series({10.0, 20.0, 15.0}))},
});

auto out = df.to_utc({"ts"});
```

Output

```
ts
2024-01-01T09:00:00Z
2024-01-01T10:00:00Z
```

5.1.240 top

Signature

```
DataFrame top(self, std::string_view feature, std::int64_t n) const
```

Parameters

`feature` : `std::string_view` See the signature for the supported argument form.

`n` : `std::int64_t` See the signature for the supported argument form.

Returns

`DataFrame` Value produced by this call.

Notes

This call is part of the public `DataFrame` API and is documented here using a concrete usage example.

Examples

```
auto df = pluss::table::DataFrame({
    {"id", pluss::table::Thread(pluss::make_series({1, 2, 3}))},
    {"sales", pluss::table::Thread(pluss::make_series({100.0, 120.0, 90.0}))},
    {"city", pluss::table::Thread(pluss::make_note({"Accra", "Tema", "Kumasi"}))},
});

auto out = df.top("sales", 2);
```

Output

```
id  sales  city
1   100.0  Accra
2   120.0  Tema
3    90.0  Kumasi
```

5.1.241 `transform`

Signature

```
DataFrame transform(self, const std::function<Thread(const Thread&)>& fn) const
```

Parameters

`fn` : `const std::function<Thread(const Thread&)>&` See the signature for the supported argument form.

Returns

`DataFrame` Value produced by this call.

Notes

This mutates or derives structure while keeping the surrounding object model intact.

See also

- `DataFrame transform(self, std::string_view feature, const std::function<Thread(const Thread&)>& fn) const`
- `DataFrame transform(self, std::initializer_list<TransformSpec> specs) const`

Examples

```
auto df = pluss::table::DataFrame({
    {"id", pluss::table::Thread(pluss::make_series({1, 2, 3}))},
    {"sales", pluss::table::Thread(pluss::make_series({100.0, 120.0, 90.0}))},
    {"city", pluss::table::Thread(pluss::make_note({"Accra", "Tema", "Kumasi"}))},
});
```

(continues on next page)

(continued from previous page)

```
auto out = df.transform([](const pluss::table::Thread& thread) { return thread.kind() ==␣
↪pluss::table::ThreadKind::Series ? thread.clip(0.0, 110.0) : thread; });
```

Output

```
id  sales  city
1   100.0  Accra
2   120.0  Tema
3    90.0  Kumasi
```

5.1.242 `transform`

Signature

```
DataFrame transform(self, std::initializer_list<TransformSpec> specs) const
```

Parameters

`specs` : `std::initializer_list<TransformSpec>` See the signature for the supported argument form.

Returns

`DataFrame` Value produced by this call.

Notes

This mutates or derives structure while keeping the surrounding object model intact.

See also

- `DataFrame transform(self, const std::function<Thread(const Thread&)>& fn) const`
- `DataFrame transform(self, std::string_view feature, const std::function<Thread(const Thread&)>& fn) const`

Examples

```
auto df = pluss::table::DataFrame({
    {"id", pluss::table::Thread(pluss::make_series({1, 2, 3}))},
    {"sales", pluss::table::Thread(pluss::make_series({100.0, 120.0, 90.0}))},
    {"city", pluss::table::Thread(pluss::make_note({"Accra", "Tema", "Kumasi"}))},
});

auto out = df.transform({{.feature = "sales", .expr = [](const pluss::table::Thread&␣
↪thread) { return thread.clip(0.0, 110.0); }}});
```

Output

```
id  sales  city
1   100.0  Accra
2   120.0  Tema
3    90.0  Kumasi
```

5.1.243 transform

Signature

```
DataFrame transform(self, std::string_view feature, const std::function<Thread(const␣
↪Thread&)>& fn) const
```

Parameters

`feature` : `std::string_view` See the signature for the supported argument form.

`fn` : `const std::function<Thread(const Thread&)>&` See the signature for the supported argument form.

Returns

`DataFrame` Value produced by this call.

Notes

This mutates or derives structure while keeping the surrounding object model intact.

See also

- `DataFrame transform(self, const std::function<Thread(const Thread&)>& fn) const`
- `DataFrame transform(self, std::initializer_list<TransformSpec> specs) const`

Examples

```
auto df = pluss::table::DataFrame({
    {"id", pluss::table::Thread(pluss::make_series({1, 2, 3}))},
    {"sales", pluss::table::Thread(pluss::make_series({100.0, 120.0, 90.0}))},
    {"city", pluss::table::Thread(pluss::make_note({"Accra", "Tema", "Kumasi"}))},
});

auto out = df.transform("sales", [](const pluss::table::Thread& thread) { return thread.
↪clip(0.0, 110.0); });
```

Output

```
id  sales  city
1   100.0  Accra
2   120.0  Tema
3    90.0  Kumasi
```

5.1.244 transpose

Signature

```
DataFrame transpose(self, bool render = false) const
```

Parameters

`render` : `bool` See the signature for the supported argument form.

Returns

`DataFrame` Value produced by this call.

Notes

This call is part of the public `DataFrame` API and is documented here using a concrete usage example.

Examples

```
auto df = pluss::table::DataFrame({
    {"id", pluss::table::Thread(pluss::make_series({1, 2, 3}))},
    {"sales", pluss::table::Thread(pluss::make_series({100.0, 120.0, 90.0}))},
    {"city", pluss::table::Thread(pluss::make_note({"Accra", "Tema", "Kumasi"}))},
});

auto out = df.transpose(false);
```

Output

```
feature  row0   row1   row2
id       1      2      3
sales    100.0  120.0  90.0
```

5.1.245 `truediv`

Signature

```
DataFrame truediv(self, const DataFrame& other) const
```

Parameters

`other` : `const DataFrame&` See the signature for the supported argument form.

Returns

`DataFrame` Value produced by this call.

Notes

This applies arithmetic across the compatible numeric payloads in the current object.

See also

- `DataFrame truediv(self, scalar other) const`

Examples

```cpp
auto left = pluss::table::DataFrame({
    {"id", pluss::table::Thread(pluss::make_series({1, 2, 3}))},
    {"sales", pluss::table::Thread(pluss::make_series({100.0, 120.0, 90.0}))},
});
auto right = pluss::table::DataFrame({
    {"id", pluss::table::Thread(pluss::make_series({1, 2, 3}))},
    {"cost", pluss::table::Thread(pluss::make_series({60.0, 80.0, 55.0}))},
});

auto out = left.truediv(right);
```

Output

```
id  sales  city
1   100.0  Accra
2   120.0  Tema
3    90.0  Kumasi
```

5.1.246 truediv

Signature

```cpp
DataFrame truediv(self, scalar other) const
```

Parameters

`other` : `scalar` See the signature for the supported argument form.

Returns

`DataFrame` Value produced by this call.

Notes

This applies arithmetic across the compatible numeric payloads in the current object.

See also

- `DataFrame truediv(self, const DataFrame& other) const`

Examples

```cpp
auto df = pluss::table::DataFrame({
    {"id", pluss::table::Thread(pluss::make_series({1, 2, 3}))},
    {"sales", pluss::table::Thread(pluss::make_series({100.0, 120.0, 90.0}))},
    {"city", pluss::table::Thread(pluss::make_note({"Accra", "Tema", "Kumasi"}))},
});

auto out = df.truediv(10);
```

Output

```
id  sales  city
11  110.0  Accra
12  130.0  Tema
13  100.0  Kumasi
```

5.1.247 tz_convert

Signature

```
DataFrame tz_convert(self, std::initializer_list<std::string_view> features, std::string_
↪view tz_name) const
```

Parameters

`features` : `std::initializer_list<std::string_view>` See the signature for the supported argument form.

`tz_name` : `std::string_view` See the signature for the supported argument form.

Returns

`DataFrame` Value produced by this call.

Notes

This is part of the time-aware surface and expects timeline or time-index semantics.

Examples

```
auto df = pluss::table::DataFrame({
    {"ts", pluss::table::Thread(pluss::make_timeline({
        "2024-01-01T09:10:00Z",
        "2024-01-01T09:40:00Z",
        "2024-01-01T10:05:00Z",
    }))},
    {"load", pluss::table::Thread(pluss::make_series({10.0, 20.0, 15.0}))},
});

auto out = df.tz_convert({"ts"}, "America/Edmonton");
```

Output

```
ts
2024-01-01T09:00:00Z
2024-01-01T10:00:00Z
```

5.1.248 tz_localize

Signature

```
DataFrame tz_localize(self, std::initializer_list<std::string_view> features,␣
↪std::string_view tz_name, int fold = 0, std::string_view nonexistent = "raise") const
```

Parameters

`features` : `std::initializer_list<std::string_view>` See the signature for the supported argument form.

`tz_name` : `std::string_view` See the signature for the supported argument form.

`fold` : `int` See the signature for the supported argument form.

`nonexistent` : `std::string_view` See the signature for the supported argument form.

Returns

`DataFrame` Value produced by this call.

Notes

This is part of the time-aware surface and expects timeline or time-index semantics.

Examples

```
auto df = pluss::table::DataFrame({
    {"ts", pluss::table::Thread(pluss::make_timeline({
        "2024-01-01T09:10:00Z",
        "2024-01-01T09:40:00Z",
        "2024-01-01T10:05:00Z",
    }))},
    {"load", pluss::table::Thread(pluss::make_series({10.0, 20.0, 15.0}))},
});

auto out = df.tz_localize({"ts"}, "America/Edmonton", 0, "shift_forward");
```

Output

```
ts
2024-01-01T09:00:00Z
2024-01-01T10:00:00Z
```

5.1.249 `unique`

Signature

```
bool unique(self) const
```

Parameters

`None`

Returns

`bool` Boolean status or membership result.

Notes

This call is part of the public `DataFrame` API and is documented here using a concrete usage example.

Examples

```
auto df = pluss::table::DataFrame({
    {"id", pluss::table::Thread(pluss::make_series({1, 2, 3}))},
    {"sales", pluss::table::Thread(pluss::make_series({100.0, 120.0, 90.0}))},
    {"city", pluss::table::Thread(pluss::make_note({"Accra", "Tema", "Kumasi"}))},
});

auto out = df.unique();
```

Output

```
true
```

5.1.250 unique_cases

Signature

```
DataFrame unique_cases(self, std::initializer_list<std::string_view> features =
```

Parameters

`features` : `std::initializer_list<std::string_view>` See the signature for the supported argument form.

Returns

`DataFrame` Value produced by this call.

Notes

This call is part of the public `DataFrame` API and is documented here using a concrete usage example.

Examples

```
auto df = pluss::table::DataFrame({
    {"id", pluss::table::Thread(pluss::make_series({1, 2, 3}))},
    {"sales", pluss::table::Thread(pluss::make_series({100.0, 120.0, 90.0}))},
    {"city", pluss::table::Thread(pluss::make_note({"Accra", "Tema", "Kumasi"}))},
});

auto out = df.unique_cases();
```

Output

```
id  sales  city
1   100.0  Accra
2   120.0  Tema
3    90.0  Kumasi
```

5.1.251 unsqueeze

Signature

```
Atlas unsqueeze(self) const
```

Parameters

None

Returns

`Atlas` Value produced by this call.

Notes

This call is part of the public `DataFrame` API and is documented here using a concrete usage example.

See also

- `Atlas unsqueeze(self, const Thread& depth_labels) const`

Examples

```
auto df = pluss::table::DataFrame({
    {"id", pluss::table::Thread(pluss::make_series({1, 2, 3}))},
    {"sales", pluss::table::Thread(pluss::make_series({100.0, 120.0, 90.0}))},
    {"city", pluss::table::Thread(pluss::make_note({"Accra", "Tema", "Kumasi"}))},
});

auto out = df.unsqueeze();
```

Output

```
frame0:
sales
10.0
20.0

frame1:
sales
30.0
40.0
```

5.1.252 unsqueeze

Signature

```
Atlas unsqueeze(self, const Thread& depth_labels) const
```

Parameters

depth_labels : const Thread& See the signature for the supported argument form.

Returns

Atlas Value produced by this call.

Notes

This call is part of the public DataFrame API and is documented here using a concrete usage example.

See also

- Atlas unsqueeze(self) const

Examples

```
auto df = pluss::table::DataFrame({
    {"id", pluss::table::Thread(pluss::make_series({1, 2, 3}))},
    {"sales", pluss::table::Thread(pluss::make_series({100.0, 120.0, 90.0}))},
    {"city", pluss::table::Thread(pluss::make_note({"Accra", "Tema", "Kumasi"}))},
});

auto labels = pluss::table::Thread(pluss::make_note({"train"}));
auto out = df.unsqueeze(labels);
```

Output

```
frame0:
sales
10.0
20.0

frame1:
sales
30.0
40.0
```

5.1.253 update

Signature

```
DataFrame update(self, const DataFrame& other, bool overwrite = true) const
```

Parameters

other : const DataFrame& See the signature for the supported argument form.

overwrite : bool See the signature for the supported argument form.

Returns

DataFrame Value produced by this call.

Notes

Use this when you need to merge compatible objects or extend the current one with another input.

Examples

```
auto left = pluss::table::DataFrame({
    {"id", pluss::table::Thread(pluss::make_series({1, 2, 3}))},
    {"sales", pluss::table::Thread(pluss::make_series({100.0, 120.0, 90.0}))},
});
auto right = pluss::table::DataFrame({
    {"id", pluss::table::Thread(pluss::make_series({1, 2, 3}))},
    {"cost", pluss::table::Thread(pluss::make_series({60.0, 80.0, 55.0}))},
});

auto out = left.update(right);
```

Output

```
id  sales  cost
1   100.0  60.0
2   120.0  80.0
3    90.0  55.0
```

5.1.254 values

Signature

```
std::vector<Thread> values(self) const
```

Parameters

None

Returns

std::vector<Thread> Materialized collection of thread objects.

Notes

This call is part of the public DataFrame API and is documented here using a concrete usage example.

Examples

```
auto df = pluss::table::DataFrame({
    {"id", pluss::table::Thread(pluss::make_series({1, 2, 3}))},
    {"sales", pluss::table::Thread(pluss::make_series({100.0, 120.0, 90.0}))},
    {"city", pluss::table::Thread(pluss::make_note({"Accra", "Tema", "Kumasi"}))},
});
```

(continues on next page)

(continued from previous page)

```
auto out = df.values();
```

Output

```
[Thread(id), Thread(sales), Thread(city)]
```

5.1.255 var

Signature

```
DataFrame var(self, bool render = true) const
```

Parameters

`render` : `bool` See the signature for the supported argument form.

Returns

`DataFrame` Value produced by this call.

Notes

Use this when you want a summary view instead of carrying the full input forward unchanged.

Examples

```
auto df = pluss::table::DataFrame({
    {"id", pluss::table::Thread(pluss::make_series({1, 2, 3}))},
    {"sales", pluss::table::Thread(pluss::make_series({100.0, 120.0, 90.0}))},
    {"city", pluss::table::Thread(pluss::make_note({"Accra", "Tema", "Kumasi"}))},
});

auto out = df.var(false);
```

Output

```
feature  value
sales    310.0
```

5.1.256 view

Signature

```
DataFrame view(self, std::int64_t start, std::int64_t stop, bool render = true) const
```

Parameters

`start` : `std::int64_t` See the signature for the supported argument form.

`stop` : `std::int64_t` See the signature for the supported argument form.

`render` : `bool` See the signature for the supported argument form.

Returns

`DataFrame` Value produced by this call.

Notes

Use this to narrow the current object to the rows or depth range you want to inspect next.

Examples

```
auto df = pluss::table::DataFrame({
    {"id", pluss::table::Thread(pluss::make_series({1, 2, 3}))},
    {"sales", pluss::table::Thread(pluss::make_series({100.0, 120.0, 90.0}))},
    {"city", pluss::table::Thread(pluss::make_note({"Accra", "Tema", "Kumasi"}))},
});

auto out = df.view(0, 2, false);
```

Output

```
id  sales  city
1   100.0  Accra
2   120.0  Tema
3    90.0  Kumasi
```

5.1.257 where

Signature

```
DataFrame where(self, const pluss::Series& bool_mask) const
```

Parameters

`bool_mask` : `const pluss::Series&` See the signature for the supported argument form.

Returns

`DataFrame` Value produced by this call.

Notes

Use this to reduce the current object to the rows or fields that matter for the next step.

Examples

```
auto df = pluss::table::DataFrame({
    {"id", pluss::table::Thread(pluss::make_series({1, 2, 3}))},
    {"sales", pluss::table::Thread(pluss::make_series({100.0, 120.0, 90.0}))},
    {"city", pluss::table::Thread(pluss::make_note({"Accra", "Tema", "Kumasi"}))},
});

auto mask = pluss::make_series({true, false, true}, pluss::codem::ScalarType::Bool);
auto out = df.where(mask);
```

Output

```
id  sales  city
1   100.0  Accra
3    90.0  Kumasi
```

5.1.258 window

Signature

```
DataFrame window(self, std::initializer_list<WindowSpec> specs, bool keep_input = true)␣
↪const
```

Parameters

specs : std::initializer_list<WindowSpec> See the signature for the supported argument form.

keep_input : bool See the signature for the supported argument form.

Returns

DataFrame Value produced by this call.

Notes

This call is part of the public DataFrame API and is documented here using a concrete usage example.

Examples

```
auto df = pluss::table::DataFrame({
    {"id", pluss::table::Thread(pluss::make_series({1, 2, 3}))},
    {"sales", pluss::table::Thread(pluss::make_series({100.0, 120.0, 90.0}))},
    {"city", pluss::table::Thread(pluss::make_note({"Accra", "Tema", "Kumasi"}))},
});

auto out = df.window({{.output = "sales_roll", .feature = "sales", .op =␣
↪pluss::table::ThreadWindowOp::RollingMean, .window = 2, .min_periods = 1}}, true);
```

Output

```
id  sales  city
1   100.0  Accra
2   120.0  Tema
3    90.0  Kumasi
```

5.1.259 with_cases

Signature

```
DataFrame with_cases(self, const Thread& cases) const
```

Parameters

`cases` : `const Thread&` See the signature for the supported argument form.

Returns

`DataFrame` Value produced by this call.

Notes

This mutates or derives structure while keeping the surrounding object model intact.

Examples

```
auto df = pluss::table::DataFrame({
    {"id", pluss::table::Thread(pluss::make_series({1, 2, 3}))},
    {"sales", pluss::table::Thread(pluss::make_series({100.0, 120.0, 90.0}))},
    {"city", pluss::table::Thread(pluss::make_note({"Accra", "Tema", "Kumasi"}))},
});

auto cases = pluss::table::Thread(pluss::make_note({"cs0", "cs1", "cs2"}));
auto out = df.with_cases(cases);
```

Output

```
id  sales  city
1   100.0  Accra
2   120.0  Tema
3    90.0  Kumasi
```

5.1.260 with_cases_

Signature

```
DataFrame& with_cases_(self, const Thread& cases)
```

Parameters

`cases` : `const Thread&` See the signature for the supported argument form.

Returns

`DataFrame&` Value produced by this call.

Notes

This mutates or derives structure while keeping the surrounding object model intact.

Examples

```
auto df = pluss::table::DataFrame({
    {"id", pluss::table::Thread(pluss::make_series({1, 2, 3}))},
    {"sales", pluss::table::Thread(pluss::make_series({100.0, 120.0, 90.0}))},
    {"city", pluss::table::Thread(pluss::make_note({"Accra", "Tema", "Kumasi"}))},
```

(continues on next page)

(continued from previous page)

```
});

auto out = df.with_cases_();
```

Output

```
id  sales  city
1   100.0  Accra
2   120.0  Tema
3    90.0  Kumasi
```

5.1.261 with_column

Signature

```
DataFrame with_column(self, std::string name, const Thread& column) const
```

Parameters

`name` : `std::string` See the signature for the supported argument form.

`column` : `const Thread&` See the signature for the supported argument form.

Returns

`DataFrame` Value produced by this call.

Notes

This mutates or derives structure while keeping the surrounding object model intact.

Examples

```
auto df = pluss::table::DataFrame({
    {"id", pluss::table::Thread(pluss::make_series({1, 2, 3}))},
    {"sales", pluss::table::Thread(pluss::make_series({100.0, 120.0, 90.0}))},
    {"city", pluss::table::Thread(pluss::make_note({"Accra", "Tema", "Kumasi"}))},
});

auto out = df.with_column("profit", pluss::table::Thread(pluss::make_series({40.0, 35.0,␣
↪30.0})));
```

Output

```
id  sales  city    profit
1   100.0  Accra   40.0
2   120.0  Tema    35.0
3    90.0  Kumasi  30.0
```

5.1.262 with_column_

Signature

```
DataFrame& with_column_(self, std::string name, const Thread& column)
```

Parameters

`name` : `std::string` See the signature for the supported argument form.

`column` : `const Thread&` See the signature for the supported argument form.

Returns

`DataFrame&` Value produced by this call.

Notes

This mutates or derives structure while keeping the surrounding object model intact.

Examples

```
auto df = pluss::table::DataFrame({
    {"id", pluss::table::Thread(pluss::make_series({1, 2, 3}))},
    {"sales", pluss::table::Thread(pluss::make_series({100.0, 120.0, 90.0}))},
    {"city", pluss::table::Thread(pluss::make_note({"Accra", "Tema", "Kumasi"}))},
});

df.with_column_("profit", pluss::table::Thread(pluss::make_series({40.0, 35.0, 30.0})));
```

Output

```
id  sales  city    profit
1   100.0  Accra   40.0
2   120.0  Tema    35.0
3    90.0  Kumasi  30.0
```

CHAPTER

SIX

COLUMNREF API REFERENCE

CHAPTER

SEVEN

PRACTICAL COLUMNREF API REFERENCE

7.1 API Reference Style

This page documents the public `ColumnRef` helper returned by DataFrame column-reference calls.

7.1.1 head

Signature

```
Thread head(self, std::int64_t n) const
```

Parameters

`n` : `std::int64_t` See the signature for the supported argument form.

Returns

`Thread` Value produced by this call.

Notes

Use this to narrow the current object to the rows or depth range you want to inspect next.

Examples

```
auto df = pluss::table::DataFrame({
    {"id", pluss::table::Thread(pluss::make_series({1, 2, 3}))},
    {"sales", pluss::table::Thread(pluss::make_series({100.0, 120.0, 90.0}))},
    {"city", pluss::table::Thread(pluss::make_note({"Accra", "Tema", "Kumasi"}))},
});
auto ref = df.column_ref("sales");

auto out = ref.head(2);
```

Output

```
[100.0, 120.0]
```

7.1.2 mean

Signature

```
pluss::Series mean(self) const
```

Parameters

None

Returns

`pluss::Series` Value produced by this call.

Notes

Use this when you want a summary view instead of carrying the full input forward unchanged.

Examples

```
auto df = pluss::table::DataFrame({
    {"id", pluss::table::Thread(pluss::make_series({1, 2, 3}))},
    {"sales", pluss::table::Thread(pluss::make_series({100.0, 120.0, 90.0}))},
    {"city", pluss::table::Thread(pluss::make_note({"Accra", "Tema", "Kumasi"}))},
});
auto ref = df.column_ref("sales");

auto out = ref.mean();
```

Output

```
103.3
```

7.1.3 name

Signature

```
const std::string& name(self) const
```

Parameters

None

Returns

`const std::string&` Value produced by this call.

Notes

This call is part of the public `ColumnRef` API and is documented here using a concrete usage example.

Examples

```
auto df = pluss::table::DataFrame({
    {"id", pluss::table::Thread(pluss::make_series({1, 2, 3}))},
    {"sales", pluss::table::Thread(pluss::make_series({100.0, 120.0, 90.0}))},
    {"city", pluss::table::Thread(pluss::make_note({"Accra", "Tema", "Kumasi"}))},
});
auto ref = df.column_ref("sales");

auto out = ref.name();
```

Output

```
sales
```

7.1.4 thread

Signature

```
const Thread& thread(self) const
```

Parameters

None

Returns

`const Thread&` Value produced by this call.

Notes

This call is part of the public `ColumnRef` API and is documented here using a concrete usage example.

Examples

```
auto df = pluss::table::DataFrame({
    {"id", pluss::table::Thread(pluss::make_series({1, 2, 3}))},
    {"sales", pluss::table::Thread(pluss::make_series({100.0, 120.0, 90.0}))},
    {"city", pluss::table::Thread(pluss::make_note({"Accra", "Tema", "Kumasi"}))},
});
auto ref = df.column_ref("sales");

auto out = ref.thread();
```

Output

```
[10.0, 20.0, 30.0]
```

7.1.5 value

Signature

```
ThreadValue value(self) const
```

Parameters

None

Returns

`ThreadValue` Value produced by this call.

Notes

This call is part of the public `ColumnRef` API and is documented here using a concrete usage example.

Examples

```
auto df = pluss::table::DataFrame({
    {"id", pluss::table::Thread(pluss::make_series({1, 2, 3}))},
    {"sales", pluss::table::Thread(pluss::make_series({100.0, 120.0, 90.0}))},
    {"city", pluss::table::Thread(pluss::make_note({"Accra", "Tema", "Kumasi"}))},
});
auto ref = df.column_ref("sales");

auto out = ref.value();
```

Output

```
100.0
```

7.1.6 values

Signature

```
Thread values(self) const
```

Parameters

None

Returns

`Thread` Value produced by this call.

Notes

This call is part of the public `ColumnRef` API and is documented here using a concrete usage example.

Examples

```
auto df = pluss::table::DataFrame({
    {"id", pluss::table::Thread(pluss::make_series({1, 2, 3}))},
    {"sales", pluss::table::Thread(pluss::make_series({100.0, 120.0, 90.0}))},
    {"city", pluss::table::Thread(pluss::make_note({"Accra", "Tema", "Kumasi"}))},
});
auto ref = df.column_ref("sales");

auto out = ref.values();
```

Output

```
[10.0, 20.0, 30.0]
```

CHAPTER

EIGHT

DATAFRAMEPLOTACCESSOR API REFERENCE

CHAPTER

NINE

PRACTICAL DATAFRAMEPLOTACCESSOR API REFERENCE

9.1 API Reference Style

This page documents the public plotting accessor returned by `DataFrame::plot()` and `Atlas::plot()`.

9.1.1 bar

Signature

```
pluss::plotism::Figure bar(self, std::string_view y_feature, std::optional<std::string_
↪view> x_feature = std::nullopt) const
```

Parameters

`y_feature` : `std::string_view` See the signature for the supported argument form.

`x_feature` : `std::optional<std::string_view>` See the signature for the supported argument form.

Returns

`pluss::plotism::Figure` Value produced by this call.

Notes

This call is part of the public `DataFramePlotAccessor` API and is documented here using a concrete usage example.

Examples

```
auto df = pluss::table::DataFrame({
    {"id", pluss::table::Thread(pluss::make_series({1, 2, 3}))},
    {"sales", pluss::table::Thread(pluss::make_series({100.0, 120.0, 90.0}))},
    {"city", pluss::table::Thread(pluss::make_note({"Accra", "Tema", "Kumasi"}))},
});
auto plot = df.plot();

auto fig = plot.bar("sales", std::nullopt);
```

Output

```
Figure(...)
```

9.1.2 hist

Signature

```
pluss::plotism::Figure hist(self, std::string_view feature, std::int64_t bins = 20) const
```

Parameters

`feature` : `std::string_view` See the signature for the supported argument form.

`bins` : `std::int64_t` See the signature for the supported argument form.

Returns

`pluss::plotism::Figure` Value produced by this call.

Notes

This call is part of the public `DataFramePlotAccessor` API and is documented here using a concrete usage example.

Examples

```
auto df = pluss::table::DataFrame({
    {"id", pluss::table::Thread(pluss::make_series({1, 2, 3}))},
    {"sales", pluss::table::Thread(pluss::make_series({100.0, 120.0, 90.0}))},
    {"city", pluss::table::Thread(pluss::make_note({"Accra", "Tema", "Kumasi"}))},
});
auto plot = df.plot();

auto fig = plot.hist("sales", 12);
```

Output

```
Figure(...)
```

9.1.3 line

Signature

```
pluss::plotism::Figure line(self, std::initializer_list<std::string_view> y_features,␣
↪std::optional<std::string_view> x_feature = std::nullopt) const
```

Parameters

`y_features` : `std::initializer_list<std::string_view>` See the signature for the supported argument form.

`x_feature` : `std::optional<std::string_view>` See the signature for the supported argument form.

Returns

`pluss::plotism::Figure` Value produced by this call.

Notes

This call is part of the public `DataFramePlotAccessor` API and is documented here using a concrete usage example.

See also

- `pluss::plotism::Figure line(self, std::string_view y_feature, std::optional<std::string_view> x_feature = std::nullopt) const`

Examples

```
auto df = pluss::table::DataFrame({
    {"id", pluss::table::Thread(pluss::make_series({1, 2, 3}))},
    {"sales", pluss::table::Thread(pluss::make_series({100.0, 120.0, 90.0}))},
    {"city", pluss::table::Thread(pluss::make_note({"Accra", "Tema", "Kumasi"}))},
});
auto plot = df.plot();

auto fig = plot.line({"sales"}, std::nullopt);
```

Output

```
Figure(...)
```

9.1.4 `line`

Signature

```
pluss::plotism::Figure line(self, std::string_view y_feature, std::optional<std::string_
↪view> x_feature = std::nullopt) const
```

Parameters

`y_feature` : `std::string_view` See the signature for the supported argument form.

`x_feature` : `std::optional<std::string_view>` See the signature for the supported argument form.

Returns

`pluss::plotism::Figure` Value produced by this call.

Notes

This call is part of the public `DataFramePlotAccessor` API and is documented here using a concrete usage example.

See also

- `pluss::plotism::Figure line(self, std::initializer_list<std::string_view> y_features, std::optional<std::string_view> x_feature = std::nullopt) const`

Examples

```
auto df = pluss::table::DataFrame({
    {"id", pluss::table::Thread(pluss::make_series({1, 2, 3}))},
    {"sales", pluss::table::Thread(pluss::make_series({100.0, 120.0, 90.0}))},
    {"city", pluss::table::Thread(pluss::make_note({"Accra", "Tema", "Kumasi"}))},
});
auto plot = df.plot();

auto fig = plot.line("sales");
```

Output

```
Figure(...)
```

9.1.5 scatter

Signature

```
pluss::plotism::Figure scatter(self, std::string_view x_feature, std::string_view y_
↪feature) const
```

Parameters

`x_feature` : `std::string_view` See the signature for the supported argument form.

`y_feature` : `std::string_view` See the signature for the supported argument form.

Returns

`pluss::plotism::Figure` Value produced by this call.

Notes

This call is part of the public `DataFramePlotAccessor` API and is documented here using a concrete usage example.

Examples

```
auto df = pluss::table::DataFrame({
    {"id", pluss::table::Thread(pluss::make_series({1, 2, 3}))},
    {"sales", pluss::table::Thread(pluss::make_series({100.0, 120.0, 90.0}))},
    {"city", pluss::table::Thread(pluss::make_note({"Accra", "Tema", "Kumasi"}))},
});
auto plot = df.plot();

auto fig = plot.scatter("id", "sales");
```

Output

```
Figure(...)
```

CHAPTER

TEN

DATAFRAMESQLACCESSOR API REFERENCE

CHAPTER

ELEVEN

PRACTICAL DATAFRAMESQLACCESSOR API REFERENCE

11.1 API Reference Style

This page documents the public SQL accessor returned by `DataFrame::sql()`.

11.1.1 execute

Signature

```
DataFrame execute(self, std::string_view statement) const
```

Parameters

`statement` : `std::string_view` See the signature for the supported argument form.

Returns

`DataFrame` Value produced by this call.

Notes

This call is part of the public `DataFrameSqlAccessor` API and is documented here using a concrete usage example.

Examples

```
auto df = pluss::table::DataFrame({
    {"id", pluss::table::Thread(pluss::make_series({1, 2, 3}))},
    {"sales", pluss::table::Thread(pluss::make_series({100.0, 120.0, 90.0}))},
    {"city", pluss::table::Thread(pluss::make_note({"Accra", "Tema", "Kumasi"}))},
});
auto sql = df.sql();

auto out = sql.execute("SELECT id, sales FROM self WHERE sales > 95");
```

Output

```
id  sales  city
1   100.0  Accra
3    90.0  Kumasi
```

11.1.2 prepare

Signature

```
DataFrameSqlPrepared prepare(self, std::string_view statement) const
```

Parameters

`statement` : `std::string_view` See the signature for the supported argument form.

Returns

`DataFrameSqlPrepared` Value produced by this call.

Notes

This call is part of the public `DataFrameSqlAccessor` API and is documented here using a concrete usage example.

Examples

```
auto df = pluss::table::DataFrame({
    {"id", pluss::table::Thread(pluss::make_series({1, 2, 3}))},
    {"sales", pluss::table::Thread(pluss::make_series({100.0, 120.0, 90.0}))},
    {"city", pluss::table::Thread(pluss::make_note({"Accra", "Tema", "Kumasi"}))},
});
auto sql = df.sql();

auto out = sql.prepare("SELECT id, sales FROM self WHERE sales > {min_sales}");
```

Output

```
Prepared statement ready
```

CHAPTER

TWELVE

DATAFRAMESQLPREPARED API REFERENCE

CHAPTER

THIRTEEN

PRACTICAL DATAFRAMESQLPREPARED API REFERENCE

13.1 API Reference Style

This page documents the public prepared-statement helper returned by `DataFrame::prepare_sql()` or `DataFrameSqlAccessor::prepare()`.

13.1.1 execute

Signature

```
DataFrame execute(self) const
```

Parameters

None

Returns

`DataFrame` Value produced by this call.

Notes

This call is part of the public `DataFrameSqlPrepared` API and is documented here using a concrete usage example.

See also

- `DataFrame execute(self, std::unordered_map<std::string, DataFrame::SqlOptions::SqlParam> params) const`
- `DataFrame execute(self, std::unordered_map<std::string, DataFrame::SqlOptions::SqlParam> params, std::unordered_map<std::string, std::string> identifier_params) const`

Examples

```
auto df = pluss::table::DataFrame({
    {"id", pluss::table::Thread(pluss::make_series({1, 2, 3}))},
    {"sales", pluss::table::Thread(pluss::make_series({100.0, 120.0, 90.0}))},
    {"city", pluss::table::Thread(pluss::make_note({"Accra", "Tema", "Kumasi"}))},
});
auto stmt = df.prepare_sql("SELECT id, sales FROM self WHERE sales > {min_sales}");

auto out = stmt.execute();
```

Output

```
id  sales  city
1   100.0  Accra
3    90.0  Kumasi
```

13.1.2 execute

Signature

```
DataFrame execute(self, std::unordered_map<std::string, DataFrame::SqlOptions::SqlParam>␣
↪params) const
```

Parameters

params : std::unordered_map<std::string, DataFrame::SqlOptions::SqlParam> See the signature for the supported argument form.

Returns

DataFrame Value produced by this call.

Notes

This call is part of the public DataFrameSqlPrepared API and is documented here using a concrete usage example.

See also

- DataFrame execute(self) const
- DataFrame execute(self, std::unordered_map<std::string, DataFrame::SqlOptions::SqlParam> params, std::unordered_map<std::string, std::string> identifier_params) const

Examples

```
auto df = pluss::table::DataFrame({
    {"id", pluss::table::Thread(pluss::make_series({1, 2, 3}))},
    {"sales", pluss::table::Thread(pluss::make_series({100.0, 120.0, 90.0}))},
    {"city", pluss::table::Thread(pluss::make_note({"Accra", "Tema", "Kumasi"}))},
});
auto stmt = df.prepare_sql("SELECT id, sales FROM self WHERE sales > {min_sales}");

auto out = stmt.execute({{"min_sales", 95.0}});
```

Output

```
id  sales  city
1   100.0  Accra
3    90.0  Kumasi
```

13.1.3 execute

Signature

```
DataFrame execute(self, std::unordered_map<std::string, DataFrame::SqlOptions::SqlParam>␣
↪params, std::unordered_map<std::string, std::string> identifier_params) const
```

Parameters

`params` : `std::unordered_map<std::string, DataFrame::SqlOptions::SqlParam>` See the signature for the supported argument form.

`identifier_params` : `std::unordered_map<std::string, std::string>` See the signature for the supported argument form.

Returns

`DataFrame` Value produced by this call.

Notes

This call is part of the public `DataFrameSqlPrepared` API and is documented here using a concrete usage example.

See also

- `DataFrame execute(self) const`
- `DataFrame execute(self, std::unordered_map<std::string, DataFrame::SqlOptions::SqlParam> params) const`

Examples

```
auto df = pluss::table::DataFrame({
    {"id", pluss::table::Thread(pluss::make_series({1, 2, 3}))},
    {"sales", pluss::table::Thread(pluss::make_series({100.0, 120.0, 90.0}))},
    {"city", pluss::table::Thread(pluss::make_note({"Accra", "Tema", "Kumasi"}))},
});
auto stmt = df.prepare_sql("SELECT id, sales FROM self WHERE sales > {min_sales}");

auto out = stmt.execute({{"min_sales", 95.0}}, {{"table_name", "self"}});
```

Output

```
id  sales  city
1   100.0  Accra
3    90.0  Kumasi
```

CHAPTER

FOURTEEN

THREAD AND ATLAS GUIDE

14.1 Thread

`Thread` is a typed column wrapper around `Series`, `Note`, `Timeline`, or `TimeIndex`. Use it when you want column operations without carrying the full `DataFrame` around.

```
auto sales = pluss::table::Thread(pluss::make_series({100.0, 120.0, 90.0}));
auto cities = pluss::table::Thread(pluss::make_note({"Accra", "Tema", "Kumasi"}));

auto cleaned = sales.fillna(0.0).clip(0.0, 150.0);
auto avg = cleaned.mean();
auto city_mask = cities.contains("Accra");
```

Notes:

- `Thread` is the easiest bridge point between the Pluss domain types and table verbs
- when you want the original typed payload back, use `as_series()`, `as_note()`, `as_timeline()`, or `as_timeindex()`

14.2 Row-Space Helpers

```
auto head = sales.head(2);
auto tail = sales.tail(2);
auto slice = sales.slice(0, 2);
auto taken = sales.take(std::array<std::int64_t, 2>{0, 2});
auto filtered = sales.filter(city_mask);
```

14.3 Atlas

`Atlas` is the frame-stack model in PlussMic. It is useful when you want a small set of aligned `DataFrame` objects with an optional depth axis.

```
auto f0 = pluss::table::DataFrame({
    {"city", pluss::table::Thread(pluss::make_note({"Accra", "Tema"}))},
    {"sales", pluss::table::Thread(pluss::make_series({100.0, 120.0}))},
});

auto f1 = pluss::table::DataFrame({
    {"city", pluss::table::Thread(pluss::make_note({"Accra", "Tema"}))},
```

(continues on next page)

(continued from previous page)

```
    {"sales", pluss::table::Thread(pluss::make_series({130.0, 125.0}))},
});

auto labels = pluss::table::Thread(pluss::make_note({"train", "test"}));
auto atlas = pluss::table::Atlas({f0, f1}, labels);

atlas.head(1).show();
auto sorted = atlas.sort_depth();
auto first = atlas.frame(0);
```

14.4 Atlas To Kindle

```
auto archive = atlas.kindelize();
auto roundtrip = archive.tabelize();
```

CHAPTER

FIFTEEN

THREAD API REFERENCE

CHAPTER

SIXTEEN

PRACTICAL THREAD API REFERENCE

16.1 API Reference Style

This page documents the full public `Thread` surface with direct typed-column examples.

16.1.1 abs

Signature

```
Thread abs(self) const
```

Parameters

None

Returns

`Thread` Value produced by this call.

Notes

This call is part of the public `Thread` API and is documented here using a concrete usage example.

Examples

```
auto thread = pluss::table::Thread(pluss::make_series({10.0, 20.0, 30.0}));

auto out = thread.abs();
```

Output

```
[10.0, 20.0, 30.0]
```

16.1.2 as_note

Signature

```
const pluss::Note& as_note(self) const
```

Parameters

None

Returns

`const pluss::Note&` Value produced by this call.

Notes

This call is part of the public `Thread` API and is documented here using a concrete usage example.

Examples

```
auto thread = pluss::table::Thread(pluss::make_note({"Accra", "Tema", "Kumasi"}));

auto out = thread.as_note();
```

Output

```
["Accra", "Tema", "Kumasi"]
```

16.1.3 as_series

Signature

```
const pluss::Series& as_series(self) const
```

Parameters

None

Returns

`const pluss::Series&` Value produced by this call.

Notes

This call is part of the public `Thread` API and is documented here using a concrete usage example.

Examples

```
auto thread = pluss::table::Thread(pluss::make_series({10.0, 20.0, 30.0}));

auto out = thread.as_series();
```

Output

```
[10.0, 20.0, 30.0]
```

16.1.4 as_timeindex

Signature

```
const pluss::TimeIndex& as_timeindex(self) const
```

Parameters

None

Returns

const pluss::TimeIndex& Value produced by this call.

Notes

This call is part of the public Thread API and is documented here using a concrete usage example.

Examples

```
auto thread = pluss::table::Thread(pluss::make_timeline({"2024-01-01T09:10:00Z", "2024-
↪02-01T11:00:00Z", "2024-03-05T15:30:00Z"}));

auto out = thread.as_timeindex();
```

Output

```
["2024-01-01T09:10:00Z", "2024-02-01T11:00:00Z", "2024-03-05T15:30:00Z"]
```

16.1.5 as_timeline

Signature

```
const pluss::Timeline& as_timeline(self) const
```

Parameters

None

Returns

const pluss::Timeline& Value produced by this call.

Notes

This call is part of the public Thread API and is documented here using a concrete usage example.

Examples

```
auto thread = pluss::table::Thread(pluss::make_timeline({"2024-01-01T09:10:00Z", "2024-
↪02-01T11:00:00Z", "2024-03-05T15:30:00Z"}));

auto out = thread.as_timeline();
```

Output

```
["2024-01-01T09:10:00Z", "2024-02-01T11:00:00Z", "2024-03-05T15:30:00Z"]
```

16.1.6 astype

Signature

```
Thread astype(self, pluss::codem::ScalarType dtype) const
```

Parameters

`dtype` : `pluss::codem::ScalarType` See the signature for the supported argument form.

Returns

`Thread` Value produced by this call.

Notes

This changes the semantic or numeric type so later operations route through the correct implementation path.

Examples

```
auto thread = pluss::table::Thread(pluss::make_series({10.0, 20.0, 30.0}));

auto out = thread.astype(pluss::codem::ScalarType::Float64);
```

Output

```
[10.0, 20.0, 30.0]
```

16.1.7 bfill

Signature

```
Thread bfill(self) const
```

Parameters

`None`

Returns

`Thread` Value produced by this call.

Notes

This is part of the missing-data cleanup path and is usually applied before summaries, grouping, or export.

Examples

```
auto thread = pluss::table::Thread(pluss::make_series({10.0, 20.0, 30.0}));

auto out = thread.bfill();
```

Output

```
[10.0, 20.0, 30.0]
```

16.1.8 cast

Signature

```
Thread cast(self, ThreadCastTarget target, std::optional<pluss::codem::ScalarType>␣
↪series_dtype = std::nullopt) const
```

Parameters

`target` : `ThreadCastTarget` See the signature for the supported argument form.

`series_dtype` : `std::optional<pluss::codem::ScalarType>` See the signature for the supported argument form.

Returns

`Thread` Value produced by this call.

Notes

This changes the semantic or numeric type so later operations route through the correct implementation path.

Examples

```
auto thread = pluss::table::Thread(pluss::make_note({"1.0", "2.0", "3.0"}));

auto out = thread.cast(pluss::table::ThreadCastTarget::Series,␣
↪pluss::codem::ScalarType::Float64);
```

Output

```
[10.0, 20.0, 30.0]
```

16.1.9 clear_validity_mask

Signature

```
Thread clear_validity_mask(self) const
```

Parameters

None

Returns

Thread Value produced by this call.

Notes

This call is part of the public Thread API and is documented here using a concrete usage example.

Examples

```
auto mask = pluss::make_series({true, false, true}, pluss::codem::ScalarType::Bool);
auto thread = pluss::table::Thread(pluss::make_series({10.0, 20.0, 30.0}));

auto out = thread.clear_validity_mask();
```

16.1.10 clear_validity_mask_

Signature

```
Thread& clear_validity_mask_(self)
```

Parameters

None

Returns

Thread& Value produced by this call.

Notes

This call is part of the public Thread API and is documented here using a concrete usage example.

Examples

```
auto mask = pluss::make_series({true, false, true}, pluss::codem::ScalarType::Bool);
auto thread = pluss::table::Thread(pluss::make_series({10.0, 20.0, 30.0}));

thread.clear_validity_mask_();
```

16.1.11 clip

Signature

```
Thread clip(self, scalar min_value, scalar max_value) const
```

Parameters

`min_value` : `scalar` See the signature for the supported argument form.

`max_value` : `scalar` See the signature for the supported argument form.

Returns

`Thread` Value produced by this call.

Notes

This call is part of the public `Thread` API and is documented here using a concrete usage example.

Examples

```
auto thread = pluss::table::Thread(pluss::make_series({10.0, 20.0, 30.0}));

auto out = thread.clip(0.0, 25.0);
```

Output

```
[10.0, 20.0, 30.0]
```

16.1.12 `compare`

Signature

```
Thread compare(self, const Thread& other, ThreadBinaryCompareOp op) const
```

Parameters

`other` : `const Thread&` See the signature for the supported argument form.

`op` : `ThreadBinaryCompareOp` See the signature for the supported argument form.

Returns

`Thread` Value produced by this call.

Notes

This produces a comparison-style result you can inspect directly or reuse as a mask.

See also

- `Thread compare(self, scalar other, ThreadBinaryCompareOp op) const`

Examples

```
auto thread = pluss::table::Thread(pluss::make_series({10.0, 20.0, 30.0}));

auto other = pluss::table::Thread(pluss::make_series({15.0, 10.0, 35.0}));
auto out = thread.compare(other, pluss::table::ThreadBinaryCompareOp::Gt);
```

Output

```
1
```

16.1.13 compare

Signature

```
Thread compare(self, scalar other, ThreadBinaryCompareOp op) const
```

Parameters

`other` : `scalar` See the signature for the supported argument form.

`op` : `ThreadBinaryCompareOp` See the signature for the supported argument form.

Returns

`Thread` Value produced by this call.

Notes

This produces a comparison-style result you can inspect directly or reuse as a mask.

See also

- `Thread compare(self, const Thread& other, ThreadBinaryCompareOp op) const`

Examples

```
auto thread = pluss::table::Thread(pluss::make_series({10.0, 20.0, 30.0}));

auto out = thread.compare(15.0, pluss::table::ThreadBinaryCompareOp::Gt);
```

Output

```
1
```

16.1.14 compare_rows

Signature

```
int compare_rows(self, std::int64_t lhs_row, std::int64_t rhs_row, bool nulls_last =␣
↪true) const
```

Parameters

`lhs_row` : `std::int64_t` See the signature for the supported argument form.

`rhs_row` : `std::int64_t` See the signature for the supported argument form.

`nulls_last` : `bool` See the signature for the supported argument form.

Returns

`int` Value produced by this call.

Notes

This call is part of the public `Thread` API and is documented here using a concrete usage example.

Examples

```
auto thread = pluss::table::Thread(pluss::make_series({10.0, 20.0, 30.0}));

auto out = thread.compare_rows(0, 1, true);
```

Output

```
1
```

16.1.15 contains

Signature

```
pluss::Series contains(self, std::string_view needle) const
```

Parameters

`needle` : `std::string_view` See the signature for the supported argument form.

Returns

`pluss::Series` Value produced by this call.

Notes

This is part of the text-processing surface for note-backed or text-oriented data.

Examples

```
auto thread = pluss::table::Thread(pluss::make_note({"Accra", "Tema", "Kumasi"}));

auto out = thread.contains("Accra");
```

Output

```
[true, false, true]
```

16.1.16 day

Signature

```
Thread day(self) const
```

Parameters

None

Returns

Thread Value produced by this call.

Notes

This is part of the time-aware surface and expects timeline or time-index semantics.

Examples

```
auto thread = pluss::table::Thread(pluss::make_timeline({"2024-01-01T09:10:00Z", "2024-
↪02-01T11:00:00Z", "2024-03-05T15:30:00Z"}));

auto out = thread.day();
```

Output

```
[1, 2, 3]
```

16.1.17 defined

Signature

```
bool defined(self) const
```

Parameters

None

Returns

bool Boolean status or membership result.

Notes

This call is part of the public Thread API and is documented here using a concrete usage example.

Examples

```
auto thread = pluss::table::Thread(pluss::make_series({10.0, 20.0, 30.0}));

auto out = thread.defined();
```

Output

```
true
```

16.1.18 dropna

Signature

```
Thread dropna(self) const
```

Parameters

None

Returns

Thread Value produced by this call.

Notes

This is part of the missing-data cleanup path and is usually applied before summaries, grouping, or export.

Examples

```
auto thread = pluss::table::Thread(pluss::make_series({10.0, 20.0, 30.0}));

auto out = thread.dropna();
```

Output

```
[10.0, 20.0, 30.0]
```

16.1.19 dtype_name

Signature

```
std::string dtype_name(self) const
```

Parameters

None

Returns

std::string Rendered string result.

Notes

This call is part of the public Thread API and is documented here using a concrete usage example.

Examples

```
auto thread = pluss::table::Thread(pluss::make_series({10.0, 20.0, 30.0}));

auto out = thread.dtype_name();
```

Output

```
Float64
```

16.1.20 exp

Signature

```
Thread exp(self) const
```

Parameters

None

Returns

Thread Value produced by this call.

Notes

This call is part of the public Thread API and is documented here using a concrete usage example.

Examples

```
auto thread = pluss::table::Thread(pluss::make_series({10.0, 20.0, 30.0}));

auto out = thread.exp();
```

Output

```
[10.0, 20.0, 30.0]
```

16.1.21 ffill

Signature

```
Thread ffill(self) const
```

Parameters

None

Returns

Thread Value produced by this call.

Notes

This is part of the missing-data cleanup path and is usually applied before summaries, grouping, or export.

Examples

```
auto thread = pluss::table::Thread(pluss::make_series({10.0, 20.0, 30.0}));

auto out = thread.ffill();
```

Output

```
[10.0, 20.0, 30.0]
```

16.1.22 fillna

Signature

```
Thread fillna(self, const char* value) const
```

Parameters

`value` : `const char*` See the signature for the supported argument form.

Returns

`Thread` Value produced by this call.

Notes

This is part of the missing-data cleanup path and is usually applied before summaries, grouping, or export.

See also

- `Thread fillna(self, scalar value) const`
- `Thread fillna(self, std::string_view value) const`
- `Thread fillna(self, const pluss::Timeline& value) const`
- `Thread fillna(self, const pluss::TimeIndex& value) const`

Examples

```
auto thread = pluss::table::Thread(pluss::make_note({"Accra", "Tema", "Kumasi"}));

auto out = thread.fillna("Unknown");
```

Output

```
[10.0, 20.0, 30.0]
```

16.1.23 fillna

Signature

```
Thread fillna(self, const pluss::TimeIndex& value) const
```

Parameters

`value` : `const pluss::TimeIndex&` See the signature for the supported argument form.

Returns

`Thread` Value produced by this call.

Notes

This is part of the missing-data cleanup path and is usually applied before summaries, grouping, or export.

See also

- `Thread fillna(self, scalar value) const`
- `Thread fillna(self, std::string_view value) const`
- `Thread fillna(self, const char* value) const`
- `Thread fillna(self, const pluss::Timeline& value) const`

Examples

```
auto thread = pluss::table::Thread(pluss::make_timeindex({"2024-01-01T09:10:00Z", "2024-
↪02-01T11:00:00Z"}));

auto out = thread.fillna(pluss::make_timeindex("2024-01-01T00:00:00Z"));
```

Output

```
[10.0, 20.0, 30.0]
```

16.1.24 `fillna`

Signature

```
Thread fillna(self, const pluss::Timeline& value) const
```

Parameters

`value` : `const pluss::Timeline&` See the signature for the supported argument form.

Returns

`Thread` Value produced by this call.

Notes

This is part of the missing-data cleanup path and is usually applied before summaries, grouping, or export.

See also

- `Thread fillna(self, scalar value) const`
- `Thread fillna(self, std::string_view value) const`
- `Thread fillna(self, const char* value) const`

- `Thread fillna(self, const pluss::TimeIndex& value) const`

Examples

```
auto thread = pluss::table::Thread(pluss::make_timeline({"2024-01-01T09:10:00Z", "2024-
↪02-01T11:00:00Z", "2024-03-05T15:30:00Z"}));

auto out = thread.fillna(pluss::make_timeline("2024-01-01T00:00:00Z"));
```

Output

```
[10.0, 20.0, 30.0]
```

16.1.25 `fillna`

Signature

```
Thread fillna(self, scalar value) const
```

Parameters

`value` : `scalar` See the signature for the supported argument form.

Returns

`Thread` Value produced by this call.

Notes

This is part of the missing-data cleanup path and is usually applied before summaries, grouping, or export.

See also

- `Thread fillna(self, std::string_view value) const`
- `Thread fillna(self, const char* value) const`
- `Thread fillna(self, const pluss::Timeline& value) const`
- `Thread fillna(self, const pluss::TimeIndex& value) const`

Examples

```
auto thread = pluss::table::Thread(pluss::make_series({10.0, 20.0, 30.0}));

auto out = thread.fillna(0.0);
```

Output

```
[10.0, 20.0, 30.0]
```

16.1.26 fillna

Signature

```
Thread fillna(self, std::string_view value) const
```

Parameters

value : std::string_view See the signature for the supported argument form.

Returns

Thread Value produced by this call.

Notes

This is part of the missing-data cleanup path and is usually applied before summaries, grouping, or export.

See also

- Thread fillna(self, scalar value) const
- Thread fillna(self, const char* value) const
- Thread fillna(self, const pluss::Timeline& value) const
- Thread fillna(self, const pluss::TimeIndex& value) const

Examples

```
auto thread = pluss::table::Thread(pluss::make_note({"Accra", "Tema", "Kumasi"}));

auto out = thread.fillna("Unknown");
```

Output

```
[10.0, 20.0, 30.0]
```

16.1.27 fillna_

Signature

```
Thread& fillna_(self, const char* value)
```

Parameters

value : const char* See the signature for the supported argument form.

Returns

Thread& Value produced by this call.

Notes

This is part of the missing-data cleanup path and is usually applied before summaries, grouping, or export.

See also

- `Thread& fillna_(self, scalar value)`
- `Thread& fillna_(self, std::string_view value)`
- `Thread& fillna_(self, const pluss::Timeline& value)`
- `Thread& fillna_(self, const pluss::TimeIndex& value)`

Examples

```
auto thread = pluss::table::Thread(pluss::make_note({"Accra", "Tema", "Kumasi"}));

thread.fillna_("Unknown");
```

Output

```
[10.0, 20.0, 30.0]
```

16.1.28 `fillna_`

Signature

```
Thread& fillna_(self, const pluss::TimeIndex& value)
```

Parameters

`value` : `const pluss::TimeIndex&` See the signature for the supported argument form.

Returns

`Thread&` Value produced by this call.

Notes

This is part of the missing-data cleanup path and is usually applied before summaries, grouping, or export.

See also

- `Thread& fillna_(self, scalar value)`
- `Thread& fillna_(self, std::string_view value)`
- `Thread& fillna_(self, const char* value)`
- `Thread& fillna_(self, const pluss::Timeline& value)`

Examples

```
auto thread = pluss::table::Thread(pluss::make_timeindex({"2024-01-01T09:10:00Z", "2024-
↪02-01T11:00:00Z"}));

thread.fillna_(pluss::make_timeindex("2024-01-01T00:00:00Z"));
```

Output

```
[10.0, 20.0, 30.0]
```

16.1.29 fillna_

Signature

```
Thread& fillna_(self, const pluss::Timeline& value)
```

Parameters

value : const pluss::Timeline& See the signature for the supported argument form.

Returns

Thread& Value produced by this call.

Notes

This is part of the missing-data cleanup path and is usually applied before summaries, grouping, or export.

See also

- Thread& fillna_(self, scalar value)
- Thread& fillna_(self, std::string_view value)
- Thread& fillna_(self, const char* value)
- Thread& fillna_(self, const pluss::TimeIndex& value)

Examples

```
auto thread = pluss::table::Thread(pluss::make_timeline({"2024-01-01T09:10:00Z", "2024-
↪02-01T11:00:00Z", "2024-03-05T15:30:00Z"}));

thread.fillna_(pluss::make_timeline("2024-01-01T00:00:00Z"));
```

Output

```
[10.0, 20.0, 30.0]
```

16.1.30 fillna_

Signature

```
Thread& fillna_(self, scalar value)
```

Parameters

value : scalar See the signature for the supported argument form.

Returns

Thread& Value produced by this call.

Notes

This is part of the missing-data cleanup path and is usually applied before summaries, grouping, or export.

See also

- Thread& fillna_(self, std::string_view value)
- Thread& fillna_(self, const char* value)
- Thread& fillna_(self, const pluss::Timeline& value)
- Thread& fillna_(self, const pluss::TimeIndex& value)

Examples

```
auto thread = pluss::table::Thread(pluss::make_series({10.0, 20.0, 30.0}));

thread.fillna_(0.0);
```

Output

```
[10.0, 20.0, 30.0]
```

16.1.31 fillna_

Signature

```
Thread& fillna_(self, std::string_view value)
```

Parameters

value : std::string_view See the signature for the supported argument form.

Returns

Thread& Value produced by this call.

Notes

This is part of the missing-data cleanup path and is usually applied before summaries, grouping, or export.

See also

- Thread& fillna_(self, scalar value)
- Thread& fillna_(self, const char* value)
- Thread& fillna_(self, const pluss::Timeline& value)
- Thread& fillna_(self, const pluss::TimeIndex& value)

Examples

```
auto thread = pluss::table::Thread(pluss::make_note({"Accra", "Tema", "Kumasi"}));

thread.fillna_("Unknown");
```

Output

```
[10.0, 20.0, 30.0]
```

16.1.32 filter

Signature

```
Thread filter(self, const pluss::Series& bool_mask) const
```

Parameters

bool_mask : const pluss::Series& See the signature for the supported argument form.

Returns

Thread Value produced by this call.

Notes

Use this to reduce the current object to the rows or fields that matter for the next step.

Examples

```
auto thread = pluss::table::Thread(pluss::make_series({10.0, 20.0, 30.0}));

auto mask = pluss::make_series({true, false, true}, pluss::codem::ScalarType::Bool);
auto out = thread.filter(mask);
```

Output

```
[10.0, 20.0, 30.0]
```

16.1.33 has_validity_mask

Signature

```
bool has_validity_mask(self) const
```

Parameters

None

Returns

`bool` Boolean status or membership result.

Notes

This call is part of the public `Thread` API and is documented here using a concrete usage example.

Examples

```cpp
auto thread = pluss::table::Thread(pluss::make_series({10.0, 20.0, 30.0}));

auto out = thread.has_validity_mask();
```

16.1.34 head

Signature

```cpp
Thread head(self, std::int64_t n) const
```

Parameters

`n` : `std::int64_t` See the signature for the supported argument form.

Returns

`Thread` Value produced by this call.

Notes

Use this to narrow the current object to the rows or depth range you want to inspect next.

Examples

```cpp
auto thread = pluss::table::Thread(pluss::make_series({10.0, 20.0, 30.0}));

auto out = thread.head(2);
```

Output

```
[10.0, 20.0, 30.0]
```

16.1.35 hour

Signature

```cpp
Thread hour(self) const
```

Parameters

`None`

Returns

Thread Value produced by this call.

Notes

This is part of the time-aware surface and expects timeline or time-index semantics.

Examples

```
auto thread = pluss::table::Thread(pluss::make_timeline({"2024-01-01T09:10:00Z", "2024-
↪02-01T11:00:00Z", "2024-03-05T15:30:00Z"}));

auto out = thread.hour();
```

Output

```
[1, 2, 3]
```

16.1.36 is_valid

Signature

```
bool is_valid(self, std::int64_t row) const
```

Parameters

row : std::int64_t See the signature for the supported argument form.

Returns

bool Boolean status or membership result.

Notes

This call is part of the public Thread API and is documented here using a concrete usage example.

Examples

```
auto thread = pluss::table::Thread(pluss::make_series({10.0, 20.0, 30.0}));

auto out = thread.is_valid(0);
```

Output

```
true
```

16.1.37 isfinite

Signature

```
pluss::Series isfinite(self) const
```

Parameters

None

Returns

`pluss::Series` Value produced by this call.

Notes

This is part of the missing-data cleanup path and is usually applied before summaries, grouping, or export.

Examples

```
auto thread = pluss::table::Thread(pluss::make_series({10.0, 20.0, 30.0}));

auto out = thread.isfinite();
```

Output

```
[true, false, true]
```

16.1.38 `isnan`

Signature

```
pluss::Series isnan(self) const
```

Parameters

None

Returns

`pluss::Series` Value produced by this call.

Notes

This is part of the missing-data cleanup path and is usually applied before summaries, grouping, or export.

Examples

```
auto thread = pluss::table::Thread(pluss::make_series({10.0, 20.0, 30.0}));

auto out = thread.isnan();
```

Output

```
[true, false, true]
```

16.1.39 isnull

Signature

```
pluss::Series isnull(self) const
```

Parameters

None

Returns

pluss::Series Value produced by this call.

Notes

This is part of the missing-data cleanup path and is usually applied before summaries, grouping, or export.

Examples

```
auto thread = pluss::table::Thread(pluss::make_series({10.0, 20.0, 30.0}));

auto out = thread.isnull();
```

Output

```
[true, false, true]
```

16.1.40 kind

Signature

```
ThreadKind kind(self) const
```

Parameters

None

Returns

ThreadKind Value produced by this call.

Notes

This call is part of the public Thread API and is documented here using a concrete usage example.

Examples

```
auto thread = pluss::table::Thread(pluss::make_series({10.0, 20.0, 30.0}));

auto out = thread.kind();
```

Output

```
Series
```

16.1.41 log

Signature

```
Thread log(self) const
```

Parameters

None

Returns

Thread Value produced by this call.

Notes

This call is part of the public Thread API and is documented here using a concrete usage example.

Examples

```
auto thread = pluss::table::Thread(pluss::make_series({10.0, 20.0, 30.0}));

auto out = thread.log();
```

Output

```
[10.0, 20.0, 30.0]
```

16.1.42 lower

Signature

```
Thread lower(self) const
```

Parameters

None

Returns

Thread Value produced by this call.

Notes

This is part of the text-processing surface for note-backed or text-oriented data.

Examples

```
auto thread = pluss::table::Thread(pluss::make_note({"Accra", "Tema", "Kumasi"}));

auto out = thread.lower();
```

Output

```
["Accra", "Tema", "Kumasi"]
```

16.1.43 max

Signature

```
pluss::Series max(self) const
```

Parameters

None

Returns

`pluss::Series` Value produced by this call.

Notes

Use this when you want a summary view instead of carrying the full input forward unchanged.

Examples

```
auto thread = pluss::table::Thread(pluss::make_series({10.0, 20.0, 30.0}));

auto out = thread.max();
```

Output

```
20.0
```

16.1.44 mean

Signature

```
pluss::Series mean(self) const
```

Parameters

None

Returns

`pluss::Series` Value produced by this call.

Notes

Use this when you want a summary view instead of carrying the full input forward unchanged.

Examples

```
auto thread = pluss::table::Thread(pluss::make_series({10.0, 20.0, 30.0}));

auto out = thread.mean();
```

Output

```
20.0
```

16.1.45 min

Signature

```
pluss::Series min(self) const
```

Parameters

None

Returns

`pluss::Series` Value produced by this call.

Notes

Use this when you want a summary view instead of carrying the full input forward unchanged.

Examples

```
auto thread = pluss::table::Thread(pluss::make_series({10.0, 20.0, 30.0}));

auto out = thread.min();
```

Output

```
20.0
```

16.1.46 month

Signature

```
Thread month(self) const
```

Parameters

None

Returns

Thread Value produced by this call.

Notes

This is part of the time-aware surface and expects timeline or time-index semantics.

Examples

```
auto thread = pluss::table::Thread(pluss::make_timeline({"2024-01-01T09:10:00Z", "2024-
↪02-01T11:00:00Z", "2024-03-05T15:30:00Z"}));

auto out = thread.month();
```

Output

```
[1, 2, 3]
```

16.1.47 note

Signature

```
const pluss::Note& note(self) const
```

Parameters

None

Returns

const pluss::Note& Value produced by this call.

Notes

This call is part of the public Thread API and is documented here using a concrete usage example.

Examples

```
auto thread = pluss::table::Thread(pluss::make_note({"Accra", "Tema", "Kumasi"}));

auto out = thread.note();
```

Output

```
["Accra", "Tema", "Kumasi"]
```

16.1.48 notnull

Signature

```
pluss::Series notnull(self) const
```

Parameters

None

Returns

pluss::Series Value produced by this call.

Notes

This is part of the missing-data cleanup path and is usually applied before summaries, grouping, or export.

Examples

```
auto thread = pluss::table::Thread(pluss::make_series({10.0, 20.0, 30.0}));

auto out = thread.notnull();
```

Output

```
[true, false, true]
```

16.1.49 numeric_binary

Signature

```
Thread numeric_binary(self, const Thread& other, ThreadBinaryMathOp op) const
```

Parameters

other : const Thread& See the signature for the supported argument form.

op : ThreadBinaryMathOp See the signature for the supported argument form.

Returns

Thread Value produced by this call.

Notes

This applies arithmetic across the compatible numeric payloads in the current object.

See also

- Thread numeric_binary(self, scalar other, ThreadBinaryMathOp op, bool reverse = false) const

Examples

```
auto thread = pluss::table::Thread(pluss::make_series({10.0, 20.0, 30.0}));

auto out = thread.numeric_binary(10.0, pluss::table::ThreadBinaryMathOp::Add, false);
```

Output

```
[20.0, 30.0, 40.0]
```

16.1.50 numeric_binary

Signature

```
Thread numeric_binary(self, scalar other, ThreadBinaryMathOp op, bool reverse = false)␣
↪const
```

Parameters

`other` : `scalar` See the signature for the supported argument form.

`op` : `ThreadBinaryMathOp` See the signature for the supported argument form.

`reverse` : `bool` See the signature for the supported argument form.

Returns

`Thread` Value produced by this call.

Notes

This applies arithmetic across the compatible numeric payloads in the current object.

See also

- `Thread numeric_binary(self, const Thread& other, ThreadBinaryMathOp op) const`

Examples

```
auto thread = pluss::table::Thread(pluss::make_series({10.0, 20.0, 30.0}));

auto out = thread.numeric_binary(10.0, pluss::table::ThreadBinaryMathOp::Add, false);
```

Output

```
[20.0, 30.0, 40.0]
```

16.1.51 replace

Signature

```
Thread replace(self, std::string_view from, std::string_view to) const
```

Parameters

`from` : `std::string_view` See the signature for the supported argument form.

`to` : `std::string_view` See the signature for the supported argument form.

Returns

Thread Value produced by this call.

Notes

This is part of the text-processing surface for note-backed or text-oriented data.

Examples

```
auto thread = pluss::table::Thread(pluss::make_note({"Accra", "Tema", "Kumasi"}));

auto out = thread.replace("Tema", "Tamale");
```

Output

```
["Accra", "Tema", "Kumasi"]
```

16.1.52 series

Signature

```
const pluss::Series& series(self) const
```

Parameters

None

Returns

const pluss::Series& Value produced by this call.

Notes

This call is part of the public Thread API and is documented here using a concrete usage example.

Examples

```
auto thread = pluss::table::Thread(pluss::make_series({10.0, 20.0, 30.0}));

auto out = thread.series();
```

Output

```
[10.0, 20.0, 30.0]
```

16.1.53 show

Signature

```
void show(self, const display::DisplayOptions& options =
```

Parameters

`options` : `const display::DisplayOptions&` See the signature for the supported argument form.

Returns

`void` Performs the side effect and returns no value.

Notes

This renders a human-readable view for inspection instead of creating a new analytic object.

See also

- `void show(self, std::ostream& os, const display::DisplayOptions& options =`

Examples

```
auto thread = pluss::table::Thread(pluss::make_series({10.0, 20.0, 30.0}));

thread.show();
```

Output

```
[10.0, 20.0, 30.0]
```

16.1.54 show

Signature

```
void show(self, std::ostream& os, const display::DisplayOptions& options =
```

Parameters

`os` : `std::ostream&` See the signature for the supported argument form.

`options` : `const display::DisplayOptions&` See the signature for the supported argument form.

Returns

`void` Performs the side effect and returns no value.

Notes

This renders a human-readable view for inspection instead of creating a new analytic object.

See also

- `void show(self, const display::DisplayOptions& options =`

Examples

```
auto thread = pluss::table::Thread(pluss::make_series({10.0, 20.0, 30.0}));

thread.show();
```

Output

```
[10.0, 20.0, 30.0]
```

16.1.55 size

Signature

```
std::int64_t size(self) const
```

Parameters

None

Returns

`std::int64_t` Integer count, size, or positional result.

Notes

This call is part of the public `Thread` API and is documented here using a concrete usage example.

Examples

```
auto thread = pluss::table::Thread(pluss::make_series({10.0, 20.0, 30.0}));

auto out = thread.size();
```

Output

```
3
```

16.1.56 slice

Signature

```
Thread slice(self, std::int64_t start, std::int64_t stop) const
```

Parameters

`start` : `std::int64_t` See the signature for the supported argument form.

`stop` : `std::int64_t` See the signature for the supported argument form.

Returns

`Thread` Value produced by this call.

Notes

Use this to narrow the current object to the rows or depth range you want to inspect next.

Examples

```
auto thread = pluss::table::Thread(pluss::make_series({10.0, 20.0, 30.0}));

auto out = thread.slice(0, 2);
```

Output

```
[10.0, 20.0, 30.0]
```

16.1.57 std

Signature

```
pluss::Series std(self, bool unbiased = true) const
```

Parameters

`unbiased` : `bool` See the signature for the supported argument form.

Returns

`pluss::Series` Value produced by this call.

Notes

Use this when you want a summary view instead of carrying the full input forward unchanged.

Examples

```
auto thread = pluss::table::Thread(pluss::make_series({10.0, 20.0, 30.0}));

auto out = thread.std(true);
```

Output

```
20.0
```

16.1.58 sum

Signature

```
pluss::Series sum(self) const
```

Parameters

None

Returns

`pluss::Series` Value produced by this call.

Notes

Use this when you want a summary view instead of carrying the full input forward unchanged.

Examples

```
auto thread = pluss::table::Thread(pluss::make_series({10.0, 20.0, 30.0}));

auto out = thread.sum();
```

Output

```
20.0
```

16.1.59 tail

Signature

```
Thread tail(self, std::int64_t n) const
```

Parameters

n : std::int64_t See the signature for the supported argument form.

Returns

Thread Value produced by this call.

Notes

Use this to narrow the current object to the rows or depth range you want to inspect next.

Examples

```
auto thread = pluss::table::Thread(pluss::make_series({10.0, 20.0, 30.0}));

auto out = thread.tail(2);
```

Output

```
[10.0, 20.0, 30.0]
```

16.1.60 take

Signature

```
Thread take(self, std::span<const std::int64_t> indices) const
```

Parameters

indices : std::span<const std::int64_t> See the signature for the supported argument form.

Returns

Thread Value produced by this call.

Notes

Use this to narrow the current object to the rows or depth range you want to inspect next.

Examples

```
auto thread = pluss::table::Thread(pluss::make_series({10.0, 20.0, 30.0}));

std::vector<std::int64_t> picks{0, 2};
auto out = thread.take(picks);
```

Output

```
[10.0, 20.0, 30.0]
```

16.1.61 timeindex

Signature

```
const pluss::TimeIndex& timeindex(self) const
```

Parameters

None

Returns

const pluss::TimeIndex& Value produced by this call.

Notes

This call is part of the public Thread API and is documented here using a concrete usage example.

Examples

```
auto thread = pluss::table::Thread(pluss::make_timeline({"2024-01-01T09:10:00Z", "2024-
↪02-01T11:00:00Z", "2024-03-05T15:30:00Z"}));

auto out = thread.timeindex();
```

Output

```
["2024-01-01T09:10:00Z", "2024-02-01T11:00:00Z", "2024-03-05T15:30:00Z"]
```

16.1.62 timeline

Signature

```
const pluss::Timeline& timeline(self) const
```

Parameters

None

Returns

`const pluss::Timeline&` Value produced by this call.

Notes

This call is part of the public `Thread` API and is documented here using a concrete usage example.

Examples

```
auto thread = pluss::table::Thread(pluss::make_timeline({"2024-01-01T09:10:00Z", "2024-
↪02-01T11:00:00Z", "2024-03-05T15:30:00Z"}));

auto out = thread.timeline();
```

Output

```
["2024-01-01T09:10:00Z", "2024-02-01T11:00:00Z", "2024-03-05T15:30:00Z"]
```

16.1.63 to_note_weekday

Signature

```
Thread to_note_weekday(self) const
```

Parameters

None

Returns

`Thread` Value produced by this call.

Notes

This is part of the time-aware surface and expects timeline or time-index semantics.

Examples

```
auto thread = pluss::table::Thread(pluss::make_timeline({"2024-01-01T09:10:00Z", "2024-
↪02-01T11:00:00Z", "2024-03-05T15:30:00Z"}));

auto out = thread.to_note_weekday();
```

Output

```
[1, 2, 3]
```

16.1.64 to_string

Signature

```
std::string to_string(self, const display::DisplayOptions& options =
```

Parameters

`options` : `const display::DisplayOptions&` See the signature for the supported argument form.

Returns

`std::string` Rendered string result.

Notes

This call is part of the public `Thread` API and is documented here using a concrete usage example.

Examples

```
auto thread = pluss::table::Thread(pluss::make_series({10.0, 20.0, 30.0}));

auto out = thread.to_string();
```

Output

```
[10.0, 20.0, 30.0]
```

16.1.65 unique

Signature

```
bool unique(self) const
```

Parameters

None

Returns

`bool` Boolean status or membership result.

Notes

This call is part of the public `Thread` API and is documented here using a concrete usage example.

Examples

```
auto thread = pluss::table::Thread(pluss::make_series({10.0, 20.0, 30.0}));

auto out = thread.unique();
```

Output

```
true
```

16.1.66 validity_mask

Signature

```
const std::optional<pluss::Series>& validity_mask(self) const
```

Parameters

None

Returns

const std::optional<pluss::Series>& Value produced by this call.

Notes

This call is part of the public Thread API and is documented here using a concrete usage example.

Examples

```
auto thread = pluss::table::Thread(pluss::make_series({10.0, 20.0, 30.0}));

auto out = thread.validity_mask();
```

16.1.67 value

Signature

```
ThreadValue value(self) const
```

Parameters

None

Returns

ThreadValue Value produced by this call.

Notes

This call is part of the public Thread API and is documented here using a concrete usage example.

Examples

```
auto thread = pluss::table::Thread(pluss::make_series({10.0, 20.0, 30.0}));

auto out = thread.value();
```

Output

```
10.0
```

16.1.68 values

Signature

```
const Thread& values(self) const
```

Parameters

None

Returns

const Thread& Value produced by this call.

Notes

This call is part of the public Thread API and is documented here using a concrete usage example.

Examples

```
auto thread = pluss::table::Thread(pluss::make_series({10.0, 20.0, 30.0}));

auto out = thread.values();
```

Output

```
[10.0, 20.0, 30.0]
```

16.1.69 var

Signature

```
pluss::Series var(self, bool unbiased = true) const
```

Parameters

unbiased : bool See the signature for the supported argument form.

Returns

pluss::Series Value produced by this call.

Notes

Use this when you want a summary view instead of carrying the full input forward unchanged.

Examples

```
auto thread = pluss::table::Thread(pluss::make_series({10.0, 20.0, 30.0}));

auto out = thread.var(true);
```

Output

```
20.0
```

16.1.70 window

Signature

```
Thread window(self, std::int64_t window, ThreadWindowOp op, std::int64_t min_periods =␣
↪1) const
```

Parameters

`window` : `std::int64_t` See the signature for the supported argument form.

`op` : `ThreadWindowOp` See the signature for the supported argument form.

`min_periods` : `std::int64_t` See the signature for the supported argument form.

Returns

`Thread` Value produced by this call.

Notes

This call is part of the public `Thread` API and is documented here using a concrete usage example.

Examples

```
auto thread = pluss::table::Thread(pluss::make_series({10.0, 20.0, 30.0}));

auto out = thread.window(2, pluss::table::ThreadWindowOp::RollingMean, 1);
```

Output

```
[10.0, 20.0, 30.0]
```

16.1.71 with_validity_mask

Signature

```
Thread with_validity_mask(self, const pluss::Series& validity_mask) const
```

Parameters

validity_mask : const pluss::Series& See the signature for the supported argument form.

Returns

Thread Value produced by this call.

Notes

This mutates or derives structure while keeping the surrounding object model intact.

Examples

```
auto mask = pluss::make_series({true, false, true}, pluss::codem::ScalarType::Bool);
auto thread = pluss::table::Thread(pluss::make_series({10.0, 20.0, 30.0}));

auto out = thread.with_validity_mask(mask);
```

16.1.72 with_validity_mask_

Signature

```
Thread& with_validity_mask_(self, const pluss::Series& validity_mask)
```

Parameters

validity_mask : const pluss::Series& See the signature for the supported argument form.

Returns

Thread& Value produced by this call.

Notes

This mutates or derives structure while keeping the surrounding object model intact.

Examples

```
auto mask = pluss::make_series({true, false, true}, pluss::codem::ScalarType::Bool);
auto thread = pluss::table::Thread(pluss::make_series({10.0, 20.0, 30.0}));

thread.with_validity_mask_(mask);
```

16.1.73 year

Signature

```
Thread year(self) const
```

Parameters

None

Returns

Thread Value produced by this call.

Notes

This is part of the time-aware surface and expects timeline or time-index semantics.

Examples

```
auto thread = pluss::table::Thread(pluss::make_timeline({"2024-01-01T09:10:00Z", "2024-
↪02-01T11:00:00Z", "2024-03-05T15:30:00Z"}));

auto out = thread.year();
```

Output

```
[2024, 2024, 2024]
```

CHAPTER

SEVENTEEN

THREADVALUE API REFERENCE

CHAPTER

EIGHTEEN

PRACTICAL THREADVALUE API REFERENCE

18.1 API Reference Style

This page documents the public scalar-like `ThreadValue` wrapper returned by thread and column-value accessors.

18.1.1 as_note

Signature

```
pluss::Note as_note(self) const
```

Parameters

None

Returns

`pluss::Note` Value produced by this call.

Notes

This call is part of the public `ThreadValue` API and is documented here using a concrete usage example.

Examples

```
auto thread = pluss::table::Thread(pluss::make_note({"Accra"}));
auto value = thread.value();

auto out = value.as_note();
```

Output

```
["Accra"]
```

18.1.2 as_series

Signature

```
pluss::Series as_series(self) const
```

Parameters

None

Returns

`pluss::Series` Value produced by this call.

Notes

This call is part of the public `ThreadValue` API and is documented here using a concrete usage example.

Examples

```
auto thread = pluss::table::Thread(pluss::make_series({10.0}));
auto value = thread.value();

auto out = value.as_series();
```

Output

```
[10.0]
```

18.1.3 as_timeindex

Signature

```
pluss::TimeIndex as_timeindex(self) const
```

Parameters

None

Returns

`pluss::TimeIndex` Value produced by this call.

Notes

This call is part of the public `ThreadValue` API and is documented here using a concrete usage example.

Examples

```
auto thread = pluss::table::Thread(pluss::make_timeindex({"2024-01-01T09:10:00Z"}));
auto value = thread.value();

auto out = value.as_timeindex();
```

Output

```
["2024-01-01T09:10:00Z"]
```

18.1.4 as_timeline

Signature

```
pluss::Timeline as_timeline(self) const
```

Parameters

None

Returns

pluss::Timeline Value produced by this call.

Notes

This call is part of the public ThreadValue API and is documented here using a concrete usage example.

Examples

```
auto thread = pluss::table::Thread(pluss::make_timeline({"2024-01-01T09:10:00Z"}));
auto value = thread.value();

auto out = value.as_timeline();
```

Output

```
["2024-01-01T09:10:00Z"]
```

18.1.5 defined

Signature

```
bool defined(self) const
```

Parameters

None

Returns

bool Boolean status or membership result.

Notes

This call is part of the public ThreadValue API and is documented here using a concrete usage example.

Examples

```
auto thread = pluss::table::Thread(pluss::make_series({10.0}));
auto value = thread.value();

auto out = value.defined();
```

Output

```
true
```

18.1.6 kind

Signature

```
ThreadKind kind(self) const
```

Parameters

None

Returns

ThreadKind Value produced by this call.

Notes

This call is part of the public ThreadValue API and is documented here using a concrete usage example.

Examples

```
auto thread = pluss::table::Thread(pluss::make_series({10.0}));
auto value = thread.value();

auto out = value.kind();
```

Output

```
Series
```

18.1.7 pluss::Note

Signature

```
operator pluss::Note(self) const
```

Parameters

None

Returns

operator Value produced by this call.

Notes

This call is part of the public ThreadValue API and is documented here using a concrete usage example.

Examples

```
auto thread = pluss::table::Thread(pluss::make_series({10.0}));
auto value = thread.value();

auto out = value.pluss::Note();
```

18.1.8 pluss::Series

Signature

```
operator pluss::Series(self) const
```

Parameters

None

Returns

operator Value produced by this call.

Notes

This call is part of the public ThreadValue API and is documented here using a concrete usage example.

Examples

```
auto thread = pluss::table::Thread(pluss::make_series({10.0}));
auto value = thread.value();

auto out = value.pluss::Series();
```

18.1.9 pluss::TimeIndex

Signature

```
operator pluss::TimeIndex(self) const
```

Parameters

None

Returns

operator Value produced by this call.

Notes

This call is part of the public ThreadValue API and is documented here using a concrete usage example.

Examples

```
auto thread = pluss::table::Thread(pluss::make_series({10.0}));
auto value = thread.value();

auto out = value.pluss::TimeIndex();
```

18.1.10 pluss::Timeline

Signature

```
operator pluss::Timeline(self) const
```

Parameters

None

Returns

operator Value produced by this call.

Notes

This call is part of the public ThreadValue API and is documented here using a concrete usage example.

Examples

```
auto thread = pluss::table::Thread(pluss::make_series({10.0}));
auto value = thread.value();

auto out = value.pluss::Timeline();
```

18.1.11 thread

Signature

```
const Thread& thread(self) const
```

Parameters

None

Returns

const Thread& Value produced by this call.

Notes

This call is part of the public ThreadValue API and is documented here using a concrete usage example.

Examples

```
auto thread = pluss::table::Thread(pluss::make_series({10.0}));
auto value = thread.value();

auto out = value.thread();
```

Output

```
[10.0]
```

CHAPTER

NINETEEN

ATLAS API REFERENCE

CHAPTER

TWENTY

PRACTICAL ATLAS API REFERENCE

20.1 API Reference Style

This page documents the full public `Atlas` surface for frame-stack workflows.

20.1.1 aggregate_over_frames

Signature

```
DataFrame aggregate_over_frames(self, const std::function<DataFrame(const std::vector
↪<DataFrame>&)>& fn) const
```

Parameters

`fn` : `const std::function<DataFrame(const std::vector<DataFrame>&)>&` See the signature for the supported argument form.

Returns

`DataFrame` Value produced by this call.

Notes

Use this when you want a summary view instead of carrying the full input forward unchanged.

Examples

```
auto f0 = pluss::table::DataFrame({
    {"sales", pluss::table::Thread(pluss::make_series({10.0, 20.0}))},
});
auto f1 = pluss::table::DataFrame({
    {"sales", pluss::table::Thread(pluss::make_series({30.0, 40.0}))},
});
auto atlas = pluss::table::Atlas({f0, f1});

auto out = atlas.aggregate_over_frames([](const std::vector<pluss::table::DataFrame>&␣
↪frames) { return frames.front(); });
```

Output

```
sales
10.0
20.0
```

20.1.2 align_depth

Signature

```
Atlas align_depth(self, const Thread& labels) const
```

Parameters

`labels` : `const Thread&` See the signature for the supported argument form.

Returns

`Atlas` Value produced by this call.

Notes

This call is part of the public `Atlas` API and is documented here using a concrete usage example.

Examples

```
auto f0 = pluss::table::DataFrame({
    {"sales", pluss::table::Thread(pluss::make_series({10.0, 20.0}))},
});
auto f1 = pluss::table::DataFrame({
    {"sales", pluss::table::Thread(pluss::make_series({30.0, 40.0}))},
});
auto atlas = pluss::table::Atlas({f0, f1});

auto labels = pluss::table::Thread(pluss::make_note({"baseline", "stress"}));
auto out = atlas.align_depth(labels);
```

Output

```
frame0:
sales
10.0
20.0

frame1:
sales
30.0
40.0
```

20.1.3 axis0_labels

Signature

```
const std::optional<Thread>& axis0_labels(self) const
```

Parameters

None

Returns

const std::optional<Thread>& Value produced by this call.

Notes

This call is part of the public Atlas API and is documented here using a concrete usage example.

Examples

```
auto f0 = pluss::table::DataFrame({
    {"sales", pluss::table::Thread(pluss::make_series({10.0, 20.0}))},
});
auto f1 = pluss::table::DataFrame({
    {"sales", pluss::table::Thread(pluss::make_series({30.0, 40.0}))},
});
auto atlas = pluss::table::Atlas({f0, f1});

auto out = atlas.axis0_labels();
```

Output

```
["train", "test"]
```

20.1.4 column_names

Signature

```
const std::vector<std::string>& column_names(self) const
```

Parameters

None

Returns

const std::vector<std::string>& Borrowed list of names.

Notes

This call is part of the public Atlas API and is documented here using a concrete usage example.

Examples

```
auto f0 = pluss::table::DataFrame({
    {"sales", pluss::table::Thread(pluss::make_series({10.0, 20.0}))},
});
auto f1 = pluss::table::DataFrame({
    {"sales", pluss::table::Thread(pluss::make_series({30.0, 40.0}))},
});
auto atlas = pluss::table::Atlas({f0, f1});

auto out = atlas.column_names();
```

Output

```
["sales"]
```

20.1.5 defined

Signature

```
bool defined(self) const
```

Parameters

None

Returns

bool Boolean status or membership result.

Notes

This call is part of the public Atlas API and is documented here using a concrete usage example.

Examples

```
auto f0 = pluss::table::DataFrame({
    {"sales", pluss::table::Thread(pluss::make_series({10.0, 20.0}))},
});
auto f1 = pluss::table::DataFrame({
    {"sales", pluss::table::Thread(pluss::make_series({30.0, 40.0}))},
});
auto atlas = pluss::table::Atlas({f0, f1});

auto out = atlas.defined();
```

Output

```
true
```

20.1.6 depth

Signature

```
std::int64_t depth(self) const
```

Parameters

None

Returns

std::int64_t Integer count, size, or positional result.

Notes

This call is part of the public Atlas API and is documented here using a concrete usage example.

Examples

```
auto f0 = pluss::table::DataFrame({
    {"sales", pluss::table::Thread(pluss::make_series({10.0, 20.0}))},
});
auto f1 = pluss::table::DataFrame({
    {"sales", pluss::table::Thread(pluss::make_series({30.0, 40.0}))},
});
auto atlas = pluss::table::Atlas({f0, f1});

auto out = atlas.depth();
```

Output

```
2
```

20.1.7 empty

Signature

```
bool empty(self) const
```

Parameters

None

Returns

bool Boolean status or membership result.

Notes

This call is part of the public Atlas API and is documented here using a concrete usage example.

Examples

```
auto f0 = pluss::table::DataFrame({
    {"sales", pluss::table::Thread(pluss::make_series({10.0, 20.0}))},
});
auto f1 = pluss::table::DataFrame({
    {"sales", pluss::table::Thread(pluss::make_series({30.0, 40.0}))},
});
auto atlas = pluss::table::Atlas({f0, f1});

auto out = atlas.empty();
```

Output

```
false
```

20.1.8 `frame`

Signature

```
DataFrame frame(self, std::int64_t index) const
```

Parameters

`index` : `std::int64_t` See the signature for the supported argument form.

Returns

`DataFrame` Value produced by this call.

Notes

This call is part of the public `Atlas` API and is documented here using a concrete usage example.

Examples

```
auto f0 = pluss::table::DataFrame({
    {"sales", pluss::table::Thread(pluss::make_series({10.0, 20.0}))},
});
auto f1 = pluss::table::DataFrame({
    {"sales", pluss::table::Thread(pluss::make_series({30.0, 40.0}))},
});
auto atlas = pluss::table::Atlas({f0, f1});

auto out = atlas.frame(0);
```

Output

```
sales
10.0
20.0
```

20.1.9 frames

Signature

```
const std::vector<DataFrame>& frames(self) const
```

Parameters

None

Returns

const std::vector<DataFrame>& Borrowed collection of frames.

Notes

This call is part of the public Atlas API and is documented here using a concrete usage example.

Examples

```
auto f0 = pluss::table::DataFrame({
    {"sales", pluss::table::Thread(pluss::make_series({10.0, 20.0}))},
});
auto f1 = pluss::table::DataFrame({
    {"sales", pluss::table::Thread(pluss::make_series({30.0, 40.0}))},
});
auto atlas = pluss::table::Atlas({f0, f1});

auto out = atlas.frames();
```

Output

```
[frame0, frame1]
```

20.1.10 head

Signature

```
Atlas head(self, std::int64_t n = 5) const
```

Parameters

n : std::int64_t See the signature for the supported argument form.

Returns

Atlas Value produced by this call.

Notes

Use this to narrow the current object to the rows or depth range you want to inspect next.

Examples

```
auto f0 = pluss::table::DataFrame({
    {"sales", pluss::table::Thread(pluss::make_series({10.0, 20.0}))},
});
auto f1 = pluss::table::DataFrame({
    {"sales", pluss::table::Thread(pluss::make_series({30.0, 40.0}))},
});
auto atlas = pluss::table::Atlas({f0, f1});

auto out = atlas.head(1);
```

Output

```
frame0:
sales
10.0
20.0

frame1:
sales
30.0
40.0
```

20.1.11 iloc

Signature

```
DataFrame iloc(self, std::int64_t index) const
```

Parameters

`index` : `std::int64_t` See the signature for the supported argument form.

Returns

`DataFrame` Value produced by this call.

Notes

Use this to narrow the current object to the rows or depth range you want to inspect next.

Examples

```
auto f0 = pluss::table::DataFrame({
    {"sales", pluss::table::Thread(pluss::make_series({10.0, 20.0}))},
});
auto f1 = pluss::table::DataFrame({
    {"sales", pluss::table::Thread(pluss::make_series({30.0, 40.0}))},
});
auto atlas = pluss::table::Atlas({f0, f1});

auto out = atlas.iloc(0);
```

Output

```
sales
10.0
20.0
```

20.1.12 kindelize

Signature

```
pluss::kindle::Archive kindelize(self) const
```

Parameters

None

Returns

`pluss::kindle::Archive` Value produced by this call.

Notes

This is an explicit bridge between PlussMic models so you can switch to the representation that fits the next task.

See also

- `pluss::kindle::Archive kindelize(self, const pluss::kindle::KindelizePolicy& policy) const`

Examples

```
auto f0 = pluss::table::DataFrame({
    {"sales", pluss::table::Thread(pluss::make_series({10.0, 20.0}))},
});
auto f1 = pluss::table::DataFrame({
    {"sales", pluss::table::Thread(pluss::make_series({30.0, 40.0}))},
});
auto atlas = pluss::table::Atlas({f0, f1});

auto out = atlas.kindelize();
```

Output

```
train -> Catalog(metric -> Series([10.0, 20.0]))
test  -> Catalog(metric -> Series([30.0, 40.0]))
```

20.1.13 kindelize

Signature

```
pluss::kindle::Archive kindelize(self, const pluss::kindle::KindelizePolicy& policy)␣
↪const
```

Parameters

`policy` : `const pluss::kindle::KindelizePolicy&` See the signature for the supported argument form.

Returns

`pluss::kindle::Archive` Value produced by this call.

Notes

This is an explicit bridge between PlussMic models so you can switch to the representation that fits the next task.

See also

- `pluss::kindle::Archive kindelize(self) const`

Examples

```
auto f0 = pluss::table::DataFrame({
    {"sales", pluss::table::Thread(pluss::make_series({10.0, 20.0}))},
});
auto f1 = pluss::table::DataFrame({
    {"sales", pluss::table::Thread(pluss::make_series({30.0, 40.0}))},
});
auto atlas = pluss::table::Atlas({f0, f1});

auto out = atlas.kindelize();
```

Output

```
train -> Catalog(metric -> Series([10.0, 20.0]))
test  -> Catalog(metric -> Series([30.0, 40.0]))
```

20.1.14 `loc`

Signature

```
DataFrame loc(self, const pluss::Timeline& label) const
```

Parameters

`label` : `const pluss::Timeline&` See the signature for the supported argument form.

Returns

`DataFrame` Value produced by this call.

Notes

Use this to narrow the current object to the rows or depth range you want to inspect next.

See also

- DataFrame loc(self, std::string_view label) const
- DataFrame loc(self, std::int64_t label) const

Examples

```
auto f0 = pluss::table::DataFrame({
    {"sales", pluss::table::Thread(pluss::make_series({10.0, 20.0}))},
});
auto f1 = pluss::table::DataFrame({
    {"sales", pluss::table::Thread(pluss::make_series({30.0, 40.0}))},
});
auto atlas = pluss::table::Atlas({f0, f1});

auto labeled = pluss::table::Atlas({f0, f1}, pluss::table::Thread(pluss::make_note({
↪"train", "test"})));
auto out = labeled.loc("test");
```

Output

```
sales
10.0
20.0
```

20.1.15 loc

Signature

```
DataFrame loc(self, std::int64_t label) const
```

Parameters

label : std::int64_t See the signature for the supported argument form.

Returns

DataFrame Value produced by this call.

Notes

Use this to narrow the current object to the rows or depth range you want to inspect next.

See also

- DataFrame loc(self, std::string_view label) const
- DataFrame loc(self, const pluss::Timeline& label) const

Examples

```
auto f0 = pluss::table::DataFrame({
    {"sales", pluss::table::Thread(pluss::make_series({10.0, 20.0}))},
});
```

(continues on next page)

(continued from previous page)

```
auto f1 = pluss::table::DataFrame({
    {"sales", pluss::table::Thread(pluss::make_series({30.0, 40.0}))},
});
auto atlas = pluss::table::Atlas({f0, f1});

auto labeled = pluss::table::Atlas({f0, f1}, pluss::table::Thread(pluss::make_note({
↪"train", "test"})));
auto out = labeled.loc("test");
```

Output

```
sales
10.0
20.0
```

20.1.16 loc

Signature

```
DataFrame loc(self, std::string_view label) const
```

Parameters

`label` : `std::string_view` See the signature for the supported argument form.

Returns

`DataFrame` Value produced by this call.

Notes

Use this to narrow the current object to the rows or depth range you want to inspect next.

See also

- `DataFrame loc(self, std::int64_t label) const`
- `DataFrame loc(self, const pluss::Timeline& label) const`

Examples

```
auto f0 = pluss::table::DataFrame({
    {"sales", pluss::table::Thread(pluss::make_series({10.0, 20.0}))},
});
auto f1 = pluss::table::DataFrame({
    {"sales", pluss::table::Thread(pluss::make_series({30.0, 40.0}))},
});
auto atlas = pluss::table::Atlas({f0, f1});

auto labeled = pluss::table::Atlas({f0, f1}, pluss::table::Thread(pluss::make_note({
↪"train", "test"})));
auto out = labeled.loc("test");
```

Output

```
sales
10.0
20.0
```

20.1.17 map_frames

Signature

```
Atlas map_frames(self, const std::function<DataFrame(const DataFrame&)>& fn) const
```

Parameters

`fn` : `const std::function<DataFrame(const DataFrame&)>&` See the signature for the supported argument form.

Returns

`Atlas` Value produced by this call.

Notes

This mutates or derives structure while keeping the surrounding object model intact.

Examples

```
auto f0 = pluss::table::DataFrame({
    {"sales", pluss::table::Thread(pluss::make_series({10.0, 20.0}))},
});
auto f1 = pluss::table::DataFrame({
    {"sales", pluss::table::Thread(pluss::make_series({30.0, 40.0}))},
});
auto atlas = pluss::table::Atlas({f0, f1});

auto out = atlas.map_frames([](const pluss::table::DataFrame& frame) { return frame.
↪head(1, false); });
```

Output

```
frame0:
sales
10.0
20.0

frame1:
sales
30.0
40.0
```

20.1.18 ncols

Signature

```
std::int64_t ncols(self) const
```

Parameters

None

Returns

std::int64_t Integer count, size, or positional result.

Notes

This call is part of the public Atlas API and is documented here using a concrete usage example.

Examples

```
auto f0 = pluss::table::DataFrame({
    {"sales", pluss::table::Thread(pluss::make_series({10.0, 20.0}))},
});
auto f1 = pluss::table::DataFrame({
    {"sales", pluss::table::Thread(pluss::make_series({30.0, 40.0}))},
});
auto atlas = pluss::table::Atlas({f0, f1});

auto out = atlas.ncols();
```

Output

```
2
```

20.1.19 nrows

Signature

```
std::int64_t nrows(self) const
```

Parameters

None

Returns

std::int64_t Integer count, size, or positional result.

Notes

This call is part of the public Atlas API and is documented here using a concrete usage example.

Examples

```
auto f0 = pluss::table::DataFrame({
    {"sales", pluss::table::Thread(pluss::make_series({10.0, 20.0}))},
});
auto f1 = pluss::table::DataFrame({
    {"sales", pluss::table::Thread(pluss::make_series({30.0, 40.0}))},
});
auto atlas = pluss::table::Atlas({f0, f1});

auto out = atlas.nrows();
```

Output

```
2
```

20.1.20 plot

Signature

```
DataFramePlotAccessor plot(self, std::int64_t frame_index = 0) const
```

Parameters

`frame_index` : `std::int64_t` See the signature for the supported argument form.

Returns

`DataFramePlotAccessor` Value produced by this call.

Notes

This opens the plotting accessor for the current data object.

Examples

```
auto f0 = pluss::table::DataFrame({
    {"sales", pluss::table::Thread(pluss::make_series({10.0, 20.0}))},
});
auto f1 = pluss::table::DataFrame({
    {"sales", pluss::table::Thread(pluss::make_series({30.0, 40.0}))},
});
auto atlas = pluss::table::Atlas({f0, f1});

auto fig = atlas.plot(0).line("sales");
```

Output

```
Figure(line)
```

20.1.21 relabel_depth

Signature

```
Atlas relabel_depth(self, const Thread& labels) const
```

Parameters

`labels` : `const Thread&` See the signature for the supported argument form.

Returns

`Atlas` Value produced by this call.

Notes

This adjusts labels without changing the underlying payload values.

Examples

```
auto f0 = pluss::table::DataFrame({
    {"sales", pluss::table::Thread(pluss::make_series({10.0, 20.0}))},
});
auto f1 = pluss::table::DataFrame({
    {"sales", pluss::table::Thread(pluss::make_series({30.0, 40.0}))},
});
auto atlas = pluss::table::Atlas({f0, f1});

auto labels = pluss::table::Thread(pluss::make_note({"baseline", "stress"}));
auto out = atlas.relabel_depth(labels);
```

Output

```
frame0:
sales
10.0
20.0

frame1:
sales
30.0
40.0
```

20.1.22 show

Signature

```
void show(self, const display::DisplayOptions& options =
```

Parameters

`options` : `const display::DisplayOptions&` See the signature for the supported argument form.

Returns

`void` Performs the side effect and returns no value.

Notes

This renders a human-readable view for inspection instead of creating a new analytic object.

See also

- `void show(self, std::ostream& os, const display::DisplayOptions& options =`

Examples

```
auto f0 = pluss::table::DataFrame({
    {"sales", pluss::table::Thread(pluss::make_series({10.0, 20.0}))},
});
auto f1 = pluss::table::DataFrame({
    {"sales", pluss::table::Thread(pluss::make_series({30.0, 40.0}))},
});
auto atlas = pluss::table::Atlas({f0, f1});

atlas.show();
```

Output

```
frame0:
sales
10.0
20.0

frame1:
sales
30.0
40.0
```

20.1.23 show

Signature

```
void show(self, std::ostream& os, const display::DisplayOptions& options =
```

Parameters

`os` : `std::ostream&` See the signature for the supported argument form.

`options` : `const display::DisplayOptions&` See the signature for the supported argument form.

Returns

`void` Performs the side effect and returns no value.

Notes

This renders a human-readable view for inspection instead of creating a new analytic object.

See also

- `void show(self, const display::DisplayOptions& options =`

Examples

```
auto f0 = pluss::table::DataFrame({
    {"sales", pluss::table::Thread(pluss::make_series({10.0, 20.0}))},
});
auto f1 = pluss::table::DataFrame({
    {"sales", pluss::table::Thread(pluss::make_series({30.0, 40.0}))},
});
auto atlas = pluss::table::Atlas({f0, f1});

atlas.show();
```

Output

```
frame0:
sales
10.0
20.0

frame1:
sales
30.0
40.0
```

20.1.24 slice_frames

Signature

```
Atlas slice_frames(self, std::int64_t start, std::int64_t stop) const
```

Parameters

`start` : `std::int64_t` See the signature for the supported argument form.

`stop` : `std::int64_t` See the signature for the supported argument form.

Returns

`Atlas` Value produced by this call.

Notes

Use this to narrow the current object to the rows or depth range you want to inspect next.

Examples

```
auto f0 = pluss::table::DataFrame({
    {"sales", pluss::table::Thread(pluss::make_series({10.0, 20.0}))},
});
auto f1 = pluss::table::DataFrame({
    {"sales", pluss::table::Thread(pluss::make_series({30.0, 40.0}))},
});
auto atlas = pluss::table::Atlas({f0, f1});

auto out = atlas.slice_frames(0, 1);
```

Output

```
frame0:
sales
10.0
20.0

frame1:
sales
30.0
40.0
```

20.1.25 `sort_depth`

Signature

```
Atlas sort_depth(self, bool ascending = true, bool nulls_last = true) const
```

Parameters

`ascending` : `bool` See the signature for the supported argument form.

`nulls_last` : `bool` See the signature for the supported argument form.

Returns

`Atlas` Value produced by this call.

Notes

This call is part of the public `Atlas` API and is documented here using a concrete usage example.

Examples

```
auto f0 = pluss::table::DataFrame({
    {"sales", pluss::table::Thread(pluss::make_series({10.0, 20.0}))},
});
auto f1 = pluss::table::DataFrame({
    {"sales", pluss::table::Thread(pluss::make_series({30.0, 40.0}))},
});
auto atlas = pluss::table::Atlas({f0, f1});
```

(continues on next page)

(continued from previous page)

```
auto labeled = pluss::table::Atlas({f0, f1}, pluss::table::Thread(pluss::make_note({"test
↪", "train"})));
auto out = labeled.sort_depth(true, true);
```

Output

```
frame0:
sales
10.0
20.0

frame1:
sales
30.0
40.0
```

20.1.26 squeeze

Signature

```
DataFrame squeeze(self) const
```

Parameters

None

Returns

`DataFrame` Value produced by this call.

Notes

This call is part of the public `Atlas` API and is documented here using a concrete usage example.

Examples

```
auto f0 = pluss::table::DataFrame({
    {"sales", pluss::table::Thread(pluss::make_series({10.0, 20.0}))},
});
auto f1 = pluss::table::DataFrame({
    {"sales", pluss::table::Thread(pluss::make_series({30.0, 40.0}))},
});
auto atlas = pluss::table::Atlas({f0, f1});

auto out = atlas.squeeze();
```

Output

```
sales
10.0
20.0
```

20.1.27 `tabelize`

Signature

```
static Atlas tabelize(const pluss::kindle::Archive& archive)
```

Parameters

`archive` : `const pluss::kindle::Archive&` See the signature for the supported argument form.

Returns

`Atlas` Value produced by this call.

Notes

This is an explicit bridge between PlussMic models so you can switch to the representation that fits the next task.

See also

- `static Atlas tabelize(const pluss::kindle::Archive& archive, const pluss::kindle::TabelizePolicy& policy)`

Examples

```
using namespace pluss::kindle::literals;
auto c0 = pluss::kindle::Catalog({"sales"_k = pluss::kindle::Stream(pluss::make_series(
↪{10.0, 20.0}))});
auto c1 = pluss::kindle::Catalog({"sales"_k = pluss::kindle::Stream(pluss::make_series(
↪{30.0, 40.0}))});
auto archive = pluss::kindle::Archive({c0, c1});
auto out = pluss::table::Atlas::tabelize(archive);
```

Output

```
frame0:
sales
10.0
20.0

frame1:
sales
30.0
40.0
```

20.1.28 `tabelize`

Signature

```
static Atlas tabelize(const pluss::kindle::Archive& archive, const␣
↪pluss::kindle::TabelizePolicy& policy)
```

Parameters

`archive` : `const pluss::kindle::Archive&` See the signature for the supported argument form.

`policy` : `const pluss::kindle::TabelizePolicy&` See the signature for the supported argument form.

Returns

`Atlas` Value produced by this call.

Notes

This is an explicit bridge between PlussMic models so you can switch to the representation that fits the next task.

See also

- `static Atlas tabelize(const pluss::kindle::Archive& archive)`

Examples

```
using namespace pluss::kindle::literals;
auto c0 = pluss::kindle::Catalog({"sales"_k = pluss::kindle::Stream(pluss::make_series(
↪{10.0, 20.0}))});
auto c1 = pluss::kindle::Catalog({"sales"_k = pluss::kindle::Stream(pluss::make_series(
↪{30.0, 40.0}))});
auto archive = pluss::kindle::Archive({c0, c1});
auto out = pluss::table::Atlas::tabelize(archive);
```

Output

```
frame0:
sales
10.0
20.0

frame1:
sales
30.0
40.0
```

20.1.29 `tail`

Signature

```
Atlas tail(self, std::int64_t n = 5) const
```

Parameters

`n` : `std::int64_t` See the signature for the supported argument form.

Returns

`Atlas` Value produced by this call.

Notes

Use this to narrow the current object to the rows or depth range you want to inspect next.

Examples

```
auto f0 = pluss::table::DataFrame({
    {"sales", pluss::table::Thread(pluss::make_series({10.0, 20.0}))},
});
auto f1 = pluss::table::DataFrame({
    {"sales", pluss::table::Thread(pluss::make_series({30.0, 40.0}))},
});
auto atlas = pluss::table::Atlas({f0, f1});

auto out = atlas.tail(1);
```

Output

```
frame0:
sales
10.0
20.0

frame1:
sales
30.0
40.0
```

20.1.30 `to_string`

Signature

```
std::string to_string(self, const display::DisplayOptions& options =
```

Parameters

`options` : `const display::DisplayOptions&` See the signature for the supported argument form.

Returns

`std::string` Rendered string result.

Notes

This call is part of the public `Atlas` API and is documented here using a concrete usage example.

Examples

```
auto f0 = pluss::table::DataFrame({
    {"sales", pluss::table::Thread(pluss::make_series({10.0, 20.0}))},
});
auto f1 = pluss::table::DataFrame({
    {"sales", pluss::table::Thread(pluss::make_series({30.0, 40.0}))},
```

(continues on next page)

(continued from previous page)

```
});
auto atlas = pluss::table::Atlas({f0, f1});

auto out = atlas.to_string();
```

Output

```
frame0:
sales
10.0
20.0

frame1:
sales
30.0
40.0
```

20.1.31 unique

Signature

```
bool unique(self) const
```

Parameters

None

Returns

bool Boolean status or membership result.

Notes

This call is part of the public Atlas API and is documented here using a concrete usage example.

Examples

```
auto f0 = pluss::table::DataFrame({
    {"sales", pluss::table::Thread(pluss::make_series({10.0, 20.0}))},
});
auto f1 = pluss::table::DataFrame({
    {"sales", pluss::table::Thread(pluss::make_series({30.0, 40.0}))},
});
auto atlas = pluss::table::Atlas({f0, f1});

auto out = atlas.unique();
```

Output

```
true
```

CHAPTER

TWENTYONE

KINDLE MODEL GUIDE

21.1 When To Use Kindle Types

The kindle model is stream-first:

- `Stream`: one typed vector plus optional validity mask and case labels
- `Catalog`: named collection of streams
- `Archive`: depth stack of catalogs

This is the right model when case alignment and stream-native transforms matter more than table presentation.

21.2 Stream Basics

```
auto metric = pluss::kindle::Stream(
    pluss::make_series({10.0, 20.0, 30.0}),
    std::nullopt,
    pluss::table::Thread(pluss::make_note({"cs0", "cs1", "cs2"})));

metric = metric.with_meta("source", "demo").with_meta("unit", "km");

auto cleaned = metric.fillna(0.0).clip(0.0, 35.0);
auto filtered = cleaned.filter(cleaned.as_series().gt(15.0));
```

21.3 Catalog Basics

```
using namespace pluss::kindle::literals;

auto id = pluss::kindle::Stream(pluss::make_series({1, 2, 3}));
auto city = pluss::kindle::Stream(pluss::make_note({"Accra", "Tema", "Kumasi"}));
auto value = pluss::kindle::Stream(pluss::make_series({10.0, 15.0, 20.0}));

auto catalog = pluss::kindle::Catalog({
    "id"_k = id,
    "city"_k = city,
    "value"_k = value,
});

catalog.select({"id", "value"}).show();
catalog.sort("value").show();
```

21.4 Text And Similarity Helpers

```
auto text_catalog = pluss::kindle::Catalog({
    "text"_k = pluss::kindle::Stream(pluss::make_note({
        "heavy rain on road",
        "clear weather downtown",
        "storm warning issued",
    })),
});

auto clean = text_catalog.text_preprocess("text");
auto keywords = text_catalog.extract_keywords("text", 5);
auto tfidf = text_catalog.tfidf("text");
```

21.5 Archive Basics

```
auto c0 = catalog;
auto c1 = catalog.rename({{"value", "value_test"}});
auto depth = pluss::kindle::Stream(pluss::make_note({"train", "test"}));

auto archive = pluss::kindle::Archive({c0, c1}, depth);

auto first = archive.catalog(0);
auto mapped = archive.map_catalogs([](const pluss::kindle::Catalog& c) {
  return c.head(2);
});
```

21.6 Convert Back To Table Model

```
pluss::kindle::TabelizePolicy policy;
policy.align = pluss::kindle::AlignPolicy::Outer;
policy.index_mode = pluss::kindle::IndexMode::Positional;

auto df = catalog.tabelize(policy);
auto atlas = archive.tabelize(policy);
```

CHAPTER

TWENTYTWO

STREAM API REFERENCE

CHAPTER

TWENTYTHREE

PRACTICAL STREAM API REFERENCE

23.1 API Reference Style

This page documents the full public `Stream` surface for the Kindle model.

23.1.1 abs

Signature

```
Stream abs(self) const
```

Parameters

None

Returns

`Stream` Value produced by this call.

Notes

This call is part of the public `Stream` API and is documented here using a concrete usage example.

Examples

```
auto stream = pluss::kindle::Stream(pluss::make_series({10.0, 20.0, 30.0}));

auto out = stream.abs();
```

Output

```
Series([10.0, 20.0, 30.0])
```

23.1.2 as_note

Signature

```
const pluss::Note& as_note(self) const
```

Parameters

None

Returns

`const pluss::Note&` Value produced by this call.

Notes

This call is part of the public `Stream` API and is documented here using a concrete usage example.

Examples

```
auto stream = pluss::kindle::Stream(pluss::make_note({"Accra", "Tema", "Kumasi"}));

auto out = stream.as_note();
```

Output

```
Note(["Accra", "Tema", "Kumasi"])
```

23.1.3 as_series

Signature

```
const pluss::Series& as_series(self) const
```

Parameters

None

Returns

`const pluss::Series&` Value produced by this call.

Notes

This call is part of the public `Stream` API and is documented here using a concrete usage example.

Examples

```
auto stream = pluss::kindle::Stream(pluss::make_series({10.0, 20.0, 30.0}));

auto out = stream.as_series();
```

Output

```
Series([10.0, 20.0, 30.0])
```

23.1.4 as_timeindex

Signature

```
const pluss::TimeIndex& as_timeindex(self) const
```

Parameters

None

Returns

const pluss::TimeIndex& Value produced by this call.

Notes

This call is part of the public Stream API and is documented here using a concrete usage example.

Examples

```
auto stream = pluss::kindle::Stream(pluss::make_timeindex({"2024-01-01T09:10:00Z", "2024-
↪01-01T09:40:00Z"}));

auto out = stream.as_timeindex();
```

Output

```
Timeline([...])
```

23.1.5 as_timeline

Signature

```
const pluss::Timeline& as_timeline(self) const
```

Parameters

None

Returns

const pluss::Timeline& Value produced by this call.

Notes

This call is part of the public Stream API and is documented here using a concrete usage example.

Examples

```
auto stream = pluss::kindle::Stream(pluss::make_timeline({"2024-01-01T09:10:00Z", "2024-
↪01-01T09:40:00Z", "2024-01-01T10:05:00Z"}));

auto out = stream.as_timeline();
```

Output

```
Timeline([...])
```

23.1.6 astype

Signature

```
Stream astype(self, pluss::codem::ScalarType dtype) const
```

Parameters

`dtype` : `pluss::codem::ScalarType` See the signature for the supported argument form.

Returns

`Stream` Value produced by this call.

Notes

This changes the semantic or numeric type so later operations route through the correct implementation path.

Examples

```
auto stream = pluss::kindle::Stream(pluss::make_series({10.0, 20.0, 30.0}));

auto out = stream.astype(pluss::codem::ScalarType::Float64);
```

Output

```
Series([10.0, 20.0, 30.0])
```

23.1.7 bfill

Signature

```
Stream bfill(self) const
```

Parameters

`None`

Returns

`Stream` Value produced by this call.

Notes

This is part of the missing-data cleanup path and is usually applied before summaries, grouping, or export.

Examples

```
auto stream = pluss::kindle::Stream(pluss::make_series({10.0, 20.0, 30.0}));

auto out = stream.bfill();
```

Output

```
Series([10.0, 20.0, 30.0])
```

23.1.8 cases

Signature

```
const std::optional<pluss::table::Thread>& cases(self) const
```

Parameters

None

Returns

const std::optional<pluss::table::Thread>& Value produced by this call.

Notes

This call is part of the public Stream API and is documented here using a concrete usage example.

Examples

```
auto stream = pluss::kindle::Stream(pluss::make_series({10.0, 20.0, 30.0}));

auto out = stream.cases();
```

Output

```
["cs0", "cs1", "cs2"]
```

23.1.9 cast

Signature

```
Stream cast(self, pluss::table::ThreadCastTarget target, std::optional
↪<pluss::codem::ScalarType> series_dtype = std::nullopt) const
```

Parameters

target : pluss::table::ThreadCastTarget See the signature for the supported argument form.

series_dtype : std::optional<pluss::codem::ScalarType> See the signature for the supported argument form.

Returns

`Stream` Value produced by this call.

Notes

This changes the semantic or numeric type so later operations route through the correct implementation path.

Examples

```
auto stream = pluss::kindle::Stream(pluss::make_note({"1.0", "2.0", "3.0"}));

auto out = stream.cast(pluss::table::ThreadCastTarget::Series,␣
↪pluss::codem::ScalarType::Float64);
```

Output

```
Series([10.0, 20.0, 30.0])
```

23.1.10 clear_cases

Signature

```
Stream clear_cases(self) const
```

Parameters

None

Returns

`Stream` Value produced by this call.

Notes

This call is part of the public `Stream` API and is documented here using a concrete usage example.

Examples

```
auto stream = pluss::kindle::Stream(pluss::make_series({10.0, 20.0, 30.0}));

auto out = stream.clear_cases();
```

Output

```
Series([10.0, 20.0, 30.0])
```

23.1.11 clear_cases_

Signature

```
Stream& clear_cases_(self)
```

Parameters

None

Returns

Stream& Value produced by this call.

Notes

This call is part of the public Stream API and is documented here using a concrete usage example.

Examples

```
auto stream = pluss::kindle::Stream(pluss::make_series({10.0, 20.0, 30.0}));

stream.clear_cases_();
```

23.1.12 clear_validity_mask

Signature

```
Stream clear_validity_mask(self) const
```

Parameters

None

Returns

Stream Value produced by this call.

Notes

This call is part of the public Stream API and is documented here using a concrete usage example.

Examples

```
auto stream = pluss::kindle::Stream(pluss::make_series({10.0, 20.0, 30.0}));

auto out = stream.clear_validity_mask();
```

Output

```
Series([10.0, 20.0, 30.0])
```

23.1.13 clear_validity_mask_

Signature

```
Stream& clear_validity_mask_(self)
```

Parameters

None

Returns

Stream& Value produced by this call.

Notes

This call is part of the public Stream API and is documented here using a concrete usage example.

Examples

```
auto stream = pluss::kindle::Stream(pluss::make_series({10.0, 20.0, 30.0}));

stream.clear_validity_mask_();
```

23.1.14 clip

Signature

```
Stream clip(self, scalar min_value, scalar max_value) const
```

Parameters

min_value : scalar See the signature for the supported argument form.

max_value : scalar See the signature for the supported argument form.

Returns

Stream Value produced by this call.

Notes

This call is part of the public Stream API and is documented here using a concrete usage example.

Examples

```
auto stream = pluss::kindle::Stream(pluss::make_series({10.0, 20.0, 30.0}));

auto out = stream.clip(0.0, 25.0);
```

Output

```
Series([10.0, 20.0, 30.0])
```

23.1.15 defined

Signature

```
bool defined(self) const
```

Parameters

None

Returns

bool Boolean status or membership result.

Notes

This call is part of the public Stream API and is documented here using a concrete usage example.

Examples

```
auto stream = pluss::kindle::Stream(pluss::make_series({10.0, 20.0, 30.0}));

auto out = stream.defined();
```

Output

```
true
```

23.1.16 drop_meta

Signature

```
Stream drop_meta(self, std::string_view key) const
```

Parameters

key : std::string_view See the signature for the supported argument form.

Returns

Stream Value produced by this call.

Notes

This call is part of the public Stream API and is documented here using a concrete usage example.

Examples

```
auto stream = pluss::kindle::Stream(pluss::make_series({10.0, 20.0, 30.0}));
stream = stream.with_meta("unit", "km");

auto out = stream.drop_meta("unit");
```

Output

```
Series([10.0, 20.0, 30.0])
```

23.1.17 drop_meta_

Signature

```
Stream& drop_meta_(self, std::string_view key)
```

Parameters

key : std::string_view See the signature for the supported argument form.

Returns

Stream& Value produced by this call.

Notes

This call is part of the public Stream API and is documented here using a concrete usage example.

Examples

```
auto stream = pluss::kindle::Stream(pluss::make_series({10.0, 20.0, 30.0}));
stream = stream.with_meta("unit", "km");

stream.drop_meta_("unit");
```

23.1.18 dropna

Signature

```
Stream dropna(self) const
```

Parameters

None

Returns

Stream Value produced by this call.

Notes

This is part of the missing-data cleanup path and is usually applied before summaries, grouping, or export.

Examples

```
auto stream = pluss::kindle::Stream(pluss::make_series({10.0, 20.0, 30.0}));

auto out = stream.dropna();
```

Output

```
Series([10.0, 20.0, 30.0])
```

23.1.19 dtype_name

Signature

```
std::string dtype_name(self) const
```

Parameters

None

Returns

std::string Rendered string result.

Notes

This call is part of the public Stream API and is documented here using a concrete usage example.

Examples

```
auto stream = pluss::kindle::Stream(pluss::make_series({10.0, 20.0, 30.0}));

auto out = stream.dtype_name();
```

Output

```
Float64
```

23.1.20 exp

Signature

```
Stream exp(self) const
```

Parameters

None

Returns

Stream Value produced by this call.

Notes

This call is part of the public Stream API and is documented here using a concrete usage example.

Examples

```
auto stream = pluss::kindle::Stream(pluss::make_series({10.0, 20.0, 30.0}));

auto out = stream.exp();
```

Output

```
Series([10.0, 20.0, 30.0])
```

23.1.21 ffill

Signature

```
Stream ffill(self) const
```

Parameters

None

Returns

Stream Value produced by this call.

Notes

This is part of the missing-data cleanup path and is usually applied before summaries, grouping, or export.

Examples

```
auto stream = pluss::kindle::Stream(pluss::make_series({10.0, 20.0, 30.0}));

auto out = stream.ffill();
```

Output

```
Series([10.0, 20.0, 30.0])
```

23.1.22 fillna

Signature

```
Stream fillna(self, const char* value) const
```

Parameters

value : const char* See the signature for the supported argument form.

Returns

Stream Value produced by this call.

Notes

This is part of the missing-data cleanup path and is usually applied before summaries, grouping, or export.

See also

- Stream fillna(self, scalar value) const
- Stream fillna(self, std::string_view value) const
- Stream fillna(self, const pluss::Timeline& value) const
- Stream fillna(self, const pluss::TimeIndex& value) const

Examples

```
auto stream = pluss::kindle::Stream(pluss::make_note({"Accra", "Tema", "Kumasi"}));

auto out = stream.fillna("Unknown");
```

Output

```
Note(["Accra", "Tema", "Kumasi"])
```

23.1.23 fillna

Signature

```
Stream fillna(self, const pluss::TimeIndex& value) const
```

Parameters

value : const pluss::TimeIndex& See the signature for the supported argument form.

Returns

Stream Value produced by this call.

Notes

This is part of the missing-data cleanup path and is usually applied before summaries, grouping, or export.

See also

- Stream fillna(self, scalar value) const
- Stream fillna(self, std::string_view value) const
- Stream fillna(self, const char* value) const
- Stream fillna(self, const pluss::Timeline& value) const

Examples

```
auto stream = pluss::kindle::Stream(pluss::make_timeindex({"2024-01-01T09:10:00Z", "2024-
↪01-01T09:40:00Z"}));

auto out = stream.fillna(pluss::make_timeindex("2024-01-01T00:00:00Z"));
```

Output

```
Note(["Accra", "Tema", "Kumasi"])
```

23.1.24 fillna

Signature

```
Stream fillna(self, const pluss::Timeline& value) const
```

Parameters

`value` : `const pluss::Timeline&` See the signature for the supported argument form.

Returns

`Stream` Value produced by this call.

Notes

This is part of the missing-data cleanup path and is usually applied before summaries, grouping, or export.

See also

- `Stream fillna(self, scalar value) const`
- `Stream fillna(self, std::string_view value) const`
- `Stream fillna(self, const char* value) const`
- `Stream fillna(self, const pluss::TimeIndex& value) const`

Examples

```
auto stream = pluss::kindle::Stream(pluss::make_timeline({"2024-01-01T09:10:00Z", "2024-
↪01-01T09:40:00Z", "2024-01-01T10:05:00Z"}));

auto out = stream.fillna(pluss::make_timeline("2024-01-01T00:00:00Z"));
```

Output

```
Note(["Accra", "Tema", "Kumasi"])
```

23.1.25 fillna

Signature

```
Stream fillna(self, scalar value) const
```

Parameters

`value` : `scalar` See the signature for the supported argument form.

Returns

`Stream` Value produced by this call.

Notes

This is part of the missing-data cleanup path and is usually applied before summaries, grouping, or export.

See also

- `Stream fillna(self, std::string_view value) const`
- `Stream fillna(self, const char* value) const`
- `Stream fillna(self, const pluss::Timeline& value) const`
- `Stream fillna(self, const pluss::TimeIndex& value) const`

Examples

```
auto stream = pluss::kindle::Stream(pluss::make_series({10.0, 20.0, 30.0}));

auto out = stream.fillna(0.0);
```

Output

```
Note(["Accra", "Tema", "Kumasi"])
```

23.1.26 `fillna`

Signature

```
Stream fillna(self, std::string_view value) const
```

Parameters

`value` : `std::string_view` See the signature for the supported argument form.

Returns

`Stream` Value produced by this call.

Notes

This is part of the missing-data cleanup path and is usually applied before summaries, grouping, or export.

See also

- `Stream fillna(self, scalar value) const`
- `Stream fillna(self, const char* value) const`
- `Stream fillna(self, const pluss::Timeline& value) const`
- `Stream fillna(self, const pluss::TimeIndex& value) const`

Examples

```
auto stream = pluss::kindle::Stream(pluss::make_note({"Accra", "Tema", "Kumasi"}));

auto out = stream.fillna("Unknown");
```

Output

```
Note(["Accra", "Tema", "Kumasi"])
```

23.1.27 filter

Signature

```
Stream filter(self, const pluss::Series& bool_mask) const
```

Parameters

bool_mask : const pluss::Series& See the signature for the supported argument form.

Returns

Stream Value produced by this call.

Notes

Use this to reduce the current object to the rows or fields that matter for the next step.

Examples

```
auto stream = pluss::kindle::Stream(pluss::make_series({10.0, 20.0, 30.0}));

auto mask = pluss::make_series({true, false, true}, pluss::codem::ScalarType::Bool);
auto out = stream.filter(mask);
```

Output

```
Series([10.0, 20.0, 30.0])
```

23.1.28 from_thread

Signature

```
static Stream from_thread(const pluss::table::Thread& thread, std::optional
↪<pluss::table::Thread> cases = std::nullopt)
```

Parameters

thread : const pluss::table::Thread& See the signature for the supported argument form.

cases : std::optional<pluss::table::Thread> See the signature for the supported argument form.

Returns

`Stream` Value produced by this call.

Notes

This call is part of the public `Stream` API and is documented here using a concrete usage example.

Examples

```cpp
auto thread = pluss::table::Thread(pluss::make_series({10.0, 20.0, 30.0}));

auto out = pluss::kindle::Stream::from_thread(thread);
```

Output

```
Series([10.0, 20.0, 30.0])
```

23.1.29 has_cases

Signature

```cpp
bool has_cases(self) const
```

Parameters

None

Returns

`bool` Boolean status or membership result.

Notes

This call is part of the public `Stream` API and is documented here using a concrete usage example.

Examples

```cpp
auto stream = pluss::kindle::Stream(pluss::make_series({10.0, 20.0, 30.0}));

auto out = stream.has_cases();
```

Output

```
true
```

23.1.30 has_meta

Signature

```cpp
bool has_meta(self, std::string_view key) const
```

Parameters

key : `std::string_view` See the signature for the supported argument form.

Returns

`bool` Boolean status or membership result.

Notes

This call is part of the public `Stream` API and is documented here using a concrete usage example.

Examples

```
auto stream = pluss::kindle::Stream(pluss::make_series({10.0, 20.0, 30.0}));

stream = stream.with_meta("unit", "km");
auto out = stream.has_meta("unit");
```

Output

```
true
```

23.1.31 has_validity_mask

Signature

```
bool has_validity_mask(self) const
```

Parameters

None

Returns

`bool` Boolean status or membership result.

Notes

This call is part of the public `Stream` API and is documented here using a concrete usage example.

Examples

```
auto stream = pluss::kindle::Stream(pluss::make_series({10.0, 20.0, 30.0}));

auto out = stream.has_validity_mask();
```

Output

```
true
```

23.1.32 head

Signature

```
Stream head(self, std::int64_t n) const
```

Parameters

n : std::int64_t See the signature for the supported argument form.

Returns

Stream Value produced by this call.

Notes

Use this to narrow the current object to the rows or depth range you want to inspect next.

Examples

```
auto stream = pluss::kindle::Stream(pluss::make_series({10.0, 20.0, 30.0}));

auto out = stream.head();
```

Output

```
Series([10.0, 20.0, 30.0])
```

23.1.33 isnull

Signature

```
Stream isnull(self) const
```

Parameters

None

Returns

Stream Value produced by this call.

Notes

This is part of the missing-data cleanup path and is usually applied before summaries, grouping, or export.

Examples

```
auto stream = pluss::kindle::Stream(pluss::make_series({10.0, 20.0, 30.0}));

auto out = stream.isnull();
```

Output

```
Series([10.0, 20.0, 30.0])
```

23.1.34 kind

Signature

```
StreamKind kind(self) const
```

Parameters

None

Returns

StreamKind Value produced by this call.

Notes

This call is part of the public Stream API and is documented here using a concrete usage example.

Examples

```
auto stream = pluss::kindle::Stream(pluss::make_series({10.0, 20.0, 30.0}));

auto out = stream.kind();
```

Output

```
Series
```

23.1.35 log

Signature

```
Stream log(self) const
```

Parameters

None

Returns

Stream Value produced by this call.

Notes

This call is part of the public Stream API and is documented here using a concrete usage example.

Examples

```
auto stream = pluss::kindle::Stream(pluss::make_series({10.0, 20.0, 30.0}));

auto out = stream.log();
```

Output

```
Series([10.0, 20.0, 30.0])
```

23.1.36 max

Signature

```
Stream max(self) const
```

Parameters

None

Returns

`Stream` Value produced by this call.

Notes

Use this when you want a summary view instead of carrying the full input forward unchanged.

Examples

```
auto stream = pluss::kindle::Stream(pluss::make_series({10.0, 20.0, 30.0}));

auto out = stream.max();
```

Output

```
Series([20.0])
```

23.1.37 mean

Signature

```
Stream mean(self) const
```

Parameters

None

Returns

`Stream` Value produced by this call.

Notes

Use this when you want a summary view instead of carrying the full input forward unchanged.

Examples

```
auto stream = pluss::kindle::Stream(pluss::make_series({10.0, 20.0, 30.0}));

auto out = stream.mean();
```

Output

```
Series([20.0])
```

23.1.38 meta

Signature

```
const std::unordered_map<std::string, std::string>& meta(self) const
```

Parameters

None

Returns

const std::unordered_map<std::string, std::string>& Value produced by this call.

Notes

This call is part of the public Stream API and is documented here using a concrete usage example.

See also

- std::optional<std::string> meta(self, std::string_view key) const

Examples

```
auto stream = pluss::kindle::Stream(pluss::make_series({10.0, 20.0, 30.0}));

stream = stream.with_meta("unit", "km");
auto out = stream.meta("unit");
```

Output

```
km
```

23.1.39 meta

Signature

```
std::optional<std::string> meta(self, std::string_view key) const
```

Parameters

key : `std::string_view` See the signature for the supported argument form.

Returns

`std::optional<std::string>` Value produced by this call.

Notes

This call is part of the public `Stream` API and is documented here using a concrete usage example.

See also

- `const std::unordered_map<std::string, std::string>& meta(self) const`

Examples

```
auto stream = pluss::kindle::Stream(pluss::make_series({10.0, 20.0, 30.0}));

stream = stream.with_meta("unit", "km");
auto out = stream.meta("unit");
```

Output

```
km
```

23.1.40 `min`

Signature

```
Stream min(self) const
```

Parameters

None

Returns

`Stream` Value produced by this call.

Notes

Use this when you want a summary view instead of carrying the full input forward unchanged.

Examples

```
auto stream = pluss::kindle::Stream(pluss::make_series({10.0, 20.0, 30.0}));

auto out = stream.min();
```

Output

```
Series([20.0])
```

23.1.41 notnull

Signature

```
Stream notnull(self) const
```

Parameters

None

Returns

Stream Value produced by this call.

Notes

This is part of the missing-data cleanup path and is usually applied before summaries, grouping, or export.

Examples

```
auto stream = pluss::kindle::Stream(pluss::make_series({10.0, 20.0, 30.0}));

auto out = stream.notnull();
```

Output

```
Series([10.0, 20.0, 30.0])
```

23.1.42 show

Signature

```
void show(self, std::ostream& os) const
```

Parameters

os : std::ostream& See the signature for the supported argument form.

Returns

void Performs the side effect and returns no value.

Notes

This renders a human-readable view for inspection instead of creating a new analytic object.

Examples

```
auto stream = pluss::kindle::Stream(pluss::make_series({10.0, 20.0, 30.0}));

stream.show(std::cout);
```

Output

```
Series([10.0, 20.0, 30.0])
```

23.1.43 size

Signature

```
std::int64_t size(self) const
```

Parameters

None

Returns

`std::int64_t` Integer count, size, or positional result.

Notes

This call is part of the public `Stream` API and is documented here using a concrete usage example.

Examples

```
auto stream = pluss::kindle::Stream(pluss::make_series({10.0, 20.0, 30.0}));

auto out = stream.size();
```

Output

```
3
```

23.1.44 slice

Signature

```
Stream slice(self, std::int64_t start, std::int64_t stop) const
```

Parameters

`start` : `std::int64_t` See the signature for the supported argument form.

`stop` : `std::int64_t` See the signature for the supported argument form.

Returns

`Stream` Value produced by this call.

Notes

Use this to narrow the current object to the rows or depth range you want to inspect next.

Examples

```
auto stream = pluss::kindle::Stream(pluss::make_series({10.0, 20.0, 30.0}));

auto out = stream.slice(0, 2);
```

Output

```
Series([10.0, 20.0, 30.0])
```

23.1.45 `std`

Signature

```
Stream std(self, bool unbiased = true) const
```

Parameters

`unbiased` : `bool` See the signature for the supported argument form.

Returns

`Stream` Value produced by this call.

Notes

Use this when you want a summary view instead of carrying the full input forward unchanged.

Examples

```
auto stream = pluss::kindle::Stream(pluss::make_series({10.0, 20.0, 30.0}));

auto out = stream.std(true);
```

Output

```
Series([20.0])
```

23.1.46 `sum`

Signature

```
Stream sum(self) const
```

Parameters

None

Returns

Stream Value produced by this call.

Notes

Use this when you want a summary view instead of carrying the full input forward unchanged.

Examples

```
auto stream = pluss::kindle::Stream(pluss::make_series({10.0, 20.0, 30.0}));

auto out = stream.sum();
```

Output

```
Series([20.0])
```

23.1.47 tail

Signature

```
Stream tail(self, std::int64_t n) const
```

Parameters

n : std::int64_t See the signature for the supported argument form.

Returns

Stream Value produced by this call.

Notes

Use this to narrow the current object to the rows or depth range you want to inspect next.

Examples

```
auto stream = pluss::kindle::Stream(pluss::make_series({10.0, 20.0, 30.0}));

auto out = stream.tail();
```

Output

```
Series([10.0, 20.0, 30.0])
```

23.1.48 take

Signature

```
Stream take(self, std::span<const std::int64_t> indices) const
```

Parameters

indices : std::span<const std::int64_t> See the signature for the supported argument form.

Returns

Stream Value produced by this call.

Notes

Use this to narrow the current object to the rows or depth range you want to inspect next.

Examples

```
auto stream = pluss::kindle::Stream(pluss::make_series({10.0, 20.0, 30.0}));

std::vector<std::int64_t> picks{0, 2};
auto out = stream.take(picks);
```

Output

```
Series([10.0, 20.0, 30.0])
```

23.1.49 to_json

Signature

```
std::string to_json(self) const
```

Parameters

None

Returns

std::string Rendered string result.

Notes

This call is part of the public Stream API and is documented here using a concrete usage example.

Examples

```
auto stream = pluss::kindle::Stream(pluss::make_series({10.0, 20.0, 30.0}));

auto out = stream.to_json();
```

Output

```
Series([10.0, 20.0, 30.0])
```

23.1.50 to_string

Signature

```
std::string to_string(self) const
```

Parameters

None

Returns

`std::string` Rendered string result.

Notes

This call is part of the public `Stream` API and is documented here using a concrete usage example.

Examples

```
auto stream = pluss::kindle::Stream(pluss::make_series({10.0, 20.0, 30.0}));

auto out = stream.to_string();
```

Output

```
Series([10.0, 20.0, 30.0])
```

23.1.51 to_thread

Signature

```
pluss::table::Thread to_thread(self) const
```

Parameters

None

Returns

`pluss::table::Thread` Value produced by this call.

Notes

This is an explicit bridge between PlussMic models so you can switch to the representation that fits the next task.

See also

- pluss::table::Thread to_thread(self, std::int64_t start, std::int64_t stop) const

Examples

```
auto stream = pluss::kindle::Stream(pluss::make_series({10.0, 20.0, 30.0}));

auto out = stream.to_thread();
```

Output

```
[10.0, 20.0, 30.0]
```

23.1.52 to_thread

Signature

```
pluss::table::Thread to_thread(self, std::int64_t start, std::int64_t stop) const
```

Parameters

start : std::int64_t See the signature for the supported argument form.

stop : std::int64_t See the signature for the supported argument form.

Returns

pluss::table::Thread Value produced by this call.

Notes

This is an explicit bridge between PlussMic models so you can switch to the representation that fits the next task.

See also

- pluss::table::Thread to_thread(self) const

Examples

```
auto stream = pluss::kindle::Stream(pluss::make_series({10.0, 20.0, 30.0}));

auto out = stream.to_thread(0, 2);
```

Output

```
[10.0, 20.0, 30.0]
```

23.1.53 unique

Signature

```
bool unique(self) const
```

Parameters

None

Returns

bool Boolean status or membership result.

Notes

This call is part of the public Stream API and is documented here using a concrete usage example.

Examples

```
auto stream = pluss::kindle::Stream(pluss::make_series({10.0, 20.0, 30.0}));

auto out = stream.unique();
```

Output

```
true
```

23.1.54 validity_mask

Signature

```
const std::optional<pluss::Series>& validity_mask(self) const
```

Parameters

None

Returns

const std::optional<pluss::Series>& Value produced by this call.

Notes

This call is part of the public Stream API and is documented here using a concrete usage example.

Examples

```
auto stream = pluss::kindle::Stream(pluss::make_series({10.0, 20.0, 30.0}));

auto out = stream.validity_mask();
```

Output

```
[true, false, true]
```

23.1.55 var

Signature

```
Stream var(self, bool unbiased = true) const
```

Parameters

unbiased : bool See the signature for the supported argument form.

Returns

Stream Value produced by this call.

Notes

Use this when you want a summary view instead of carrying the full input forward unchanged.

Examples

```
auto stream = pluss::kindle::Stream(pluss::make_series({10.0, 20.0, 30.0}));

auto out = stream.var(true);
```

Output

```
Series([20.0])
```

23.1.56 with_cases

Signature

```
Stream with_cases(self, const pluss::table::Thread& cases) const
```

Parameters

cases : const pluss::table::Thread& See the signature for the supported argument form.

Returns

Stream Value produced by this call.

Notes

This mutates or derives structure while keeping the surrounding object model intact.

Examples

```
auto cases = pluss::table::Thread(pluss::make_note({"cs0", "cs1", "cs2"}));
auto stream = pluss::kindle::Stream(pluss::make_series({10.0, 20.0, 30.0}));

auto out = stream.with_cases(cases);
```

Output

```
Series([10.0, 20.0, 30.0])
```

23.1.57 with_cases_

Signature

```
Stream& with_cases_(self, const pluss::table::Thread& cases)
```

Parameters

`cases` : `const pluss::table::Thread&` See the signature for the supported argument form.

Returns

`Stream&` Value produced by this call.

Notes

This mutates or derives structure while keeping the surrounding object model intact.

Examples

```
auto cases = pluss::table::Thread(pluss::make_note({"cs0", "cs1", "cs2"}));
auto stream = pluss::kindle::Stream(pluss::make_series({10.0, 20.0, 30.0}));

stream.with_cases_(cases);
```

23.1.58 with_meta

Signature

```
Stream with_meta(self, std::initializer_list<std::pair<std::string_view, std::string_
↪view>> items) const
```

Parameters

`items` : `std::initializer_list<std::pair<std::string_view, std::string_view>>` See the signature for the supported argument form.

Returns

`Stream` Value produced by this call.

Notes

This mutates or derives structure while keeping the surrounding object model intact.

See also

- `Stream with_meta(self, std::string key, std::string value) const`

Examples

```
auto stream = pluss::kindle::Stream(pluss::make_series({10.0, 20.0, 30.0}));

auto out = stream.with_meta("unit", "km");
```

Output

```
Series([10.0, 20.0, 30.0])
```

23.1.59 with_meta

Signature

```
Stream with_meta(self, std::string key, std::string value) const
```

Parameters

key : std::string See the signature for the supported argument form.

value : std::string See the signature for the supported argument form.

Returns

Stream Value produced by this call.

Notes

This mutates or derives structure while keeping the surrounding object model intact.

See also

- Stream with_meta(self, std::initializer_list<std::pair<std::string_view, std::string_view>> items) const

Examples

```
auto stream = pluss::kindle::Stream(pluss::make_series({10.0, 20.0, 30.0}));

auto out = stream.with_meta("unit", "km");
```

Output

```
Series([10.0, 20.0, 30.0])
```

23.1.60 with_meta_

Signature

```
Stream& with_meta_(self, std::initializer_list<std::pair<std::string_view, std::string_
↪view>> items)
```

Parameters

`items` : `std::initializer_list<std::pair<std::string_view, std::string_view>>` See the signature for the supported argument form.

Returns

`Stream&` Value produced by this call.

Notes

This mutates or derives structure while keeping the surrounding object model intact.

See also

- `Stream& with_meta_(self, std::string key, std::string value)`

Examples

```
auto stream = pluss::kindle::Stream(pluss::make_series({10.0, 20.0, 30.0}));

stream.with_meta_("unit", "km");
```

23.1.61 `with_meta_`

Signature

```
Stream& with_meta_(self, std::string key, std::string value)
```

Parameters

`key` : `std::string` See the signature for the supported argument form.

`value` : `std::string` See the signature for the supported argument form.

Returns

`Stream&` Value produced by this call.

Notes

This mutates or derives structure while keeping the surrounding object model intact.

See also

- `Stream& with_meta_(self, std::initializer_list<std::pair<std::string_view, std::string_view>> items)`

Examples

```
auto stream = pluss::kindle::Stream(pluss::make_series({10.0, 20.0, 30.0}));

stream.with_meta_("unit", "km");
```

23.1.62 with_validity_mask

Signature

```
Stream with_validity_mask(self, const pluss::Series& mask) const
```

Parameters

mask : const pluss::Series& See the signature for the supported argument form.

Returns

Stream Value produced by this call.

Notes

This mutates or derives structure while keeping the surrounding object model intact.

Examples

```
auto mask = pluss::make_series({true, false, true}, pluss::codem::ScalarType::Bool);
auto stream = pluss::kindle::Stream(pluss::make_series({10.0, 20.0, 30.0}));

auto out = stream.with_validity_mask(mask);
```

Output

```
Series([10.0, 20.0, 30.0])
```

23.1.63 with_validity_mask_

Signature

```
Stream& with_validity_mask_(self, const pluss::Series& mask)
```

Parameters

mask : const pluss::Series& See the signature for the supported argument form.

Returns

Stream& Value produced by this call.

Notes

This mutates or derives structure while keeping the surrounding object model intact.

Examples

```
auto mask = pluss::make_series({true, false, true}, pluss::codem::ScalarType::Bool);
auto stream = pluss::kindle::Stream(pluss::make_series({10.0, 20.0, 30.0}));

stream.with_validity_mask_(mask);
```

CHAPTER

TWENTYFOUR

CATALOG API REFERENCE

CHAPTER

TWENTYFIVE

PRACTICAL CATALOG API REFERENCE

25.1 API Reference Style

This page documents the full public `Catalog` surface for named stream collections.

25.1.1 add_text_features

Signature

```
Catalog add_text_features(self, std::string_view field) const
```

Parameters

`field` : `std::string_view` See the signature for the supported argument form.

Returns

`Catalog` Value produced by this call.

Notes

This is part of the text-processing surface for note-backed or text-oriented data.

Examples

```
using namespace pluss::kindle::literals;

auto catalog = pluss::kindle::Catalog({
    "text"_k = pluss::kindle::Stream(pluss::make_note({
        "heavy rain on road",
        "clear weather downtown",
        "storm warning issued",
    })),
});

auto out = catalog.add_text_features("text");
```

Output

```
id     -> Series([1, 2, 3])
metric -> Series([10.0, 20.0, 30.0])
city   -> Note(["Accra", "Tema", "Kumasi"])
```

25.1.2 aggregate_text_stats

Signature

```
Catalog aggregate_text_stats(self, std::string_view field, std::string_view stat_type)␣
↪const
```

Parameters

`field` : `std::string_view` See the signature for the supported argument form.

`stat_type` : `std::string_view` See the signature for the supported argument form.

Returns

`Catalog` Value produced by this call.

Notes

This is part of the text-processing surface for note-backed or text-oriented data.

Examples

```
using namespace pluss::kindle::literals;

auto catalog = pluss::kindle::Catalog({
    "text"_k = pluss::kindle::Stream(pluss::make_note({
        "heavy rain on road",
        "clear weather downtown",
        "storm warning issued",
    })),
});

auto out = catalog.aggregate_text_stats("text", "mean");
```

Output

```
id     -> Series([1, 2, 3])
metric -> Series([10.0, 20.0, 30.0])
city   -> Note(["Accra", "Tema", "Kumasi"])
```

25.1.3 apply

Signature

```
Catalog apply(self, std::string_view field, const std::function<Stream(const Stream&)>&␣
↪func) const
```

Parameters

`field` : `std::string_view` See the signature for the supported argument form.

`func` : `const std::function<Stream(const Stream&)>&` See the signature for the supported argument form.

Returns

`Catalog` Value produced by this call.

Notes

This mutates or derives structure while keeping the surrounding object model intact.

Examples

```
using namespace pluss::kindle::literals;

auto catalog = pluss::kindle::Catalog({
    "id"_k = pluss::kindle::Stream(pluss::make_series({1, 2, 3})),
    "metric"_k = pluss::kindle::Stream(pluss::make_series({10.0, 20.0, 30.0})),
    "city"_k = pluss::kindle::Stream(pluss::make_note({"Accra", "Tema", "Kumasi"})),
});

auto out = catalog.apply("metric", [](const pluss::kindle::Stream& s) { return s.clip(0.
↪0, 25.0); });
```

Output

```
id     -> Series([1, 2, 3])
metric -> Series([10.0, 20.0, 30.0])
city   -> Note(["Accra", "Tema", "Kumasi"])
```

25.1.4 apply_

Signature

```
Catalog& apply_(self, std::string_view field, const std::function<Stream(const Stream&)>&
↪ func)
```

Parameters

`field` : `std::string_view` See the signature for the supported argument form.

`func` : `const std::function<Stream(const Stream&)>&` See the signature for the supported argument form.

Returns

`Catalog&` Value produced by this call.

Notes

This mutates or derives structure while keeping the surrounding object model intact.

Examples

```
using namespace pluss::kindle::literals;

auto catalog = pluss::kindle::Catalog({
    "id"_k = pluss::kindle::Stream(pluss::make_series({1, 2, 3})),
    "metric"_k = pluss::kindle::Stream(pluss::make_series({10.0, 20.0, 30.0})),
    "city"_k = pluss::kindle::Stream(pluss::make_note({"Accra", "Tema", "Kumasi"})),
});

catalog.apply_("metric", [](const pluss::kindle::Stream& s) { return s.clip(0.0, 25.0); }
↪);
```

25.1.5 apply_all

Signature

```
Catalog apply_all(self, const std::function<Stream(const Stream&)>& fn) const
```

Parameters

`fn` : `const std::function<Stream(const Stream&)>&` See the signature for the supported argument form.

Returns

`Catalog` Value produced by this call.

Notes

This mutates or derives structure while keeping the surrounding object model intact.

Examples

```
using namespace pluss::kindle::literals;

auto catalog = pluss::kindle::Catalog({
    "id"_k = pluss::kindle::Stream(pluss::make_series({1, 2, 3})),
    "metric"_k = pluss::kindle::Stream(pluss::make_series({10.0, 20.0, 30.0})),
    "city"_k = pluss::kindle::Stream(pluss::make_note({"Accra", "Tema", "Kumasi"})),
});

auto out = catalog.apply_all([](const pluss::kindle::Stream& s) { return s.kind() ==␣
↪pluss::kindle::StreamKind::Series ? s.clip(0.0, 25.0) : s; });
```

Output

```
id     -> Series([1, 2, 3])
metric -> Series([10.0, 20.0, 30.0])
city   -> Note(["Accra", "Tema", "Kumasi"])
```

25.1.6 apply_all_

Signature

```
Catalog& apply_all_(self, const std::function<Stream(const Stream&)>& fn)
```

Parameters

fn : const std::function<Stream(const Stream&)>& See the signature for the supported argument form.

Returns

Catalog& Value produced by this call.

Notes

This mutates or derives structure while keeping the surrounding object model intact.

Examples

```
using namespace pluss::kindle::literals;

auto catalog = pluss::kindle::Catalog({
    "id"_k = pluss::kindle::Stream(pluss::make_series({1, 2, 3})),
    "metric"_k = pluss::kindle::Stream(pluss::make_series({10.0, 20.0, 30.0})),
    "city"_k = pluss::kindle::Stream(pluss::make_note({"Accra", "Tema", "Kumasi"})),
});

catalog.apply_all_([](const pluss::kindle::Stream& s) { return s.kind() ==␣
↪pluss::kindle::StreamKind::Series ? s.clip(0.0, 25.0) : s; });
```

25.1.7 astype

Signature

```
Catalog astype(self, std::initializer_list<std::string_view> keys,␣
↪pluss::codem::ScalarType dtype) const
```

Parameters

keys : std::initializer_list<std::string_view> See the signature for the supported argument form.

dtype : pluss::codem::ScalarType See the signature for the supported argument form.

Returns

Catalog Value produced by this call.

Notes

This changes the semantic or numeric type so later operations route through the correct implementation path.

Examples

```
using namespace pluss::kindle::literals;

auto catalog = pluss::kindle::Catalog({
    "id"_k = pluss::kindle::Stream(pluss::make_series({1, 2, 3})),
    "metric"_k = pluss::kindle::Stream(pluss::make_series({10.0, 20.0, 30.0})),
    "city"_k = pluss::kindle::Stream(pluss::make_note({"Accra", "Tema", "Kumasi"})),
});

auto out = catalog.astype({"metric"}, pluss::codem::ScalarType::Float64);
```

Output

```
id     -> Series([1, 2, 3])
metric -> Series([10.0, 20.0, 30.0])
city   -> Note(["Accra", "Tema", "Kumasi"])
```

25.1.8 `at`

Signature

```
Stream at(self, std::string_view key) const
```

Parameters

`key` : `std::string_view` See the signature for the supported argument form.

Returns

`Stream` Value produced by this call.

Notes

This call is part of the public `Catalog` API and is documented here using a concrete usage example.

Examples

```
using namespace pluss::kindle::literals;

auto catalog = pluss::kindle::Catalog({
    "id"_k = pluss::kindle::Stream(pluss::make_series({1, 2, 3})),
    "metric"_k = pluss::kindle::Stream(pluss::make_series({10.0, 20.0, 30.0})),
    "city"_k = pluss::kindle::Stream(pluss::make_note({"Accra", "Tema", "Kumasi"})),
});

auto out = catalog.at("metric");
```

Output

```
Series([10.0, 20.0, 30.0])
```

25.1.9 begin

Signature

```
const_iterator begin(self) const
```

Parameters

None

Returns

const_iterator Value produced by this call.

Notes

This call is part of the public Catalog API and is documented here using a concrete usage example.

Examples

```
using namespace pluss::kindle::literals;

auto catalog = pluss::kindle::Catalog({
    "id"_k = pluss::kindle::Stream(pluss::make_series({1, 2, 3})),
    "metric"_k = pluss::kindle::Stream(pluss::make_series({10.0, 20.0, 30.0})),
    "city"_k = pluss::kindle::Stream(pluss::make_note({"Accra", "Tema", "Kumasi"})),
});

for (const auto& key : catalog) {
  (void)key;
}
```

25.1.10 bfill

Signature

```
Catalog bfill(self, std::initializer_list<std::string_view> keys =
```

Parameters

keys : std::initializer_list<std::string_view> See the signature for the supported argument form.

Returns

Catalog Value produced by this call.

Notes

This is part of the missing-data cleanup path and is usually applied before summaries, grouping, or export.

Examples

```
using namespace pluss::kindle::literals;

auto catalog = pluss::kindle::Catalog({
    "id"_k = pluss::kindle::Stream(pluss::make_series({1, 2, 3})),
    "metric"_k = pluss::kindle::Stream(pluss::make_series({10.0, 20.0, 30.0})),
    "city"_k = pluss::kindle::Stream(pluss::make_note({"Accra", "Tema", "Kumasi"})),
});

auto out = catalog.bfill();
```

Output

```
id     -> Series([1, 2, 3])
metric -> Series([10.0, 20.0, 30.0])
city   -> Note(["Accra", "Tema", "Kumasi"])
```

25.1.11 cast_note

Signature

```
Catalog cast_note(self, std::initializer_list<std::string_view> keys) const
```

Parameters

keys : std::initializer_list<std::string_view> See the signature for the supported argument form.

Returns

Catalog Value produced by this call.

Notes

This changes the semantic or numeric type so later operations route through the correct implementation path.

Examples

```
using namespace pluss::kindle::literals;

auto catalog = pluss::kindle::Catalog({
    "id"_k = pluss::kindle::Stream(pluss::make_series({1, 2, 3})),
    "metric"_k = pluss::kindle::Stream(pluss::make_series({10.0, 20.0, 30.0})),
    "city"_k = pluss::kindle::Stream(pluss::make_note({"Accra", "Tema", "Kumasi"})),
});

auto out = catalog.cast_note({"metric"});
```

Output

```
id     -> Series([1, 2, 3])
metric -> Series([10.0, 20.0, 30.0])
city   -> Note(["Accra", "Tema", "Kumasi"])
```

25.1.12 cast_series

Signature

```
Catalog cast_series(self, std::initializer_list<std::string_view> keys, std::optional
↪<pluss::codem::ScalarType> dtype = std::nullopt) const
```

Parameters

keys : std::initializer_list<std::string_view> See the signature for the supported argument form.

dtype : std::optional<pluss::codem::ScalarType> See the signature for the supported argument form.

Returns

Catalog Value produced by this call.

Notes

This changes the semantic or numeric type so later operations route through the correct implementation path.

Examples

```
using namespace pluss::kindle::literals;
auto catalog = pluss::kindle::Catalog({"metric"_k = pluss::kindle::Stream(pluss::make_
↪note({"10.0", "20.0", "30.0"}))});

auto out = catalog.cast_series({"metric"}, pluss::codem::ScalarType::Float64);
```

Output

```
id     -> Series([1, 2, 3])
metric -> Series([10.0, 20.0, 30.0])
city   -> Note(["Accra", "Tema", "Kumasi"])
```

25.1.13 cast_timeindex

Signature

```
Catalog cast_timeindex(self, std::initializer_list<std::string_view> keys) const
```

Parameters

keys : std::initializer_list<std::string_view> See the signature for the supported argument form.

Returns

Catalog Value produced by this call.

Notes

This changes the semantic or numeric type so later operations route through the correct implementation path.

Examples

```
using namespace pluss::kindle::literals;

auto catalog = pluss::kindle::Catalog({
    "id"_k = pluss::kindle::Stream(pluss::make_series({1, 2, 3})),
    "metric"_k = pluss::kindle::Stream(pluss::make_series({10.0, 20.0, 30.0})),
    "city"_k = pluss::kindle::Stream(pluss::make_note({"Accra", "Tema", "Kumasi"})),
});

auto out = catalog.cast_timeindex({"metric"});
```

Output

```
id     -> Series([1, 2, 3])
metric -> Series([10.0, 20.0, 30.0])
city   -> Note(["Accra", "Tema", "Kumasi"])
```

25.1.14 cast_timeline

Signature

```
Catalog cast_timeline(self, std::initializer_list<std::string_view> keys) const
```

Parameters

keys : std::initializer_list<std::string_view> See the signature for the supported argument form.

Returns

Catalog Value produced by this call.

Notes

This changes the semantic or numeric type so later operations route through the correct implementation path.

Examples

```
using namespace pluss::kindle::literals;

auto catalog = pluss::kindle::Catalog({
    "id"_k = pluss::kindle::Stream(pluss::make_series({1, 2, 3})),
    "metric"_k = pluss::kindle::Stream(pluss::make_series({10.0, 20.0, 30.0})),
    "city"_k = pluss::kindle::Stream(pluss::make_note({"Accra", "Tema", "Kumasi"})),
});

auto out = catalog.cast_timeline({"metric"});
```

Output

```
id     -> Series([1, 2, 3])
metric -> Series([10.0, 20.0, 30.0])
city   -> Note(["Accra", "Tema", "Kumasi"])
```

25.1.15 compute_similarity

Signature

```
Catalog compute_similarity(self, std::string_view field1, std::string_view field2,␣
↪std::string_view method = "cosine") const
```

Parameters

field1 : std::string_view See the signature for the supported argument form.

field2 : std::string_view See the signature for the supported argument form.

method : std::string_view See the signature for the supported argument form.

Returns

Catalog Value produced by this call.

Notes

This is part of the text-processing surface for note-backed or text-oriented data.

Examples

```
using namespace pluss::kindle::literals;

auto catalog = pluss::kindle::Catalog({
    "text"_k = pluss::kindle::Stream(pluss::make_note({
        "heavy rain on road",
        "clear weather downtown",
        "storm warning issued",
    })),
});

auto out = catalog.compute_similarity("text", "text", "cosine");
```

Output

```
id     -> Series([1, 2, 3])
metric -> Series([10.0, 20.0, 30.0])
city   -> Note(["Accra", "Tema", "Kumasi"])
```

25.1.16 count

Signature

```
std::int64_t count(self) const
```

Parameters

None

Returns

`std::int64_t` Integer count, size, or positional result.

Notes

Use this when you want a summary view instead of carrying the full input forward unchanged.

Examples

```
using namespace pluss::kindle::literals;

auto catalog = pluss::kindle::Catalog({
    "id"_k = pluss::kindle::Stream(pluss::make_series({1, 2, 3})),
    "metric"_k = pluss::kindle::Stream(pluss::make_series({10.0, 20.0, 30.0})),
    "city"_k = pluss::kindle::Stream(pluss::make_note({"Accra", "Tema", "Kumasi"})),
});

auto out = catalog.count();
```

Output

```
3
```

25.1.17 defined

Signature

```
bool defined(self) const
```

Parameters

None

Returns

`bool` Boolean status or membership result.

Notes

This call is part of the public `Catalog` API and is documented here using a concrete usage example.

Examples

```
using namespace pluss::kindle::literals;

auto catalog = pluss::kindle::Catalog({
    "id"_k = pluss::kindle::Stream(pluss::make_series({1, 2, 3})),
    "metric"_k = pluss::kindle::Stream(pluss::make_series({10.0, 20.0, 30.0})),
    "city"_k = pluss::kindle::Stream(pluss::make_note({"Accra", "Tema", "Kumasi"})),
});

auto out = catalog.defined();
```

Output

```
true
```

25.1.18 display

Signature

```
void display(self) const
```

Parameters

None

Returns

void Performs the side effect and returns no value.

Notes

This renders a human-readable view for inspection instead of creating a new analytic object.

Examples

```
using namespace pluss::kindle::literals;

auto catalog = pluss::kindle::Catalog({
    "id"_k = pluss::kindle::Stream(pluss::make_series({1, 2, 3})),
    "metric"_k = pluss::kindle::Stream(pluss::make_series({10.0, 20.0, 30.0})),
    "city"_k = pluss::kindle::Stream(pluss::make_note({"Accra", "Tema", "Kumasi"})),
});

catalog.display();
```

Output

```
id     -> Series([1, 2, 3])
metric -> Series([10.0, 20.0, 30.0])
city   -> Note(["Accra", "Tema", "Kumasi"])
```

25.1.19 drop

Signature

```
Catalog drop(self, const std::vector<std::string>& field_names) const
```

Parameters

field_names : const std::vector<std::string>& See the signature for the supported argument form.

Returns

`Catalog` Value produced by this call.

Notes

Use this to reduce the current object to the rows or fields that matter for the next step.

See also

- `Catalog drop(self, std::string_view field_name) const`
- `Catalog drop(self, std::initializer_list<std::string_view> keys) const`

Examples

```
using namespace pluss::kindle::literals;

auto catalog = pluss::kindle::Catalog({
    "id"_k = pluss::kindle::Stream(pluss::make_series({1, 2, 3})),
    "metric"_k = pluss::kindle::Stream(pluss::make_series({10.0, 20.0, 30.0})),
    "city"_k = pluss::kindle::Stream(pluss::make_note({"Accra", "Tema", "Kumasi"})),
});

std::vector<std::string> names{"city"};
auto out = catalog.drop(names);
```

Output

```
id     -> Series([1, 2, 3])
metric -> Series([10.0, 20.0, 30.0])
city   -> Note(["Accra", "Tema", "Kumasi"])
```

25.1.20 drop

Signature

```
Catalog drop(self, std::initializer_list<std::string_view> keys) const
```

Parameters

`keys` : `std::initializer_list<std::string_view>` See the signature for the supported argument form.

Returns

`Catalog` Value produced by this call.

Notes

Use this to reduce the current object to the rows or fields that matter for the next step.

See also

- `Catalog drop(self, std::string_view field_name) const`
- `Catalog drop(self, const std::vector<std::string>& field_names) const`

Examples

```
using namespace pluss::kindle::literals;

auto catalog = pluss::kindle::Catalog({
    "id"_k = pluss::kindle::Stream(pluss::make_series({1, 2, 3})),
    "metric"_k = pluss::kindle::Stream(pluss::make_series({10.0, 20.0, 30.0})),
    "city"_k = pluss::kindle::Stream(pluss::make_note({"Accra", "Tema", "Kumasi"})),
});

auto out = catalog.drop({"city"});
```

Output

```
id     -> Series([1, 2, 3])
metric -> Series([10.0, 20.0, 30.0])
city   -> Note(["Accra", "Tema", "Kumasi"])
```

25.1.21 drop

Signature

```
Catalog drop(self, std::string_view field_name) const
```

Parameters

`field_name` : `std::string_view` See the signature for the supported argument form.

Returns

`Catalog` Value produced by this call.

Notes

Use this to reduce the current object to the rows or fields that matter for the next step.

See also

- `Catalog drop(self, const std::vector<std::string>& field_names) const`
- `Catalog drop(self, std::initializer_list<std::string_view> keys) const`

Examples

```
using namespace pluss::kindle::literals;

auto catalog = pluss::kindle::Catalog({
    "id"_k = pluss::kindle::Stream(pluss::make_series({1, 2, 3})),
    "metric"_k = pluss::kindle::Stream(pluss::make_series({10.0, 20.0, 30.0})),
```

(continues on next page)

(continued from previous page)

```
    "city"_k = pluss::kindle::Stream(pluss::make_note({"Accra", "Tema", "Kumasi"})),
});

auto out = catalog.drop("city");
```

Output

```
id     -> Series([1, 2, 3])
metric -> Series([10.0, 20.0, 30.0])
city   -> Note(["Accra", "Tema", "Kumasi"])
```

25.1.22 dropna

Signature

```
Catalog dropna(self, std::initializer_list<std::string_view> keys =
```

Parameters

keys : std::initializer_list<std::string_view> See the signature for the supported argument form.

Returns

Catalog Value produced by this call.

Notes

This is part of the missing-data cleanup path and is usually applied before summaries, grouping, or export.

Examples

```
using namespace pluss::kindle::literals;

auto catalog = pluss::kindle::Catalog({
    "id"_k = pluss::kindle::Stream(pluss::make_series({1, 2, 3})),
    "metric"_k = pluss::kindle::Stream(pluss::make_series({10.0, 20.0, 30.0})),
    "city"_k = pluss::kindle::Stream(pluss::make_note({"Accra", "Tema", "Kumasi"})),
});

auto out = catalog.dropna();
```

Output

```
id     -> Series([1, 2, 3])
metric -> Series([10.0, 20.0, 30.0])
city   -> Note(["Accra", "Tema", "Kumasi"])
```

25.1.23 empty

Signature

```
bool empty(self) const
```

Parameters

None

Returns

bool Boolean status or membership result.

Notes

This call is part of the public Catalog API and is documented here using a concrete usage example.

Examples

```
using namespace pluss::kindle::literals;

auto catalog = pluss::kindle::Catalog({
    "id"_k = pluss::kindle::Stream(pluss::make_series({1, 2, 3})),
    "metric"_k = pluss::kindle::Stream(pluss::make_series({10.0, 20.0, 30.0})),
    "city"_k = pluss::kindle::Stream(pluss::make_note({"Accra", "Tema", "Kumasi"})),
});

auto out = catalog.empty();
```

Output

```
false
```

25.1.24 end

Signature

```
const_iterator end(self) const
```

Parameters

None

Returns

const_iterator Value produced by this call.

Notes

This call is part of the public Catalog API and is documented here using a concrete usage example.

Examples

```cpp
using namespace pluss::kindle::literals;

auto catalog = pluss::kindle::Catalog({
    "id"_k = pluss::kindle::Stream(pluss::make_series({1, 2, 3})),
    "metric"_k = pluss::kindle::Stream(pluss::make_series({10.0, 20.0, 30.0})),
    "city"_k = pluss::kindle::Stream(pluss::make_note({"Accra", "Tema", "Kumasi"})),
});

for (const auto& key : catalog) {
  (void)key;
}
```

25.1.25 extract_keywords

Signature

```cpp
Catalog extract_keywords(self, std::string_view field, int top_n = 10) const
```

Parameters

`field` : `std::string_view` See the signature for the supported argument form.

`top_n` : `int` See the signature for the supported argument form.

Returns

`Catalog` Value produced by this call.

Notes

This is part of the text-processing surface for note-backed or text-oriented data.

Examples

```cpp
using namespace pluss::kindle::literals;

auto catalog = pluss::kindle::Catalog({
    "text"_k = pluss::kindle::Stream(pluss::make_note({
        "heavy rain on road",
        "clear weather downtown",
        "storm warning issued",
    })),
});

auto out = catalog.extract_keywords("text", 5);
```

Output

```
id     -> Series([1, 2, 3])
metric -> Series([10.0, 20.0, 30.0])
city   -> Note(["Accra", "Tema", "Kumasi"])
```

25.1.26 ffill

Signature

```
Catalog ffill(self, std::initializer_list<std::string_view> keys =
```

Parameters

keys : std::initializer_list<std::string_view> See the signature for the supported argument form.

Returns

Catalog Value produced by this call.

Notes

This is part of the missing-data cleanup path and is usually applied before summaries, grouping, or export.

Examples

```
using namespace pluss::kindle::literals;

auto catalog = pluss::kindle::Catalog({
    "id"_k = pluss::kindle::Stream(pluss::make_series({1, 2, 3})),
    "metric"_k = pluss::kindle::Stream(pluss::make_series({10.0, 20.0, 30.0})),
    "city"_k = pluss::kindle::Stream(pluss::make_note({"Accra", "Tema", "Kumasi"})),
});

auto out = catalog.ffill();
```

Output

```
id     -> Series([1, 2, 3])
metric -> Series([10.0, 20.0, 30.0])
city   -> Note(["Accra", "Tema", "Kumasi"])
```

25.1.27 fillna

Signature

```
Catalog fillna(self, const char* value, std::initializer_list<std::string_view> keys =
```

Parameters

value : const char* See the signature for the supported argument form.

keys : std::initializer_list<std::string_view> See the signature for the supported argument form.

Returns

Catalog Value produced by this call.

Notes

This is part of the missing-data cleanup path and is usually applied before summaries, grouping, or export.

See also

- `Catalog fillna(self, scalar value, std::initializer_list<std::string_view> keys =`
- `Catalog fillna(self, std::string_view value, std::initializer_list<std::string_view> keys =`
- `Catalog fillna(self, const pluss::Timeline& value, std::initializer_list<std::string_view> keys =`
- `Catalog fillna(self, const pluss::TimeIndex& value, std::initializer_list<std::string_view> keys =`

Examples

```
using namespace pluss::kindle::literals;
auto catalog = pluss::kindle::Catalog({"city"_k = pluss::kindle::Stream(pluss::make_note(
↪{"Accra", "Tema", "Kumasi"}))});

auto out = catalog.fillna("Unknown", {"city"});
```

Output

```
id     -> Series([1, 2, 3])
metric -> Series([10.0, 20.0, 30.0])
city   -> Note(["Accra", "Tema", "Kumasi"])
```

25.1.28 `fillna`

Signature

```
Catalog fillna(self, const pluss::TimeIndex& value, std::initializer_list<std::string_
↪view> keys =
```

Parameters

`value` : `const pluss::TimeIndex&` See the signature for the supported argument form.

`keys` : `std::initializer_list<std::string_view>` See the signature for the supported argument form.

Returns

`Catalog` Value produced by this call.

Notes

This is part of the missing-data cleanup path and is usually applied before summaries, grouping, or export.

See also

- `Catalog fillna(self, scalar value, std::initializer_list<std::string_view> keys =`
- `Catalog fillna(self, std::string_view value, std::initializer_list<std::string_view> keys =`
- `Catalog fillna(self, const char* value, std::initializer_list<std::string_view> keys =`
- `Catalog fillna(self, const pluss::Timeline& value, std::initializer_list<std::string_view> keys =`

Examples

```
using namespace pluss::kindle::literals;
auto catalog = pluss::kindle::Catalog({"city"_k = pluss::kindle::Stream(pluss::make_note(
↪{"Accra", "Tema", "Kumasi"}))});

auto out = catalog.fillna("Unknown", {"city"});
```

Output

```
id     -> Series([1, 2, 3])
metric -> Series([10.0, 20.0, 30.0])
city   -> Note(["Accra", "Tema", "Kumasi"])
```

25.1.29 `fillna`

Signature

```
Catalog fillna(self, const pluss::Timeline& value, std::initializer_list<std::string_
↪view> keys =
```

Parameters

`value` : `const pluss::Timeline&` See the signature for the supported argument form.

`keys` : `std::initializer_list<std::string_view>` See the signature for the supported argument form.

Returns

`Catalog` Value produced by this call.

Notes

This is part of the missing-data cleanup path and is usually applied before summaries, grouping, or export.

See also

- `Catalog fillna(self, scalar value, std::initializer_list<std::string_view> keys =`
- `Catalog fillna(self, std::string_view value, std::initializer_list<std::string_view> keys =`
- `Catalog fillna(self, const char* value, std::initializer_list<std::string_view> keys =`

- `Catalog fillna(self, const pluss::TimeIndex& value, std::initializer_list<std::string_view> keys =`

Examples

```
using namespace pluss::kindle::literals;
auto catalog = pluss::kindle::Catalog({"city"_k = pluss::kindle::Stream(pluss::make_note(
↪{"Accra", "Tema", "Kumasi"}))});

auto out = catalog.fillna("Unknown", {"city"});
```

Output

```
id     -> Series([1, 2, 3])
metric -> Series([10.0, 20.0, 30.0])
city   -> Note(["Accra", "Tema", "Kumasi"])
```

25.1.30 `fillna`

Signature

```
Catalog fillna(self, scalar value, std::initializer_list<std::string_view> keys =
```

Parameters

`value` : `scalar` See the signature for the supported argument form.

`keys` : `std::initializer_list<std::string_view>` See the signature for the supported argument form.

Returns

`Catalog` Value produced by this call.

Notes

This is part of the missing-data cleanup path and is usually applied before summaries, grouping, or export.

See also

- `Catalog fillna(self, std::string_view value, std::initializer_list<std::string_view> keys =`
- `Catalog fillna(self, const char* value, std::initializer_list<std::string_view> keys =`
- `Catalog fillna(self, const pluss::Timeline& value, std::initializer_list<std::string_view> keys =`
- `Catalog fillna(self, const pluss::TimeIndex& value, std::initializer_list<std::string_view> keys =`

Examples

```
using namespace pluss::kindle::literals;
auto catalog = pluss::kindle::Catalog({"city"_k = pluss::kindle::Stream(pluss::make_note(
↪{"Accra", "Tema", "Kumasi"}))});

auto out = catalog.fillna("Unknown", {"city"});
```

Output

```
id     -> Series([1, 2, 3])
metric -> Series([10.0, 20.0, 30.0])
city   -> Note(["Accra", "Tema", "Kumasi"])
```

25.1.31 fillna

Signature

```
Catalog fillna(self, std::string_view value, std::initializer_list<std::string_view>␣
↪keys =
```

Parameters

`value` : `std::string_view` See the signature for the supported argument form.

`keys` : `std::initializer_list<std::string_view>` See the signature for the supported argument form.

Returns

`Catalog` Value produced by this call.

Notes

This is part of the missing-data cleanup path and is usually applied before summaries, grouping, or export.

See also

- `Catalog fillna(self, scalar value, std::initializer_list<std::string_view> keys =`
- `Catalog fillna(self, const char* value, std::initializer_list<std::string_view> keys =`
- `Catalog fillna(self, const pluss::Timeline& value, std::initializer_list<std::string_view> keys =`
- `Catalog fillna(self, const pluss::TimeIndex& value, std::initializer_list<std::string_view> keys =`

Examples

```
using namespace pluss::kindle::literals;
auto catalog = pluss::kindle::Catalog({"city"_k = pluss::kindle::Stream(pluss::make_note(
↪{"Accra", "Tema", "Kumasi"}))});

auto out = catalog.fillna("Unknown", {"city"});
```

Output

```
id     -> Series([1, 2, 3])
metric -> Series([10.0, 20.0, 30.0])
city   -> Note(["Accra", "Tema", "Kumasi"])
```

25.1.32 filter

Signature

```
Catalog filter(self, const pluss::Series& condition) const
```

Parameters

`condition` : `const pluss::Series&` See the signature for the supported argument form.

Returns

`Catalog` Value produced by this call.

Notes

Use this to reduce the current object to the rows or fields that matter for the next step.

Examples

```
using namespace pluss::kindle::literals;

auto catalog = pluss::kindle::Catalog({
    "id"_k = pluss::kindle::Stream(pluss::make_series({1, 2, 3})),
    "metric"_k = pluss::kindle::Stream(pluss::make_series({10.0, 20.0, 30.0})),
    "city"_k = pluss::kindle::Stream(pluss::make_note({"Accra", "Tema", "Kumasi"})),
});

auto mask = pluss::make_series({true, false, true}, pluss::codem::ScalarType::Bool);
auto out = catalog.filter(mask);
```

Output

```
id     -> Series([1, 2, 3])
metric -> Series([10.0, 20.0, 30.0])
city   -> Note(["Accra", "Tema", "Kumasi"])
```

25.1.33 filter_by_text

Signature

```
Catalog filter_by_text(self, std::string_view field, std::string_view pattern) const
```

Parameters

`field` : `std::string_view` See the signature for the supported argument form.

`pattern` : `std::string_view` See the signature for the supported argument form.

Returns

`Catalog` Value produced by this call.

Notes

This is part of the text-processing surface for note-backed or text-oriented data.

Examples

```
using namespace pluss::kindle::literals;

auto catalog = pluss::kindle::Catalog({
    "text"_k = pluss::kindle::Stream(pluss::make_note({
        "heavy rain on road",
        "clear weather downtown",
        "storm warning issued",
    })),
});

auto out = catalog.filter_by_text("text", "rain");
```

Output

```
id     -> Series([1, 2, 3])
metric -> Series([10.0, 20.0, 30.0])
city   -> Note(["Accra", "Tema", "Kumasi"])
```

25.1.34 `from_json`

Signature

```
static Catalog from_json(std::string_view json_text)
```

Parameters

`json_text` : `std::string_view` See the signature for the supported argument form.

Returns

`Catalog` Value produced by this call.

Notes

This call is part of the public `Catalog` API and is documented here using a concrete usage example.

Examples

```
auto out = pluss::kindle::Catalog::from_json(R"([{"id":1,"city":"Accra"},{"id":2,"city":
↪"Tema"}])");
```

Output

```
id     -> Series([1, 2, 3])
metric -> Series([10.0, 20.0, 30.0])
city   -> Note(["Accra", "Tema", "Kumasi"])
```

25.1.35 get

Signature

```
std::optional<Stream> get(self, std::string_view key) const
```

Parameters

key : std::string_view See the signature for the supported argument form.

Returns

std::optional<Stream> Value produced by this call.

Notes

This call is part of the public Catalog API and is documented here using a concrete usage example.

Examples

```
using namespace pluss::kindle::literals;

auto catalog = pluss::kindle::Catalog({
    "id"_k = pluss::kindle::Stream(pluss::make_series({1, 2, 3})),
    "metric"_k = pluss::kindle::Stream(pluss::make_series({10.0, 20.0, 30.0})),
    "city"_k = pluss::kindle::Stream(pluss::make_note({"Accra", "Tema", "Kumasi"})),
});

auto out = catalog.get("metric");
```

Output

```
Series([10.0, 20.0, 30.0])
```

25.1.36 get_stream

Signature

```
std::optional<Stream> get_stream(self, std::string_view key) const
```

Parameters

key : std::string_view See the signature for the supported argument form.

Returns

std::optional<Stream> Value produced by this call.

Notes

This call is part of the public Catalog API and is documented here using a concrete usage example.

Examples

```
using namespace pluss::kindle::literals;

auto catalog = pluss::kindle::Catalog({
    "id"_k = pluss::kindle::Stream(pluss::make_series({1, 2, 3})),
    "metric"_k = pluss::kindle::Stream(pluss::make_series({10.0, 20.0, 30.0})),
    "city"_k = pluss::kindle::Stream(pluss::make_note({"Accra", "Tema", "Kumasi"})),
});

auto out = catalog.get_stream("metric");
```

Output

```
Series([10.0, 20.0, 30.0])
```

25.1.37 group_by

Signature

```
std::unordered_map<std::string, Catalog> group_by(self, std::string_view key_field) const
```

Parameters

key_field : std::string_view See the signature for the supported argument form.

Returns

std::unordered_map<std::string, Catalog> Value produced by this call.

Notes

Use this when you want a summary view instead of carrying the full input forward unchanged.

Examples

```
using namespace pluss::kindle::literals;

auto catalog = pluss::kindle::Catalog({
    "id"_k = pluss::kindle::Stream(pluss::make_series({1, 2, 3})),
    "metric"_k = pluss::kindle::Stream(pluss::make_series({10.0, 20.0, 30.0})),
    "city"_k = pluss::kindle::Stream(pluss::make_note({"Accra", "Tema", "Kumasi"})),
```

(continues on next page)

(continued from previous page)

```
});

auto out = catalog.group_by("city");
```

Output

```
{"Accra": Catalog(...), "Tema": Catalog(...)}
```

25.1.38 has

Signature

```
bool has(self, std::string_view key) const
```

Parameters

key : std::string_view See the signature for the supported argument form.

Returns

bool Boolean status or membership result.

Notes

This call is part of the public Catalog API and is documented here using a concrete usage example.

Examples

```
using namespace pluss::kindle::literals;

auto catalog = pluss::kindle::Catalog({
    "id"_k = pluss::kindle::Stream(pluss::make_series({1, 2, 3})),
    "metric"_k = pluss::kindle::Stream(pluss::make_series({10.0, 20.0, 30.0})),
    "city"_k = pluss::kindle::Stream(pluss::make_note({"Accra", "Tema", "Kumasi"})),
});

auto out = catalog.has("metric");
```

Output

```
true
```

25.1.39 has_stream

Signature

```
bool has_stream(self, std::string_view key) const
```

Parameters

key : std::string_view See the signature for the supported argument form.

Returns

bool Boolean status or membership result.

Notes

This call is part of the public Catalog API and is documented here using a concrete usage example.

Examples

```
using namespace pluss::kindle::literals;

auto catalog = pluss::kindle::Catalog({
    "id"_k = pluss::kindle::Stream(pluss::make_series({1, 2, 3})),
    "metric"_k = pluss::kindle::Stream(pluss::make_series({10.0, 20.0, 30.0})),
    "city"_k = pluss::kindle::Stream(pluss::make_note({"Accra", "Tema", "Kumasi"})),
});

auto out = catalog.has_stream("metric");
```

Output

```
true
```

25.1.40 head

Signature

```
Catalog head(self, std::int64_t n) const
```

Parameters

n : std::int64_t See the signature for the supported argument form.

Returns

Catalog Value produced by this call.

Notes

Use this to narrow the current object to the rows or depth range you want to inspect next.

Examples

```
using namespace pluss::kindle::literals;

auto catalog = pluss::kindle::Catalog({
    "id"_k = pluss::kindle::Stream(pluss::make_series({1, 2, 3})),
    "metric"_k = pluss::kindle::Stream(pluss::make_series({10.0, 20.0, 30.0})),
    "city"_k = pluss::kindle::Stream(pluss::make_note({"Accra", "Tema", "Kumasi"})),
```

(continues on next page)

(continued from previous page)

```
});

auto out = catalog.head(2);
```

Output

```
id     -> Series([1, 2, 3])
metric -> Series([10.0, 20.0, 30.0])
city   -> Note(["Accra", "Tema", "Kumasi"])
```

25.1.41 iloc

Signature

```
Catalog iloc(self, std::int64_t row) const
```

Parameters

`row` : `std::int64_t` See the signature for the supported argument form.

Returns

`Catalog` Value produced by this call.

Notes

Use this to narrow the current object to the rows or depth range you want to inspect next.

See also

- `Catalog iloc(self, std::int64_t start, std::int64_t stop) const`

Examples

```
using namespace pluss::kindle::literals;

auto catalog = pluss::kindle::Catalog({
    "id"_k = pluss::kindle::Stream(pluss::make_series({1, 2, 3})),
    "metric"_k = pluss::kindle::Stream(pluss::make_series({10.0, 20.0, 30.0})),
    "city"_k = pluss::kindle::Stream(pluss::make_note({"Accra", "Tema", "Kumasi"})),
});

auto out = catalog.iloc(1);
```

Output

```
id     -> Series([1, 2, 3])
metric -> Series([10.0, 20.0, 30.0])
city   -> Note(["Accra", "Tema", "Kumasi"])
```

25.1.42 iloc

Signature

```
Catalog iloc(self, std::int64_t start, std::int64_t stop) const
```

Parameters

`start` : `std::int64_t` See the signature for the supported argument form.

`stop` : `std::int64_t` See the signature for the supported argument form.

Returns

`Catalog` Value produced by this call.

Notes

Use this to narrow the current object to the rows or depth range you want to inspect next.

See also

- `Catalog iloc(self, std::int64_t row) const`

Examples

```
using namespace pluss::kindle::literals;

auto catalog = pluss::kindle::Catalog({
    "id"_k = pluss::kindle::Stream(pluss::make_series({1, 2, 3})),
    "metric"_k = pluss::kindle::Stream(pluss::make_series({10.0, 20.0, 30.0})),
    "city"_k = pluss::kindle::Stream(pluss::make_note({"Accra", "Tema", "Kumasi"})),
});

auto out = catalog.iloc(0, 2);
```

Output

```
id     -> Series([1, 2, 3])
metric -> Series([10.0, 20.0, 30.0])
city   -> Note(["Accra", "Tema", "Kumasi"])
```

25.1.43 info

Signature

```
void info(self) const
```

Parameters

None

Returns

`void` Performs the side effect and returns no value.

Notes

This renders a human-readable view for inspection instead of creating a new analytic object.

Examples

```
using namespace pluss::kindle::literals;

auto catalog = pluss::kindle::Catalog({
    "id"_k = pluss::kindle::Stream(pluss::make_series({1, 2, 3})),
    "metric"_k = pluss::kindle::Stream(pluss::make_series({10.0, 20.0, 30.0})),
    "city"_k = pluss::kindle::Stream(pluss::make_note({"Accra", "Tema", "Kumasi"})),
});

catalog.info();
```

Output

```
id     -> Series([1, 2, 3])
metric -> Series([10.0, 20.0, 30.0])
city   -> Note(["Accra", "Tema", "Kumasi"])
```

25.1.44 `isnull`

Signature

```
Catalog isnull(self, std::initializer_list<std::string_view> keys =
```

Parameters

`keys` : `std::initializer_list<std::string_view>` See the signature for the supported argument form.

Returns

`Catalog` Value produced by this call.

Notes

This is part of the missing-data cleanup path and is usually applied before summaries, grouping, or export.

Examples

```
using namespace pluss::kindle::literals;

auto catalog = pluss::kindle::Catalog({
    "id"_k = pluss::kindle::Stream(pluss::make_series({1, 2, 3})),
    "metric"_k = pluss::kindle::Stream(pluss::make_series({10.0, 20.0, 30.0})),
    "city"_k = pluss::kindle::Stream(pluss::make_note({"Accra", "Tema", "Kumasi"})),
});
```

(continues on next page)

(continued from previous page)

```
auto out = catalog.isnull();
```

Output

```
id     -> Series([1, 2, 3])
metric -> Series([10.0, 20.0, 30.0])
city   -> Note(["Accra", "Tema", "Kumasi"])
```

25.1.45 keys

Signature

```
const std::vector<std::string>& keys(self) const
```

Parameters

None

Returns

`const std::vector<std::string>&` Borrowed list of names.

Notes

This call is part of the public `Catalog` API and is documented here using a concrete usage example.

Examples

```
using namespace pluss::kindle::literals;

auto catalog = pluss::kindle::Catalog({
    "id"_k = pluss::kindle::Stream(pluss::make_series({1, 2, 3})),
    "metric"_k = pluss::kindle::Stream(pluss::make_series({10.0, 20.0, 30.0})),
    "city"_k = pluss::kindle::Stream(pluss::make_note({"Accra", "Tema", "Kumasi"})),
});

auto out = catalog.keys();
```

Output

```
["id", "metric", "city"]
```

25.1.46 kindelize

Signature

```
static Catalog kindelize(const pluss::table::DataFrame& frame, const KindelizePolicy&␣
↪policy =
```

Parameters

`frame` : `const pluss::table::DataFrame&` See the signature for the supported argument form.

`policy` : `const KindelizePolicy&` See the signature for the supported argument form.

Returns

`Catalog` Value produced by this call.

Notes

This is an explicit bridge between PlussMic models so you can switch to the representation that fits the next task.

Examples

```
auto df = pluss::table::DataFrame({
    {"id", pluss::table::Thread(pluss::make_series({1, 2, 3}))},
    {"sales", pluss::table::Thread(pluss::make_series({100.0, 120.0, 90.0}))},
    {"city", pluss::table::Thread(pluss::make_note({"Accra", "Tema", "Kumasi"}))},
});

auto out = pluss::kindle::Catalog::kindelize(df);
```

Output

```
id     -> Series([1, 2, 3])
metric -> Series([10.0, 20.0, 30.0])
city   -> Note(["Accra", "Tema", "Kumasi"])
```

25.1.47 load

Signature

```
static Catalog load(std::string_view source_type, std::string_view filepath)
```

Parameters

`source_type` : `std::string_view` See the signature for the supported argument form.

`filepath` : `std::string_view` See the signature for the supported argument form.

Returns

`Catalog` Value produced by this call.

Notes

Loads the object from disk using the corresponding PlussMic loader route.

See also

- `static Catalog load(std::string_view source_type, std::string_view filepath, const pluss::loader::Config& config)`

Examples

```
auto out = pluss::kindle::Catalog::load("csv", "PlussMic/data/messy_data.csv");
```

Output

```
id     -> Series([1, 2, 3])
metric -> Series([10.0, 20.0, 30.0])
city   -> Note(["Accra", "Tema", "Kumasi"])
```

25.1.48 load

Signature

```
static Catalog load(std::string_view source_type, std::string_view filepath, const␣
↪pluss::loader::Config& config)
```

Parameters

`source_type` : `std::string_view` See the signature for the supported argument form.

`filepath` : `std::string_view` See the signature for the supported argument form.

`config` : `const pluss::loader::Config&` See the signature for the supported argument form.

Returns

`Catalog` Value produced by this call.

Notes

Loads the object from disk using the corresponding PlussMic loader route.

See also

- `static Catalog load(std::string_view source_type, std::string_view filepath)`

Examples

```
auto out = pluss::kindle::Catalog::load("csv", "PlussMic/data/messy_data.csv");
```

Output

```
id     -> Series([1, 2, 3])
metric -> Series([10.0, 20.0, 30.0])
city   -> Note(["Accra", "Tema", "Kumasi"])
```

25.1.49 load_csv

Signature

```
static Catalog load_csv(std::string_view filepath)
```

Parameters

`filepath` : `std::string_view` See the signature for the supported argument form.

Returns

`Catalog` Value produced by this call.

Notes

Loads the object from disk using the corresponding PlussMic loader route.

See also

- `static Catalog load_csv(std::string_view filepath, const pluss::loader::ReadCsvOptions& options)`

Examples

```
auto out = pluss::kindle::Catalog::load_csv("PlussMic/data/messy_data.csv");
```

Output

```
id     -> Series([1, 2, 3])
metric -> Series([10.0, 20.0, 30.0])
city   -> Note(["Accra", "Tema", "Kumasi"])
```

25.1.50 load_csv

Signature

```
static Catalog load_csv(std::string_view filepath, const pluss::loader::ReadCsvOptions&␣
↪options)
```

Parameters

`filepath` : `std::string_view` See the signature for the supported argument form.

`options` : `const pluss::loader::ReadCsvOptions&` See the signature for the supported argument form.

Returns

`Catalog` Value produced by this call.

Notes

Loads the object from disk using the corresponding PlussMic loader route.

See also

- `static Catalog load_csv(std::string_view filepath)`

Examples

```
pluss::loader::ReadCsvOptions options;
auto out = pluss::kindle::Catalog::load_csv("PlussMic/data/messy_data.csv", options);
```

Output

```
id     -> Series([1, 2, 3])
metric -> Series([10.0, 20.0, 30.0])
city   -> Note(["Accra", "Tema", "Kumasi"])
```

25.1.51 load_csv_with_key

Signature

```
static Catalog load_csv_with_key(std::string_view filepath, std::string_view key_field,␣
↪const pluss::loader::ReadCsvOptions& options =
```

Parameters

`filepath` : `std::string_view` See the signature for the supported argument form.

`key_field` : `std::string_view` See the signature for the supported argument form.

`options` : `const pluss::loader::ReadCsvOptions&` See the signature for the supported argument form.

Returns

`Catalog` Value produced by this call.

Notes

Loads the object from disk using the corresponding PlussMic loader route.

Examples

```
auto out = pluss::kindle::Catalog::load_csv_with_key("PlussMic/data/messy_data.csv", "id
↪");
```

Output

```
id     -> Series([1, 2, 3])
metric -> Series([10.0, 20.0, 30.0])
city   -> Note(["Accra", "Tema", "Kumasi"])
```

25.1.52 load_excel

Signature

```
static Catalog load_excel(std::string_view filepath)
```

Parameters

`filepath` : `std::string_view` See the signature for the supported argument form.

Returns

`Catalog` Value produced by this call.

Notes

Loads the object from disk using the corresponding PlussMic loader route.

See also

- `static Catalog load_excel(std::string_view filepath, const pluss::loader::ReadExcelOptions& options)`

Examples

```
auto out = pluss::kindle::Catalog::load_excel("PlussMic/data/messy_data.csv");
```

Output

```
id     -> Series([1, 2, 3])
metric -> Series([10.0, 20.0, 30.0])
city   -> Note(["Accra", "Tema", "Kumasi"])
```

25.1.53 load_excel

Signature

```
static Catalog load_excel(std::string_view filepath, const␣
↪pluss::loader::ReadExcelOptions& options)
```

Parameters

`filepath` : `std::string_view` See the signature for the supported argument form.

`options` : `const pluss::loader::ReadExcelOptions&` See the signature for the supported argument form.

Returns

`Catalog` Value produced by this call.

Notes

Loads the object from disk using the corresponding PlussMic loader route.

See also

- `static Catalog load_excel(std::string_view filepath)`

Examples

```
pluss::loader::ReadExcelOptions options;
auto out = pluss::kindle::Catalog::load_excel("PlussMic/data/report.xlsx", options);
```

Output

```
id     -> Series([1, 2, 3])
metric -> Series([10.0, 20.0, 30.0])
city   -> Note(["Accra", "Tema", "Kumasi"])
```

25.1.54 load_json

Signature

```
static Catalog load_json(std::string_view filepath)
```

Parameters

`filepath` : `std::string_view` See the signature for the supported argument form.

Returns

`Catalog` Value produced by this call.

Notes

Loads the object from disk using the corresponding PlussMic loader route.

See also

- `static Catalog load_json(std::string_view filepath, const pluss::loader::Config& config)`

Examples

```
auto out = pluss::kindle::Catalog::load_json("PlussMic/data/messy_data.csv");
```

Output

```
id     -> Series([1, 2, 3])
metric -> Series([10.0, 20.0, 30.0])
city   -> Note(["Accra", "Tema", "Kumasi"])
```

25.1.55 load_json

Signature

```
static Catalog load_json(std::string_view filepath, const pluss::loader::Config& config)
```

Parameters

`filepath` : `std::string_view` See the signature for the supported argument form.

`config` : `const pluss::loader::Config&` See the signature for the supported argument form.

Returns

`Catalog` Value produced by this call.

Notes

Loads the object from disk using the corresponding PlussMic loader route.

See also

- `static Catalog load_json(std::string_view filepath)`

Examples

```
auto out = pluss::kindle::Catalog::load_json("PlussMic/data/messy_data.csv", {"nrows",␣
↪16});
```

Output

```
id     -> Series([1, 2, 3])
metric -> Series([10.0, 20.0, 30.0])
city   -> Note(["Accra", "Tema", "Kumasi"])
```

25.1.56 load_parquet

Signature

```
static Catalog load_parquet(std::string_view filepath)
```

Parameters

`filepath` : `std::string_view` See the signature for the supported argument form.

Returns

`Catalog` Value produced by this call.

Notes

Loads the object from disk using the corresponding PlussMic loader route.

See also

- `static Catalog load_parquet(std::string_view filepath, const pluss::loader::Config& config)`

Examples

```
auto out = pluss::kindle::Catalog::load_parquet("PlussMic/data/messy_data.csv");
```

Output

```
id     -> Series([1, 2, 3])
metric -> Series([10.0, 20.0, 30.0])
city   -> Note(["Accra", "Tema", "Kumasi"])
```

25.1.57 load_parquet

Signature

```
static Catalog load_parquet(std::string_view filepath, const pluss::loader::Config&␣
↪config)
```

Parameters

`filepath` : `std::string_view` See the signature for the supported argument form.

`config` : `const pluss::loader::Config&` See the signature for the supported argument form.

Returns

`Catalog` Value produced by this call.

Notes

Loads the object from disk using the corresponding PlussMic loader route.

See also

- `static Catalog load_parquet(std::string_view filepath)`

Examples

```
auto out = pluss::kindle::Catalog::load_parquet("PlussMic/data/messy_data.csv", {"nrows",
↪ 16});
```

Output

```
id     -> Series([1, 2, 3])
metric -> Series([10.0, 20.0, 30.0])
city   -> Note(["Accra", "Tema", "Kumasi"])
```

25.1.58 load_txt

Signature

```
static Catalog load_txt(std::string_view filepath)
```

Parameters

`filepath` : `std::string_view` See the signature for the supported argument form.

Returns

`Catalog` Value produced by this call.

Notes

Loads the object from disk using the corresponding PlussMic loader route.

See also

- `static Catalog load_txt(std::string_view filepath, const pluss::loader::Config& config)`

Examples

```
auto out = pluss::kindle::Catalog::load_txt("PlussMic/data/messy_data.csv");
```

Output

```
id     -> Series([1, 2, 3])
metric -> Series([10.0, 20.0, 30.0])
city   -> Note(["Accra", "Tema", "Kumasi"])
```

25.1.59 load_txt

Signature

```
static Catalog load_txt(std::string_view filepath, const pluss::loader::Config& config)
```

Parameters

`filepath` : `std::string_view` See the signature for the supported argument form.

`config` : `const pluss::loader::Config&` See the signature for the supported argument form.

Returns

`Catalog` Value produced by this call.

Notes

Loads the object from disk using the corresponding PlussMic loader route.

See also

- `static Catalog load_txt(std::string_view filepath)`

Examples

```
auto out = pluss::kindle::Catalog::load_txt("PlussMic/data/messy_data.csv", {"nrows", 16}
↪);
```

Output

```
id     -> Series([1, 2, 3])
metric -> Series([10.0, 20.0, 30.0])
city   -> Note(["Accra", "Tema", "Kumasi"])
```

25.1.60 masked_select

Signature

```
Catalog masked_select(self, const pluss::Series& mask) const
```

Parameters

mask : const pluss::Series& See the signature for the supported argument form.

Returns

Catalog Value produced by this call.

Notes

Use this to reduce the current object to the rows or fields that matter for the next step.

Examples

```
using namespace pluss::kindle::literals;

auto catalog = pluss::kindle::Catalog({
    "id"_k = pluss::kindle::Stream(pluss::make_series({1, 2, 3})),
    "metric"_k = pluss::kindle::Stream(pluss::make_series({10.0, 20.0, 30.0})),
    "city"_k = pluss::kindle::Stream(pluss::make_note({"Accra", "Tema", "Kumasi"})),
});

auto mask = pluss::make_series({true, false, true}, pluss::codem::ScalarType::Bool);
auto out = catalog.masked_select(mask);
```

Output

```
id     -> Series([1, 2, 3])
metric -> Series([10.0, 20.0, 30.0])
city   -> Note(["Accra", "Tema", "Kumasi"])
```

25.1.61 max

Signature

```
Stream max(self, std::string_view field) const
```

Parameters

`field` : `std::string_view` See the signature for the supported argument form.

Returns

`Stream` Value produced by this call.

Notes

Use this when you want a summary view instead of carrying the full input forward unchanged.

Examples

```
using namespace pluss::kindle::literals;

auto catalog = pluss::kindle::Catalog({
    "id"_k = pluss::kindle::Stream(pluss::make_series({1, 2, 3})),
    "metric"_k = pluss::kindle::Stream(pluss::make_series({10.0, 20.0, 30.0})),
    "city"_k = pluss::kindle::Stream(pluss::make_note({"Accra", "Tema", "Kumasi"})),
});

auto out = catalog.max("metric");
```

Output

```
Series([20.0])
```

25.1.62 mean

Signature

```
Stream mean(self, std::string_view field) const
```

Parameters

`field` : `std::string_view` See the signature for the supported argument form.

Returns

`Stream` Value produced by this call.

Notes

Use this when you want a summary view instead of carrying the full input forward unchanged.

Examples

```
using namespace pluss::kindle::literals;

auto catalog = pluss::kindle::Catalog({
    "id"_k = pluss::kindle::Stream(pluss::make_series({1, 2, 3})),
    "metric"_k = pluss::kindle::Stream(pluss::make_series({10.0, 20.0, 30.0})),
    "city"_k = pluss::kindle::Stream(pluss::make_note({"Accra", "Tema", "Kumasi"})),
```

(continues on next page)

(continued from previous page)

```
});

auto out = catalog.mean("metric");
```

Output

```
Series([20.0])
```

25.1.63 merge

Signature

```
Catalog merge(self, const Catalog& other, std::string_view on, std::string_view how =
↪"inner") const
```

Parameters

other : const Catalog& See the signature for the supported argument form.

on : std::string_view See the signature for the supported argument form.

how : std::string_view See the signature for the supported argument form.

Returns

Catalog Value produced by this call.

Notes

Use this when you need to merge compatible objects or extend the current one with another input.

Examples

```
using namespace pluss::kindle::literals;

auto catalog = pluss::kindle::Catalog({
    "id"_k = pluss::kindle::Stream(pluss::make_series({1, 2, 3})),
    "metric"_k = pluss::kindle::Stream(pluss::make_series({10.0, 20.0, 30.0})),
    "city"_k = pluss::kindle::Stream(pluss::make_note({"Accra", "Tema", "Kumasi"})),
});

using namespace pluss::kindle::literals;
auto right = pluss::kindle::Catalog({"id"_k = pluss::kindle::Stream(pluss::make_series(
↪{1, 2, 3})), "score"_k = pluss::kindle::Stream(pluss::make_series({0.9, 0.7, 0.8}))});
auto out = catalog.merge(right, "id", "inner");
```

Output

```
id     -> Series([1, 2, 3])
metric -> Series([10.0, 20.0, 30.0])
city   -> Note(["Accra", "Tema", "Kumasi"])
```

25.1.64 min

Signature

```
Stream min(self, std::string_view field) const
```

Parameters

`field` : `std::string_view` See the signature for the supported argument form.

Returns

`Stream` Value produced by this call.

Notes

Use this when you want a summary view instead of carrying the full input forward unchanged.

Examples

```
using namespace pluss::kindle::literals;

auto catalog = pluss::kindle::Catalog({
    "id"_k = pluss::kindle::Stream(pluss::make_series({1, 2, 3})),
    "metric"_k = pluss::kindle::Stream(pluss::make_series({10.0, 20.0, 30.0})),
    "city"_k = pluss::kindle::Stream(pluss::make_note({"Accra", "Tema", "Kumasi"})),
});

auto out = catalog.min("metric");
```

Output

```
Series([20.0])
```

25.1.65 nfields

Signature

```
std::int64_t nfields(self) const
```

Parameters

None

Returns

`std::int64_t` Integer count, size, or positional result.

Notes

This call is part of the public `Catalog` API and is documented here using a concrete usage example.

Examples

```
using namespace pluss::kindle::literals;

auto catalog = pluss::kindle::Catalog({
    "id"_k = pluss::kindle::Stream(pluss::make_series({1, 2, 3})),
    "metric"_k = pluss::kindle::Stream(pluss::make_series({10.0, 20.0, 30.0})),
    "city"_k = pluss::kindle::Stream(pluss::make_note({"Accra", "Tema", "Kumasi"})),
});

auto out = catalog.nfields();
```

Output

```
3
```

25.1.66 `note`

Signature

```
Catalog note(self) const
```

Parameters

None

Returns

`Catalog` Value produced by this call.

Notes

This call is part of the public `Catalog` API and is documented here using a concrete usage example.

Examples

```
using namespace pluss::kindle::literals;

auto catalog = pluss::kindle::Catalog({
    "id"_k = pluss::kindle::Stream(pluss::make_series({1, 2, 3})),
    "metric"_k = pluss::kindle::Stream(pluss::make_series({10.0, 20.0, 30.0})),
    "city"_k = pluss::kindle::Stream(pluss::make_note({"Accra", "Tema", "Kumasi"})),
});

auto out = catalog.note();
```

Output

```
id     -> Series([1, 2, 3])
metric -> Series([10.0, 20.0, 30.0])
city   -> Note(["Accra", "Tema", "Kumasi"])
```

25.1.67 notnull

Signature

```
Catalog notnull(self, std::initializer_list<std::string_view> keys =
```

Parameters

keys : std::initializer_list<std::string_view> See the signature for the supported argument form.

Returns

Catalog Value produced by this call.

Notes

This is part of the missing-data cleanup path and is usually applied before summaries, grouping, or export.

Examples

```
using namespace pluss::kindle::literals;

auto catalog = pluss::kindle::Catalog({
    "id"_k = pluss::kindle::Stream(pluss::make_series({1, 2, 3})),
    "metric"_k = pluss::kindle::Stream(pluss::make_series({10.0, 20.0, 30.0})),
    "city"_k = pluss::kindle::Stream(pluss::make_note({"Accra", "Tema", "Kumasi"})),
});

auto out = catalog.notnull();
```

Output

```
id     -> Series([1, 2, 3])
metric -> Series([10.0, 20.0, 30.0])
city   -> Note(["Accra", "Tema", "Kumasi"])
```

25.1.68 nstreams

Signature

```
std::int64_t nstreams(self) const
```

Parameters

None

Returns

std::int64_t Integer count, size, or positional result.

Notes

This call is part of the public `Catalog` API and is documented here using a concrete usage example.

Examples

```
using namespace pluss::kindle::literals;

auto catalog = pluss::kindle::Catalog({
    "id"_k = pluss::kindle::Stream(pluss::make_series({1, 2, 3})),
    "metric"_k = pluss::kindle::Stream(pluss::make_series({10.0, 20.0, 30.0})),
    "city"_k = pluss::kindle::Stream(pluss::make_note({"Accra", "Tema", "Kumasi"})),
});

auto out = catalog.nstreams();
```

Output

```
3
```

25.1.69 query

Signature

```
Catalog query(self, const QueryFn& fn) const
```

Parameters

`fn` : `const QueryFn&` See the signature for the supported argument form.

Returns

`Catalog` Value produced by this call.

Notes

Use this to reduce the current object to the rows or fields that matter for the next step.

Examples

```
using namespace pluss::kindle::literals;

auto catalog = pluss::kindle::Catalog({
    "id"_k = pluss::kindle::Stream(pluss::make_series({1, 2, 3})),
    "metric"_k = pluss::kindle::Stream(pluss::make_series({10.0, 20.0, 30.0})),
    "city"_k = pluss::kindle::Stream(pluss::make_note({"Accra", "Tema", "Kumasi"})),
});

auto out = catalog.query([](const pluss::kindle::Catalog& c) { return c.stream("metric").
↪as_series().gt(15.0); });
```

Output

```
id     -> Series([1, 2, 3])
metric -> Series([10.0, 20.0, 30.0])
city   -> Note(["Accra", "Tema", "Kumasi"])
```

25.1.70 rename

Signature

```
Catalog rename(self, const std::unordered_map<std::string, std::string>& mapping) const
```

Parameters

mapping : const std::unordered_map<std::string, std::string>& See the signature for the supported argument form.

Returns

Catalog Value produced by this call.

Notes

This adjusts labels without changing the underlying payload values.

See also

- Catalog rename(self, std::initializer_list<std::pair<std::string_view, std::string_view>> mapping) const
- Catalog rename(self, std::string_view from, std::string_view to) const

Examples

```
using namespace pluss::kindle::literals;

auto catalog = pluss::kindle::Catalog({
    "id"_k = pluss::kindle::Stream(pluss::make_series({1, 2, 3})),
    "metric"_k = pluss::kindle::Stream(pluss::make_series({10.0, 20.0, 30.0})),
    "city"_k = pluss::kindle::Stream(pluss::make_note({"Accra", "Tema", "Kumasi"})),
});

std::unordered_map<std::string, std::string> renames{{"metric", "distance_km"}};
auto out = catalog.rename(renames);
```

Output

```
id     -> Series([1, 2, 3])
metric -> Series([10.0, 20.0, 30.0])
city   -> Note(["Accra", "Tema", "Kumasi"])
```

25.1.71 rename

Signature

```
Catalog rename(self, std::initializer_list<std::pair<std::string_view, std::string_view>>
↪ mapping) const
```

Parameters

`mapping` : `std::initializer_list<std::pair<std::string_view, std::string_view>>` See the signature for the supported argument form.

Returns

`Catalog` Value produced by this call.

Notes

This adjusts labels without changing the underlying payload values.

See also

- `Catalog rename(self, std::string_view from, std::string_view to) const`
- `Catalog rename(self, const std::unordered_map<std::string, std::string>& mapping) const`

Examples

```
using namespace pluss::kindle::literals;

auto catalog = pluss::kindle::Catalog({
    "id"_k = pluss::kindle::Stream(pluss::make_series({1, 2, 3})),
    "metric"_k = pluss::kindle::Stream(pluss::make_series({10.0, 20.0, 30.0})),
    "city"_k = pluss::kindle::Stream(pluss::make_note({"Accra", "Tema", "Kumasi"})),
});

auto out = catalog.rename({{"metric", "distance_km"}});
```

Output

```
id     -> Series([1, 2, 3])
metric -> Series([10.0, 20.0, 30.0])
city   -> Note(["Accra", "Tema", "Kumasi"])
```

25.1.72 rename

Signature

```
Catalog rename(self, std::string_view from, std::string_view to) const
```

Parameters

`from` : `std::string_view` See the signature for the supported argument form.

`to` : `std::string_view` See the signature for the supported argument form.

Returns

`Catalog` Value produced by this call.

Notes

This adjusts labels without changing the underlying payload values.

See also

- `Catalog rename(self, std::initializer_list<std::pair<std::string_view, std::string_view>> mapping) const`
- `Catalog rename(self, const std::unordered_map<std::string, std::string>& mapping) const`

Examples

```
using namespace pluss::kindle::literals;

auto catalog = pluss::kindle::Catalog({
    "id"_k = pluss::kindle::Stream(pluss::make_series({1, 2, 3})),
    "metric"_k = pluss::kindle::Stream(pluss::make_series({10.0, 20.0, 30.0})),
    "city"_k = pluss::kindle::Stream(pluss::make_note({"Accra", "Tema", "Kumasi"})),
});

auto out = catalog.rename("metric", "distance_km");
```

Output

```
id     -> Series([1, 2, 3])
metric -> Series([10.0, 20.0, 30.0])
city   -> Note(["Accra", "Tema", "Kumasi"])
```

25.1.73 `rename_`

Signature

```
Catalog& rename_(self, std::string_view from, std::string_view to)
```

Parameters

`from` : `std::string_view` See the signature for the supported argument form.

`to` : `std::string_view` See the signature for the supported argument form.

Returns

`Catalog&` Value produced by this call.

Notes

This adjusts labels without changing the underlying payload values.

Examples

```
using namespace pluss::kindle::literals;

auto catalog = pluss::kindle::Catalog({
    "id"_k = pluss::kindle::Stream(pluss::make_series({1, 2, 3})),
    "metric"_k = pluss::kindle::Stream(pluss::make_series({10.0, 20.0, 30.0})),
    "city"_k = pluss::kindle::Stream(pluss::make_note({"Accra", "Tema", "Kumasi"})),
});

catalog.rename_("metric", "distance_km");
```

25.1.74 `sample`

Signature

```
Catalog sample(self, std::int64_t n, std::optional<std::uint64_t> random_state =␣
↪std::nullopt) const
```

Parameters

`n` : `std::int64_t` See the signature for the supported argument form.

`random_state` : `std::optional<std::uint64_t>` See the signature for the supported argument form.

Returns

`Catalog` Value produced by this call.

Notes

Use this to narrow the current object to the rows or depth range you want to inspect next.

Examples

```
using namespace pluss::kindle::literals;

auto catalog = pluss::kindle::Catalog({
    "id"_k = pluss::kindle::Stream(pluss::make_series({1, 2, 3})),
    "metric"_k = pluss::kindle::Stream(pluss::make_series({10.0, 20.0, 30.0})),
    "city"_k = pluss::kindle::Stream(pluss::make_note({"Accra", "Tema", "Kumasi"})),
});

auto out = catalog.sample(2, 7);
```

Output

```
id     -> Series([1, 2, 3])
metric -> Series([10.0, 20.0, 30.0])
city   -> Note(["Accra", "Tema", "Kumasi"])
```

25.1.75 save_as

Signature

```
void save_as(self, std::string_view target_type, std::string_view filename, const␣
↪pluss::loader::Config& config =
```

Parameters

`target_type` : `std::string_view` See the signature for the supported argument form.

`filename` : `std::string_view` See the signature for the supported argument form.

`config` : `const pluss::loader::Config&` See the signature for the supported argument form.

Returns

`void` Performs the side effect and returns no value.

Notes

Writes the current object to disk using the corresponding PlussMic saver route.

Examples

```
using namespace pluss::kindle::literals;

auto catalog = pluss::kindle::Catalog({
    "id"_k = pluss::kindle::Stream(pluss::make_series({1, 2, 3})),
    "metric"_k = pluss::kindle::Stream(pluss::make_series({10.0, 20.0, 30.0})),
    "city"_k = pluss::kindle::Stream(pluss::make_note({"Accra", "Tema", "Kumasi"})),
});

catalog.save_as("json", "out.json");
```

Output

```
saved: out.*
```

25.1.76 save_csv

Signature

```
void save_csv(self, std::string_view filename) const
```

Parameters

`filename` : `std::string_view` See the signature for the supported argument form.

Returns

`void` Performs the side effect and returns no value.

Notes

Writes the current object to disk using the corresponding PlussMic saver route.

See also

- `void save_csv(self, std::string_view filename, const pluss::loader::Config& config) const`

Examples

```
using namespace pluss::kindle::literals;

auto catalog = pluss::kindle::Catalog({
    "id"_k = pluss::kindle::Stream(pluss::make_series({1, 2, 3})),
    "metric"_k = pluss::kindle::Stream(pluss::make_series({10.0, 20.0, 30.0})),
    "city"_k = pluss::kindle::Stream(pluss::make_note({"Accra", "Tema", "Kumasi"})),
});

catalog.save_csv("out.csv");
```

Output

```
saved: out.*
```

25.1.77 save_csv

Signature

```
void save_csv(self, std::string_view filename, const pluss::loader::Config& config) const
```

Parameters

`filename` : `std::string_view` See the signature for the supported argument form.

`config` : `const pluss::loader::Config&` See the signature for the supported argument form.

Returns

`void` Performs the side effect and returns no value.

Notes

Writes the current object to disk using the corresponding PlussMic saver route.

See also

- `void save_csv(self, std::string_view filename) const`

Examples

```
using namespace pluss::kindle::literals;

auto catalog = pluss::kindle::Catalog({
    "id"_k = pluss::kindle::Stream(pluss::make_series({1, 2, 3})),
    "metric"_k = pluss::kindle::Stream(pluss::make_series({10.0, 20.0, 30.0})),
    "city"_k = pluss::kindle::Stream(pluss::make_note({"Accra", "Tema", "Kumasi"})),
});

catalog.save_csv("out.csv");
```

Output

```
saved: out.*
```

25.1.78 save_excel

Signature

```
void save_excel(self, std::string_view filename) const
```

Parameters

`filename` : `std::string_view` See the signature for the supported argument form.

Returns

`void` Performs the side effect and returns no value.

Notes

Writes the current object to disk using the corresponding PlussMic saver route.

See also

- `void save_excel(self, std::string_view filename, const pluss::loader::Config& config) const`

Examples

```
using namespace pluss::kindle::literals;

auto catalog = pluss::kindle::Catalog({
    "id"_k = pluss::kindle::Stream(pluss::make_series({1, 2, 3})),
    "metric"_k = pluss::kindle::Stream(pluss::make_series({10.0, 20.0, 30.0})),
    "city"_k = pluss::kindle::Stream(pluss::make_note({"Accra", "Tema", "Kumasi"})),
});

catalog.save_excel("out.xlsx");
```

Output

```
saved: out.*
```

25.1.79 save_excel

Signature

```
void save_excel(self, std::string_view filename, const pluss::loader::Config& config)␣
↪const
```

Parameters

`filename` : `std::string_view` See the signature for the supported argument form.

`config` : `const pluss::loader::Config&` See the signature for the supported argument form.

Returns

`void` Performs the side effect and returns no value.

Notes

Writes the current object to disk using the corresponding PlussMic saver route.

See also

- `void save_excel(self, std::string_view filename) const`

Examples

```
using namespace pluss::kindle::literals;

auto catalog = pluss::kindle::Catalog({
    "id"_k = pluss::kindle::Stream(pluss::make_series({1, 2, 3})),
    "metric"_k = pluss::kindle::Stream(pluss::make_series({10.0, 20.0, 30.0})),
    "city"_k = pluss::kindle::Stream(pluss::make_note({"Accra", "Tema", "Kumasi"})),
});

catalog.save_excel("out.xlsx");
```

Output

```
saved: out.*
```

25.1.80 save_json

Signature

```
void save_json(self, std::string_view filename) const
```

Parameters

`filename` : `std::string_view` See the signature for the supported argument form.

Returns

`void` Performs the side effect and returns no value.

Notes

Writes the current object to disk using the corresponding PlussMic saver route.

See also

- `void save_json(self, std::string_view filename, const pluss::loader::Config& config) const`

Examples

```
using namespace pluss::kindle::literals;

auto catalog = pluss::kindle::Catalog({
    "id"_k = pluss::kindle::Stream(pluss::make_series({1, 2, 3})),
    "metric"_k = pluss::kindle::Stream(pluss::make_series({10.0, 20.0, 30.0})),
    "city"_k = pluss::kindle::Stream(pluss::make_note({"Accra", "Tema", "Kumasi"})),
});

catalog.save_json("out.json");
```

Output

```
saved: out.*
```

25.1.81 save_json

Signature

```
void save_json(self, std::string_view filename, const pluss::loader::Config& config)␣
↪const
```

Parameters

`filename` : `std::string_view` See the signature for the supported argument form.

`config` : `const pluss::loader::Config&` See the signature for the supported argument form.

Returns

`void` Performs the side effect and returns no value.

Notes

Writes the current object to disk using the corresponding PlussMic saver route.

See also

- `void save_json(self, std::string_view filename) const`

Examples

```
using namespace pluss::kindle::literals;

auto catalog = pluss::kindle::Catalog({
    "id"_k = pluss::kindle::Stream(pluss::make_series({1, 2, 3})),
    "metric"_k = pluss::kindle::Stream(pluss::make_series({10.0, 20.0, 30.0})),
    "city"_k = pluss::kindle::Stream(pluss::make_note({"Accra", "Tema", "Kumasi"})),
});

catalog.save_json("out.json");
```

Output

```
saved: out.*
```

25.1.82 save_parquet

Signature

```
void save_parquet(self, std::string_view filename) const
```

Parameters

`filename` : `std::string_view` See the signature for the supported argument form.

Returns

`void` Performs the side effect and returns no value.

Notes

Writes the current object to disk using the corresponding PlussMic saver route.

See also

- `void save_parquet(self, std::string_view filename, const pluss::loader::Config& config) const`

Examples

```
using namespace pluss::kindle::literals;

auto catalog = pluss::kindle::Catalog({
    "id"_k = pluss::kindle::Stream(pluss::make_series({1, 2, 3})),
    "metric"_k = pluss::kindle::Stream(pluss::make_series({10.0, 20.0, 30.0})),
    "city"_k = pluss::kindle::Stream(pluss::make_note({"Accra", "Tema", "Kumasi"})),
});

catalog.save_parquet("out.parquet");
```

Output

```
saved: out.*
```

25.1.83 save_parquet

Signature

```
void save_parquet(self, std::string_view filename, const pluss::loader::Config& config)␣
↪const
```

Parameters

`filename` : `std::string_view` See the signature for the supported argument form.

`config` : `const pluss::loader::Config&` See the signature for the supported argument form.

Returns

`void` Performs the side effect and returns no value.

Notes

Writes the current object to disk using the corresponding PlussMic saver route.

See also

- `void save_parquet(self, std::string_view filename) const`

Examples

```
using namespace pluss::kindle::literals;

auto catalog = pluss::kindle::Catalog({
    "id"_k = pluss::kindle::Stream(pluss::make_series({1, 2, 3})),
    "metric"_k = pluss::kindle::Stream(pluss::make_series({10.0, 20.0, 30.0})),
    "city"_k = pluss::kindle::Stream(pluss::make_note({"Accra", "Tema", "Kumasi"})),
});

catalog.save_parquet("out.parquet");
```

Output

```
saved: out.*
```

25.1.84 save_txt

Signature

```
void save_txt(self, std::string_view filename) const
```

Parameters

`filename` : `std::string_view` See the signature for the supported argument form.

Returns

`void` Performs the side effect and returns no value.

Notes

Writes the current object to disk using the corresponding PlussMic saver route.

See also

- `void save_txt(self, std::string_view filename, const pluss::loader::Config& config) const`

Examples

```
using namespace pluss::kindle::literals;

auto catalog = pluss::kindle::Catalog({
    "id"_k = pluss::kindle::Stream(pluss::make_series({1, 2, 3})),
    "metric"_k = pluss::kindle::Stream(pluss::make_series({10.0, 20.0, 30.0})),
    "city"_k = pluss::kindle::Stream(pluss::make_note({"Accra", "Tema", "Kumasi"})),
});

catalog.save_txt("out.txt");
```

Output

```
saved: out.*
```

25.1.85 `save_txt`

Signature

```
void save_txt(self, std::string_view filename, const pluss::loader::Config& config) const
```

Parameters

`filename` : `std::string_view` See the signature for the supported argument form.

`config` : `const pluss::loader::Config&` See the signature for the supported argument form.

Returns

`void` Performs the side effect and returns no value.

Notes

Writes the current object to disk using the corresponding PlussMic saver route.

See also

- `void save_txt(self, std::string_view filename) const`

Examples

```
using namespace pluss::kindle::literals;

auto catalog = pluss::kindle::Catalog({
    "id"_k = pluss::kindle::Stream(pluss::make_series({1, 2, 3})),
    "metric"_k = pluss::kindle::Stream(pluss::make_series({10.0, 20.0, 30.0})),
    "city"_k = pluss::kindle::Stream(pluss::make_note({"Accra", "Tema", "Kumasi"})),
});

catalog.save_txt("out.txt");
```

Output

```
saved: out.*
```

25.1.86 `select`

Signature

```
Catalog select(self, const std::vector<std::string>& field_names) const
```

Parameters

`field_names` : `const std::vector<std::string>&` See the signature for the supported argument form.

Returns

`Catalog` Value produced by this call.

Notes

Use this to reduce the current object to the rows or fields that matter for the next step.

See also

- `Catalog select(self, std::initializer_list<std::string_view> field_names) const`

Examples

```
using namespace pluss::kindle::literals;

auto catalog = pluss::kindle::Catalog({
    "id"_k = pluss::kindle::Stream(pluss::make_series({1, 2, 3})),
    "metric"_k = pluss::kindle::Stream(pluss::make_series({10.0, 20.0, 30.0})),
    "city"_k = pluss::kindle::Stream(pluss::make_note({"Accra", "Tema", "Kumasi"})),
});

std::vector<std::string> names{"id", "metric"};
auto out = catalog.select(names);
```

Output

```
id     -> Series([1, 2, 3])
metric -> Series([10.0, 20.0, 30.0])
city   -> Note(["Accra", "Tema", "Kumasi"])
```

25.1.87 select

Signature

```
Catalog select(self, std::initializer_list<std::string_view> field_names) const
```

Parameters

`field_names` : `std::initializer_list<std::string_view>` See the signature for the supported argument form.

Returns

`Catalog` Value produced by this call.

Notes

Use this to reduce the current object to the rows or fields that matter for the next step.

See also

- `Catalog select(self, const std::vector<std::string>& field_names) const`

Examples

```
using namespace pluss::kindle::literals;

auto catalog = pluss::kindle::Catalog({
    "id"_k = pluss::kindle::Stream(pluss::make_series({1, 2, 3})),
    "metric"_k = pluss::kindle::Stream(pluss::make_series({10.0, 20.0, 30.0})),
    "city"_k = pluss::kindle::Stream(pluss::make_note({"Accra", "Tema", "Kumasi"})),
});

auto out = catalog.select({"id", "metric"});
```

Output

```
id     -> Series([1, 2, 3])
metric -> Series([10.0, 20.0, 30.0])
city   -> Note(["Accra", "Tema", "Kumasi"])
```

25.1.88 series

Signature

```
Catalog series(self) const
```

Parameters

None

Returns

`Catalog` Value produced by this call.

Notes

This call is part of the public `Catalog` API and is documented here using a concrete usage example.

Examples

```
using namespace pluss::kindle::literals;

auto catalog = pluss::kindle::Catalog({
    "id"_k = pluss::kindle::Stream(pluss::make_series({1, 2, 3})),
    "metric"_k = pluss::kindle::Stream(pluss::make_series({10.0, 20.0, 30.0})),
    "city"_k = pluss::kindle::Stream(pluss::make_note({"Accra", "Tema", "Kumasi"})),
});

auto out = catalog.series();
```

Output

```
id     -> Series([1, 2, 3])
metric -> Series([10.0, 20.0, 30.0])
city   -> Note(["Accra", "Tema", "Kumasi"])
```

25.1.89 `set`

Signature

```
Catalog set(self, std::string_view key, const Stream& value) const
```

Parameters

`key` : `std::string_view` See the signature for the supported argument form.

`value` : `const Stream&` See the signature for the supported argument form.

Returns

`Catalog` Value produced by this call.

Notes

This call is part of the public `Catalog` API and is documented here using a concrete usage example.

Examples

```cpp
using namespace pluss::kindle::literals;

auto catalog = pluss::kindle::Catalog({
    "id"_k = pluss::kindle::Stream(pluss::make_series({1, 2, 3})),
    "metric"_k = pluss::kindle::Stream(pluss::make_series({10.0, 20.0, 30.0})),
    "city"_k = pluss::kindle::Stream(pluss::make_note({"Accra", "Tema", "Kumasi"})),
});

auto out = catalog.set_stream("flag", pluss::kindle::Stream(pluss::make_series({1, 0, 1}
↪)));
```

Output

```
id     -> Series([1, 2, 3])
metric -> Series([10.0, 20.0, 30.0])
city   -> Note(["Accra", "Tema", "Kumasi"])
```

25.1.90 `set_`

Signature

```cpp
Catalog& set_(self, std::string_view key, const Stream& value)
```

Parameters

`key` : `std::string_view` See the signature for the supported argument form.

`value` : `const Stream&` See the signature for the supported argument form.

Returns

`Catalog&` Value produced by this call.

Notes

This call is part of the public `Catalog` API and is documented here using a concrete usage example.

Examples

```cpp
using namespace pluss::kindle::literals;

auto catalog = pluss::kindle::Catalog({
    "id"_k = pluss::kindle::Stream(pluss::make_series({1, 2, 3})),
    "metric"_k = pluss::kindle::Stream(pluss::make_series({10.0, 20.0, 30.0})),
    "city"_k = pluss::kindle::Stream(pluss::make_note({"Accra", "Tema", "Kumasi"})),
});

catalog.set_stream_("flag", pluss::kindle::Stream(pluss::make_series({1, 0, 1})));
```

25.1.91 set_stream

Signature

```
Catalog set_stream(self, std::string_view key, const Stream& value) const
```

Parameters

key : std::string_view See the signature for the supported argument form.

value : const Stream& See the signature for the supported argument form.

Returns

Catalog Value produced by this call.

Notes

This mutates or derives structure while keeping the surrounding object model intact.

Examples

```
using namespace pluss::kindle::literals;

auto catalog = pluss::kindle::Catalog({
    "id"_k = pluss::kindle::Stream(pluss::make_series({1, 2, 3})),
    "metric"_k = pluss::kindle::Stream(pluss::make_series({10.0, 20.0, 30.0})),
    "city"_k = pluss::kindle::Stream(pluss::make_note({"Accra", "Tema", "Kumasi"})),
});

auto out = catalog.set_stream("flag", pluss::kindle::Stream(pluss::make_series({1, 0, 1}
↪)));
```

Output

```
id     -> Series([1, 2, 3])
metric -> Series([10.0, 20.0, 30.0])
city   -> Note(["Accra", "Tema", "Kumasi"])
```

25.1.92 set_stream_

Signature

```
Catalog& set_stream_(self, std::string_view key, const Stream& value)
```

Parameters

key : std::string_view See the signature for the supported argument form.

value : const Stream& See the signature for the supported argument form.

Returns

`Catalog&` Value produced by this call.

Notes

This mutates or derives structure while keeping the surrounding object model intact.

Examples

```
using namespace pluss::kindle::literals;

auto catalog = pluss::kindle::Catalog({
    "id"_k = pluss::kindle::Stream(pluss::make_series({1, 2, 3})),
    "metric"_k = pluss::kindle::Stream(pluss::make_series({10.0, 20.0, 30.0})),
    "city"_k = pluss::kindle::Stream(pluss::make_note({"Accra", "Tema", "Kumasi"})),
});

catalog.set_stream_("flag", pluss::kindle::Stream(pluss::make_series({1, 0, 1})));
```

25.1.93 `set_streams`

Signature

```
Catalog set_streams(self, std::initializer_list<NamedStream> items) const
```

Parameters

`items` : `std::initializer_list<NamedStream>` See the signature for the supported argument form.

Returns

`Catalog` Value produced by this call.

Notes

This mutates or derives structure while keeping the surrounding object model intact.

See also

- `Catalog set_streams(self, std::initializer_list<std::pair<std::string_view, Stream>> items) const`

Examples

```
using namespace pluss::kindle::literals;

auto catalog = pluss::kindle::Catalog({
    "id"_k = pluss::kindle::Stream(pluss::make_series({1, 2, 3})),
    "metric"_k = pluss::kindle::Stream(pluss::make_series({10.0, 20.0, 30.0})),
    "city"_k = pluss::kindle::Stream(pluss::make_note({"Accra", "Tema", "Kumasi"})),
});

using namespace pluss::kindle::literals;
```

(continues on next page)

(continued from previous page)

```
auto out = catalog.set_streams({
    "flag"_k = pluss::kindle::Stream(pluss::make_series({1, 0, 1})),
});
```

Output

```
id     -> Series([1, 2, 3])
metric -> Series([10.0, 20.0, 30.0])
city   -> Note(["Accra", "Tema", "Kumasi"])
```

25.1.94 set_streams

Signature

```
Catalog set_streams(self, std::initializer_list<std::pair<std::string_view, Stream>>␣
↪items) const
```

Parameters

`items` : `std::initializer_list<std::pair<std::string_view, Stream>>` See the signature for the supported argument form.

Returns

`Catalog` Value produced by this call.

Notes

This mutates or derives structure while keeping the surrounding object model intact.

See also

- `Catalog set_streams(self, std::initializer_list<NamedStream> items) const`

Examples

```
using namespace pluss::kindle::literals;

auto catalog = pluss::kindle::Catalog({
    "id"_k = pluss::kindle::Stream(pluss::make_series({1, 2, 3})),
    "metric"_k = pluss::kindle::Stream(pluss::make_series({10.0, 20.0, 30.0})),
    "city"_k = pluss::kindle::Stream(pluss::make_note({"Accra", "Tema", "Kumasi"})),
});

auto out = catalog.set_streams({
    {"flag", pluss::kindle::Stream(pluss::make_series({1, 0, 1}))},
});
```

Output

```
id     -> Series([1, 2, 3])
metric -> Series([10.0, 20.0, 30.0])
city   -> Note(["Accra", "Tema", "Kumasi"])
```

25.1.95 set_streams_

Signature

```
Catalog& set_streams_(self, std::initializer_list<NamedStream> items)
```

Parameters

`items` : `std::initializer_list<NamedStream>` See the signature for the supported argument form.

Returns

`Catalog&` Value produced by this call.

Notes

This mutates or derives structure while keeping the surrounding object model intact.

See also

- `Catalog& set_streams_(self, std::initializer_list<std::pair<std::string_view, Stream>> items)`

Examples

```
using namespace pluss::kindle::literals;

auto catalog = pluss::kindle::Catalog({
    "id"_k = pluss::kindle::Stream(pluss::make_series({1, 2, 3})),
    "metric"_k = pluss::kindle::Stream(pluss::make_series({10.0, 20.0, 30.0})),
    "city"_k = pluss::kindle::Stream(pluss::make_note({"Accra", "Tema", "Kumasi"})),
});

using namespace pluss::kindle::literals;
catalog.set_streams_({
    "flag"_k = pluss::kindle::Stream(pluss::make_series({1, 0, 1})),
});
```

25.1.96 set_streams_

Signature

```
Catalog& set_streams_(self, std::initializer_list<std::pair<std::string_view, Stream>>␣
↪items)
```

Parameters

`items` : `std::initializer_list<std::pair<std::string_view, Stream>>` See the signature for the supported argument form.

Returns

`Catalog&` Value produced by this call.

Notes

This mutates or derives structure while keeping the surrounding object model intact.

See also

- `Catalog& set_streams_(self, std::initializer_list<NamedStream> items)`

Examples

```
using namespace pluss::kindle::literals;

auto catalog = pluss::kindle::Catalog({
    "id"_k = pluss::kindle::Stream(pluss::make_series({1, 2, 3})),
    "metric"_k = pluss::kindle::Stream(pluss::make_series({10.0, 20.0, 30.0})),
    "city"_k = pluss::kindle::Stream(pluss::make_note({"Accra", "Tema", "Kumasi"})),
});

catalog.set_streams_({
    {"flag", pluss::kindle::Stream(pluss::make_series({1, 0, 1}))},
});
```

25.1.97 show

Signature

```
void show(self, std::ostream& os) const
```

Parameters

`os` : `std::ostream&` See the signature for the supported argument form.

Returns

`void` Performs the side effect and returns no value.

Notes

This renders a human-readable view for inspection instead of creating a new analytic object.

Examples

```
using namespace pluss::kindle::literals;

auto catalog = pluss::kindle::Catalog({
    "id"_k = pluss::kindle::Stream(pluss::make_series({1, 2, 3})),
```

(continues on next page)

(continued from previous page)

```
    "metric"_k = pluss::kindle::Stream(pluss::make_series({10.0, 20.0, 30.0})),
    "city"_k = pluss::kindle::Stream(pluss::make_note({"Accra", "Tema", "Kumasi"})),
});

auto out = catalog.show();
```

25.1.98 slice

Signature

```
Catalog slice(self, std::int64_t start, std::int64_t stop) const
```

Parameters

`start` : `std::int64_t` See the signature for the supported argument form.

`stop` : `std::int64_t` See the signature for the supported argument form.

Returns

`Catalog` Value produced by this call.

Notes

Use this to narrow the current object to the rows or depth range you want to inspect next.

Examples

```
using namespace pluss::kindle::literals;

auto catalog = pluss::kindle::Catalog({
    "id"_k = pluss::kindle::Stream(pluss::make_series({1, 2, 3})),
    "metric"_k = pluss::kindle::Stream(pluss::make_series({10.0, 20.0, 30.0})),
    "city"_k = pluss::kindle::Stream(pluss::make_note({"Accra", "Tema", "Kumasi"})),
});

auto out = catalog.slice(0, 2);
```

Output

```
id     -> Series([1, 2, 3])
metric -> Series([10.0, 20.0, 30.0])
city   -> Note(["Accra", "Tema", "Kumasi"])
```

25.1.99 sort

Signature

```
Catalog sort(self, std::string_view by_field, bool ascending = true) const
```

Parameters

`by_field` : `std::string_view` See the signature for the supported argument form.

`ascending` : `bool` See the signature for the supported argument form.

Returns

`Catalog` Value produced by this call.

Notes

This call is part of the public `Catalog` API and is documented here using a concrete usage example.

Examples

```
using namespace pluss::kindle::literals;

auto catalog = pluss::kindle::Catalog({
    "id"_k = pluss::kindle::Stream(pluss::make_series({1, 2, 3})),
    "metric"_k = pluss::kindle::Stream(pluss::make_series({10.0, 20.0, 30.0})),
    "city"_k = pluss::kindle::Stream(pluss::make_note({"Accra", "Tema", "Kumasi"})),
});

auto out = catalog.sort("metric", false);
```

Output

```
id     -> Series([1, 2, 3])
metric -> Series([10.0, 20.0, 30.0])
city   -> Note(["Accra", "Tema", "Kumasi"])
```

25.1.100 sort_by

Signature

```
Catalog sort_by(self, const std::vector<std::string>& fields, const std::vector<bool>&␣
↪ascending =
```

Parameters

`fields` : `const std::vector<std::string>&` See the signature for the supported argument form.

`ascending` : `const std::vector<bool>&` See the signature for the supported argument form.

Returns

`Catalog` Value produced by this call.

Notes

This call is part of the public `Catalog` API and is documented here using a concrete usage example.

Examples

```
using namespace pluss::kindle::literals;

auto catalog = pluss::kindle::Catalog({
    "id"_k = pluss::kindle::Stream(pluss::make_series({1, 2, 3})),
    "metric"_k = pluss::kindle::Stream(pluss::make_series({10.0, 20.0, 30.0})),
    "city"_k = pluss::kindle::Stream(pluss::make_note({"Accra", "Tema", "Kumasi"})),
});

auto out = catalog.sort_by({"city", "id"}, {true, false});
```

Output

```
id     -> Series([1, 2, 3])
metric -> Series([10.0, 20.0, 30.0])
city   -> Note(["Accra", "Tema", "Kumasi"])
```

25.1.101 `stream`

Signature

```
Stream stream(self, std::string_view key) const
```

Parameters

`key` : `std::string_view` See the signature for the supported argument form.

Returns

`Stream` Value produced by this call.

Notes

This call is part of the public `Catalog` API and is documented here using a concrete usage example.

Examples

```
using namespace pluss::kindle::literals;

auto catalog = pluss::kindle::Catalog({
    "id"_k = pluss::kindle::Stream(pluss::make_series({1, 2, 3})),
    "metric"_k = pluss::kindle::Stream(pluss::make_series({10.0, 20.0, 30.0})),
    "city"_k = pluss::kindle::Stream(pluss::make_note({"Accra", "Tema", "Kumasi"})),
});

auto out = catalog.stream("metric");
```

Output

```
Series([10.0, 20.0, 30.0])
```

25.1.102 streams

Signature

```
std::vector<Stream> streams(self) const
```

Parameters

None

Returns

std::vector<Stream> Materialized collection of stream objects.

Notes

This call is part of the public Catalog API and is documented here using a concrete usage example.

Examples

```
using namespace pluss::kindle::literals;

auto catalog = pluss::kindle::Catalog({
    "id"_k = pluss::kindle::Stream(pluss::make_series({1, 2, 3})),
    "metric"_k = pluss::kindle::Stream(pluss::make_series({10.0, 20.0, 30.0})),
    "city"_k = pluss::kindle::Stream(pluss::make_note({"Accra", "Tema", "Kumasi"})),
});

auto out = catalog.streams();
```

Output

```
[Stream(id), Stream(metric), Stream(city)]
```

25.1.103 sum

Signature

```
Stream sum(self, std::string_view field) const
```

Parameters

field : std::string_view See the signature for the supported argument form.

Returns

Stream Value produced by this call.

Notes

Use this when you want a summary view instead of carrying the full input forward unchanged.

Examples

```
using namespace pluss::kindle::literals;

auto catalog = pluss::kindle::Catalog({
    "id"_k = pluss::kindle::Stream(pluss::make_series({1, 2, 3})),
    "metric"_k = pluss::kindle::Stream(pluss::make_series({10.0, 20.0, 30.0})),
    "city"_k = pluss::kindle::Stream(pluss::make_note({"Accra", "Tema", "Kumasi"})),
});

auto out = catalog.sum("metric");
```

Output

```
Series([20.0])
```

25.1.104 tabelize

Signature

```
pluss::table::DataFrame tabelize(self, const TabelizePolicy& policy =
```

Parameters

`policy` : `const TabelizePolicy&` See the signature for the supported argument form.

Returns

`pluss::table::DataFrame` Value produced by this call.

Notes

This is an explicit bridge between PlussMic models so you can switch to the representation that fits the next task.

Examples

```
using namespace pluss::kindle::literals;

auto catalog = pluss::kindle::Catalog({
    "id"_k = pluss::kindle::Stream(pluss::make_series({1, 2, 3})),
    "metric"_k = pluss::kindle::Stream(pluss::make_series({10.0, 20.0, 30.0})),
    "city"_k = pluss::kindle::Stream(pluss::make_note({"Accra", "Tema", "Kumasi"})),
});

pluss::kindle::TabelizePolicy policy;
policy.align = pluss::kindle::AlignPolicy::Inner;
auto out = catalog.tabelize(policy);
```

Output

```
id  sales  city
1   100.0  Accra
```

(continues on next page)

(continued from previous page)

```
2   120.0  Tema
3    90.0  Kumasi
```

25.1.105 tail

Signature

```
Catalog tail(self, std::int64_t n) const
```

Parameters

n : std::int64_t See the signature for the supported argument form.

Returns

Catalog Value produced by this call.

Notes

Use this to narrow the current object to the rows or depth range you want to inspect next.

Examples

```
using namespace pluss::kindle::literals;

auto catalog = pluss::kindle::Catalog({
    "id"_k = pluss::kindle::Stream(pluss::make_series({1, 2, 3})),
    "metric"_k = pluss::kindle::Stream(pluss::make_series({10.0, 20.0, 30.0})),
    "city"_k = pluss::kindle::Stream(pluss::make_note({"Accra", "Tema", "Kumasi"})),
});

auto out = catalog.tail(2);
```

Output

```
id     -> Series([1, 2, 3])
metric -> Series([10.0, 20.0, 30.0])
city   -> Note(["Accra", "Tema", "Kumasi"])
```

25.1.106 take

Signature

```
Catalog take(self, std::span<const std::int64_t> indices) const
```

Parameters

indices : std::span<const std::int64_t> See the signature for the supported argument form.

Returns

`Catalog` Value produced by this call.

Notes

Use this to narrow the current object to the rows or depth range you want to inspect next.

Examples

```
using namespace pluss::kindle::literals;

auto catalog = pluss::kindle::Catalog({
    "id"_k = pluss::kindle::Stream(pluss::make_series({1, 2, 3})),
    "metric"_k = pluss::kindle::Stream(pluss::make_series({10.0, 20.0, 30.0})),
    "city"_k = pluss::kindle::Stream(pluss::make_note({"Accra", "Tema", "Kumasi"})),
});

std::vector<std::int64_t> picks{0, 2};
auto out = catalog.take(picks);
```

Output

```
id     -> Series([1, 2, 3])
metric -> Series([10.0, 20.0, 30.0])
city   -> Note(["Accra", "Tema", "Kumasi"])
```

25.1.107 `text_preprocess`

Signature

```
Catalog text_preprocess(self, std::string_view field) const
```

Parameters

`field` : `std::string_view` See the signature for the supported argument form.

Returns

`Catalog` Value produced by this call.

Notes

This is part of the text-processing surface for note-backed or text-oriented data.

Examples

```
using namespace pluss::kindle::literals;

auto catalog = pluss::kindle::Catalog({
    "text"_k = pluss::kindle::Stream(pluss::make_note({
        "heavy rain on road",
        "clear weather downtown",
        "storm warning issued",
```

(continues on next page)

(continued from previous page)

```
    })),
});

auto out = catalog.text_preprocess("text");
```

Output

```
id     -> Series([1, 2, 3])
metric -> Series([10.0, 20.0, 30.0])
city   -> Note(["Accra", "Tema", "Kumasi"])
```

25.1.108 `tfidf`

Signature

```
Catalog tfidf(self, std::string_view field) const
```

Parameters

`field` : `std::string_view` See the signature for the supported argument form.

Returns

`Catalog` Value produced by this call.

Notes

This is part of the text-processing surface for note-backed or text-oriented data.

Examples

```
using namespace pluss::kindle::literals;

auto catalog = pluss::kindle::Catalog({
    "text"_k = pluss::kindle::Stream(pluss::make_note({
        "heavy rain on road",
        "clear weather downtown",
        "storm warning issued",
    })),
});

auto out = catalog.tfidf("text");
```

Output

```
id     -> Series([1, 2, 3])
metric -> Series([10.0, 20.0, 30.0])
city   -> Note(["Accra", "Tema", "Kumasi"])
```

25.1.109 `timeindex`

Signature

```
Catalog timeindex(self) const
```

Parameters

None

Returns

`Catalog` Value produced by this call.

Notes

This call is part of the public `Catalog` API and is documented here using a concrete usage example.

Examples

```
using namespace pluss::kindle::literals;

auto catalog = pluss::kindle::Catalog({
    "id"_k = pluss::kindle::Stream(pluss::make_series({1, 2, 3})),
    "metric"_k = pluss::kindle::Stream(pluss::make_series({10.0, 20.0, 30.0})),
    "city"_k = pluss::kindle::Stream(pluss::make_note({"Accra", "Tema", "Kumasi"})),
});

auto out = catalog.timeindex();
```

Output

```
id     -> Series([1, 2, 3])
metric -> Series([10.0, 20.0, 30.0])
city   -> Note(["Accra", "Tema", "Kumasi"])
```

25.1.110 `timeline`

Signature

```
Catalog timeline(self) const
```

Parameters

None

Returns

`Catalog` Value produced by this call.

Notes

This call is part of the public `Catalog` API and is documented here using a concrete usage example.

Examples

```
using namespace pluss::kindle::literals;

auto catalog = pluss::kindle::Catalog({
    "id"_k = pluss::kindle::Stream(pluss::make_series({1, 2, 3})),
    "metric"_k = pluss::kindle::Stream(pluss::make_series({10.0, 20.0, 30.0})),
    "city"_k = pluss::kindle::Stream(pluss::make_note({"Accra", "Tema", "Kumasi"})),
});

auto out = catalog.timeline();
```

Output

```
id     -> Series([1, 2, 3])
metric -> Series([10.0, 20.0, 30.0])
city   -> Note(["Accra", "Tema", "Kumasi"])
```

25.1.111 `to_json`

Signature

```
std::string to_json(self) const
```

Parameters

None

Returns

`std::string` Rendered string result.

Notes

This call is part of the public `Catalog` API and is documented here using a concrete usage example.

Examples

```
using namespace pluss::kindle::literals;

auto catalog = pluss::kindle::Catalog({
    "id"_k = pluss::kindle::Stream(pluss::make_series({1, 2, 3})),
    "metric"_k = pluss::kindle::Stream(pluss::make_series({10.0, 20.0, 30.0})),
    "city"_k = pluss::kindle::Stream(pluss::make_note({"Accra", "Tema", "Kumasi"})),
});

auto out = catalog.to_json();
```

Output

```
id     -> Series([1, 2, 3])
metric -> Series([10.0, 20.0, 30.0])
city   -> Note(["Accra", "Tema", "Kumasi"])
```

25.1.112 to_string

Signature

```
std::string to_string(self) const
```

Parameters

None

Returns

`std::string` Rendered string result.

Notes

This call is part of the public `Catalog` API and is documented here using a concrete usage example.

Examples

```
using namespace pluss::kindle::literals;

auto catalog = pluss::kindle::Catalog({
    "id"_k = pluss::kindle::Stream(pluss::make_series({1, 2, 3})),
    "metric"_k = pluss::kindle::Stream(pluss::make_series({10.0, 20.0, 30.0})),
    "city"_k = pluss::kindle::Stream(pluss::make_note({"Accra", "Tema", "Kumasi"})),
});

auto out = catalog.to_string();
```

Output

```
id     -> Series([1, 2, 3])
metric -> Series([10.0, 20.0, 30.0])
city   -> Note(["Accra", "Tema", "Kumasi"])
```

25.1.113 to_table

Signature

```
pluss::table::DataFrame to_table(self, const TabelizePolicy& policy =
```

Parameters

`policy` : `const TabelizePolicy&` See the signature for the supported argument form.

Returns

`pluss::table::DataFrame` Value produced by this call.

Notes

This is an explicit bridge between PlussMic models so you can switch to the representation that fits the next task.

Examples

```
using namespace pluss::kindle::literals;

auto catalog = pluss::kindle::Catalog({
    "id"_k = pluss::kindle::Stream(pluss::make_series({1, 2, 3})),
    "metric"_k = pluss::kindle::Stream(pluss::make_series({10.0, 20.0, 30.0})),
    "city"_k = pluss::kindle::Stream(pluss::make_note({"Accra", "Tema", "Kumasi"})),
});

pluss::kindle::TabelizePolicy policy;
policy.align = pluss::kindle::AlignPolicy::Inner;
auto out = catalog.to_table(policy, true);
```

Output

```
id  sales  city
1   100.0  Accra
2   120.0  Tema
3    90.0  Kumasi
```

25.1.114 transform

Signature

```
Catalog transform(self, const std::function<Stream(const Stream&)>& fn) const
```

Parameters

`fn` : `const std::function<Stream(const Stream&)>&` See the signature for the supported argument form.

Returns

`Catalog` Value produced by this call.

Notes

This mutates or derives structure while keeping the surrounding object model intact.

See also

- `Catalog transform(self, std::string_view field, const std::function<Stream(const Stream&)>& fn) const`

Examples

```
using namespace pluss::kindle::literals;

auto catalog = pluss::kindle::Catalog({
    "id"_k = pluss::kindle::Stream(pluss::make_series({1, 2, 3})),
    "metric"_k = pluss::kindle::Stream(pluss::make_series({10.0, 20.0, 30.0})),
    "city"_k = pluss::kindle::Stream(pluss::make_note({"Accra", "Tema", "Kumasi"})),
});

auto out = catalog.transform([](const pluss::kindle::Stream& s) { return s.kind() ==␣
↪pluss::kindle::StreamKind::Series ? s.clip(0.0, 25.0) : s; });
```

Output

```
id     -> Series([1, 2, 3])
metric -> Series([10.0, 20.0, 30.0])
city   -> Note(["Accra", "Tema", "Kumasi"])
```

25.1.115 transform

Signature

```
Catalog transform(self, std::string_view field, const std::function<Stream(const Stream&
↪)>& fn) const
```

Parameters

`field` : `std::string_view` See the signature for the supported argument form.

`fn` : `const std::function<Stream(const Stream&)>&` See the signature for the supported argument form.

Returns

`Catalog` Value produced by this call.

Notes

This mutates or derives structure while keeping the surrounding object model intact.

See also

- `Catalog transform(self, const std::function<Stream(const Stream&)>& fn) const`

Examples

```
using namespace pluss::kindle::literals;

auto catalog = pluss::kindle::Catalog({
    "id"_k = pluss::kindle::Stream(pluss::make_series({1, 2, 3})),
    "metric"_k = pluss::kindle::Stream(pluss::make_series({10.0, 20.0, 30.0})),
    "city"_k = pluss::kindle::Stream(pluss::make_note({"Accra", "Tema", "Kumasi"})),
});
```

(continues on next page)

(continued from previous page)

```
auto out = catalog.transform("metric", [](const pluss::kindle::Stream& s) { return s.
↪clip(0.0, 25.0); });
```

Output

```
id     -> Series([1, 2, 3])
metric -> Series([10.0, 20.0, 30.0])
city   -> Note(["Accra", "Tema", "Kumasi"])
```

25.1.116 unique

Signature

```
bool unique(self) const
```

Parameters

None

Returns

bool Boolean status or membership result.

Notes

This call is part of the public Catalog API and is documented here using a concrete usage example.

Examples

```
using namespace pluss::kindle::literals;

auto catalog = pluss::kindle::Catalog({
    "id"_k = pluss::kindle::Stream(pluss::make_series({1, 2, 3})),
    "metric"_k = pluss::kindle::Stream(pluss::make_series({10.0, 20.0, 30.0})),
    "city"_k = pluss::kindle::Stream(pluss::make_note({"Accra", "Tema", "Kumasi"})),
});

auto out = catalog.unique();
```

Output

```
true
```

25.1.117 where

Signature

```
Catalog where(self, const pluss::Series& condition) const
```

Parameters

`condition` : `const pluss::Series&` See the signature for the supported argument form.

Returns

`Catalog` Value produced by this call.

Notes

Use this to reduce the current object to the rows or fields that matter for the next step.

Examples

```
using namespace pluss::kindle::literals;

auto catalog = pluss::kindle::Catalog({
    "id"_k = pluss::kindle::Stream(pluss::make_series({1, 2, 3})),
    "metric"_k = pluss::kindle::Stream(pluss::make_series({10.0, 20.0, 30.0})),
    "city"_k = pluss::kindle::Stream(pluss::make_note({"Accra", "Tema", "Kumasi"})),
});

auto mask = pluss::make_series({true, false, true}, pluss::codem::ScalarType::Bool);
auto out = catalog.where(mask);
```

Output

```
id     -> Series([1, 2, 3])
metric -> Series([10.0, 20.0, 30.0])
city   -> Note(["Accra", "Tema", "Kumasi"])
```

CHAPTER

TWENTYSIX

ARCHIVE API REFERENCE

CHAPTER

TWENTYSEVEN

PRACTICAL ARCHIVE API REFERENCE

27.1 API Reference Style

This page documents the full public `Archive` surface for depth-stacked catalogs.

27.1.1 aggregate_over_catalogs

Signature

```
Catalog aggregate_over_catalogs(self, const std::function<Catalog(const std::vector
↪<Catalog>&)>& fn) const
```

Parameters

`fn` : `const std::function<Catalog(const std::vector<Catalog>&)>&` See the signature for the supported argument form.

Returns

`Catalog` Value produced by this call.

Notes

Use this when you want a summary view instead of carrying the full input forward unchanged.

Examples

```
using namespace pluss::kindle::literals;

auto c0 = pluss::kindle::Catalog({
    "metric"_k = pluss::kindle::Stream(pluss::make_series({10.0, 20.0})),
});
auto c1 = pluss::kindle::Catalog({
    "metric"_k = pluss::kindle::Stream(pluss::make_series({30.0, 40.0})),
});
auto archive = pluss::kindle::Archive({c0, c1}, pluss::kindle::Stream(pluss::make_note({
↪"train", "test"})));

auto out = archive.aggregate_over_catalogs([](const std::vector<pluss::kindle::Catalog>&␣
↪catalogs) { return catalogs.front(); });
```

Output

```
id     -> Series([1, 2, 3])
metric -> Series([10.0, 20.0, 30.0])
city   -> Note(["Accra", "Tema", "Kumasi"])
```

27.1.2 align_depth

Signature

```
Archive align_depth(self, const Stream& labels) const
```

Parameters

`labels` : `const Stream&` See the signature for the supported argument form.

Returns

`Archive` Value produced by this call.

Notes

This call is part of the public `Archive` API and is documented here using a concrete usage example.

Examples

```
using namespace pluss::kindle::literals;

auto c0 = pluss::kindle::Catalog({
    "metric"_k = pluss::kindle::Stream(pluss::make_series({10.0, 20.0})),
});
auto c1 = pluss::kindle::Catalog({
    "metric"_k = pluss::kindle::Stream(pluss::make_series({30.0, 40.0})),
});
auto archive = pluss::kindle::Archive({c0, c1}, pluss::kindle::Stream(pluss::make_note({
↪"train", "test"})));

auto labels = pluss::kindle::Stream(pluss::make_note({"baseline", "stress"}));
auto out = archive.align_depth(labels);
```

Output

```
train -> Catalog(metric -> Series([10.0, 20.0]))
test  -> Catalog(metric -> Series([30.0, 40.0]))
```

27.1.3 catalog

Signature

```
Catalog catalog(self, std::int64_t i) const
```

Parameters

`i` : `std::int64_t` See the signature for the supported argument form.

Returns

`Catalog` Value produced by this call.

Notes

This call is part of the public `Archive` API and is documented here using a concrete usage example.

Examples

```
using namespace pluss::kindle::literals;

auto c0 = pluss::kindle::Catalog({
    "metric"_k = pluss::kindle::Stream(pluss::make_series({10.0, 20.0})),
});
auto c1 = pluss::kindle::Catalog({
    "metric"_k = pluss::kindle::Stream(pluss::make_series({30.0, 40.0})),
});
auto archive = pluss::kindle::Archive({c0, c1}, pluss::kindle::Stream(pluss::make_note({
↪"train", "test"})));

auto out = archive.catalog(0);
```

Output

```
id     -> Series([1, 2, 3])
metric -> Series([10.0, 20.0, 30.0])
city   -> Note(["Accra", "Tema", "Kumasi"])
```

27.1.4 `catalogs`

Signature

```
const std::vector<Catalog>& catalogs(self) const
```

Parameters

None

Returns

`const std::vector<Catalog>&` Borrowed collection of catalogs.

Notes

This call is part of the public `Archive` API and is documented here using a concrete usage example.

Examples

```
using namespace pluss::kindle::literals;

auto c0 = pluss::kindle::Catalog({
    "metric"_k = pluss::kindle::Stream(pluss::make_series({10.0, 20.0})),
});
auto c1 = pluss::kindle::Catalog({
    "metric"_k = pluss::kindle::Stream(pluss::make_series({30.0, 40.0})),
});
auto archive = pluss::kindle::Archive({c0, c1}, pluss::kindle::Stream(pluss::make_note({
↪"train", "test"})));

auto out = archive.catalogs();
```

Output

```
[Catalog(train), Catalog(test)]
```

27.1.5 defined

Signature

```
bool defined(self) const
```

Parameters

None

Returns

bool Boolean status or membership result.

Notes

This call is part of the public `Archive` API and is documented here using a concrete usage example.

Examples

```
using namespace pluss::kindle::literals;

auto c0 = pluss::kindle::Catalog({
    "metric"_k = pluss::kindle::Stream(pluss::make_series({10.0, 20.0})),
});
auto c1 = pluss::kindle::Catalog({
    "metric"_k = pluss::kindle::Stream(pluss::make_series({30.0, 40.0})),
});
auto archive = pluss::kindle::Archive({c0, c1}, pluss::kindle::Stream(pluss::make_note({
↪"train", "test"})));

auto out = archive.defined();
```

Output

```
true
```

27.1.6 depth

Signature

```
std::int64_t depth(self) const
```

Parameters

None

Returns

`std::int64_t` Integer count, size, or positional result.

Notes

This call is part of the public `Archive` API and is documented here using a concrete usage example.

Examples

```
using namespace pluss::kindle::literals;

auto c0 = pluss::kindle::Catalog({
    "metric"_k = pluss::kindle::Stream(pluss::make_series({10.0, 20.0})),
});
auto c1 = pluss::kindle::Catalog({
    "metric"_k = pluss::kindle::Stream(pluss::make_series({30.0, 40.0})),
});
auto archive = pluss::kindle::Archive({c0, c1}, pluss::kindle::Stream(pluss::make_note({
↪"train", "test"})));

auto out = archive.depth();
```

Output

```
2
```

27.1.7 depth_labels

Signature

```
const std::optional<Stream>& depth_labels(self) const
```

Parameters

None

Returns

const std::optional<Stream>& Value produced by this call.

Notes

This call is part of the public Archive API and is documented here using a concrete usage example.

Examples

```
using namespace pluss::kindle::literals;

auto c0 = pluss::kindle::Catalog({
    "metric"_k = pluss::kindle::Stream(pluss::make_series({10.0, 20.0})),
});
auto c1 = pluss::kindle::Catalog({
    "metric"_k = pluss::kindle::Stream(pluss::make_series({30.0, 40.0})),
});
auto archive = pluss::kindle::Archive({c0, c1}, pluss::kindle::Stream(pluss::make_note({
↪"train", "test"})));

auto out = archive.depth_labels();
```

Output

```
["train", "test"]
```

27.1.8 empty

Signature

```
bool empty(self) const
```

Parameters

None

Returns

bool Boolean status or membership result.

Notes

This call is part of the public Archive API and is documented here using a concrete usage example.

Examples

```
using namespace pluss::kindle::literals;

auto c0 = pluss::kindle::Catalog({
    "metric"_k = pluss::kindle::Stream(pluss::make_series({10.0, 20.0})),
});
auto c1 = pluss::kindle::Catalog({
    "metric"_k = pluss::kindle::Stream(pluss::make_series({30.0, 40.0})),
```

(continues on next page)

(continued from previous page)

```
});
auto archive = pluss::kindle::Archive({c0, c1}, pluss::kindle::Stream(pluss::make_note({
↪"train", "test"})));

auto out = archive.empty();
```

Output

```
false
```

27.1.9 head

Signature

```
Archive head(self, std::int64_t n = 5) const
```

Parameters

`n` : `std::int64_t` See the signature for the supported argument form.

Returns

`Archive` Value produced by this call.

Notes

Use this to narrow the current object to the rows or depth range you want to inspect next.

Examples

```
using namespace pluss::kindle::literals;

auto c0 = pluss::kindle::Catalog({
    "metric"_k = pluss::kindle::Stream(pluss::make_series({10.0, 20.0})),
});
auto c1 = pluss::kindle::Catalog({
    "metric"_k = pluss::kindle::Stream(pluss::make_series({30.0, 40.0})),
});
auto archive = pluss::kindle::Archive({c0, c1}, pluss::kindle::Stream(pluss::make_note({
↪"train", "test"})));

auto out = archive.head(1);
```

Output

```
train -> Catalog(metric -> Series([10.0, 20.0]))
test  -> Catalog(metric -> Series([30.0, 40.0]))
```

27.1.10 iloc

Signature

```
Catalog iloc(self, std::int64_t i) const
```

Parameters

i : std::int64_t See the signature for the supported argument form.

Returns

Catalog Value produced by this call.

Notes

Use this to narrow the current object to the rows or depth range you want to inspect next.

Examples

```
using namespace pluss::kindle::literals;

auto c0 = pluss::kindle::Catalog({
    "metric"_k = pluss::kindle::Stream(pluss::make_series({10.0, 20.0})),
});
auto c1 = pluss::kindle::Catalog({
    "metric"_k = pluss::kindle::Stream(pluss::make_series({30.0, 40.0})),
});
auto archive = pluss::kindle::Archive({c0, c1}, pluss::kindle::Stream(pluss::make_note({
↪"train", "test"})));

auto out = archive.iloc(0);
```

Output

```
id     -> Series([1, 2, 3])
metric -> Series([10.0, 20.0, 30.0])
city   -> Note(["Accra", "Tema", "Kumasi"])
```

27.1.11 kindelize

Signature

```
static Archive kindelize(const pluss::table::Atlas& atlas, const KindelizePolicy& policy␣
↪=
```

Parameters

atlas : const pluss::table::Atlas& See the signature for the supported argument form.

policy : const KindelizePolicy& See the signature for the supported argument form.

Returns

`Archive` Value produced by this call.

Notes

This is an explicit bridge between PlussMic models so you can switch to the representation that fits the next task.

Examples

```
auto f0 = pluss::table::DataFrame({
    {"sales", pluss::table::Thread(pluss::make_series({10.0, 20.0}))},
});
auto f1 = pluss::table::DataFrame({
    {"sales", pluss::table::Thread(pluss::make_series({30.0, 40.0}))},
});
auto atlas = pluss::table::Atlas({f0, f1});

auto out = pluss::kindle::Archive::kindelize(atlas);
```

Output

```
train -> Catalog(metric -> Series([10.0, 20.0]))
test  -> Catalog(metric -> Series([30.0, 40.0]))
```

27.1.12 `loc`

Signature

```
Catalog loc(self, const pluss::Timeline& label) const
```

Parameters

`label` : `const pluss::Timeline&` See the signature for the supported argument form.

Returns

`Catalog` Value produced by this call.

Notes

Use this to narrow the current object to the rows or depth range you want to inspect next.

See also

- `Catalog loc(self, std::string_view label) const`
- `Catalog loc(self, std::int64_t label) const`

Examples

```
using namespace pluss::kindle::literals;

auto c0 = pluss::kindle::Catalog({
```

(continues on next page)

(continued from previous page)

```
    "metric"_k = pluss::kindle::Stream(pluss::make_series({10.0, 20.0})),
});
auto c1 = pluss::kindle::Catalog({
    "metric"_k = pluss::kindle::Stream(pluss::make_series({30.0, 40.0})),
});
auto archive = pluss::kindle::Archive({c0, c1}, pluss::kindle::Stream(pluss::make_note({
↪"train", "test"})));

auto out = archive.loc("test");
```

Output

```
id     -> Series([1, 2, 3])
metric -> Series([10.0, 20.0, 30.0])
city   -> Note(["Accra", "Tema", "Kumasi"])
```

27.1.13 loc

Signature

```
Catalog loc(self, std::int64_t label) const
```

Parameters

`label` : `std::int64_t` See the signature for the supported argument form.

Returns

`Catalog` Value produced by this call.

Notes

Use this to narrow the current object to the rows or depth range you want to inspect next.

See also

- `Catalog loc(self, std::string_view label) const`
- `Catalog loc(self, const pluss::Timeline& label) const`

Examples

```
using namespace pluss::kindle::literals;

auto c0 = pluss::kindle::Catalog({
    "metric"_k = pluss::kindle::Stream(pluss::make_series({10.0, 20.0})),
});
auto c1 = pluss::kindle::Catalog({
    "metric"_k = pluss::kindle::Stream(pluss::make_series({30.0, 40.0})),
});
auto archive = pluss::kindle::Archive({c0, c1}, pluss::kindle::Stream(pluss::make_note({
↪"train", "test"})));
```

(continues on next page)

(continued from previous page)

```
auto out = archive.loc("test");
```

Output

```
id     -> Series([1, 2, 3])
metric -> Series([10.0, 20.0, 30.0])
city   -> Note(["Accra", "Tema", "Kumasi"])
```

27.1.14 loc

Signature

```
Catalog loc(self, std::string_view label) const
```

Parameters

`label` : `std::string_view` See the signature for the supported argument form.

Returns

`Catalog` Value produced by this call.

Notes

Use this to narrow the current object to the rows or depth range you want to inspect next.

See also

- `Catalog loc(self, std::int64_t label) const`
- `Catalog loc(self, const pluss::Timeline& label) const`

Examples

```
using namespace pluss::kindle::literals;

auto c0 = pluss::kindle::Catalog({
    "metric"_k = pluss::kindle::Stream(pluss::make_series({10.0, 20.0})),
});
auto c1 = pluss::kindle::Catalog({
    "metric"_k = pluss::kindle::Stream(pluss::make_series({30.0, 40.0})),
});
auto archive = pluss::kindle::Archive({c0, c1}, pluss::kindle::Stream(pluss::make_note({
↪"train", "test"})));

auto out = archive.loc("test");
```

Output

```
id     -> Series([1, 2, 3])
metric -> Series([10.0, 20.0, 30.0])
city   -> Note(["Accra", "Tema", "Kumasi"])
```

27.1.15 map_catalogs

Signature

```
Archive map_catalogs(self, const std::function<Catalog(const Catalog&)>& fn) const
```

Parameters

fn : const std::function<Catalog(const Catalog&)>& See the signature for the supported argument form.

Returns

Archive Value produced by this call.

Notes

This mutates or derives structure while keeping the surrounding object model intact.

Examples

```
using namespace pluss::kindle::literals;

auto c0 = pluss::kindle::Catalog({
    "metric"_k = pluss::kindle::Stream(pluss::make_series({10.0, 20.0})),
});
auto c1 = pluss::kindle::Catalog({
    "metric"_k = pluss::kindle::Stream(pluss::make_series({30.0, 40.0})),
});
auto archive = pluss::kindle::Archive({c0, c1}, pluss::kindle::Stream(pluss::make_note({
↪"train", "test"})));

auto out = archive.map_catalogs([](const pluss::kindle::Catalog& catalog) { return␣
↪catalog.head(1); });
```

Output

```
train -> Catalog(metric -> Series([10.0, 20.0]))
test  -> Catalog(metric -> Series([30.0, 40.0]))
```

27.1.16 relabel_depth

Signature

```
Archive relabel_depth(self, const Stream& labels) const
```

Parameters

labels : const Stream& See the signature for the supported argument form.

Returns

Archive Value produced by this call.

Notes

This adjusts labels without changing the underlying payload values.

Examples

```
using namespace pluss::kindle::literals;

auto c0 = pluss::kindle::Catalog({
    "metric"_k = pluss::kindle::Stream(pluss::make_series({10.0, 20.0})),
});
auto c1 = pluss::kindle::Catalog({
    "metric"_k = pluss::kindle::Stream(pluss::make_series({30.0, 40.0})),
});
auto archive = pluss::kindle::Archive({c0, c1}, pluss::kindle::Stream(pluss::make_note({
↪"train", "test"})));

auto labels = pluss::kindle::Stream(pluss::make_note({"baseline", "stress"}));
auto out = archive.relabel_depth(labels);
```

Output

```
train -> Catalog(metric -> Series([10.0, 20.0]))
test  -> Catalog(metric -> Series([30.0, 40.0]))
```

27.1.17 show

Signature

```
void show(self, std::ostream& os) const
```

Parameters

`os` : `std::ostream&` See the signature for the supported argument form.

Returns

`void` Performs the side effect and returns no value.

Notes

This renders a human-readable view for inspection instead of creating a new analytic object.

Examples

```
using namespace pluss::kindle::literals;

auto c0 = pluss::kindle::Catalog({
    "metric"_k = pluss::kindle::Stream(pluss::make_series({10.0, 20.0})),
});
auto c1 = pluss::kindle::Catalog({
    "metric"_k = pluss::kindle::Stream(pluss::make_series({30.0, 40.0})),
});
```

(continues on next page)

(continued from previous page)

```
auto archive = pluss::kindle::Archive({c0, c1}, pluss::kindle::Stream(pluss::make_note({
↪"train", "test"})));

archive.show(std::cout);
```

Output

```
train -> Catalog(metric -> Series([10.0, 20.0]))
test  -> Catalog(metric -> Series([30.0, 40.0]))
```

27.1.18 slice_depth

Signature

```
Archive slice_depth(self, std::int64_t start, std::int64_t stop) const
```

Parameters

`start` : `std::int64_t` See the signature for the supported argument form.

`stop` : `std::int64_t` See the signature for the supported argument form.

Returns

`Archive` Value produced by this call.

Notes

Use this to narrow the current object to the rows or depth range you want to inspect next.

Examples

```
using namespace pluss::kindle::literals;

auto c0 = pluss::kindle::Catalog({
    "metric"_k = pluss::kindle::Stream(pluss::make_series({10.0, 20.0})),
});
auto c1 = pluss::kindle::Catalog({
    "metric"_k = pluss::kindle::Stream(pluss::make_series({30.0, 40.0})),
});
auto archive = pluss::kindle::Archive({c0, c1}, pluss::kindle::Stream(pluss::make_note({
↪"train", "test"})));

auto out = archive.slice_depth(0, 1);
```

Output

```
train -> Catalog(metric -> Series([10.0, 20.0]))
test  -> Catalog(metric -> Series([30.0, 40.0]))
```

27.1.19 sort_depth

Signature

```
Archive sort_depth(self, bool ascending = true, bool nulls_last = true) const
```

Parameters

`ascending` : `bool` See the signature for the supported argument form.

`nulls_last` : `bool` See the signature for the supported argument form.

Returns

`Archive` Value produced by this call.

Notes

This call is part of the public `Archive` API and is documented here using a concrete usage example.

Examples

```
using namespace pluss::kindle::literals;

auto c0 = pluss::kindle::Catalog({
    "metric"_k = pluss::kindle::Stream(pluss::make_series({10.0, 20.0})),
});
auto c1 = pluss::kindle::Catalog({
    "metric"_k = pluss::kindle::Stream(pluss::make_series({30.0, 40.0})),
});
auto archive = pluss::kindle::Archive({c0, c1}, pluss::kindle::Stream(pluss::make_note({
↪"train", "test"})));

auto out = archive.sort_depth(true, true);
```

Output

```
train -> Catalog(metric -> Series([10.0, 20.0]))
test  -> Catalog(metric -> Series([30.0, 40.0]))
```

27.1.20 tabelize

Signature

```
pluss::table::Atlas tabelize(self, const TabelizePolicy& policy =
```

Parameters

`policy` : `const TabelizePolicy&` See the signature for the supported argument form.

Returns

`pluss::table::Atlas` Value produced by this call.

Notes

This is an explicit bridge between PlussMic models so you can switch to the representation that fits the next task.

Examples

```
using namespace pluss::kindle::literals;

auto c0 = pluss::kindle::Catalog({
    "metric"_k = pluss::kindle::Stream(pluss::make_series({10.0, 20.0})),
});
auto c1 = pluss::kindle::Catalog({
    "metric"_k = pluss::kindle::Stream(pluss::make_series({30.0, 40.0})),
});
auto archive = pluss::kindle::Archive({c0, c1}, pluss::kindle::Stream(pluss::make_note({
↪"train", "test"})));

pluss::kindle::TabelizePolicy policy;
policy.align = pluss::kindle::AlignPolicy::Inner;
auto out = archive.tabelize(policy);
```

Output

```
frame0:
sales
10.0
20.0

frame1:
sales
30.0
40.0
```

27.1.21 `tail`

Signature

```
Archive tail(self, std::int64_t n = 5) const
```

Parameters

`n` : `std::int64_t` See the signature for the supported argument form.

Returns

`Archive` Value produced by this call.

Notes

Use this to narrow the current object to the rows or depth range you want to inspect next.

Examples

```
using namespace pluss::kindle::literals;

auto c0 = pluss::kindle::Catalog({
    "metric"_k = pluss::kindle::Stream(pluss::make_series({10.0, 20.0})),
});
auto c1 = pluss::kindle::Catalog({
    "metric"_k = pluss::kindle::Stream(pluss::make_series({30.0, 40.0})),
});
auto archive = pluss::kindle::Archive({c0, c1}, pluss::kindle::Stream(pluss::make_note({
↪"train", "test"})));

auto out = archive.tail(1);
```

Output

```
train -> Catalog(metric -> Series([10.0, 20.0]))
test  -> Catalog(metric -> Series([30.0, 40.0]))
```

27.1.22 to_string

Signature

```
std::string to_string(self) const
```

Parameters

None

Returns

`std::string` Rendered string result.

Notes

This call is part of the public `Archive` API and is documented here using a concrete usage example.

Examples

```
using namespace pluss::kindle::literals;

auto c0 = pluss::kindle::Catalog({
    "metric"_k = pluss::kindle::Stream(pluss::make_series({10.0, 20.0})),
});
auto c1 = pluss::kindle::Catalog({
    "metric"_k = pluss::kindle::Stream(pluss::make_series({30.0, 40.0})),
});
auto archive = pluss::kindle::Archive({c0, c1}, pluss::kindle::Stream(pluss::make_note({
```

(continues on next page)

(continued from previous page)

```
↪"train", "test"})));

auto out = archive.to_string();
```

Output

```
train -> Catalog(metric -> Series([10.0, 20.0]))
test  -> Catalog(metric -> Series([30.0, 40.0]))
```

27.1.23 to_table

Signature

```
pluss::table::Atlas to_table(self, const TabelizePolicy& policy =
```

Parameters

`policy` : `const TabelizePolicy&` See the signature for the supported argument form.

Returns

`pluss::table::Atlas` Value produced by this call.

Notes

This is an explicit bridge between PlussMic models so you can switch to the representation that fits the next task.

Examples

```
using namespace pluss::kindle::literals;

auto c0 = pluss::kindle::Catalog({
    "metric"_k = pluss::kindle::Stream(pluss::make_series({10.0, 20.0})),
});
auto c1 = pluss::kindle::Catalog({
    "metric"_k = pluss::kindle::Stream(pluss::make_series({30.0, 40.0})),
});
auto archive = pluss::kindle::Archive({c0, c1}, pluss::kindle::Stream(pluss::make_note({
↪"train", "test"})));

pluss::kindle::TabelizePolicy policy;
policy.align = pluss::kindle::AlignPolicy::Inner;
auto out = archive.to_table(policy, true);
```

Output

```
frame0:
sales
10.0
20.0
```

(continues on next page)

(continued from previous page)

```
frame1:
sales
30.0
40.0
```

27.1.24 unique

Signature

```
bool unique(self) const
```

Parameters

None

Returns

bool Boolean status or membership result.

Notes

This call is part of the public Archive API and is documented here using a concrete usage example.

Examples

```
using namespace pluss::kindle::literals;

auto c0 = pluss::kindle::Catalog({
    "metric"_k = pluss::kindle::Stream(pluss::make_series({10.0, 20.0})),
});
auto c1 = pluss::kindle::Catalog({
    "metric"_k = pluss::kindle::Stream(pluss::make_series({30.0, 40.0})),
});
auto archive = pluss::kindle::Archive({c0, c1}, pluss::kindle::Stream(pluss::make_note({
↪"train", "test"})));

auto out = archive.unique();
```

Output

```
true
```

CHAPTER

TWENTYEIGHT

BUNDLE MODEL GUIDE

28.1 When To Use Bundle Types

The bundle model is the heterogeneous record layer:

- `Slot`: sequence of heterogeneous Pluss values
- `Registry`: named set of slots
- `Aisle`: depth stack of registries

Use it when one field can hold different domain types, or when you want a lighter record/batch abstraction than `DataFrame`.

28.2 Slot

```
auto ids = pluss::bundle::Slot({
    pluss::make_series(1, pluss::codem::ScalarType::Int64),
    pluss::make_series(2, pluss::codem::ScalarType::Int64),
    pluss::make_series(3, pluss::codem::ScalarType::Int64),
});

ids.push_(pluss::make_series(4, pluss::codem::ScalarType::Int64));
auto even = ids.filter([](const pluss::bundle::Slot::Value& v) {
  return std::get<pluss::Series>(v).to_vec<std::int64_t>()[0] % 2 == 0;
});
```

28.3 Registry

```
auto cities = pluss::bundle::Slot({
    pluss::make_note("Accra"),
    pluss::make_note("Tema"),
    pluss::make_note("Kumasi"),
});

auto registry = pluss::bundle::Registry({
    {"id", ids},
    {"city", cities},
});

auto selected = registry.select({"id"});
```

(continues on next page)

(continued from previous page)

```
auto renamed = registry.rename({{"city", "location"}});
auto profiled = registry.type_profiles();
```

28.4 Aisle

```
auto train = registry;
auto test = registry.rename({{"city", "city_test"}});
auto labels = pluss::bundle::Slot({pluss::make_note("train"), pluss::make_note("test")});

auto aisle = pluss::bundle::Aisle({train, test}, labels);

auto first = aisle.head(1);
auto combined = aisle.concat_depth(first);
```

28.5 Kindle Bridge

```
auto stream = ids.to_kindle();
auto catalog = registry.to_kindle();
auto archive = aisle.to_kindle();

auto back_slot = pluss::bundle::Slot::from_kindle(stream);
auto back_registry = pluss::bundle::Registry::from_kindle(catalog);
auto back_aisle = pluss::bundle::Aisle::from_kindle(archive);
```

CHAPTER

TWENTYNINE

SLOT API REFERENCE

CHAPTER

THIRTY

PRACTICAL SLOT API REFERENCE

30.1 API Reference Style

This page documents the full public `Slot` surface for one-dimensional heterogeneous bundles.

30.1.1 as_kindle

Signature

```
pluss::kindle::Stream as_kindle(self) const
```

Parameters

None

Returns

`pluss::kindle::Stream` Value produced by this call.

Notes

This is an explicit bridge between PlussMic models so you can switch to the representation that fits the next task.

Examples

```
auto slot = pluss::bundle::Slot({pluss::make_series(1), pluss::make_series(2),␣
↪pluss::make_series(3)});

auto out = slot.as_kindle();
```

Output

```
Series([10.0, 20.0, 30.0])
```

30.1.2 at

Signature

```
const Value& at(self, std::int64_t i) const
```

Parameters

`i` : `std::int64_t` See the signature for the supported argument form.

Returns

`const Value&` Value produced by this call.

Notes

This call is part of the public `Slot` API and is documented here using a concrete usage example.

Examples

```
auto slot = pluss::bundle::Slot({pluss::make_series(1), pluss::make_series(2),␣
↪pluss::make_series(3)});

auto out = slot.at(0);
```

Output

```
Series(1)
```

30.1.3 concat

Signature

```
Slot concat(self, const Slot& other) const
```

Parameters

`other` : `const Slot&` See the signature for the supported argument form.

Returns

`Slot` Value produced by this call.

Notes

Use this when you need to merge compatible objects or extend the current one with another input.

Examples

```
auto slot = pluss::bundle::Slot({pluss::make_series(1), pluss::make_series(2),␣
↪pluss::make_series(3)});

auto other = pluss::bundle::Slot({pluss::make_series(4), pluss::make_series(5)});
auto out = slot.concat(other);
```

Output

```
Slot([Series(1), Series(2), Series(3)])
```

30.1.4 defined

Signature

```
bool defined(self) const
```

Parameters

None

Returns

bool Boolean status or membership result.

Notes

This call is part of the public Slot API and is documented here using a concrete usage example.

Examples

```
auto slot = pluss::bundle::Slot({pluss::make_series(1), pluss::make_series(2),␣
↪pluss::make_series(3)});

auto out = slot.defined();
```

Output

```
true
```

30.1.5 empty

Signature

```
bool empty(self) const
```

Parameters

None

Returns

bool Boolean status or membership result.

Notes

This call is part of the public Slot API and is documented here using a concrete usage example.

Examples

```
auto slot = pluss::bundle::Slot({pluss::make_series(1), pluss::make_series(2),␣
↪pluss::make_series(3)});

auto out = slot.empty();
```

Output

```
false
```

30.1.6 filter

Signature

```
Slot filter(self, const std::function<bool(const Value&)>& predicate) const
```

Parameters

`predicate` : `const std::function<bool(const Value&)>&` See the signature for the supported argument form.

Returns

`Slot` Value produced by this call.

Notes

Use this to reduce the current object to the rows or fields that matter for the next step.

Examples

```
auto slot = pluss::bundle::Slot({pluss::make_series(1), pluss::make_note("ok"),␣
↪pluss::make_series(3)});

auto out = slot.filter([](const pluss::bundle::Slot::Value& value) { return std::holds_
↪alternative<pluss::Series>(value); });
```

Output

```
Slot([Series(1), Series(2), Series(3)])
```

30.1.7 from_kindle

Signature

```
static Slot from_kindle(const pluss::kindle::Stream& stream)
```

Parameters

`stream` : `const pluss::kindle::Stream&` See the signature for the supported argument form.

Returns

`Slot` Value produced by this call.

Notes

This is an explicit bridge between PlussMic models so you can switch to the representation that fits the next task.

Examples

```
auto stream = pluss::kindle::Stream(pluss::make_series({10.0, 20.0, 30.0}));

auto out = pluss::bundle::Slot::from_kindle(stream);
```

Output

```
Slot([Series(1), Series(2), Series(3)])
```

30.1.8 head

Signature

```
Slot head(self, std::int64_t n) const
```

Parameters

n : std::int64_t See the signature for the supported argument form.

Returns

Slot Value produced by this call.

Notes

Use this to narrow the current object to the rows or depth range you want to inspect next.

Examples

```
auto slot = pluss::bundle::Slot({pluss::make_series(1), pluss::make_series(2),␣
↪pluss::make_series(3)});

auto out = slot.head(2);
```

Output

```
Slot([Series(1), Series(2), Series(3)])
```

30.1.9 kindelize

Signature

```
pluss::kindle::Stream kindelize(self) const
```

Parameters

None

Returns

`pluss::kindle::Stream` Value produced by this call.

Notes

This is an explicit bridge between PlussMic models so you can switch to the representation that fits the next task.

Examples

```
auto slot = pluss::bundle::Slot({pluss::make_series(1), pluss::make_series(2),␣
↪pluss::make_series(3)});

auto out = slot.kindelize();
```

Output

```
Series([10.0, 20.0, 30.0])
```

30.1.10 push

Signature

```
Slot push(self, const Value& value) const
```

Parameters

`value` : `const Value&` See the signature for the supported argument form.

Returns

`Slot` Value produced by this call.

Notes

This mutates or derives structure while keeping the surrounding object model intact.

Examples

```
auto slot = pluss::bundle::Slot({pluss::make_series(1), pluss::make_series(2),␣
↪pluss::make_series(3)});

auto out = slot.push(pluss::make_note("tail"));
```

Output

```
Slot([Series(1), Series(2), Series(3)])
```

30.1.11 push_

Signature

```
Slot& push_(self, const Value& value)
```

Parameters

`value` : `const Value&` See the signature for the supported argument form.

Returns

`Slot&` Value produced by this call.

Notes

This mutates or derives structure while keeping the surrounding object model intact.

Examples

```
auto slot = pluss::bundle::Slot({pluss::make_series(1), pluss::make_series(2),␣
↪pluss::make_series(3)});

slot.push_(pluss::make_note("tail"));
```

Output

```
Slot([Series(1), Series(2), Series(3)])
```

30.1.12 show

Signature

```
void show(self, std::ostream& os) const
```

Parameters

`os` : `std::ostream&` See the signature for the supported argument form.

Returns

`void` Performs the side effect and returns no value.

Notes

This renders a human-readable view for inspection instead of creating a new analytic object.

Examples

```
auto slot = pluss::bundle::Slot({pluss::make_series(1), pluss::make_series(2),␣
↪pluss::make_series(3)});

slot.show(std::cout);
```

Output

```
Slot([Series(1), Series(2), Series(3)])
```

30.1.13 size

Signature

```
std::int64_t size(self) const
```

Parameters

None

Returns

`std::int64_t` Integer count, size, or positional result.

Notes

This call is part of the public `Slot` API and is documented here using a concrete usage example.

Examples

```
auto slot = pluss::bundle::Slot({pluss::make_series(1), pluss::make_series(2),␣
↪pluss::make_series(3)});

auto out = slot.size();
```

Output

```
3
```

30.1.14 slice

Signature

```
Slot slice(self, std::int64_t start, std::int64_t stop) const
```

Parameters

`start` : `std::int64_t` See the signature for the supported argument form.

`stop` : `std::int64_t` See the signature for the supported argument form.

Returns

`Slot` Value produced by this call.

Notes

Use this to narrow the current object to the rows or depth range you want to inspect next.

Examples

```
auto slot = pluss::bundle::Slot({pluss::make_series(1), pluss::make_series(2),␣
↪pluss::make_series(3)});

auto out = slot.slice(0, 2);
```

Output

```
Slot([Series(1), Series(2), Series(3)])
```

30.1.15 `tail`

Signature

```
Slot tail(self, std::int64_t n) const
```

Parameters

`n` : `std::int64_t` See the signature for the supported argument form.

Returns

`Slot` Value produced by this call.

Notes

Use this to narrow the current object to the rows or depth range you want to inspect next.

Examples

```
auto slot = pluss::bundle::Slot({pluss::make_series(1), pluss::make_series(2),␣
↪pluss::make_series(3)});

auto out = slot.tail(2);
```

Output

```
Slot([Series(1), Series(2), Series(3)])
```

30.1.16 `take`

Signature

```
Slot take(self, std::span<const std::int64_t> indices) const
```

Parameters

`indices` : `std::span<const std::int64_t>` See the signature for the supported argument form.

Returns

`Slot` Value produced by this call.

Notes

Use this to narrow the current object to the rows or depth range you want to inspect next.

Examples

```
auto slot = pluss::bundle::Slot({pluss::make_series(1), pluss::make_series(2),␣
↪pluss::make_series(3)});

std::vector<std::int64_t> picks{0, 2};
auto out = slot.take(picks);
```

Output

```
Slot([Series(1), Series(2), Series(3)])
```

30.1.17 `to_json`

Signature

```
std::string to_json(self) const
```

Parameters

None

Returns

`std::string` Rendered string result.

Notes

This call is part of the public `Slot` API and is documented here using a concrete usage example.

Examples

```
auto slot = pluss::bundle::Slot({pluss::make_series(1), pluss::make_series(2),␣
↪pluss::make_series(3)});

auto out = slot.to_json();
```

Output

```
Slot([Series(1), Series(2), Series(3)])
```

30.1.18 `to_kindle`

Signature

```
pluss::kindle::Stream to_kindle(self) const
```

Parameters

None

Returns

`pluss::kindle::Stream` Value produced by this call.

Notes

This is an explicit bridge between PlussMic models so you can switch to the representation that fits the next task.

Examples

```
auto slot = pluss::bundle::Slot({pluss::make_series(1), pluss::make_series(2),␣
↪pluss::make_series(3)});

auto out = slot.to_kindle();
```

Output

```
Series([10.0, 20.0, 30.0])
```

30.1.19 `to_string`

Signature

```
std::string to_string(self) const
```

Parameters

None

Returns

`std::string` Rendered string result.

Notes

This call is part of the public `Slot` API and is documented here using a concrete usage example.

Examples

```
auto slot = pluss::bundle::Slot({pluss::make_series(1), pluss::make_series(2),␣
↪pluss::make_series(3)});

auto out = slot.to_string();
```

Output

```
Slot([Series(1), Series(2), Series(3)])
```

30.1.20 transform

Signature

```
Slot transform(self, const std::function<Value(const Value&)>& fn) const
```

Parameters

fn : const std::function<Value(const Value&)>& See the signature for the supported argument form.

Returns

Slot Value produced by this call.

Notes

This mutates or derives structure while keeping the surrounding object model intact.

Examples

```
auto slot = pluss::bundle::Slot({pluss::make_series(1), pluss::make_series(2),␣
↪pluss::make_series(3)});

auto out = slot.transform([](const pluss::bundle::Slot::Value& value) ->␣
↪pluss::bundle::Slot::Value { if (std::holds_alternative<pluss::Series>(value)) return␣
↪std::get<pluss::Series>(value).add(10); return value; });
```

Output

```
Slot([Series(1), Series(2), Series(3)])
```

30.1.21 type_counts

Signature

```
std::unordered_map<std::string, std::int64_t> type_counts(self) const
```

Parameters

None

Returns

std::unordered_map<std::string, std::int64_t> Value produced by this call.

Notes

Use this when you want a summary view instead of carrying the full input forward unchanged.

Examples

```
auto slot = pluss::bundle::Slot({pluss::make_series(1), pluss::make_note("ok"),␣
↪pluss::make_series(3)});

auto out = slot.type_counts();
```

Output

```
{"Series": 2, "Note": 1}
```

30.1.22 unique

Signature

```
bool unique(self) const
```

Parameters

None

Returns

bool Boolean status or membership result.

Notes

This call is part of the public Slot API and is documented here using a concrete usage example.

Examples

```
auto slot = pluss::bundle::Slot({pluss::make_series(1), pluss::make_series(2),␣
↪pluss::make_series(3)});

auto out = slot.unique();
```

Output

```
true
```

30.1.23 values

Signature

```
const std::vector<Value>& values(self) const
```

Parameters

None

Returns

`const std::vector<Value>&` Value produced by this call.

Notes

This call is part of the public `Slot` API and is documented here using a concrete usage example.

Examples

```
auto slot = pluss::bundle::Slot({pluss::make_series(1), pluss::make_series(2),␣
↪pluss::make_series(3)});

auto out = slot.values();
```

Output

```
Slot([Series(1), Series(2), Series(3)])
```

CHAPTER

THIRTYONE

REGISTRY API REFERENCE

CHAPTER

THIRTYTWO

PRACTICAL REGISTRY API REFERENCE

32.1 API Reference Style

This page documents the full public `Registry` surface for named bundle slots.

32.1.1 apply

Signature

```
Registry apply(self, std::initializer_list<std::string_view> keys, const std::function
↪<Slot(const Slot&)>& fn) const
```

Parameters

`keys` : `std::initializer_list<std::string_view>` See the signature for the supported argument form.

`fn` : `const std::function<Slot(const Slot&)>&` See the signature for the supported argument form.

Returns

`Registry` Value produced by this call.

Notes

This mutates or derives structure while keeping the surrounding object model intact.

See also

- `Registry apply(self, std::string_view key, const std::function<Slot(const Slot&)>& fn) const`

Examples

```
auto registry = pluss::bundle::Registry({
    {"ids", pluss::bundle::Slot({pluss::make_series(1), pluss::make_series(2),␣
↪pluss::make_series(3)})},
    {"city", pluss::bundle::Slot({pluss::make_note("Accra"), pluss::make_note("Tema"),␣
↪pluss::make_note("Kumasi")})},
});

auto out = registry.apply({"ids"}, [](const pluss::bundle::Slot& slot) { return slot; });
```

Output

```
ids  -> Slot([Series(1), Series(2), Series(3)])
city -> Slot([Note("Accra"), Note("Tema"), Note("Kumasi")])
```

32.1.2 apply

Signature

```
Registry apply(self, std::string_view key, const std::function<Slot(const Slot&)>& fn)␣
↪const
```

Parameters

key : std::string_view See the signature for the supported argument form.

fn : const std::function<Slot(const Slot&)>& See the signature for the supported argument form.

Returns

Registry Value produced by this call.

Notes

This mutates or derives structure while keeping the surrounding object model intact.

See also

- Registry apply(self, std::initializer_list<std::string_view> keys, const std::function<Slot(const Slot&)>& fn) const

Examples

```
auto registry = pluss::bundle::Registry({
    {"ids", pluss::bundle::Slot({pluss::make_series(1), pluss::make_series(2),␣
↪pluss::make_series(3)})},
    {"city", pluss::bundle::Slot({pluss::make_note("Accra"), pluss::make_note("Tema"),␣
↪pluss::make_note("Kumasi")})},
});

auto out = registry.apply("ids", [](const pluss::bundle::Slot& slot) { return slot; });
```

Output

```
ids  -> Slot([Series(1), Series(2), Series(3)])
city -> Slot([Note("Accra"), Note("Tema"), Note("Kumasi")])
```

32.1.3 apply_

Signature

```
Registry& apply_(self, std::initializer_list<std::string_view> keys, const std::function
↪<Slot(const Slot&)>& fn)
```

Parameters

keys : std::initializer_list<std::string_view> See the signature for the supported argument form.

fn : const std::function<Slot(const Slot&)>& See the signature for the supported argument form.

Returns

Registry& Value produced by this call.

Notes

This mutates or derives structure while keeping the surrounding object model intact.

See also

- Registry& apply_(self, std::string_view key, const std::function<Slot(const Slot&)>& fn)

Examples

```
auto registry = pluss::bundle::Registry({
    {"ids", pluss::bundle::Slot({pluss::make_series(1), pluss::make_series(2),␣
↪pluss::make_series(3)})},
    {"city", pluss::bundle::Slot({pluss::make_note("Accra"), pluss::make_note("Tema"),␣
↪pluss::make_note("Kumasi")})},
});

registry.apply_({"ids"}, [](const pluss::bundle::Slot& slot) { return slot; });
```

32.1.4 apply_

Signature

```
Registry& apply_(self, std::string_view key, const std::function<Slot(const Slot&)>& fn)
```

Parameters

key : std::string_view See the signature for the supported argument form.

fn : const std::function<Slot(const Slot&)>& See the signature for the supported argument form.

Returns

Registry& Value produced by this call.

Notes

This mutates or derives structure while keeping the surrounding object model intact.

See also

- Registry& apply_(self, std::initializer_list<std::string_view> keys, const std::function<Slot(const Slot&)>& fn)

Examples

```
auto registry = pluss::bundle::Registry({
    {"ids", pluss::bundle::Slot({pluss::make_series(1), pluss::make_series(2),␣
↪pluss::make_series(3)})},
    {"city", pluss::bundle::Slot({pluss::make_note("Accra"), pluss::make_note("Tema"),␣
↪pluss::make_note("Kumasi")})},
});

registry.apply_("ids", [](const pluss::bundle::Slot& slot) { return slot; });
```

32.1.5 apply_all

Signature

```
Registry apply_all(self, const std::function<Slot(const Slot&)>& fn) const
```

Parameters

fn : const std::function<Slot(const Slot&)>& See the signature for the supported argument form.

Returns

Registry Value produced by this call.

Notes

This mutates or derives structure while keeping the surrounding object model intact.

Examples

```
auto registry = pluss::bundle::Registry({
    {"ids", pluss::bundle::Slot({pluss::make_series(1), pluss::make_series(2),␣
↪pluss::make_series(3)})},
    {"city", pluss::bundle::Slot({pluss::make_note("Accra"), pluss::make_note("Tema"),␣
↪pluss::make_note("Kumasi")})},
});

auto out = registry.apply_all([](const pluss::bundle::Slot& slot) { return slot; });
```

Output

```
ids  -> Slot([Series(1), Series(2), Series(3)])
city -> Slot([Note("Accra"), Note("Tema"), Note("Kumasi")])
```

32.1.6 as_kindle

Signature

```
pluss::kindle::Catalog as_kindle(self) const
```

Parameters

None

Returns

`pluss::kindle::Catalog` Value produced by this call.

Notes

This is an explicit bridge between PlussMic models so you can switch to the representation that fits the next task.

Examples

```
auto registry = pluss::bundle::Registry({
    {"ids", pluss::bundle::Slot({pluss::make_series(1), pluss::make_series(2),␣
↪pluss::make_series(3)})},
    {"city", pluss::bundle::Slot({pluss::make_note("Accra"), pluss::make_note("Tema"),␣
↪pluss::make_note("Kumasi")})},
});

auto out = registry.as_kindle();
```

Output

```
id     -> Series([1, 2, 3])
metric -> Series([10.0, 20.0, 30.0])
city   -> Note(["Accra", "Tema", "Kumasi"])
```

32.1.7 `at`

Signature

```
Slot at(self, std::string_view key) const
```

Parameters

`key` : `std::string_view` See the signature for the supported argument form.

Returns

`Slot` Value produced by this call.

Notes

This call is part of the public `Registry` API and is documented here using a concrete usage example.

Examples

```
auto registry = pluss::bundle::Registry({
    {"ids", pluss::bundle::Slot({pluss::make_series(1), pluss::make_series(2),␣
↪pluss::make_series(3)})},
    {"city", pluss::bundle::Slot({pluss::make_note("Accra"), pluss::make_note("Tema"),␣
↪pluss::make_note("Kumasi")})},
```

(continues on next page)

(continued from previous page)

```
});

auto out = registry.at("ids");
```

Output

```
Slot([Series(1), Series(2), Series(3)])
```

32.1.8 begin

Signature

```
const_iterator begin(self) const
```

Parameters

None

Returns

`const_iterator` Value produced by this call.

Notes

This call is part of the public `Registry` API and is documented here using a concrete usage example.

Examples

```
auto registry = pluss::bundle::Registry({
    {"ids", pluss::bundle::Slot({pluss::make_series(1), pluss::make_series(2),␣
↪pluss::make_series(3)})},
    {"city", pluss::bundle::Slot({pluss::make_note("Accra"), pluss::make_note("Tema"),␣
↪pluss::make_note("Kumasi")})},
});

for (const auto& key : registry) {
  (void)key;
}
```

32.1.9 clear

Signature

```
Registry clear(self) const
```

Parameters

None

Returns

`Registry` Value produced by this call.

Notes

This call is part of the public `Registry` API and is documented here using a concrete usage example.

Examples

```
auto registry = pluss::bundle::Registry({
    {"ids", pluss::bundle::Slot({pluss::make_series(1), pluss::make_series(2),␣
↪pluss::make_series(3)})},
    {"city", pluss::bundle::Slot({pluss::make_note("Accra"), pluss::make_note("Tema"),␣
↪pluss::make_note("Kumasi")})},
});

auto out = registry.clear();
```

Output

```
ids  -> Slot([Series(1), Series(2), Series(3)])
city -> Slot([Note("Accra"), Note("Tema"), Note("Kumasi")])
```

32.1.10 `clear_`

Signature

```
Registry& clear_(self)
```

Parameters

None

Returns

`Registry&` Value produced by this call.

Notes

This call is part of the public `Registry` API and is documented here using a concrete usage example.

Examples

```
auto registry = pluss::bundle::Registry({
    {"ids", pluss::bundle::Slot({pluss::make_series(1), pluss::make_series(2),␣
↪pluss::make_series(3)})},
    {"city", pluss::bundle::Slot({pluss::make_note("Accra"), pluss::make_note("Tema"),␣
↪pluss::make_note("Kumasi")})},
});

registry.clear_();
```

32.1.11 combine

Signature

```
Registry combine(self, const Registry& other, std::string suffix = "_other") const
```

Parameters

`other` : `const Registry&` See the signature for the supported argument form.

`suffix` : `std::string` See the signature for the supported argument form.

Returns

`Registry` Value produced by this call.

Notes

Use this when you need to merge compatible objects or extend the current one with another input.

Examples

```
auto registry = pluss::bundle::Registry({
    {"ids", pluss::bundle::Slot({pluss::make_series(1), pluss::make_series(2),␣
↪pluss::make_series(3)})},
    {"city", pluss::bundle::Slot({pluss::make_note("Accra"), pluss::make_note("Tema"),␣
↪pluss::make_note("Kumasi")})},
});

auto other = pluss::bundle::Registry({{"flag", pluss::bundle::Slot({pluss::make_
↪series(true, pluss::codem::ScalarType::Bool), pluss::make_series(false,␣
↪pluss::codem::ScalarType::Bool), pluss::make_series(true,␣
↪pluss::codem::ScalarType::Bool)})}});
auto out = registry.combine(other, "_rhs");
```

Output

```
ids  -> Slot([Series(1), Series(2), Series(3)])
city -> Slot([Note("Accra"), Note("Tema"), Note("Kumasi")])
```

32.1.12 concat_values

Signature

```
Registry concat_values(self, std::string_view key, const Slot& other) const
```

Parameters

`key` : `std::string_view` See the signature for the supported argument form.

`other` : `const Slot&` See the signature for the supported argument form.

Returns

`Registry` Value produced by this call.

Notes

Use this when you need to merge compatible objects or extend the current one with another input.

Examples

```
auto registry = pluss::bundle::Registry({
    {"ids", pluss::bundle::Slot({pluss::make_series(1), pluss::make_series(2),␣
↪pluss::make_series(3)})},
    {"city", pluss::bundle::Slot({pluss::make_note("Accra"), pluss::make_note("Tema"),␣
↪pluss::make_note("Kumasi")})},
});

auto extra = pluss::bundle::Slot({pluss::make_series(4), pluss::make_series(5)});
auto out = registry.concat_values("ids", extra);
```

Output

```
ids  -> Slot([Series(1), Series(2), Series(3)])
city -> Slot([Note("Accra"), Note("Tema"), Note("Kumasi")])
```

32.1.13 concat_values_

Signature

```
Registry& concat_values_(self, std::string_view key, const Slot& other)
```

Parameters

`key` : `std::string_view` See the signature for the supported argument form.

`other` : `const Slot&` See the signature for the supported argument form.

Returns

`Registry&` Value produced by this call.

Notes

This call is part of the public `Registry` API and is documented here using a concrete usage example.

Examples

```
auto registry = pluss::bundle::Registry({
    {"ids", pluss::bundle::Slot({pluss::make_series(1), pluss::make_series(2),␣
↪pluss::make_series(3)})},
    {"city", pluss::bundle::Slot({pluss::make_note("Accra"), pluss::make_note("Tema"),␣
↪pluss::make_note("Kumasi")})},
});
```

(continues on next page)

(continued from previous page)

```
auto extra = pluss::bundle::Slot({pluss::make_series(4), pluss::make_series(5)});
registry.concat_values_("ids", extra);
```

32.1.14 defined

Signature

```
bool defined(self) const
```

Parameters

None

Returns

bool Boolean status or membership result.

Notes

This call is part of the public Registry API and is documented here using a concrete usage example.

Examples

```
auto registry = pluss::bundle::Registry({
    {"ids", pluss::bundle::Slot({pluss::make_series(1), pluss::make_series(2),␣
↪pluss::make_series(3)})},
    {"city", pluss::bundle::Slot({pluss::make_note("Accra"), pluss::make_note("Tema"),␣
↪pluss::make_note("Kumasi")})},
});

auto out = registry.defined();
```

Output

```
true
```

32.1.15 drop

Signature

```
Registry drop(self, const std::vector<std::string>& keys) const
```

Parameters

keys : const std::vector<std::string>& See the signature for the supported argument form.

Returns

Registry Value produced by this call.

Notes

Use this to reduce the current object to the rows or fields that matter for the next step.

See also

- `Registry drop(self, std::initializer_list<std::string_view> keys) const`
- `Registry drop(self, std::string_view key) const`

Examples

```
auto registry = pluss::bundle::Registry({
    {"ids", pluss::bundle::Slot({pluss::make_series(1), pluss::make_series(2),␣
↪pluss::make_series(3)})},
    {"city", pluss::bundle::Slot({pluss::make_note("Accra"), pluss::make_note("Tema"),␣
↪pluss::make_note("Kumasi")})},
});

std::vector<std::string> names{"city"};
auto out = registry.drop(names);
```

Output

```
ids  -> Slot([Series(1), Series(2), Series(3)])
city -> Slot([Note("Accra"), Note("Tema"), Note("Kumasi")])
```

32.1.16 drop

Signature

```
Registry drop(self, std::initializer_list<std::string_view> keys) const
```

Parameters

`keys` : `std::initializer_list<std::string_view>` See the signature for the supported argument form.

Returns

`Registry` Value produced by this call.

Notes

Use this to reduce the current object to the rows or fields that matter for the next step.

See also

- `Registry drop(self, const std::vector<std::string>& keys) const`
- `Registry drop(self, std::string_view key) const`

Examples

```
auto registry = pluss::bundle::Registry({
    {"ids", pluss::bundle::Slot({pluss::make_series(1), pluss::make_series(2),␣
↪pluss::make_series(3)})},
    {"city", pluss::bundle::Slot({pluss::make_note("Accra"), pluss::make_note("Tema"),␣
↪pluss::make_note("Kumasi")})},
});

auto out = registry.drop({"city"});
```

Output

```
ids  -> Slot([Series(1), Series(2), Series(3)])
city -> Slot([Note("Accra"), Note("Tema"), Note("Kumasi")])
```

32.1.17 drop

Signature

```
Registry drop(self, std::string_view key) const
```

Parameters

`key` : `std::string_view` See the signature for the supported argument form.

Returns

`Registry` Value produced by this call.

Notes

Use this to reduce the current object to the rows or fields that matter for the next step.

See also

- `Registry drop(self, std::initializer_list<std::string_view> keys) const`
- `Registry drop(self, const std::vector<std::string>& keys) const`

Examples

```
auto registry = pluss::bundle::Registry({
    {"ids", pluss::bundle::Slot({pluss::make_series(1), pluss::make_series(2),␣
↪pluss::make_series(3)})},
    {"city", pluss::bundle::Slot({pluss::make_note("Accra"), pluss::make_note("Tema"),␣
↪pluss::make_note("Kumasi")})},
});

auto out = registry.drop("city");
```

Output

```
ids  -> Slot([Series(1), Series(2), Series(3)])
city -> Slot([Note("Accra"), Note("Tema"), Note("Kumasi")])
```

32.1.18 drop_empty_slots

Signature

```
Registry drop_empty_slots(self) const
```

Parameters

None

Returns

`Registry` Value produced by this call.

Notes

This call is part of the public `Registry` API and is documented here using a concrete usage example.

Examples

```
auto registry = pluss::bundle::Registry({
    {"ids", pluss::bundle::Slot({pluss::make_series(1), pluss::make_series(2),␣
↪pluss::make_series(3)})},
    {"city", pluss::bundle::Slot({pluss::make_note("Accra"), pluss::make_note("Tema"),␣
↪pluss::make_note("Kumasi")})},
});

auto out = registry.drop_empty_slots();
```

Output

```
ids  -> Slot([Series(1), Series(2), Series(3)])
city -> Slot([Note("Accra"), Note("Tema"), Note("Kumasi")])
```

32.1.19 drop_empty_slots_

Signature

```
Registry& drop_empty_slots_(self)
```

Parameters

None

Returns

Registry& Value produced by this call.

Notes

This call is part of the public Registry API and is documented here using a concrete usage example.

Examples

```
auto registry = pluss::bundle::Registry({
    {"ids", pluss::bundle::Slot({pluss::make_series(1), pluss::make_series(2),␣
↪pluss::make_series(3)})},
    {"city", pluss::bundle::Slot({pluss::make_note("Accra"), pluss::make_note("Tema"),␣
↪pluss::make_note("Kumasi")})},
});

registry.drop_empty_slots_();
```

32.1.20 empty

Signature

```
bool empty(self) const
```

Parameters

None

Returns

bool Boolean status or membership result.

Notes

This call is part of the public Registry API and is documented here using a concrete usage example.

Examples

```
auto registry = pluss::bundle::Registry({
    {"ids", pluss::bundle::Slot({pluss::make_series(1), pluss::make_series(2),␣
↪pluss::make_series(3)})},
    {"city", pluss::bundle::Slot({pluss::make_note("Accra"), pluss::make_note("Tema"),␣
↪pluss::make_note("Kumasi")})},
});

auto out = registry.empty();
```

Output

```
false
```

32.1.21 end

Signature

```
const_iterator end(self) const
```

Parameters

None

Returns

const_iterator Value produced by this call.

Notes

This call is part of the public Registry API and is documented here using a concrete usage example.

Examples

```
auto registry = pluss::bundle::Registry({
    {"ids", pluss::bundle::Slot({pluss::make_series(1), pluss::make_series(2),␣
↪pluss::make_series(3)})},
    {"city", pluss::bundle::Slot({pluss::make_note("Accra"), pluss::make_note("Tema"),␣
↪pluss::make_note("Kumasi")})},
});

for (const auto& key : registry) {
  (void)key;
}
```

32.1.22 ensure_stream_compatible

Signature

```
void ensure_stream_compatible(self, std::string_view where = "Registry") const
```

Parameters

where : std::string_view See the signature for the supported argument form.

Returns

void Performs the side effect and returns no value.

Notes

This call is part of the public Registry API and is documented here using a concrete usage example.

Examples

```
auto registry = pluss::bundle::Registry({
    {"ids", pluss::bundle::Slot({pluss::make_series(1), pluss::make_series(2),␣
↪pluss::make_series(3)})},
```

(continues on next page)

(continued from previous page)

```
    {"city", pluss::bundle::Slot({pluss::make_note("Accra"), pluss::make_note("Tema"),␣
↪pluss::make_note("Kumasi")})},
});

registry.ensure_stream_compatible("demo");
```

Output

```
no error
```

32.1.23 field_names

Signature

```
const std::vector<std::string>& field_names(self) const
```

Parameters

None

Returns

const std::vector<std::string>& Borrowed list of names.

Notes

This call is part of the public Registry API and is documented here using a concrete usage example.

Examples

```
auto registry = pluss::bundle::Registry({
    {"ids", pluss::bundle::Slot({pluss::make_series(1), pluss::make_series(2),␣
↪pluss::make_series(3)})},
    {"city", pluss::bundle::Slot({pluss::make_note("Accra"), pluss::make_note("Tema"),␣
↪pluss::make_note("Kumasi")})},
});

auto out = registry.field_names();
```

Output

```
["ids", "city"]
```

32.1.24 filter_values

Signature

```
Registry filter_values(self, std::string_view key, const std::function<bool(const␣
↪Slot::Value&)>& predicate) const
```

Parameters

`key` : `std::string_view` See the signature for the supported argument form.

`predicate` : `const std::function<bool(const Slot::Value&)>&` See the signature for the supported argument form.

Returns

`Registry` Value produced by this call.

Notes

Use this to reduce the current object to the rows or fields that matter for the next step.

Examples

```
auto registry = pluss::bundle::Registry({
    {"ids", pluss::bundle::Slot({pluss::make_series(1), pluss::make_series(2),␣
↪pluss::make_series(3)})},
    {"city", pluss::bundle::Slot({pluss::make_note("Accra"), pluss::make_note("Tema"),␣
↪pluss::make_note("Kumasi")})},
});

auto out = registry.filter_values("ids", [](const pluss::bundle::Slot::Value& value) {␣
↪return std::holds_alternative<pluss::Series>(value); });
```

Output

```
ids  -> Slot([Series(1), Series(2), Series(3)])
city -> Slot([Note("Accra"), Note("Tema"), Note("Kumasi")])
```

32.1.25 `filter_values_`

Signature

```
Registry& filter_values_(self, std::string_view key, const std::function<bool(const␣
↪Slot::Value&)>& predicate)
```

Parameters

`key` : `std::string_view` See the signature for the supported argument form.

`predicate` : `const std::function<bool(const Slot::Value&)>&` See the signature for the supported argument form.

Returns

`Registry&` Value produced by this call.

Notes

This call is part of the public `Registry` API and is documented here using a concrete usage example.

Examples

```
auto registry = pluss::bundle::Registry({
    {"ids", pluss::bundle::Slot({pluss::make_series(1), pluss::make_series(2),␣
↪pluss::make_series(3)})},
    {"city", pluss::bundle::Slot({pluss::make_note("Accra"), pluss::make_note("Tema"),␣
↪pluss::make_note("Kumasi")})},
});

registry.filter_values_("ids", [](const pluss::bundle::Slot::Value& value) { return␣
↪std::holds_alternative<pluss::Series>(value); });
```

32.1.26 from_kindle

Signature

```
static Registry from_kindle(const pluss::kindle::Catalog& catalog)
```

Parameters

`catalog` : `const pluss::kindle::Catalog&` See the signature for the supported argument form.

Returns

`Registry` Value produced by this call.

Notes

This is an explicit bridge between PlussMic models so you can switch to the representation that fits the next task.

Examples

```
using namespace pluss::kindle::literals;

auto catalog = pluss::kindle::Catalog({
    "id"_k = pluss::kindle::Stream(pluss::make_series({1, 2, 3})),
    "metric"_k = pluss::kindle::Stream(pluss::make_series({10.0, 20.0, 30.0})),
    "city"_k = pluss::kindle::Stream(pluss::make_note({"Accra", "Tema", "Kumasi"})),
});

auto out = pluss::bundle::Registry::from_kindle(catalog);
```

Output

```
ids  -> Slot([Series(1), Series(2), Series(3)])
city -> Slot([Note("Accra"), Note("Tema"), Note("Kumasi")])
```

32.1.27 get

Signature

```
std::optional<Slot> get(self, std::string_view key) const
```

Parameters

key : `std::string_view` See the signature for the supported argument form.

Returns

`std::optional<Slot>` Value produced by this call.

Notes

This call is part of the public `Registry` API and is documented here using a concrete usage example.

Examples

```
auto registry = pluss::bundle::Registry({
    {"ids", pluss::bundle::Slot({pluss::make_series(1), pluss::make_series(2),␣
↪pluss::make_series(3)})},
    {"city", pluss::bundle::Slot({pluss::make_note("Accra"), pluss::make_note("Tema"),␣
↪pluss::make_note("Kumasi")})},
});

auto out = registry.get("ids");
```

Output

```
Slot([Series(1), Series(2), Series(3)])
```

32.1.28 `get_slot`

Signature

```
std::optional<Slot> get_slot(self, std::string_view key) const
```

Parameters

key : `std::string_view` See the signature for the supported argument form.

Returns

`std::optional<Slot>` Value produced by this call.

Notes

This call is part of the public `Registry` API and is documented here using a concrete usage example.

Examples

```
auto registry = pluss::bundle::Registry({
    {"ids", pluss::bundle::Slot({pluss::make_series(1), pluss::make_series(2),␣
↪pluss::make_series(3)})},
    {"city", pluss::bundle::Slot({pluss::make_note("Accra"), pluss::make_note("Tema"),␣
↪pluss::make_note("Kumasi")})},
});

auto out = registry.get_slot("ids");
```

Output

```
Slot([Series(1), Series(2), Series(3)])
```

32.1.29 has

Signature

```
bool has(self, std::string_view key) const
```

Parameters

key : std::string_view See the signature for the supported argument form.

Returns

bool Boolean status or membership result.

Notes

This call is part of the public Registry API and is documented here using a concrete usage example.

Examples

```
auto registry = pluss::bundle::Registry({
    {"ids", pluss::bundle::Slot({pluss::make_series(1), pluss::make_series(2),␣
↪pluss::make_series(3)})},
    {"city", pluss::bundle::Slot({pluss::make_note("Accra"), pluss::make_note("Tema"),␣
↪pluss::make_note("Kumasi")})},
});

auto out = registry.has("ids");
```

Output

```
true
```

32.1.30 has_all

Signature

```
bool has_all(self, std::initializer_list<std::string_view> keys) const
```

Parameters

keys : std::initializer_list<std::string_view> See the signature for the supported argument form.

Returns

bool Boolean status or membership result.

Notes

This call is part of the public `Registry` API and is documented here using a concrete usage example.

Examples

```
auto registry = pluss::bundle::Registry({
    {"ids", pluss::bundle::Slot({pluss::make_series(1), pluss::make_series(2),␣
↪pluss::make_series(3)})},
    {"city", pluss::bundle::Slot({pluss::make_note("Accra"), pluss::make_note("Tema"),␣
↪pluss::make_note("Kumasi")})},
});

auto out = registry.has_all({"ids", "city"});
```

Output

```
true
```

32.1.31 has_any

Signature

```
bool has_any(self, std::initializer_list<std::string_view> keys) const
```

Parameters

`keys` : `std::initializer_list<std::string_view>` See the signature for the supported argument form.

Returns

`bool` Boolean status or membership result.

Notes

This call is part of the public `Registry` API and is documented here using a concrete usage example.

Examples

```
auto registry = pluss::bundle::Registry({
    {"ids", pluss::bundle::Slot({pluss::make_series(1), pluss::make_series(2),␣
↪pluss::make_series(3)})},
    {"city", pluss::bundle::Slot({pluss::make_note("Accra"), pluss::make_note("Tema"),␣
↪pluss::make_note("Kumasi")})},
});

auto out = registry.has_any({"ids", "city"});
```

Output

```
true
```

32.1.32 has_slot

Signature

```
bool has_slot(self, std::string_view key) const
```

Parameters

key : std::string_view See the signature for the supported argument form.

Returns

bool Boolean status or membership result.

Notes

This call is part of the public Registry API and is documented here using a concrete usage example.

Examples

```
auto registry = pluss::bundle::Registry({
    {"ids", pluss::bundle::Slot({pluss::make_series(1), pluss::make_series(2),␣
↪pluss::make_series(3)})},
    {"city", pluss::bundle::Slot({pluss::make_note("Accra"), pluss::make_note("Tema"),␣
↪pluss::make_note("Kumasi")})},
});

auto out = registry.has_slot("ids");
```

Output

```
true
```

32.1.33 head

Signature

```
Registry head(self, std::int64_t n) const
```

Parameters

n : std::int64_t See the signature for the supported argument form.

Returns

Registry Value produced by this call.

Notes

Use this to narrow the current object to the rows or depth range you want to inspect next.

Examples

```
auto registry = pluss::bundle::Registry({
    {"ids", pluss::bundle::Slot({pluss::make_series(1), pluss::make_series(2),␣
↪pluss::make_series(3)})},
    {"city", pluss::bundle::Slot({pluss::make_note("Accra"), pluss::make_note("Tema"),␣
↪pluss::make_note("Kumasi")})},
});

auto out = registry.head(2);
```

Output

```
ids  -> Slot([Series(1), Series(2), Series(3)])
city -> Slot([Note("Accra"), Note("Tema"), Note("Kumasi")])
```

32.1.34 `iloc`

Signature

```
Registry iloc(self, std::int64_t row) const
```

Parameters

`row` : `std::int64_t` See the signature for the supported argument form.

Returns

`Registry` Value produced by this call.

Notes

Use this to narrow the current object to the rows or depth range you want to inspect next.

See also

- `Registry iloc(self, std::int64_t start, std::int64_t stop) const`

Examples

```
auto registry = pluss::bundle::Registry({
    {"ids", pluss::bundle::Slot({pluss::make_series(1), pluss::make_series(2),␣
↪pluss::make_series(3)})},
    {"city", pluss::bundle::Slot({pluss::make_note("Accra"), pluss::make_note("Tema"),␣
↪pluss::make_note("Kumasi")})},
});

auto out = registry.iloc(1);
```

Output

```
ids  -> Slot([Series(1), Series(2), Series(3)])
city -> Slot([Note("Accra"), Note("Tema"), Note("Kumasi")])
```

32.1.35 iloc

Signature

```
Registry iloc(self, std::int64_t start, std::int64_t stop) const
```

Parameters

`start` : `std::int64_t` See the signature for the supported argument form.

`stop` : `std::int64_t` See the signature for the supported argument form.

Returns

`Registry` Value produced by this call.

Notes

Use this to narrow the current object to the rows or depth range you want to inspect next.

See also

- `Registry iloc(self, std::int64_t row) const`

Examples

```
auto registry = pluss::bundle::Registry({
    {"ids", pluss::bundle::Slot({pluss::make_series(1), pluss::make_series(2),␣
↪pluss::make_series(3)})},
    {"city", pluss::bundle::Slot({pluss::make_note("Accra"), pluss::make_note("Tema"),␣
↪pluss::make_note("Kumasi")})},
});

auto out = registry.iloc(0, 2);
```

Output

```
ids  -> Slot([Series(1), Series(2), Series(3)])
city -> Slot([Note("Accra"), Note("Tema"), Note("Kumasi")])
```

32.1.36 is_stream_compatible

Signature

```
bool is_stream_compatible(self) const
```

Parameters

None

Returns

`bool` Boolean status or membership result.

Notes

This call is part of the public `Registry` API and is documented here using a concrete usage example.

Examples

```
auto registry = pluss::bundle::Registry({
    {"ids", pluss::bundle::Slot({pluss::make_series(1), pluss::make_series(2),␣
↪pluss::make_series(3)})},
    {"city", pluss::bundle::Slot({pluss::make_note("Accra"), pluss::make_note("Tema"),␣
↪pluss::make_note("Kumasi")})},
});

auto out = registry.is_stream_compatible();
```

Output

```
true
```

32.1.37 keys

Signature

```
const std::vector<std::string>& keys(self) const
```

Parameters

None

Returns

`const std::vector<std::string>&` Borrowed list of names.

Notes

This call is part of the public `Registry` API and is documented here using a concrete usage example.

Examples

```
auto registry = pluss::bundle::Registry({
    {"ids", pluss::bundle::Slot({pluss::make_series(1), pluss::make_series(2),␣
↪pluss::make_series(3)})},
    {"city", pluss::bundle::Slot({pluss::make_note("Accra"), pluss::make_note("Tema"),␣
↪pluss::make_note("Kumasi")})},
});

auto out = registry.keys();
```

Output

```
["ids", "city"]
```

32.1.38 kindelize

Signature

```
pluss::kindle::Catalog kindelize(self) const
```

Parameters

None

Returns

`pluss::kindle::Catalog` Value produced by this call.

Notes

This is an explicit bridge between PlussMic models so you can switch to the representation that fits the next task.

Examples

```
auto registry = pluss::bundle::Registry({
    {"ids", pluss::bundle::Slot({pluss::make_series(1), pluss::make_series(2),␣
↪pluss::make_series(3)})},
    {"city", pluss::bundle::Slot({pluss::make_note("Accra"), pluss::make_note("Tema"),␣
↪pluss::make_note("Kumasi")})},
});

auto out = registry.kindelize();
```

Output

```
id     -> Series([1, 2, 3])
metric -> Series([10.0, 20.0, 30.0])
city   -> Note(["Accra", "Tema", "Kumasi"])
```

32.1.39 load

Signature

```
static Registry load(std::string_view source_type, std::string_view filepath)
```

Parameters

`source_type` : `std::string_view` See the signature for the supported argument form.

`filepath` : `std::string_view` See the signature for the supported argument form.

Returns

`Registry` Value produced by this call.

Notes

Loads the object from disk using the corresponding PlussMic loader route.

See also

- `static Registry load(std::string_view source_type, std::string_view filepath, const pluss::loader::Config& config)`

Examples

```
auto out = pluss::bundle::Registry::load("csv", "PlussMic/data/messy_data.csv");
```

Output

```
ids  -> Slot([Series(1), Series(2), Series(3)])
city -> Slot([Note("Accra"), Note("Tema"), Note("Kumasi")])
```

32.1.40 load

Signature

```
static Registry load(std::string_view source_type, std::string_view filepath, const␣
↪pluss::loader::Config& config)
```

Parameters

`source_type` : `std::string_view` See the signature for the supported argument form.

`filepath` : `std::string_view` See the signature for the supported argument form.

`config` : `const pluss::loader::Config&` See the signature for the supported argument form.

Returns

`Registry` Value produced by this call.

Notes

Loads the object from disk using the corresponding PlussMic loader route.

See also

- `static Registry load(std::string_view source_type, std::string_view filepath)`

Examples

```
auto out = pluss::bundle::Registry::load("csv", "PlussMic/data/messy_data.csv");
```

Output

```
ids  -> Slot([Series(1), Series(2), Series(3)])
city -> Slot([Note("Accra"), Note("Tema"), Note("Kumasi")])
```

32.1.41 load_csv

Signature

```
static Registry load_csv(std::string_view filepath)
```

Parameters

`filepath` : `std::string_view` See the signature for the supported argument form.

Returns

`Registry` Value produced by this call.

Notes

Loads the object from disk using the corresponding PlussMic loader route.

See also

- `static Registry load_csv(std::string_view filepath, const pluss::loader::ReadCsvOptions& options)`
- `static Registry load_csv(std::string_view filepath, const pluss::loader::Config& config)`

Examples

```
auto out = pluss::bundle::Registry::load_csv("PlussMic/data/messy_data.csv");
```

Output

```
ids  -> Slot([Series(1), Series(2), Series(3)])
city -> Slot([Note("Accra"), Note("Tema"), Note("Kumasi")])
```

32.1.42 load_csv

Signature

```
static Registry load_csv(std::string_view filepath, const pluss::loader::Config& config)
```

Parameters

`filepath` : `std::string_view` See the signature for the supported argument form.

`config` : `const pluss::loader::Config&` See the signature for the supported argument form.

Returns

`Registry` Value produced by this call.

Notes

Loads the object from disk using the corresponding PlussMic loader route.

See also

- `static Registry load_csv(std::string_view filepath)`
- `static Registry load_csv(std::string_view filepath, const pluss::loader::ReadCsvOptions& options)`

Examples

```
auto out = pluss::bundle::Registry::load_csv("PlussMic/data/messy_data.csv", {"nrows",␣
↪16});
```

Output

```
ids  -> Slot([Series(1), Series(2), Series(3)])
city -> Slot([Note("Accra"), Note("Tema"), Note("Kumasi")])
```

32.1.43 load_csv

Signature

```
static Registry load_csv(std::string_view filepath, const pluss::loader::ReadCsvOptions&␣
↪options)
```

Parameters

`filepath` : `std::string_view` See the signature for the supported argument form.

`options` : `const pluss::loader::ReadCsvOptions&` See the signature for the supported argument form.

Returns

`Registry` Value produced by this call.

Notes

Loads the object from disk using the corresponding PlussMic loader route.

See also

- `static Registry load_csv(std::string_view filepath)`
- `static Registry load_csv(std::string_view filepath, const pluss::loader::Config& config)`

Examples

```
pluss::loader::ReadCsvOptions options;
auto out = pluss::bundle::Registry::load_csv("PlussMic/data/messy_data.csv", options);
```

Output

```
ids  -> Slot([Series(1), Series(2), Series(3)])
city -> Slot([Note("Accra"), Note("Tema"), Note("Kumasi")])
```

32.1.44 load_excel

Signature

```
static Registry load_excel(std::string_view filepath)
```

Parameters

`filepath` : `std::string_view` See the signature for the supported argument form.

Returns

`Registry` Value produced by this call.

Notes

Loads the object from disk using the corresponding PlussMic loader route.

See also

- `static Registry load_excel(std::string_view filepath, const pluss::loader::ReadExcelOptions& options)`
- `static Registry load_excel(std::string_view filepath, const pluss::loader::Config& config)`

Examples

```
auto out = pluss::bundle::Registry::load_excel("PlussMic/data/messy_data.csv");
```

Output

```
ids  -> Slot([Series(1), Series(2), Series(3)])
city -> Slot([Note("Accra"), Note("Tema"), Note("Kumasi")])
```

32.1.45 load_excel

Signature

```
static Registry load_excel(std::string_view filepath, const pluss::loader::Config&␣
↪config)
```

Parameters

`filepath` : `std::string_view` See the signature for the supported argument form.

`config` : `const pluss::loader::Config&` See the signature for the supported argument form.

Returns

`Registry` Value produced by this call.

Notes

Loads the object from disk using the corresponding PlussMic loader route.

See also

- `static Registry load_excel(std::string_view filepath)`
- `static Registry load_excel(std::string_view filepath, const pluss::loader::ReadExcelOptions& options)`

Examples

```
auto out = pluss::bundle::Registry::load_excel("PlussMic/data/messy_data.csv", {"nrows",␣
↪16});
```

Output

```
ids  -> Slot([Series(1), Series(2), Series(3)])
city -> Slot([Note("Accra"), Note("Tema"), Note("Kumasi")])
```

32.1.46 load_excel

Signature

```
static Registry load_excel(std::string_view filepath, const␣
↪pluss::loader::ReadExcelOptions& options)
```

Parameters

`filepath` : `std::string_view` See the signature for the supported argument form.

`options` : `const pluss::loader::ReadExcelOptions&` See the signature for the supported argument form.

Returns

`Registry` Value produced by this call.

Notes

Loads the object from disk using the corresponding PlussMic loader route.

See also

- `static Registry load_excel(std::string_view filepath)`
- `static Registry load_excel(std::string_view filepath, const pluss::loader::Config& config)`

Examples

```
pluss::loader::ReadExcelOptions options;
auto out = pluss::bundle::Registry::load_excel("PlussMic/data/report.xlsx", options);
```

Output

```
ids  -> Slot([Series(1), Series(2), Series(3)])
city -> Slot([Note("Accra"), Note("Tema"), Note("Kumasi")])
```

32.1.47 load_json

Signature

```
static Registry load_json(std::string_view filepath)
```

Parameters

`filepath` : `std::string_view` See the signature for the supported argument form.

Returns

`Registry` Value produced by this call.

Notes

Loads the object from disk using the corresponding PlussMic loader route.

See also

- `static Registry load_json(std::string_view filepath, const pluss::loader::Config& config)`

Examples

```
auto out = pluss::bundle::Registry::load_json("PlussMic/data/messy_data.csv");
```

Output

```
ids  -> Slot([Series(1), Series(2), Series(3)])
city -> Slot([Note("Accra"), Note("Tema"), Note("Kumasi")])
```

32.1.48 load_json

Signature

```
static Registry load_json(std::string_view filepath, const pluss::loader::Config& config)
```

Parameters

`filepath` : `std::string_view` See the signature for the supported argument form.

`config` : `const pluss::loader::Config&` See the signature for the supported argument form.

Returns

`Registry` Value produced by this call.

Notes

Loads the object from disk using the corresponding PlussMic loader route.

See also

- `static Registry load_json(std::string_view filepath)`

Examples

```
auto out = pluss::bundle::Registry::load_json("PlussMic/data/messy_data.csv", {"nrows",␣
↪16});
```

Output

```
ids  -> Slot([Series(1), Series(2), Series(3)])
city -> Slot([Note("Accra"), Note("Tema"), Note("Kumasi")])
```

32.1.49 `load_parquet`

Signature

```
static Registry load_parquet(std::string_view filepath)
```

Parameters

`filepath` : `std::string_view` See the signature for the supported argument form.

Returns

`Registry` Value produced by this call.

Notes

Loads the object from disk using the corresponding PlussMic loader route.

See also

- `static Registry load_parquet(std::string_view filepath, const pluss::loader::Config& config)`

Examples

```
auto out = pluss::bundle::Registry::load_parquet("PlussMic/data/messy_data.csv");
```

Output

```
ids  -> Slot([Series(1), Series(2), Series(3)])
city -> Slot([Note("Accra"), Note("Tema"), Note("Kumasi")])
```

32.1.50 load_parquet

Signature

```
static Registry load_parquet(std::string_view filepath, const pluss::loader::Config&␣
↪config)
```

Parameters

`filepath` : `std::string_view` See the signature for the supported argument form.

`config` : `const pluss::loader::Config&` See the signature for the supported argument form.

Returns

`Registry` Value produced by this call.

Notes

Loads the object from disk using the corresponding PlussMic loader route.

See also

- `static Registry load_parquet(std::string_view filepath)`

Examples

```
auto out = pluss::bundle::Registry::load_parquet("PlussMic/data/messy_data.csv", {"nrows
↪", 16});
```

Output

```
ids  -> Slot([Series(1), Series(2), Series(3)])
city -> Slot([Note("Accra"), Note("Tema"), Note("Kumasi")])
```

32.1.51 load_txt

Signature

```
static Registry load_txt(std::string_view filepath)
```

Parameters

`filepath` : `std::string_view` See the signature for the supported argument form.

Returns

`Registry` Value produced by this call.

Notes

Loads the object from disk using the corresponding PlussMic loader route.

See also

- `static Registry load_txt(std::string_view filepath, const pluss::loader::Config& config)`

Examples

```
auto out = pluss::bundle::Registry::load_txt("PlussMic/data/messy_data.csv");
```

Output

```
ids  -> Slot([Series(1), Series(2), Series(3)])
city -> Slot([Note("Accra"), Note("Tema"), Note("Kumasi")])
```

32.1.52 `load_txt`

Signature

```
static Registry load_txt(std::string_view filepath, const pluss::loader::Config& config)
```

Parameters

`filepath` : `std::string_view` See the signature for the supported argument form.

`config` : `const pluss::loader::Config&` See the signature for the supported argument form.

Returns

`Registry` Value produced by this call.

Notes

Loads the object from disk using the corresponding PlussMic loader route.

See also

- `static Registry load_txt(std::string_view filepath)`

Examples

```
auto out = pluss::bundle::Registry::load_txt("PlussMic/data/messy_data.csv", {"nrows",␣
↪16});
```

Output

```
ids  -> Slot([Series(1), Series(2), Series(3)])
city -> Slot([Note("Accra"), Note("Tema"), Note("Kumasi")])
```

32.1.53 `map`

Signature

```
Registry map(self, const std::function<Slot(const Slot&)>& fn) const
```

Parameters

`fn` : `const std::function<Slot(const Slot&)>&` See the signature for the supported argument form.

Returns

`Registry` Value produced by this call.

Notes

This mutates or derives structure while keeping the surrounding object model intact.

Examples

```
auto registry = pluss::bundle::Registry({
    {"ids", pluss::bundle::Slot({pluss::make_series(1), pluss::make_series(2),␣
↪pluss::make_series(3)})},
    {"city", pluss::bundle::Slot({pluss::make_note("Accra"), pluss::make_note("Tema"),␣
↪pluss::make_note("Kumasi")})},
});

auto out = registry.map([](const pluss::bundle::Slot& slot) { return slot; });
```

Output

```
ids  -> Slot([Series(1), Series(2), Series(3)])
city -> Slot([Note("Accra"), Note("Tema"), Note("Kumasi")])
```

32.1.54 map_

Signature

```
Registry& map_(self, const std::function<Slot(const Slot&)>& fn)
```

Parameters

`fn` : `const std::function<Slot(const Slot&)>&` See the signature for the supported argument form.

Returns

`Registry&` Value produced by this call.

Notes

This mutates or derives structure while keeping the surrounding object model intact.

Examples

```
auto registry = pluss::bundle::Registry({
    {"ids", pluss::bundle::Slot({pluss::make_series(1), pluss::make_series(2),␣
↪pluss::make_series(3)})},
    {"city", pluss::bundle::Slot({pluss::make_note("Accra"), pluss::make_note("Tema"),␣
↪pluss::make_note("Kumasi")})},
});
```

(continues on next page)

(continued from previous page)

```
registry.map_([](const pluss::bundle::Slot& slot) { return slot; });
```

32.1.55 nfields

Signature

```
std::int64_t nfields(self) const
```

Parameters

None

Returns

std::int64_t Integer count, size, or positional result.

Notes

This call is part of the public Registry API and is documented here using a concrete usage example.

Examples

```
auto registry = pluss::bundle::Registry({
    {"ids", pluss::bundle::Slot({pluss::make_series(1), pluss::make_series(2),␣
↪pluss::make_series(3)})},
    {"city", pluss::bundle::Slot({pluss::make_note("Accra"), pluss::make_note("Tema"),␣
↪pluss::make_note("Kumasi")})},
});

auto out = registry.nfields();
```

Output

```
2
```

32.1.56 nslots

Signature

```
std::int64_t nslots(self) const
```

Parameters

None

Returns

std::int64_t Integer count, size, or positional result.

Notes

This call is part of the public `Registry` API and is documented here using a concrete usage example.

Examples

```
auto registry = pluss::bundle::Registry({
    {"ids", pluss::bundle::Slot({pluss::make_series(1), pluss::make_series(2),␣
↪pluss::make_series(3)})},
    {"city", pluss::bundle::Slot({pluss::make_note("Accra"), pluss::make_note("Tema"),␣
↪pluss::make_note("Kumasi")})},
});

auto out = registry.nslots();
```

Output

```
2
```

32.1.57 push

Signature

```
Registry push(self, std::string_view key, const Slot::Value& value) const
```

Parameters

`key` : `std::string_view` See the signature for the supported argument form.

`value` : `const Slot::Value&` See the signature for the supported argument form.

Returns

`Registry` Value produced by this call.

Notes

This mutates or derives structure while keeping the surrounding object model intact.

Examples

```
auto registry = pluss::bundle::Registry({
    {"ids", pluss::bundle::Slot({pluss::make_series(1), pluss::make_series(2),␣
↪pluss::make_series(3)})},
    {"city", pluss::bundle::Slot({pluss::make_note("Accra"), pluss::make_note("Tema"),␣
↪pluss::make_note("Kumasi")})},
});

auto out = registry.push("ids", pluss::make_series(4));
```

Output

```
ids  -> Slot([Series(1), Series(2), Series(3)])
city -> Slot([Note("Accra"), Note("Tema"), Note("Kumasi")])
```

32.1.58 push_

Signature

```
Registry& push_(self, std::string_view key, const Slot::Value& value)
```

Parameters

`key` : `std::string_view` See the signature for the supported argument form.

`value` : `const Slot::Value&` See the signature for the supported argument form.

Returns

`Registry&` Value produced by this call.

Notes

This mutates or derives structure while keeping the surrounding object model intact.

Examples

```
auto registry = pluss::bundle::Registry({
    {"ids", pluss::bundle::Slot({pluss::make_series(1), pluss::make_series(2),␣
↪pluss::make_series(3)})},
    {"city", pluss::bundle::Slot({pluss::make_note("Accra"), pluss::make_note("Tema"),␣
↪pluss::make_note("Kumasi")})},
});

registry.push_("ids", pluss::make_series(4));
```

32.1.59 push_all

Signature

```
Registry push_all(self, const Slot::Value& value) const
```

Parameters

`value` : `const Slot::Value&` See the signature for the supported argument form.

Returns

`Registry` Value produced by this call.

Notes

This mutates or derives structure while keeping the surrounding object model intact.

Examples

```
auto registry = pluss::bundle::Registry({
    {"ids", pluss::bundle::Slot({pluss::make_series(1), pluss::make_series(2),␣
↪pluss::make_series(3)})},
    {"city", pluss::bundle::Slot({pluss::make_note("Accra"), pluss::make_note("Tema"),␣
↪pluss::make_note("Kumasi")})},
});

auto out = registry.push_all(pluss::make_note("ok"));
```

Output

```
ids  -> Slot([Series(1), Series(2), Series(3)])
city -> Slot([Note("Accra"), Note("Tema"), Note("Kumasi")])
```

32.1.60 push_all_

Signature

```
Registry& push_all_(self, const Slot::Value& value)
```

Parameters

`value` : `const Slot::Value&` See the signature for the supported argument form.

Returns

`Registry&` Value produced by this call.

Notes

This mutates or derives structure while keeping the surrounding object model intact.

Examples

```
auto registry = pluss::bundle::Registry({
    {"ids", pluss::bundle::Slot({pluss::make_series(1), pluss::make_series(2),␣
↪pluss::make_series(3)})},
    {"city", pluss::bundle::Slot({pluss::make_note("Accra"), pluss::make_note("Tema"),␣
↪pluss::make_note("Kumasi")})},
});

registry.push_all_(pluss::make_note("ok"));
```

32.1.61 rename

Signature

```
Registry rename(self, const std::unordered_map<std::string, std::string>& mapping) const
```

Parameters

`mapping` : `const std::unordered_map<std::string, std::string>&` See the signature for the supported argument form.

Returns

`Registry` Value produced by this call.

Notes

This adjusts labels without changing the underlying payload values.

See also

- `Registry rename(self, std::initializer_list<std::pair<std::string_view, std::string_view>> mapping) const`
- `Registry rename(self, std::string_view from, std::string_view to) const`

Examples

```
auto registry = pluss::bundle::Registry({
    {"ids", pluss::bundle::Slot({pluss::make_series(1), pluss::make_series(2),␣
↪pluss::make_series(3)})},
    {"city", pluss::bundle::Slot({pluss::make_note("Accra"), pluss::make_note("Tema"),␣
↪pluss::make_note("Kumasi")})},
});

std::unordered_map<std::string, std::string> renames{{"ids", "ids_new"}};
auto out = registry.rename(renames);
```

Output

```
ids  -> Slot([Series(1), Series(2), Series(3)])
city -> Slot([Note("Accra"), Note("Tema"), Note("Kumasi")])
```

32.1.62 `rename`

Signature

```
Registry rename(self, std::initializer_list<std::pair<std::string_view, std::string_view>
↪> mapping) const
```

Parameters

`mapping` : `std::initializer_list<std::pair<std::string_view, std::string_view>>` See the signature for the supported argument form.

Returns

`Registry` Value produced by this call.

Notes

This adjusts labels without changing the underlying payload values.

See also

- `Registry rename(self, const std::unordered_map<std::string, std::string>& mapping) const`
- `Registry rename(self, std::string_view from, std::string_view to) const`

Examples

```
auto registry = pluss::bundle::Registry({
    {"ids", pluss::bundle::Slot({pluss::make_series(1), pluss::make_series(2),␣
↪pluss::make_series(3)})},
    {"city", pluss::bundle::Slot({pluss::make_note("Accra"), pluss::make_note("Tema"),␣
↪pluss::make_note("Kumasi")})},
});

auto out = registry.rename({{"ids", "ids_new"}});
```

Output

```
ids  -> Slot([Series(1), Series(2), Series(3)])
city -> Slot([Note("Accra"), Note("Tema"), Note("Kumasi")])
```

32.1.63 `rename`

Signature

```
Registry rename(self, std::string_view from, std::string_view to) const
```

Parameters

`from` : `std::string_view` See the signature for the supported argument form.

`to` : `std::string_view` See the signature for the supported argument form.

Returns

`Registry` Value produced by this call.

Notes

This adjusts labels without changing the underlying payload values.

See also

- `Registry rename(self, std::initializer_list<std::pair<std::string_view, std::string_view>> mapping) const`
- `Registry rename(self, const std::unordered_map<std::string, std::string>& mapping) const`

Examples

```
auto registry = pluss::bundle::Registry({
    {"ids", pluss::bundle::Slot({pluss::make_series(1), pluss::make_series(2),␣
↪pluss::make_series(3)})},
    {"city", pluss::bundle::Slot({pluss::make_note("Accra"), pluss::make_note("Tema"),␣
↪pluss::make_note("Kumasi")})},
});

auto out = registry.rename("ids", "ids_new");
```

Output

```
ids  -> Slot([Series(1), Series(2), Series(3)])
city -> Slot([Note("Accra"), Note("Tema"), Note("Kumasi")])
```

32.1.64 `rename_`

Signature

```
Registry& rename_(self, std::string_view from, std::string_view to)
```

Parameters

`from` : `std::string_view` See the signature for the supported argument form.

`to` : `std::string_view` See the signature for the supported argument form.

Returns

`Registry&` Value produced by this call.

Notes

This adjusts labels without changing the underlying payload values.

Examples

```
auto registry = pluss::bundle::Registry({
    {"ids", pluss::bundle::Slot({pluss::make_series(1), pluss::make_series(2),␣
↪pluss::make_series(3)})},
    {"city", pluss::bundle::Slot({pluss::make_note("Accra"), pluss::make_note("Tema"),␣
↪pluss::make_note("Kumasi")})},
});

registry.rename_("ids", "ids_new");
```

32.1.65 sample

Signature

```
Registry sample(self, std::int64_t n, std::optional<std::uint64_t> random_state =␣
↪std::nullopt) const
```

Parameters

`n` : `std::int64_t` See the signature for the supported argument form.

`random_state` : `std::optional<std::uint64_t>` See the signature for the supported argument form.

Returns

`Registry` Value produced by this call.

Notes

Use this to narrow the current object to the rows or depth range you want to inspect next.

Examples

```
auto registry = pluss::bundle::Registry({
    {"ids", pluss::bundle::Slot({pluss::make_series(1), pluss::make_series(2),␣
↪pluss::make_series(3)})},
    {"city", pluss::bundle::Slot({pluss::make_note("Accra"), pluss::make_note("Tema"),␣
↪pluss::make_note("Kumasi")})},
});

auto out = registry.sample(2, 7);
```

Output

```
ids  -> Slot([Series(1), Series(2), Series(3)])
city -> Slot([Note("Accra"), Note("Tema"), Note("Kumasi")])
```

32.1.66 save_as

Signature

```
void save_as(self, std::string_view target_type, std::string_view filepath, const␣
↪pluss::loader::Config& config =
```

Parameters

`target_type` : `std::string_view` See the signature for the supported argument form.

`filepath` : `std::string_view` See the signature for the supported argument form.

`config` : `const pluss::loader::Config&` See the signature for the supported argument form.

Returns

`void` Performs the side effect and returns no value.

Notes

Writes the current object to disk using the corresponding PlussMic saver route.

Examples

```
auto registry = pluss::bundle::Registry({
    {"ids", pluss::bundle::Slot({pluss::make_series(1), pluss::make_series(2),␣
↪pluss::make_series(3)})},
    {"city", pluss::bundle::Slot({pluss::make_note("Accra"), pluss::make_note("Tema"),␣
↪pluss::make_note("Kumasi")})},
});

registry.save_as("json", "out.json");
```

Output

```
saved: out.*
```

32.1.67 save_csv

Signature

```
void save_csv(self, std::string_view filepath) const
```

Parameters

`filepath` : `std::string_view` See the signature for the supported argument form.

Returns

`void` Performs the side effect and returns no value.

Notes

Writes the current object to disk using the corresponding PlussMic saver route.

See also

- `void save_csv(self, std::string_view filepath, const pluss::loader::Config& config) const`

Examples

```
auto registry = pluss::bundle::Registry({
    {"ids", pluss::bundle::Slot({pluss::make_series(1), pluss::make_series(2),␣
↪pluss::make_series(3)})},
    {"city", pluss::bundle::Slot({pluss::make_note("Accra"), pluss::make_note("Tema"),␣
↪pluss::make_note("Kumasi")})},
});
```

(continues on next page)

(continued from previous page)

```
registry.save_csv("out.csv");
```

Output

```
saved: out.*
```

32.1.68 save_csv

Signature

```
void save_csv(self, std::string_view filepath, const pluss::loader::Config& config) const
```

Parameters

`filepath` : `std::string_view` See the signature for the supported argument form.

`config` : `const pluss::loader::Config&` See the signature for the supported argument form.

Returns

`void` Performs the side effect and returns no value.

Notes

Writes the current object to disk using the corresponding PlussMic saver route.

See also

- `void save_csv(self, std::string_view filepath) const`

Examples

```
auto registry = pluss::bundle::Registry({
    {"ids", pluss::bundle::Slot({pluss::make_series(1), pluss::make_series(2),␣
↪pluss::make_series(3)})},
    {"city", pluss::bundle::Slot({pluss::make_note("Accra"), pluss::make_note("Tema"),␣
↪pluss::make_note("Kumasi")})},
});

registry.save_csv("out.csv");
```

Output

```
saved: out.*
```

32.1.69 save_excel

Signature

```
void save_excel(self, std::string_view filepath) const
```

Parameters

`filepath` : `std::string_view` See the signature for the supported argument form.

Returns

`void` Performs the side effect and returns no value.

Notes

Writes the current object to disk using the corresponding PlussMic saver route.

See also

- `void save_excel(self, std::string_view filepath, const pluss::loader::Config& config) const`

Examples

```
auto registry = pluss::bundle::Registry({
    {"ids", pluss::bundle::Slot({pluss::make_series(1), pluss::make_series(2),␣
↪pluss::make_series(3)})},
    {"city", pluss::bundle::Slot({pluss::make_note("Accra"), pluss::make_note("Tema"),␣
↪pluss::make_note("Kumasi")})},
});

registry.save_excel("out.xlsx");
```

Output

```
saved: out.*
```

32.1.70 save_excel

Signature

```
void save_excel(self, std::string_view filepath, const pluss::loader::Config& config)␣
↪const
```

Parameters

`filepath` : `std::string_view` See the signature for the supported argument form.

`config` : `const pluss::loader::Config&` See the signature for the supported argument form.

Returns

`void` Performs the side effect and returns no value.

Notes

Writes the current object to disk using the corresponding PlussMic saver route.

See also

- `void save_excel(self, std::string_view filepath) const`

Examples

```
auto registry = pluss::bundle::Registry({
    {"ids", pluss::bundle::Slot({pluss::make_series(1), pluss::make_series(2),␣
↪pluss::make_series(3)})},
    {"city", pluss::bundle::Slot({pluss::make_note("Accra"), pluss::make_note("Tema"),␣
↪pluss::make_note("Kumasi")})},
});

registry.save_excel("out.xlsx");
```

Output

```
saved: out.*
```

32.1.71 save_json

Signature

```
void save_json(self, std::string_view filepath) const
```

Parameters

`filepath` : `std::string_view` See the signature for the supported argument form.

Returns

`void` Performs the side effect and returns no value.

Notes

Writes the current object to disk using the corresponding PlussMic saver route.

See also

- `void save_json(self, std::string_view filepath, const pluss::loader::Config& config) const`

Examples

```
auto registry = pluss::bundle::Registry({
    {"ids", pluss::bundle::Slot({pluss::make_series(1), pluss::make_series(2),␣
↪pluss::make_series(3)})},
    {"city", pluss::bundle::Slot({pluss::make_note("Accra"), pluss::make_note("Tema"),␣
↪pluss::make_note("Kumasi")})},
});

registry.save_json("out.json");
```

Output

```
saved: out.*
```

32.1.72 save_json

Signature

```
void save_json(self, std::string_view filepath, const pluss::loader::Config& config)␣
↪const
```

Parameters

`filepath` : `std::string_view` See the signature for the supported argument form.

`config` : `const pluss::loader::Config&` See the signature for the supported argument form.

Returns

`void` Performs the side effect and returns no value.

Notes

Writes the current object to disk using the corresponding PlussMic saver route.

See also

- `void save_json(self, std::string_view filepath) const`

Examples

```
auto registry = pluss::bundle::Registry({
    {"ids", pluss::bundle::Slot({pluss::make_series(1), pluss::make_series(2),␣
↪pluss::make_series(3)})},
    {"city", pluss::bundle::Slot({pluss::make_note("Accra"), pluss::make_note("Tema"),␣
↪pluss::make_note("Kumasi")})},
});

registry.save_json("out.json");
```

Output

```
saved: out.*
```

32.1.73 save_parquet

Signature

```
void save_parquet(self, std::string_view filepath) const
```

Parameters

`filepath` : `std::string_view` See the signature for the supported argument form.

Returns

`void` Performs the side effect and returns no value.

Notes

Writes the current object to disk using the corresponding PlussMic saver route.

See also

- `void save_parquet(self, std::string_view filepath, const pluss::loader::Config& config) const`

Examples

```
auto registry = pluss::bundle::Registry({
    {"ids", pluss::bundle::Slot({pluss::make_series(1), pluss::make_series(2),␣
↪pluss::make_series(3)})},
    {"city", pluss::bundle::Slot({pluss::make_note("Accra"), pluss::make_note("Tema"),␣
↪pluss::make_note("Kumasi")})},
});

registry.save_parquet("out.parquet");
```

Output

```
saved: out.*
```

32.1.74 save_parquet

Signature

```
void save_parquet(self, std::string_view filepath, const pluss::loader::Config& config)␣
↪const
```

Parameters

`filepath` : `std::string_view` See the signature for the supported argument form.

`config` : `const pluss::loader::Config&` See the signature for the supported argument form.

Returns

`void` Performs the side effect and returns no value.

Notes

Writes the current object to disk using the corresponding PlussMic saver route.

See also

- `void save_parquet(self, std::string_view filepath) const`

Examples

```
auto registry = pluss::bundle::Registry({
    {"ids", pluss::bundle::Slot({pluss::make_series(1), pluss::make_series(2),␣
↪pluss::make_series(3)})},
    {"city", pluss::bundle::Slot({pluss::make_note("Accra"), pluss::make_note("Tema"),␣
↪pluss::make_note("Kumasi")})},
});

registry.save_parquet("out.parquet");
```

Output

```
saved: out.*
```

32.1.75 `save_txt`

Signature

```
void save_txt(self, std::string_view filepath) const
```

Parameters

`filepath` : `std::string_view` See the signature for the supported argument form.

Returns

`void` Performs the side effect and returns no value.

Notes

Writes the current object to disk using the corresponding PlussMic saver route.

See also

- `void save_txt(self, std::string_view filepath, const pluss::loader::Config& config) const`

Examples

```
auto registry = pluss::bundle::Registry({
    {"ids", pluss::bundle::Slot({pluss::make_series(1), pluss::make_series(2),␣
↪pluss::make_series(3)})},
    {"city", pluss::bundle::Slot({pluss::make_note("Accra"), pluss::make_note("Tema"),␣
↪pluss::make_note("Kumasi")})},
});

registry.save_txt("out.txt");
```

Output

```
saved: out.*
```

32.1.76 save_txt

Signature

```
void save_txt(self, std::string_view filepath, const pluss::loader::Config& config) const
```

Parameters

`filepath` : `std::string_view` See the signature for the supported argument form.

`config` : `const pluss::loader::Config&` See the signature for the supported argument form.

Returns

`void` Performs the side effect and returns no value.

Notes

Writes the current object to disk using the corresponding PlussMic saver route.

See also

- `void save_txt(self, std::string_view filepath) const`

Examples

```
auto registry = pluss::bundle::Registry({
    {"ids", pluss::bundle::Slot({pluss::make_series(1), pluss::make_series(2),␣
↪pluss::make_series(3)})},
    {"city", pluss::bundle::Slot({pluss::make_note("Accra"), pluss::make_note("Tema"),␣
↪pluss::make_note("Kumasi")})},
});

registry.save_txt("out.txt");
```

Output

```
saved: out.*
```

32.1.77 select

Signature

```
Registry select(self, const std::vector<std::string>& keys) const
```

Parameters

`keys` : `const std::vector<std::string>&` See the signature for the supported argument form.

Returns

`Registry` Value produced by this call.

Notes

Use this to reduce the current object to the rows or fields that matter for the next step.

See also

- `Registry select(self, std::initializer_list<std::string_view> keys) const`
- `Registry select(self, std::string_view key) const`

Examples

```
auto registry = pluss::bundle::Registry({
    {"ids", pluss::bundle::Slot({pluss::make_series(1), pluss::make_series(2),␣
↪pluss::make_series(3)})},
    {"city", pluss::bundle::Slot({pluss::make_note("Accra"), pluss::make_note("Tema"),␣
↪pluss::make_note("Kumasi")})},
});

std::vector<std::string> names{"ids"};
auto out = registry.select(names);
```

Output

```
ids  -> Slot([Series(1), Series(2), Series(3)])
city -> Slot([Note("Accra"), Note("Tema"), Note("Kumasi")])
```

32.1.78 `select`

Signature

```
Registry select(self, std::initializer_list<std::string_view> keys) const
```

Parameters

`keys` : `std::initializer_list<std::string_view>` See the signature for the supported argument form.

Returns

`Registry` Value produced by this call.

Notes

Use this to reduce the current object to the rows or fields that matter for the next step.

See also

- `Registry select(self, const std::vector<std::string>& keys) const`
- `Registry select(self, std::string_view key) const`

Examples

```
auto registry = pluss::bundle::Registry({
    {"ids", pluss::bundle::Slot({pluss::make_series(1), pluss::make_series(2),␣
↪pluss::make_series(3)})},
    {"city", pluss::bundle::Slot({pluss::make_note("Accra"), pluss::make_note("Tema"),␣
↪pluss::make_note("Kumasi")})},
});

auto out = registry.select({"ids"});
```

Output

```
ids  -> Slot([Series(1), Series(2), Series(3)])
city -> Slot([Note("Accra"), Note("Tema"), Note("Kumasi")])
```

32.1.79 `select`

Signature

```
Registry select(self, std::string_view key) const
```

Parameters

`key` : `std::string_view` See the signature for the supported argument form.

Returns

`Registry` Value produced by this call.

Notes

Use this to reduce the current object to the rows or fields that matter for the next step.

See also

- `Registry select(self, std::initializer_list<std::string_view> keys) const`
- `Registry select(self, const std::vector<std::string>& keys) const`

Examples

```
auto registry = pluss::bundle::Registry({
    {"ids", pluss::bundle::Slot({pluss::make_series(1), pluss::make_series(2),␣
↪pluss::make_series(3)})},
    {"city", pluss::bundle::Slot({pluss::make_note("Accra"), pluss::make_note("Tema"),␣
↪pluss::make_note("Kumasi")})},
});

auto out = registry.select("ids");
```

Output

```
ids  -> Slot([Series(1), Series(2), Series(3)])
city -> Slot([Note("Accra"), Note("Tema"), Note("Kumasi")])
```

32.1.80 set_slot

Signature

```
Registry set_slot(self, std::string_view key, const Slot& value) const
```

Parameters

`key` : `std::string_view` See the signature for the supported argument form.

`value` : `const Slot&` See the signature for the supported argument form.

Returns

`Registry` Value produced by this call.

Notes

This mutates or derives structure while keeping the surrounding object model intact.

Examples

```
auto registry = pluss::bundle::Registry({
    {"ids", pluss::bundle::Slot({pluss::make_series(1), pluss::make_series(2),␣
↪pluss::make_series(3)})},
    {"city", pluss::bundle::Slot({pluss::make_note("Accra"), pluss::make_note("Tema"),␣
↪pluss::make_note("Kumasi")})},
});

auto out = registry.set_slot("score", pluss::bundle::Slot({pluss::make_series(0.9),␣
↪pluss::make_series(0.7), pluss::make_series(0.8)}));
```

Output

```
ids  -> Slot([Series(1), Series(2), Series(3)])
city -> Slot([Note("Accra"), Note("Tema"), Note("Kumasi")])
```

32.1.81 set_slot_

Signature

```
Registry& set_slot_(self, std::string_view key, const Slot& value)
```

Parameters

`key` : `std::string_view` See the signature for the supported argument form.

`value` : `const Slot&` See the signature for the supported argument form.

Returns

`Registry&` Value produced by this call.

Notes

This mutates or derives structure while keeping the surrounding object model intact.

Examples

```
auto registry = pluss::bundle::Registry({
    {"ids", pluss::bundle::Slot({pluss::make_series(1), pluss::make_series(2),␣
↪pluss::make_series(3)})},
    {"city", pluss::bundle::Slot({pluss::make_note("Accra"), pluss::make_note("Tema"),␣
↪pluss::make_note("Kumasi")})},
});

registry.set_slot_("score", pluss::bundle::Slot({pluss::make_series(0.9), pluss::make_
↪series(0.7), pluss::make_series(0.8)}));
```

32.1.82 `set_slots`

Signature

```
Registry set_slots(self, std::initializer_list<std::pair<std::string, Slot>> items) const
```

Parameters

`items` : `std::initializer_list<std::pair<std::string, Slot>>` See the signature for the supported argument form.

Returns

`Registry` Value produced by this call.

Notes

This mutates or derives structure while keeping the surrounding object model intact.

See also

- `Registry set_slots(self, std::initializer_list<std::pair<std::string_view, Slot>> items) const`

Examples

```
auto registry = pluss::bundle::Registry({
    {"ids", pluss::bundle::Slot({pluss::make_series(1), pluss::make_series(2),␣
↪pluss::make_series(3)})},
    {"city", pluss::bundle::Slot({pluss::make_note("Accra"), pluss::make_note("Tema"),␣
↪pluss::make_note("Kumasi")})},
});

auto out = registry.set_slots({
```

(continues on next page)

(continued from previous page)

```
    {"score", pluss::bundle::Slot({pluss::make_series(0.9), pluss::make_series(0.7),␣
↪pluss::make_series(0.8)})},
});
```

Output

```
ids  -> Slot([Series(1), Series(2), Series(3)])
city -> Slot([Note("Accra"), Note("Tema"), Note("Kumasi")])
```

32.1.83 set_slots

Signature

```
Registry set_slots(self, std::initializer_list<std::pair<std::string_view, Slot>> items)␣
↪const
```

Parameters

`items` : `std::initializer_list<std::pair<std::string_view, Slot>>` See the signature for the supported argument form.

Returns

`Registry` Value produced by this call.

Notes

This mutates or derives structure while keeping the surrounding object model intact.

See also

- `Registry set_slots(self, std::initializer_list<std::pair<std::string, Slot>> items) const`

Examples

```
auto registry = pluss::bundle::Registry({
    {"ids", pluss::bundle::Slot({pluss::make_series(1), pluss::make_series(2),␣
↪pluss::make_series(3)})},
    {"city", pluss::bundle::Slot({pluss::make_note("Accra"), pluss::make_note("Tema"),␣
↪pluss::make_note("Kumasi")})},
});

auto out = registry.set_slots({
    {"score", pluss::bundle::Slot({pluss::make_series(0.9), pluss::make_series(0.7),␣
↪pluss::make_series(0.8)})},
});
```

Output

```
ids  -> Slot([Series(1), Series(2), Series(3)])
city -> Slot([Note("Accra"), Note("Tema"), Note("Kumasi")])
```

32.1.84 set_slots_

Signature

```
Registry& set_slots_(self, std::initializer_list<std::pair<std::string, Slot>> items)
```

Parameters

`items` : `std::initializer_list<std::pair<std::string, Slot>>` See the signature for the supported argument form.

Returns

`Registry&` Value produced by this call.

Notes

This mutates or derives structure while keeping the surrounding object model intact.

See also

- `Registry& set_slots_(self, std::initializer_list<std::pair<std::string_view, Slot>> items)`

Examples

```
auto registry = pluss::bundle::Registry({
    {"ids", pluss::bundle::Slot({pluss::make_series(1), pluss::make_series(2),␣
↪pluss::make_series(3)})},
    {"city", pluss::bundle::Slot({pluss::make_note("Accra"), pluss::make_note("Tema"),␣
↪pluss::make_note("Kumasi")})},
});

registry.set_slots_({
    {"score", pluss::bundle::Slot({pluss::make_series(0.9), pluss::make_series(0.7),␣
↪pluss::make_series(0.8)})},
});
```

32.1.85 set_slots_

Signature

```
Registry& set_slots_(self, std::initializer_list<std::pair<std::string_view, Slot>>␣
↪items)
```

Parameters

`items` : `std::initializer_list<std::pair<std::string_view, Slot>>` See the signature for the supported argument form.

Returns

`Registry&` Value produced by this call.

Notes

This mutates or derives structure while keeping the surrounding object model intact.

See also

- `Registry& set_slots_(self, std::initializer_list<std::pair<std::string, Slot>> items)`

Examples

```
auto registry = pluss::bundle::Registry({
    {"ids", pluss::bundle::Slot({pluss::make_series(1), pluss::make_series(2),␣
↪pluss::make_series(3)})},
    {"city", pluss::bundle::Slot({pluss::make_note("Accra"), pluss::make_note("Tema"),␣
↪pluss::make_note("Kumasi")})},
});

registry.set_slots_({
    {"score", pluss::bundle::Slot({pluss::make_series(0.9), pluss::make_series(0.7),␣
↪pluss::make_series(0.8)})},
});
```

32.1.86 show

Signature

```
void show(self, std::ostream& os) const
```

Parameters

`os` : `std::ostream&` See the signature for the supported argument form.

Returns

`void` Performs the side effect and returns no value.

Notes

This renders a human-readable view for inspection instead of creating a new analytic object.

Examples

```
auto registry = pluss::bundle::Registry({
    {"ids", pluss::bundle::Slot({pluss::make_series(1), pluss::make_series(2),␣
↪pluss::make_series(3)})},
```

(continues on next page)

(continued from previous page)

```
    {"city", pluss::bundle::Slot({pluss::make_note("Accra"), pluss::make_note("Tema"),␣
↪pluss::make_note("Kumasi")})},
});

registry.show(std::cout);
```

Output

```
ids  -> Slot([Series(1), Series(2), Series(3)])
city -> Slot([Note("Accra"), Note("Tema"), Note("Kumasi")])
```

32.1.87 slice

Signature

```
Registry slice(self, std::int64_t start, std::int64_t stop) const
```

Parameters

`start` : `std::int64_t` See the signature for the supported argument form.

`stop` : `std::int64_t` See the signature for the supported argument form.

Returns

`Registry` Value produced by this call.

Notes

Use this to narrow the current object to the rows or depth range you want to inspect next.

Examples

```
auto registry = pluss::bundle::Registry({
    {"ids", pluss::bundle::Slot({pluss::make_series(1), pluss::make_series(2),␣
↪pluss::make_series(3)})},
    {"city", pluss::bundle::Slot({pluss::make_note("Accra"), pluss::make_note("Tema"),␣
↪pluss::make_note("Kumasi")})},
});

auto out = registry.slice(0, 2);
```

Output

```
ids  -> Slot([Series(1), Series(2), Series(3)])
city -> Slot([Note("Accra"), Note("Tema"), Note("Kumasi")])
```

32.1.88 slot

Signature

```
Slot slot(self, std::string_view key) const
```

Parameters

key : std::string_view See the signature for the supported argument form.

Returns

Slot Value produced by this call.

Notes

This call is part of the public Registry API and is documented here using a concrete usage example.

Examples

```
auto registry = pluss::bundle::Registry({
    {"ids", pluss::bundle::Slot({pluss::make_series(1), pluss::make_series(2),␣
↪pluss::make_series(3)})},
    {"city", pluss::bundle::Slot({pluss::make_note("Accra"), pluss::make_note("Tema"),␣
↪pluss::make_note("Kumasi")})},
});

auto out = registry.slot("ids");
```

Output

```
Slot([Series(1), Series(2), Series(3)])
```

32.1.89 slot_sizes

Signature

```
std::unordered_map<std::string, std::int64_t> slot_sizes(self) const
```

Parameters

None

Returns

std::unordered_map<std::string, std::int64_t> Value produced by this call.

Notes

Use this when you want a summary view instead of carrying the full input forward unchanged.

Examples

```
auto registry = pluss::bundle::Registry({
    {"ids", pluss::bundle::Slot({pluss::make_series(1), pluss::make_series(2),␣
↪pluss::make_series(3)})},
    {"city", pluss::bundle::Slot({pluss::make_note("Accra"), pluss::make_note("Tema"),␣
↪pluss::make_note("Kumasi")})},
});

auto out = registry.slot_sizes();
```

Output

```
{"ids": 3, "city": 3}
```

32.1.90 slots

Signature

```
std::vector<Slot> slots(self) const
```

Parameters

None

Returns

std::vector<Slot> Materialized collection of slot objects.

Notes

This call is part of the public Registry API and is documented here using a concrete usage example.

Examples

```
auto registry = pluss::bundle::Registry({
    {"ids", pluss::bundle::Slot({pluss::make_series(1), pluss::make_series(2),␣
↪pluss::make_series(3)})},
    {"city", pluss::bundle::Slot({pluss::make_note("Accra"), pluss::make_note("Tema"),␣
↪pluss::make_note("Kumasi")})},
});

auto out = registry.slots();
```

Output

```
[Slot(ids), Slot(city)]
```

32.1.91 tail

Signature

```
Registry tail(self, std::int64_t n) const
```

Parameters

`n` : `std::int64_t` See the signature for the supported argument form.

Returns

`Registry` Value produced by this call.

Notes

Use this to narrow the current object to the rows or depth range you want to inspect next.

Examples

```
auto registry = pluss::bundle::Registry({
    {"ids", pluss::bundle::Slot({pluss::make_series(1), pluss::make_series(2),␣
↪pluss::make_series(3)})},
    {"city", pluss::bundle::Slot({pluss::make_note("Accra"), pluss::make_note("Tema"),␣
↪pluss::make_note("Kumasi")})},
});

auto out = registry.tail(2);
```

Output

```
ids  -> Slot([Series(1), Series(2), Series(3)])
city -> Slot([Note("Accra"), Note("Tema"), Note("Kumasi")])
```

32.1.92 `take`

Signature

```
Registry take(self, const std::vector<std::int64_t>& indices) const
```

Parameters

`indices` : `const std::vector<std::int64_t>&` See the signature for the supported argument form.

Returns

`Registry` Value produced by this call.

Notes

Use this to narrow the current object to the rows or depth range you want to inspect next.

See also

- `Registry take(self, std::span<const std::int64_t> indices) const`

Examples

```cpp
auto registry = pluss::bundle::Registry({
    {"ids", pluss::bundle::Slot({pluss::make_series(1), pluss::make_series(2),␣
↪pluss::make_series(3)})},
    {"city", pluss::bundle::Slot({pluss::make_note("Accra"), pluss::make_note("Tema"),␣
↪pluss::make_note("Kumasi")})},
});

std::vector<std::int64_t> picks{0, 2};
auto out = registry.take(picks);
```

Output

```
ids  -> Slot([Series(1), Series(2), Series(3)])
city -> Slot([Note("Accra"), Note("Tema"), Note("Kumasi")])
```

32.1.93 take

Signature

```cpp
Registry take(self, std::span<const std::int64_t> indices) const
```

Parameters

`indices` : `std::span<const std::int64_t>` See the signature for the supported argument form.

Returns

`Registry` Value produced by this call.

Notes

Use this to narrow the current object to the rows or depth range you want to inspect next.

See also

- `Registry take(self, const std::vector<std::int64_t>& indices) const`

Examples

```cpp
auto registry = pluss::bundle::Registry({
    {"ids", pluss::bundle::Slot({pluss::make_series(1), pluss::make_series(2),␣
↪pluss::make_series(3)})},
    {"city", pluss::bundle::Slot({pluss::make_note("Accra"), pluss::make_note("Tema"),␣
↪pluss::make_note("Kumasi")})},
});

std::vector<std::int64_t> picks{0, 2};
auto out = registry.take(picks);
```

Output

```
ids  -> Slot([Series(1), Series(2), Series(3)])
city -> Slot([Note("Accra"), Note("Tema"), Note("Kumasi")])
```

32.1.94 take_values

Signature

```
Registry take_values(self, std::string_view key, std::span<const std::int64_t> indices)␣
↪const
```

Parameters

key : std::string_view See the signature for the supported argument form.

indices : std::span<const std::int64_t> See the signature for the supported argument form.

Returns

Registry Value produced by this call.

Notes

This call is part of the public Registry API and is documented here using a concrete usage example.

Examples

```
auto registry = pluss::bundle::Registry({
    {"ids", pluss::bundle::Slot({pluss::make_series(1), pluss::make_series(2),␣
↪pluss::make_series(3)})},
    {"city", pluss::bundle::Slot({pluss::make_note("Accra"), pluss::make_note("Tema"),␣
↪pluss::make_note("Kumasi")})},
});

std::vector<std::int64_t> picks{0, 2};
auto out = registry.take_values("ids", picks);
```

Output

```
ids  -> Slot([Series(1), Series(2), Series(3)])
city -> Slot([Note("Accra"), Note("Tema"), Note("Kumasi")])
```

32.1.95 take_values_

Signature

```
Registry& take_values_(self, std::string_view key, std::span<const std::int64_t> indices)
```

Parameters

key : `std::string_view` See the signature for the supported argument form.

indices : `std::span<const std::int64_t>` See the signature for the supported argument form.

Returns

`Registry&` Value produced by this call.

Notes

This call is part of the public `Registry` API and is documented here using a concrete usage example.

Examples

```
auto registry = pluss::bundle::Registry({
    {"ids", pluss::bundle::Slot({pluss::make_series(1), pluss::make_series(2),␣
↪pluss::make_series(3)})},
    {"city", pluss::bundle::Slot({pluss::make_note("Accra"), pluss::make_note("Tema"),␣
↪pluss::make_note("Kumasi")})},
});

std::vector<std::int64_t> picks{0, 2};
registry.take_values_("ids", picks);
```

32.1.96 `to_json`

Signature

```
std::string to_json(self) const
```

Parameters

None

Returns

`std::string` Rendered string result.

Notes

This call is part of the public `Registry` API and is documented here using a concrete usage example.

Examples

```
auto registry = pluss::bundle::Registry({
    {"ids", pluss::bundle::Slot({pluss::make_series(1), pluss::make_series(2),␣
↪pluss::make_series(3)})},
    {"city", pluss::bundle::Slot({pluss::make_note("Accra"), pluss::make_note("Tema"),␣
↪pluss::make_note("Kumasi")})},
});

auto out = registry.to_json();
```

Output

```
ids  -> Slot([Series(1), Series(2), Series(3)])
city -> Slot([Note("Accra"), Note("Tema"), Note("Kumasi")])
```

32.1.97 to_kindle

Signature

```
pluss::kindle::Catalog to_kindle(self) const
```

Parameters

None

Returns

`pluss::kindle::Catalog` Value produced by this call.

Notes

This is an explicit bridge between PlussMic models so you can switch to the representation that fits the next task.

Examples

```
auto registry = pluss::bundle::Registry({
    {"ids", pluss::bundle::Slot({pluss::make_series(1), pluss::make_series(2),␣
↪pluss::make_series(3)})},
    {"city", pluss::bundle::Slot({pluss::make_note("Accra"), pluss::make_note("Tema"),␣
↪pluss::make_note("Kumasi")})},
});

auto out = registry.to_kindle();
```

Output

```
id     -> Series([1, 2, 3])
metric -> Series([10.0, 20.0, 30.0])
city   -> Note(["Accra", "Tema", "Kumasi"])
```

32.1.98 to_string

Signature

```
std::string to_string(self) const
```

Parameters

None

Returns

`std::string` Rendered string result.

Notes

This call is part of the public `Registry` API and is documented here using a concrete usage example.

Examples

```
auto registry = pluss::bundle::Registry({
    {"ids", pluss::bundle::Slot({pluss::make_series(1), pluss::make_series(2),␣
↪pluss::make_series(3)})},
    {"city", pluss::bundle::Slot({pluss::make_note("Accra"), pluss::make_note("Tema"),␣
↪pluss::make_note("Kumasi")})},
});

auto out = registry.to_string();
```

Output

```
ids  -> Slot([Series(1), Series(2), Series(3)])
city -> Slot([Note("Accra"), Note("Tema"), Note("Kumasi")])
```

32.1.99 `transform_values`

Signature

```
Registry transform_values(self, std::string_view key, const std::function
↪<Slot::Value(const Slot::Value&)>& fn) const
```

Parameters

`key` : `std::string_view` See the signature for the supported argument form.

`fn` : `const std::function<Slot::Value(const Slot::Value&)>&` See the signature for the supported argument form.

Returns

`Registry` Value produced by this call.

Notes

This call is part of the public `Registry` API and is documented here using a concrete usage example.

Examples

```
auto registry = pluss::bundle::Registry({
    {"ids", pluss::bundle::Slot({pluss::make_series(1), pluss::make_series(2),␣
↪pluss::make_series(3)})},
    {"city", pluss::bundle::Slot({pluss::make_note("Accra"), pluss::make_note("Tema"),␣
↪pluss::make_note("Kumasi")})},
});
```

(continues on next page)

(continued from previous page)

```
auto out = registry.transform_values("ids", [](const pluss::bundle::Slot::Value& value) -
↪> pluss::bundle::Slot::Value { if (std::holds_alternative<pluss::Series>(value))␣
↪return std::get<pluss::Series>(value).add(10); return value; });
```

Output

```
ids  -> Slot([Series(1), Series(2), Series(3)])
city -> Slot([Note("Accra"), Note("Tema"), Note("Kumasi")])
```

32.1.100 transform_values_

Signature

```
Registry& transform_values_(self, std::string_view key, const std::function
↪<Slot::Value(const Slot::Value&)>& fn)
```

Parameters

key : `std::string_view` See the signature for the supported argument form.

fn : `const std::function<Slot::Value(const Slot::Value&)>&` See the signature for the supported argument form.

Returns

`Registry&` Value produced by this call.

Notes

This call is part of the public `Registry` API and is documented here using a concrete usage example.

Examples

```
auto registry = pluss::bundle::Registry({
    {"ids", pluss::bundle::Slot({pluss::make_series(1), pluss::make_series(2),␣
↪pluss::make_series(3)})},
    {"city", pluss::bundle::Slot({pluss::make_note("Accra"), pluss::make_note("Tema"),␣
↪pluss::make_note("Kumasi")})},
});

registry.transform_values_("ids", [](const pluss::bundle::Slot::Value& value) ->␣
↪pluss::bundle::Slot::Value { if (std::holds_alternative<pluss::Series>(value)) return␣
↪std::get<pluss::Series>(value).add(10); return value; });
```

32.1.101 type_profiles

Signature

```
std::unordered_map<std::string, std::unordered_map<std::string, std::int64_t>> type_
↪profiles(self) const
```

Parameters

None

Returns

`std::unordered_map<std::string, std::unordered_map<std::string, std::int64_t>>` Value produced by this call.

Notes

Use this when you want a summary view instead of carrying the full input forward unchanged.

Examples

```
auto registry = pluss::bundle::Registry({
    {"ids", pluss::bundle::Slot({pluss::make_series(1), pluss::make_series(2),␣
↪pluss::make_series(3)})},
    {"city", pluss::bundle::Slot({pluss::make_note("Accra"), pluss::make_note("Tema"),␣
↪pluss::make_note("Kumasi")})},
});

auto out = registry.type_profiles();
```

Output

```
{"ids": {"Series": 3}, "city": {"Note": 3}}
```

32.1.102 unique

Signature

```
bool unique(self) const
```

Parameters

None

Returns

`bool` Boolean status or membership result.

Notes

This call is part of the public `Registry` API and is documented here using a concrete usage example.

Examples

```
auto registry = pluss::bundle::Registry({
    {"ids", pluss::bundle::Slot({pluss::make_series(1), pluss::make_series(2),␣
↪pluss::make_series(3)})},
    {"city", pluss::bundle::Slot({pluss::make_note("Accra"), pluss::make_note("Tema"),␣
↪pluss::make_note("Kumasi")})},
});
```

(continues on next page)

(continued from previous page)

```
auto out = registry.unique();
```

Output

```
true
```

CHAPTER

THIRTYTHREE

AISLE API REFERENCE

CHAPTER

THIRTYFOUR

PRACTICAL AISLE API REFERENCE

34.1 API Reference Style

This page documents the full public `Aisle` surface for depth-stacked registries.

34.1.1 apply_slot

Signature

```
Aisle apply_slot(self, std::string_view key, const std::function<Slot(const Slot&)>& fn)␣
↪const
```

Parameters

`key` : `std::string_view` See the signature for the supported argument form.

`fn` : `const std::function<Slot(const Slot&)>&` See the signature for the supported argument form.

Returns

`Aisle` Value produced by this call.

Notes

This mutates or derives structure while keeping the surrounding object model intact.

Examples

```
auto r0 = pluss::bundle::Registry({
    {"ids", pluss::bundle::Slot({pluss::make_series(1), pluss::make_series(2)})},
});
auto r1 = pluss::bundle::Registry({
    {"ids", pluss::bundle::Slot({pluss::make_series(3), pluss::make_series(4)})},
});
auto aisle = pluss::bundle::Aisle({r0, r1});

auto out = aisle.apply_slot("ids", [](const pluss::bundle::Slot& slot) { return slot; });
```

Output

```
depth0 -> Registry(ids -> Slot([Series(1), Series(2)]))
depth1 -> Registry(ids -> Slot([Series(3), Series(4)]))
```

34.1.2 as_kindle

Signature

```
pluss::kindle::Archive as_kindle(self) const
```

Parameters

None

Returns

`pluss::kindle::Archive` Value produced by this call.

Notes

This is an explicit bridge between PlussMic models so you can switch to the representation that fits the next task.

Examples

```
auto r0 = pluss::bundle::Registry({
    {"ids", pluss::bundle::Slot({pluss::make_series(1), pluss::make_series(2)})},
});
auto r1 = pluss::bundle::Registry({
    {"ids", pluss::bundle::Slot({pluss::make_series(3), pluss::make_series(4)})},
});
auto aisle = pluss::bundle::Aisle({r0, r1});

auto out = aisle.as_kindle();
```

Output

```
train -> Catalog(metric -> Series([10.0, 20.0]))
test  -> Catalog(metric -> Series([30.0, 40.0]))
```

34.1.3 concat_depth

Signature

```
Aisle concat_depth(self, const Aisle& other) const
```

Parameters

`other` : `const Aisle&` See the signature for the supported argument form.

Returns

`Aisle` Value produced by this call.

Notes

Use this when you need to merge compatible objects or extend the current one with another input.

Examples

```
auto r0 = pluss::bundle::Registry({
    {"ids", pluss::bundle::Slot({pluss::make_series(1), pluss::make_series(2)})},
});
auto r1 = pluss::bundle::Registry({
    {"ids", pluss::bundle::Slot({pluss::make_series(3), pluss::make_series(4)})},
});
auto aisle = pluss::bundle::Aisle({r0, r1});

auto out = aisle.concat_depth(aisle);
```

Output

```
depth0 -> Registry(ids -> Slot([Series(1), Series(2)]))
depth1 -> Registry(ids -> Slot([Series(3), Series(4)]))
```

34.1.4 `defined`

Signature

```
bool defined(self) const
```

Parameters

None

Returns

`bool` Boolean status or membership result.

Notes

This call is part of the public `Aisle` API and is documented here using a concrete usage example.

Examples

```
auto r0 = pluss::bundle::Registry({
    {"ids", pluss::bundle::Slot({pluss::make_series(1), pluss::make_series(2)})},
});
auto r1 = pluss::bundle::Registry({
    {"ids", pluss::bundle::Slot({pluss::make_series(3), pluss::make_series(4)})},
});
auto aisle = pluss::bundle::Aisle({r0, r1});

auto out = aisle.defined();
```

Output

```
true
```

34.1.5 depth

Signature

```
std::int64_t depth(self) const
```

Parameters

None

Returns

`std::int64_t` Integer count, size, or positional result.

Notes

This call is part of the public `Aisle` API and is documented here using a concrete usage example.

Examples

```
auto r0 = pluss::bundle::Registry({
    {"ids", pluss::bundle::Slot({pluss::make_series(1), pluss::make_series(2)})},
});
auto r1 = pluss::bundle::Registry({
    {"ids", pluss::bundle::Slot({pluss::make_series(3), pluss::make_series(4)})},
});
auto aisle = pluss::bundle::Aisle({r0, r1});

auto out = aisle.depth();
```

Output

```
2
```

34.1.6 empty

Signature

```
bool empty(self) const
```

Parameters

None

Returns

`bool` Boolean status or membership result.

Notes

This call is part of the public `Aisle` API and is documented here using a concrete usage example.

Examples

```
auto r0 = pluss::bundle::Registry({
    {"ids", pluss::bundle::Slot({pluss::make_series(1), pluss::make_series(2)})},
});
auto r1 = pluss::bundle::Registry({
    {"ids", pluss::bundle::Slot({pluss::make_series(3), pluss::make_series(4)})},
});
auto aisle = pluss::bundle::Aisle({r0, r1});

auto out = aisle.empty();
```

Output

```
false
```

34.1.7 from_kindle

Signature

```
static Aisle from_kindle(const pluss::kindle::Archive& archive)
```

Parameters

`archive` : `const pluss::kindle::Archive&` See the signature for the supported argument form.

Returns

`Aisle` Value produced by this call.

Notes

This is an explicit bridge between PlussMic models so you can switch to the representation that fits the next task.

Examples

```
using namespace pluss::kindle::literals;

auto c0 = pluss::kindle::Catalog({
    "metric"_k = pluss::kindle::Stream(pluss::make_series({10.0, 20.0})),
});
auto c1 = pluss::kindle::Catalog({
    "metric"_k = pluss::kindle::Stream(pluss::make_series({30.0, 40.0})),
});
auto archive = pluss::kindle::Archive({c0, c1}, pluss::kindle::Stream(pluss::make_note({
↪"train", "test"})));

auto out = pluss::bundle::Aisle::from_kindle(archive);
```

Output

```
depth0 -> Registry(ids -> Slot([Series(1), Series(2)]))
depth1 -> Registry(ids -> Slot([Series(3), Series(4)]))
```

34.1.8 head

Signature

```
Aisle head(self, std::int64_t n) const
```

Parameters

n : std::int64_t See the signature for the supported argument form.

Returns

Aisle Value produced by this call.

Notes

Use this to narrow the current object to the rows or depth range you want to inspect next.

Examples

```
auto r0 = pluss::bundle::Registry({
    {"ids", pluss::bundle::Slot({pluss::make_series(1), pluss::make_series(2)})},
});
auto r1 = pluss::bundle::Registry({
    {"ids", pluss::bundle::Slot({pluss::make_series(3), pluss::make_series(4)})},
});
auto aisle = pluss::bundle::Aisle({r0, r1});

auto out = aisle.head(1);
```

Output

```
depth0 -> Registry(ids -> Slot([Series(1), Series(2)]))
depth1 -> Registry(ids -> Slot([Series(3), Series(4)]))
```

34.1.9 kindelize

Signature

```
pluss::kindle::Archive kindelize(self) const
```

Parameters

None

Returns

`pluss::kindle::Archive` Value produced by this call.

Notes

This is an explicit bridge between PlussMic models so you can switch to the representation that fits the next task.

Examples

```
auto r0 = pluss::bundle::Registry({
    {"ids", pluss::bundle::Slot({pluss::make_series(1), pluss::make_series(2)})},
});
auto r1 = pluss::bundle::Registry({
    {"ids", pluss::bundle::Slot({pluss::make_series(3), pluss::make_series(4)})},
});
auto aisle = pluss::bundle::Aisle({r0, r1});

auto out = aisle.kindelize();
```

Output

```
train -> Catalog(metric -> Series([10.0, 20.0]))
test  -> Catalog(metric -> Series([30.0, 40.0]))
```

34.1.10 `labels`

Signature

```
const std::optional<Slot>& labels(self) const
```

Parameters

None

Returns

`const std::optional<Slot>&` Value produced by this call.

Notes

This call is part of the public `Aisle` API and is documented here using a concrete usage example.

Examples

```
auto r0 = pluss::bundle::Registry({
    {"ids", pluss::bundle::Slot({pluss::make_series(1), pluss::make_series(2)})},
});
auto r1 = pluss::bundle::Registry({
    {"ids", pluss::bundle::Slot({pluss::make_series(3), pluss::make_series(4)})},
});
auto aisle = pluss::bundle::Aisle({r0, r1});

auto out = aisle.labels();
```

Output

```
["train", "test"]
```

34.1.11 map

Signature

```
Aisle map(self, const std::function<Registry(const Registry&)>& fn) const
```

Parameters

fn : const std::function<Registry(const Registry&)>& See the signature for the supported argument form.

Returns

Aisle Value produced by this call.

Notes

This mutates or derives structure while keeping the surrounding object model intact.

Examples

```
auto r0 = pluss::bundle::Registry({
    {"ids", pluss::bundle::Slot({pluss::make_series(1), pluss::make_series(2)})},
});
auto r1 = pluss::bundle::Registry({
    {"ids", pluss::bundle::Slot({pluss::make_series(3), pluss::make_series(4)})},
});
auto aisle = pluss::bundle::Aisle({r0, r1});

auto out = aisle.map([](const pluss::bundle::Registry& registry) { return registry; });
```

Output

```
depth0 -> Registry(ids -> Slot([Series(1), Series(2)]))
depth1 -> Registry(ids -> Slot([Series(3), Series(4)]))
```

34.1.12 registries

Signature

```
const std::vector<Registry>& registries(self) const
```

Parameters

None

Returns

`const std::vector<Registry>&` Borrowed collection of registries.

Notes

This call is part of the public `Aisle` API and is documented here using a concrete usage example.

Examples

```
auto r0 = pluss::bundle::Registry({
    {"ids", pluss::bundle::Slot({pluss::make_series(1), pluss::make_series(2)})},
});
auto r1 = pluss::bundle::Registry({
    {"ids", pluss::bundle::Slot({pluss::make_series(3), pluss::make_series(4)})},
});
auto aisle = pluss::bundle::Aisle({r0, r1});

auto out = aisle.registries();
```

Output

```
[Registry(depth0), Registry(depth1)]
```

34.1.13 `registry`

Signature

```
Registry registry(self, std::int64_t i) const
```

Parameters

`i` : `std::int64_t` See the signature for the supported argument form.

Returns

`Registry` Value produced by this call.

Notes

This call is part of the public `Aisle` API and is documented here using a concrete usage example.

Examples

```
auto r0 = pluss::bundle::Registry({
    {"ids", pluss::bundle::Slot({pluss::make_series(1), pluss::make_series(2)})},
});
auto r1 = pluss::bundle::Registry({
    {"ids", pluss::bundle::Slot({pluss::make_series(3), pluss::make_series(4)})},
});
auto aisle = pluss::bundle::Aisle({r0, r1});

auto out = aisle.registry(0);
```

Output

```
ids  -> Slot([Series(1), Series(2), Series(3)])
city -> Slot([Note("Accra"), Note("Tema"), Note("Kumasi")])
```

34.1.14 select

Signature

```
Aisle select(self, std::initializer_list<std::string_view> keys) const
```

Parameters

keys : std::initializer_list<std::string_view> See the signature for the supported argument form.

Returns

Aisle Value produced by this call.

Notes

Use this to reduce the current object to the rows or fields that matter for the next step.

Examples

```
auto r0 = pluss::bundle::Registry({
    {"ids", pluss::bundle::Slot({pluss::make_series(1), pluss::make_series(2)})},
});
auto r1 = pluss::bundle::Registry({
    {"ids", pluss::bundle::Slot({pluss::make_series(3), pluss::make_series(4)})},
});
auto aisle = pluss::bundle::Aisle({r0, r1});

auto out = aisle.select({"ids"});
```

Output

```
depth0 -> Registry(ids -> Slot([Series(1), Series(2)]))
depth1 -> Registry(ids -> Slot([Series(3), Series(4)]))
```

34.1.15 show

Signature

```
void show(self, std::ostream& os) const
```

Parameters

os : std::ostream& See the signature for the supported argument form.

Returns

`void` Performs the side effect and returns no value.

Notes

This renders a human-readable view for inspection instead of creating a new analytic object.

Examples

```
auto r0 = pluss::bundle::Registry({
    {"ids", pluss::bundle::Slot({pluss::make_series(1), pluss::make_series(2)})},
});
auto r1 = pluss::bundle::Registry({
    {"ids", pluss::bundle::Slot({pluss::make_series(3), pluss::make_series(4)})},
});
auto aisle = pluss::bundle::Aisle({r0, r1});

aisle.show(std::cout);
```

Output

```
depth0 -> Registry(ids -> Slot([Series(1), Series(2)]))
depth1 -> Registry(ids -> Slot([Series(3), Series(4)]))
```

34.1.16 `slice`

Signature

```
Aisle slice(self, std::int64_t start, std::int64_t stop) const
```

Parameters

`start` : `std::int64_t` See the signature for the supported argument form.

`stop` : `std::int64_t` See the signature for the supported argument form.

Returns

`Aisle` Value produced by this call.

Notes

Use this to narrow the current object to the rows or depth range you want to inspect next.

Examples

```
auto r0 = pluss::bundle::Registry({
    {"ids", pluss::bundle::Slot({pluss::make_series(1), pluss::make_series(2)})},
});
auto r1 = pluss::bundle::Registry({
    {"ids", pluss::bundle::Slot({pluss::make_series(3), pluss::make_series(4)})},
});
auto aisle = pluss::bundle::Aisle({r0, r1});
```

(continues on next page)

(continued from previous page)

```
auto out = aisle.slice(0, 1);
```

Output

```
depth0 -> Registry(ids -> Slot([Series(1), Series(2)]))
depth1 -> Registry(ids -> Slot([Series(3), Series(4)]))
```

34.1.17 tail

Signature

```
Aisle tail(self, std::int64_t n) const
```

Parameters

n : std::int64_t See the signature for the supported argument form.

Returns

Aisle Value produced by this call.

Notes

Use this to narrow the current object to the rows or depth range you want to inspect next.

Examples

```
auto r0 = pluss::bundle::Registry({
    {"ids", pluss::bundle::Slot({pluss::make_series(1), pluss::make_series(2)})},
});
auto r1 = pluss::bundle::Registry({
    {"ids", pluss::bundle::Slot({pluss::make_series(3), pluss::make_series(4)})},
});
auto aisle = pluss::bundle::Aisle({r0, r1});

auto out = aisle.tail(1);
```

Output

```
depth0 -> Registry(ids -> Slot([Series(1), Series(2)]))
depth1 -> Registry(ids -> Slot([Series(3), Series(4)]))
```

34.1.18 to_json

Signature

```
std::string to_json(self) const
```

Parameters

None

Returns

`std::string` Rendered string result.

Notes

This call is part of the public `Aisle` API and is documented here using a concrete usage example.

Examples

```
auto r0 = pluss::bundle::Registry({
    {"ids", pluss::bundle::Slot({pluss::make_series(1), pluss::make_series(2)})},
});
auto r1 = pluss::bundle::Registry({
    {"ids", pluss::bundle::Slot({pluss::make_series(3), pluss::make_series(4)})},
});
auto aisle = pluss::bundle::Aisle({r0, r1});

auto out = aisle.to_json();
```

Output

```
depth0 -> Registry(ids -> Slot([Series(1), Series(2)]))
depth1 -> Registry(ids -> Slot([Series(3), Series(4)]))
```

34.1.19 `to_kindle`

Signature

```
pluss::kindle::Archive to_kindle(self) const
```

Parameters

None

Returns

`pluss::kindle::Archive` Value produced by this call.

Notes

This is an explicit bridge between PlussMic models so you can switch to the representation that fits the next task.

Examples

```
auto r0 = pluss::bundle::Registry({
    {"ids", pluss::bundle::Slot({pluss::make_series(1), pluss::make_series(2)})},
});
auto r1 = pluss::bundle::Registry({
    {"ids", pluss::bundle::Slot({pluss::make_series(3), pluss::make_series(4)})},
```

(continues on next page)

(continued from previous page)

```
});
auto aisle = pluss::bundle::Aisle({r0, r1});

auto out = aisle.to_kindle();
```

Output

```
train -> Catalog(metric -> Series([10.0, 20.0]))
test  -> Catalog(metric -> Series([30.0, 40.0]))
```

34.1.20 to_string

Signature

```
std::string to_string(self) const
```

Parameters

None

Returns

`std::string` Rendered string result.

Notes

This call is part of the public `Aisle` API and is documented here using a concrete usage example.

Examples

```
auto r0 = pluss::bundle::Registry({
    {"ids", pluss::bundle::Slot({pluss::make_series(1), pluss::make_series(2)})},
});
auto r1 = pluss::bundle::Registry({
    {"ids", pluss::bundle::Slot({pluss::make_series(3), pluss::make_series(4)})},
});
auto aisle = pluss::bundle::Aisle({r0, r1});

auto out = aisle.to_string();
```

Output

```
depth0 -> Registry(ids -> Slot([Series(1), Series(2)]))
depth1 -> Registry(ids -> Slot([Series(3), Series(4)]))
```

34.1.21 unique

Signature

```
bool unique(self) const
```

Parameters

None

Returns

bool Boolean status or membership result.

Notes

This call is part of the public `Aisle` API and is documented here using a concrete usage example.

Examples

```
auto r0 = pluss::bundle::Registry({
    {"ids", pluss::bundle::Slot({pluss::make_series(1), pluss::make_series(2)})},
});
auto r1 = pluss::bundle::Registry({
    {"ids", pluss::bundle::Slot({pluss::make_series(3), pluss::make_series(4)})},
});
auto aisle = pluss::bundle::Aisle({r0, r1});

auto out = aisle.unique();
```

Output

```
true
```

CHAPTER

THIRTYFIVE

BUILD OUTPUTS

35.1 HTML

```
cd /Users/yawasante/software/cxx/PlussDev/PlussMic/docs/sphinx
PATH=/Users/yawasante/software/miniconda3/envs/cpp_env/bin:$PATH make html
```

Expected output:

- `_build/html/index.html`

35.2 PDF

```
cd /Users/yawasante/software/cxx/PlussDev/PlussMic/docs/sphinx
PATH=/Users/yawasante/software/miniconda3/envs/cpp_env/bin:$PATH make pdf
```

Expected output:

- `_build/latex/plussmic_user_docs.pdf`

35.3 Notes

- HTML uses the same custom left-sidebar/top-bar shell as the PlussQit docs.
- PDF is built through `latex` plus `latexmk`.
- If HTML looks stale after a rebuild, hard refresh the browser once.